

CAPÍTULO 10

Dependencias funcionales y normalización en bases de datos relacionales

En los Capítulos del 5 al 9, mostramos varios aspectos del modelo relacional y los lenguajes asociados a él. Cada *esquema de relación* consta de un número de atributos, mientras que un *esquema de base de datos relacional* está compuesto por un número de esquemas de relación. Hasta ahora sólo hemos utilizado el sentido común del diseñador de la base de datos para agrupar los atributos y formar así un esquema de relación, o bien hemos utilizado un diseño de esquema de base de datos a partir de un modelo de datos conceptual como ER o EER, o algún otro. Estos modelos hacen que el diseñador identifique los tipos de entidad y de relación y sus respectivos atributos, lo que nos lleva a un agrupamiento natural y lógico de los atributos en relaciones cuando van seguidos por los procedimientos de mapeado del Capítulo 7. Sin embargo, aún necesitamos algún tipo de medida formal que nos indique por qué un agrupamiento de atributos en el esquema de una relación puede ser mejor que otro. Hasta el momento, en nuestro debate sobre el diseño conceptual de los Capítulos 3 y 4 y su asignación en el modelo relacional del Capítulo 7, no hemos desarrollado ningún método que nos indique la idoneidad de la calidad del diseño, aparte de la intuición del diseñador. En este capítulo vamos a ver parte de la teoría desarrollada con el objetivo de evaluar esquemas relacionales encaminados a la calidad del diseño; es decir, mediremos formalmente por qué un conjunto de agrupaciones de atributos en un esquema de relación es mejor que otro.

Hay dos niveles a los que podemos explicar la *bondad* de los esquemas de relación. El primero es el **nivel lógico** (o **conceptual**): cómo los usuarios interpretan los esquemas de relación y el significado de sus atributos. Disponer de un buen esquema de relación a este nivel permite a los usuarios comprender con claridad el significado de los datos en la relaciones y, por consiguiente, formular sus consultas correctamente. El segundo nivel es el de **implementación** (o **almacenamiento**): de qué modo se almacenan y actualizan las tuplas en una relación base. Este nivel se aplica sólo a esquemas de relación base (cómo se almacenarán físicamente los ficheros), mientras que a nivel lógico nos interesan tanto las relaciones base como las vistas (relaciones virtuales). La teoría de diseño de una base de datos relacional desarrollada en este capítulo se aplica fundamentalmente a las *relaciones base*, aunque algunos criterios de idoneidad también se utilizan en las vistas (consulte la Sección 10.1).

Como ocurre con otros muchos problemas de diseño, el de una base de datos debe llevarse a cabo usando dos metodologías: ascendente (*bottom-up*) o descendente (*top-down*). Una **metodología de diseño de tipo ascendente**, llamada también *diseño por síntesis*, tiene como punto de partida las relaciones básicas entre *atributos*

individuales, y los usa para construir los esquemas de relación. Esta metodología no es muy popular en la práctica¹, ya que tiene el problema de tener que recopilar al principio una gran cantidad de relaciones binarias entre los atributos. Como contrapartida, en una **metodología descendente**, conocida también como *diseño por análisis*, se empieza con varios agrupamientos de atributos de una relación que están juntos de forma natural, como por ejemplo, en una factura, un formulario o un informe. Las relaciones son entonces analizadas individual y colectivamente, lo que conduce a una descomposición posterior que permite conocer todas las propiedades deseables. La teoría descrita en este capítulo es aplicable a ambas metodologías de diseño, aunque es más práctica cuando se emplea en la de tipo descendente.

Iniciamos este capítulo comentando de manera informal algunos criterios en la Sección 10.1 para determinar un buen o mal esquema de relación. En la Sección 10.2 definimos el concepto de *dependencia funcional*, una restricción formal entre los atributos que es la herramienta principal para la medida formal de la idoneidad del agrupamiento de atributos en los esquemas de relación. También se estudian y analizan las propiedades de las dependencias funcionales. La Sección 10.3 se centra en las formas normales y en el proceso de normalización usando dependencias funcionales. Las formas normales sucesivas están definidas para cumplir el conjunto de restricciones deseables expresadas mediante dependencias funcionales. El procedimiento de normalización consiste en la aplicación de una serie de comprobaciones de las relaciones para cumplir con unos requisitos cada vez más restrictivos y descomponer las relaciones cuando sea necesario. En la Sección 10.4 tratamos las definiciones más generales de las formas normales que pueden aplicarse directamente a un diseño concreto y que no precisan de un análisis paso a paso y una normalización.

El Capítulo 11 continúa con el desarrollo de la teoría para un buen diseño del esquema relacional. Comentamos las propiedades deseables de la descomposición relacional (propiedad de reunión no aditiva y de preservación de dependencia funcional) y después consideramos la metodología de tipo ascendente en el diseño de una base de datos que consiste en un conjunto de algoritmos. Estos algoritmos asumen como entrada un conjunto dado de dependencias funcionales y consiguen un diseño relacional en una forma normal de destino a la vez que añade las propiedades deseables antes comentadas. También se presenta un algoritmo general que verifica si una descomposición tiene o no la propiedad de reunión sin pérdida (Algoritmo 11.1). El Capítulo 11 contiene además la definición de tipos de dependencias adicionales y formas normales avanzadas que lleva más allá la *idoneidad* de un esquema de relación.

El lector que sólo está interesado en una introducción informal a la normalización puede saltarse las Secciones 10.2.3, 10.2.4 y 10.2.5. Si no se estudia el Capítulo 11 en un curso, recomendamos una introducción rápida a las propiedades deseables de la descomposición mostradas de la Sección 11.1 y un debate de la propiedad NJB, además del Capítulo 10.

10.1 Directrices de diseño informales para los esquemas de relación

Antes de entrar en detalles con la teoría formal del diseño de bases de datos relacionales, vamos a ver en esta sección cuatro *medidas informales* de calidad para el diseño de un esquema de relación:

- La semántica de los atributos.
- La reducción de información redundante en las tuplas.
- La reducción de los valores NULL en las tuplas.
- Prohibición de la posibilidad de generar tuplas falsas.

Como podremos ver, estas medidas no siempre son independientes entre sí.

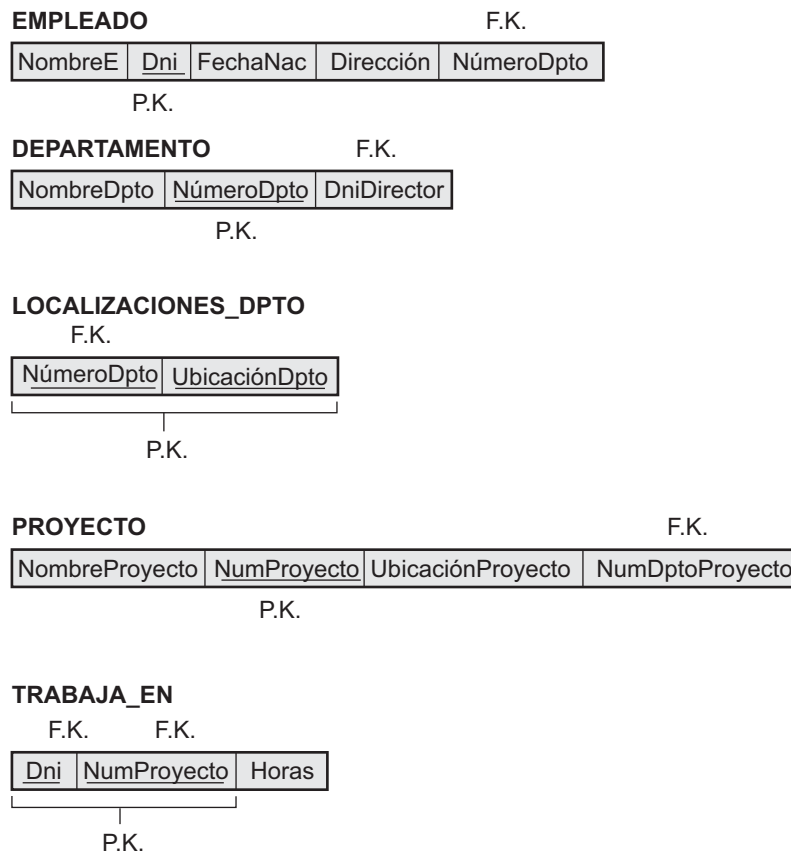
¹ El modelo relacional binario es una excepción en la que se usa esta metodología en la práctica. Un ejemplo del mismo es la metodología NIAM (Verheijen y VanBekkum 1982).

10.1.1 Impartir una semántica clara a los atributos de las relaciones

Siempre que agrupamos atributos para formar un esquema de relación asumimos que pertenecen a una relación que tiene cierta similitud con el mundo real y una interpretación propia asociada a ellos. La semántica de una relación hace referencia a la interpretación de los valores de un atributo en una tupla. En el Capítulo 5 vimos que una relación puede interpretarse como un conjunto de hechos. Si el diseño conceptual descrito en los Capítulos 3 y 4 se lleva a cabo cuidadosamente y el procedimiento de mapeado del Capítulo 7 se sigue sistemáticamente, el diseño del esquema relacional debería tener un significado claro.

En general, cuanto más sencillo es explicar la semántica de la relación, mejor será el diseño del esquema de relación. Para ilustrar esto, considere la Figura 10.1, una versión simplificada del esquema de base de datos relacional EMPRESA de la Figura 5.5, y la Figura 10.2, que muestra un ejemplo de estado de relación. El significado del esquema de relación EMPLEADO es muy simple: cada tupla representa a un empleado, con valores que contienen su nombre (NombreE), su Documento Nacional de Identidad (Dni), su fecha de nacimiento (FechaNac), su dirección (Dirección) y el número del departamento en el que trabaja (NúmeroDpto). El atributo NúmeroDpto es una *foreign key* que representa una *relación implícita* entre EMPLEADO y DEPARTAMENTO. Las semánticas de los esquemas DEPARTAMENTO y PROYECTO son también muy directas: cada tupla DEPARTAMENTO representa a una entidad departamento, y cada tupla PROYECTO es una entidad proyecto. El atributo DniDirector de DEPARTAMENTO relaciona un departamento con el empleado que es director del mismo, mientras que NumDptoProyecto de PROYECTO asocia un proyecto con el departamento que lo gestiona; ambos atributos son *foreign keys*. La facilidad con la que se pueda explicar el significado de los atributos de una relación es una *medida informal* de lo bien que está diseñada esa relación.

Figura 10.1. Una versión simplificada del esquema de base de datos relacional EMPRESA.



La semántica de los otros dos esquemas de relación de la Figura 10.1 es algo más compleja. Cada tupla de LOCALIZACIONES_DPTO consta de un número de departamento (NúmeroDpto) y una de las localizaciones del departamento (UbicaciónDpto). Cada tupla de TRABAJA_EN contiene el DNI del empleado (DniEmpleado), el número de uno de los proyectos en los que trabaja (NumProyecto) y el número de horas semanales que le dedica al mismo (Horas). Sin embargo, ambos esquemas tienen una interpretación bien definida y sin ambigüedad. El esquema LOCALIZACIONES_DPTO representa un atributo multivalor de DEPARTAMENTO, mientras que TRABAJA_EN es una relación M:N entre EMPLEADO y PROYECTO. Por consiguiente, todo el esquema de relaciones de la Figura 10.1 podría considerarse como fácil de explicar y, por tanto, bueno desde el punto de vista de contar con una semántica clara. De esta forma, podemos formular la siguiente directriz informal de diseño.

Directriz 1

Diseñar un esquema de relación para que sea fácil explicar su significado. No combine atributos de varios tipos de entidad y de relación en una única relación. Intuitivamente, si un esquema de relación se corres-

Figura 10.2. Ejemplo del estado de la base de datos para el esquema de base de datos relacional de la Figura 10.1.

EMPLEADO

NombreE	<u>Dni</u>	FechaNac	Dirección	NúmeroDpto
Pérez Pérez, José	123456789	09-01-1965	Eloy I, 98	5
Campos Sastre, Alberto	333445555	08-12-1955	Avda. Ríos, 9	5
Jiménez Celaya, Alicia	999887777	19-07-1968	Gran Vía, 38	4
Sainz Oreja, Juana	987654321	20-06-1941	Cerquillas, 67	4
Ojeda Ordóñez, Fernando.	666884444	15-09-1962	Portillo, s/n	5
Oliva Avezuela, Aurora	453453453	31-07-1972	Antón, 6	5
Pajares Morera, Luis	987987987	29-03-1969	Enebros, 90	4
Ochoa Paredes, Eduardo	888665555	10-11-1937	Las Peñas, 1	1

DEPARTAMENTO

NombreDpto	<u>NúmeroDpto</u>	DniDirector
Investigación	5	333445555
Administración	4	987654321
Sede central	1	888665555

LOCALIZACIONES_DPTO

<u>NúmeroDpto</u>	<u>UbicaciónDpto</u>
1	Madrid
4	Gijón
5	Valencia
5	Sevilla
5	Madrid

PROYECTO

NombreProyecto	<u>NumProyecto</u>	UbicaciónProyecto	NumDptoProyecto
ProductoX	1	Valencia	5
ProductoY	2	Sevilla	5
ProductoZ	3	Madrid	5
Computación	10	Gijón	4
Reorganización	20	Madrid	1
Comunicaciones	30	Gijón	4

Figura 10.2. (Continuación).**TRABAJA_EN**

<u>Dni</u>	<u>NumProyecto</u>	Horas
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	Null

Figura 10.3. Dos esquemas de relación con anomalías en la actualización. (a) EMP_DEPT y (b) EMP_PROY

(a)

EMP_DEPT

NombreE	<u>Dni</u>	FechaNac	Dirección	NúmeroDpto	NombreDpto	DniDirector
---------	------------	----------	-----------	------------	------------	-------------

(b)

EMP_PROY

<u>Dni</u>	<u>NumProyecto</u>	Horas	NombreE	NombreProyecto	UbicaciónProyecto
------------	--------------------	-------	---------	----------------	-------------------

ponde con un tipo de entidad o de relación, es correcto interpretar y explicar su significado. Por contra, si la relación está compuesta por una mezcla de múltiples entidades y relaciones, se producirá una ambigüedad semántica y la relación no podrá explicarse con claridad.

Los esquemas de relación de las Figuras 10.3(a) y 10.3(b) tienen también semánticas claras (el lector debe ignorar por el momento las líneas que aparecen bajo las relaciones; se utilizan para documentar la notación de dependencia funcional que explicamos en la Sección 10.2). Una tupla en el esquema de relación EMP_DEPT de la Figura 10.3(a) representa a un solo empleado, aunque incluye información adicional: el

nombre del departamento en el que trabaja (NombreDpto) y el DNI del director de ese departamento (DniDirector). En la relación EMP_PROY de la Figura 10.3(b), cada tupla relaciona un empleado con un proyecto, aunque incluye también el nombre del empleado (NombreE), el del proyecto (NombreProyecto) y la localización de éste (UbicaciónProyecto). Aunque desde el punto de vista lógico no existe nada erróneo en estas dos relaciones, se considera que tienen un diseño pobre porque violan la directriz 1 al mezclar atributos de dos entidades del mundo real; EMP_DEPT combina atributos de empleados y departamentos, mientras que EMP_PROY combina atributos de empleados y proyectos y la relación TRABAJA_EN. Deberían utilizarse como vistas, aunque esto provocaría problemas cuando se usasen como relaciones base, tal y como veremos en la siguiente sección.

10.1.2 Información redundante en tuplas y anomalías en la actualización

Uno de los objetivos de un esquema de diseño es reducir el espacio de almacenamiento utilizado por las relaciones base (y, por tanto, por los ficheros correspondientes). El agrupamiento de atributos en esquemas de relación tiene un efecto significativo sobre el espacio de almacenamiento. Por ejemplo, compare el espacio empleado por las dos relaciones base EMPLEADO y DEPARTAMENTO de la Figura 10.2 con el necesario para EMP_DEPT de la Figura 10.4, que es el resultado de aplicar la operación NATURAL JOIN a EMPLEADO y DEPARTAMENTO. En EMP_DEPT, los valores de atributo pertenecientes a un departamento particular (NúmeroDpto, NombreDpto, DniDirector) están repetidos para *cada empleado que trabaja en ese departamento*. Por contra, la información de cada departamento sólo aparece una vez en la relación DEPARTAMENTO de la Figura 10.2. Por cada empleado que trabaja en ese departamento, sólo se repite el número de departamento (NúmeroDpto) en la relación EMPLEADO como una *foreign key*. A la relación EMP_PROY pueden aplicársele comentarios similares (véase la Figura 10.4), que aumenta la relación TRABAJA_EN con atributos adicionales procedentes de EMPLEADO y PROYECTO.

Otro serio problema que aparece cuando se usan las relaciones de la Figura 10.4 como relaciones base son las **anomalías en la actualización**, las cuales pueden clasificarse en anomalías de inserción, de borrado y de modificación.²

Anomalías de inserción. Las anomalías de inserción pueden diferenciarse en dos tipos, que se ilustran con los siguientes ejemplos basados en la relación EMP_DEPT:

- Para insertar una nueva tupla en EMP_DEPT, debemos incluir los valores correspondientes al departamento en el que dicho empleado trabaja, o valores NULL en el caso de que no lo haga para ninguno. Por ejemplo, para insertar una nueva tupla para un empleado que trabaja en el departamento número 5, debemos introducir correctamente los valores de atributo del departamento 5, de modo que sean *coherentes* con los valores correspondientes del resto de tuplas de EMP_DEPT. En el diseño de la Figura 10.2 no tenemos que preocuparnos por el problema de la coherencia ya que sólo indicamos el número de departamento en la tupla de empleado; el resto de valores de atributo del departamento 5 sólo se graban una vez en la base de datos, como una única tupla de la relación DEPARTAMENTO.
- Es complicado insertar un nuevo departamento que aún no tenga ningún empleado en la relación EMP_DEPT. La única forma de hacerlo es colocando valores NULL en los atributos correspondiente al empleado. Esto genera un problema, ya que el DNI es la clave principal de EMP_DEPT, y se supone que cada tupla representa a una entidad empleado, no a una entidad departamento. Además, cuando se asigna el primer empleado a ese departamento, ya no necesitaremos nunca más esta tupla con valores NULL. Este problema no se da en el diseño de la Figura 10.2 porque un departamento se introduce en

² Estas anomalías fueron identificadas por Codd (1972a) para justificar la necesidad de normalización en las relaciones, como ya comentaremos en la Sección 10.3.

Figura 10.4. Ejemplo de los estados de EMP_DEPT y EMP_PROY resultantes de aplicar una NATURAL JOIN a las relaciones de la Figura 10.2. Éstas deberían almacenarse como relaciones base por motivos de rendimiento.

Redundancia

EMP_DEPT

NombreE	Dni	FechaNac	Dirección	NúmeroDpto	NombreDpto	DniDirector
Pérez Pérez, José	123456789	09-01-1965	Eloy I, 98	5	Investigación	333445555
Campos Sastre, Alberto	333445555	08-12-1955	Avda. Ríos, 9	5	Investigación	333445555
Jiménez Celaya, Alicia	999887777	19-07-1968	Gran Vía, 38	4	Administración	987654321
Sainz Oreja, Juana	987654321	20-06-1941	Cerquillas, 67	4	Administración	987654321
Ojeda Ordóñez, Fernando.	666884444	15-09-1962	Portillo, s/n	5	Investigación	333445555
Oliva Avezuela, Aurora	453453453	31-07-1972	Antón, 6	5	Investigación	333445555
Pajares Morera, Luis	987987987	29-03-1969	Enebros, 90	4	Administración	987654321
Ochoa Paredes, Eduardo	888665555	10-11-1937	Las Peñas, 1	1	Sede central	888665555

Redundancia Redundancia

EMP_PROY

Dni	NumProyecto	Horas	NombreE	NombreProyecto	UbicaciónProyecto
123456789	1	32.5	Pérez Pérez, José	ProductoX	Valencia
123456789	2	7.5	Pérez Pérez, José	ProductoY	Sevilla
666884444	3	40.0	Ojeda Ordóñez, Fernando.	ProductoZ	Madrid
453453453	1	20.0	Oliva Avezuela, Aurora	ProductoX	Valencia
453453453	2	20.0	Oliva Avezuela, Aurora	ProductoY	Sevilla
333445555	2	10.0	Campos Sastre, Alberto	ProductoY	Sevilla
333445555	3	10.0	Campos Sastre, Alberto	ProductoZ	Madrid
333445555	10	10.0	Campos Sastre, Alberto	Computación	Gijón
333445555	20	10.0	Campos Sastre, Alberto	Reorganización	Madrid
999887777	30	30.0	Jiménez Celaya, Alicia	Comunicaciones	Gijón
999887777	10	10.0	Jiménez Celaya, Alicia	Computación	Gijón
987987987	10	35.0	Pajares Morera, Luís	Computación	Gijón
987987987	30	5.0	Pajares Morera, Luís	Comunicaciones	Gijón
987654321	30	20.0	Sainz Oreja, Juana	Comunicaciones	Gijón
987654321	20	15.0	Sainz Oreja, Juana	Reorganización	Madrid
888665555	20	Null	Ochoa Paredes, Eduardo	Reorganización	Madrid

la relación DEPARTAMENTO independientemente de que existan o no empleados trabajando en él, y siempre que un empleado se asigne a ese departamento se inserta una tupla en EMPLEADO.

Anomalías de borrado. El problema de las anomalías de borrado está relacionado con la segunda anomalía de inserción comentada anteriormente. Si eliminamos de EMP_DEPT una tupla empleado que representa al último que trabaja para un departamento dado, la información concerniente a ese departamento se pierde de la base de datos. Este problema no ocurre en la base de datos de la Figura 10.2 porque las tuplas DEPARTAMENTO se almacenan de forma separada.

Anomalías de modificación. En EMP_DEPT, si cambiamos el valor de uno de los atributos de un departamento particular (por ejemplo, el director del departamento 5), debemos actualizar las tuplas de todos los empleados que trabajan en ese departamento; en caso de no hacerlo, la base de datos se volverá inconsistente. Si falla la actualización de alguna tupla, el mismo departamento tendrá dos valores diferentes como director en distintas tuplas de empleado, lo que será incorrecto.³

Basándonos en las tres anomalías precedentes, podemos enunciar la siguiente directriz.

Directriz 2

Diseñar los esquemas de relación base de forma que no se presenten anomalías de inserción, borrado o actualización en las relaciones. En caso de que aparezca alguna de ellas, anótela claramente y asegúrese de que los programas que actualizan la base de datos operarán correctamente.

La segunda directriz es coherente, en cierto modo, con una reafirmación de la primera directriz. Podemos ver también la necesidad de una metodología más formal para evaluar si un diseño cumple estas directrices. Las Secciones de la 10.2 a la 10.4 muestran estas necesidades formales. Es importante indicar que estas directrices pueden, a veces, *tener que violarse para mejorar el rendimiento* de ciertas consultas. Por ejemplo, si una consulta importante recupera información relativa al departamento de un empleado junto con atributos de ese empleado, podría usarse el esquema EMP_DEPT como relación base. Sin embargo, deben indicarse y justificarse las anomalías de EMP_DEPT (por ejemplo, usando *triggers* o procedimientos almacenados que llevarían a cabo actualizaciones automáticas) de modo que, si se actualiza la relación base, no nos encontremos con incoherencias. En general, es aconsejable usar relaciones base que estén libres de anomalías y especificar vistas que incluyan las concatenaciones necesarias para recuperar los atributos que se referencian frecuentemente en las consultas. Esto reduce el número de términos JOIN especificados en la consulta, simplificando la escritura correcta de la consulta y, en muchos casos, mejorando el rendimiento.⁴

10.1.3 Valores NULL en las tuplas

En algunos diseños podemos agrupar muchos atributos en una relación “muy grande”. Si muchos de los atributos no se aplican a todas las tuplas de la relación, nos encontraremos con muchos valores NULL en esas tuplas. Esto puede desperdiciar espacio de almacenamiento y puede inducir a problemas a la hora de entender el significado de los atributos con la especificación de operaciones JOIN a nivel lógico.⁵ Otro problema con los NULL es cómo contabilizarlos cuando se aplican operaciones de agregación como COUNT o SUM. Las operaciones SELECT o JOIN implican comparaciones. Si hay presentes valores NULL, los resultados serán impredecibles.⁶ Además, los NULL pueden tener múltiples interpretaciones:

- El atributo *no se aplica* a esta tupla.
- El valor de atributo de esta tupla es *desconocido*.
- El valor es *conocido pero está ausente*, es decir, aún no se ha grabado.

³ Esto no es tan serio como otros problemas, ya que todas las tuplas pueden actualizarse con una sola sentencia SQL.

⁴ El rendimiento de una consulta especificada en una vista que es la concatenación de varias relaciones base depende de cómo el DBMS implementa la vista. Muchos RDBMSs materializan las vistas usadas frecuentemente de forma que no se tengan que llevar a cabo las concatenaciones más habituales. El DBMS es responsable de la actualización de la vista materializada (ya sea inmediata o periódicamente) siempre que las relaciones base se modifiquen.

⁵ Esto se debe a que las concatenaciones externas e internas producen resultados diferentes cuando existen valores NULL implicados en ellas. Los usuarios deben, por tanto, tener cuidado con los distintos significados de cada tipo de concatenación. Lo que resulta razonable para usuarios sofisticados, puede ser difícil para otros.

⁶ En la Sección 8.5.1 presentamos varias comparaciones que implican valores NULL donde el resultado (en la lógica de tres valores) es TRUE, FALSE y UNKNOWN.

El tener la misma representación para todos los NULL compromete los diferentes significados que pueden tener. Por consiguiente, podemos establecer otra directriz.

Directriz 3

Hasta donde sea posible, evite situar en una relación base atributos cuyos valores sean NULL frecuentemente. En caso de no poderse evitar, asegúrese de que se aplican sólo en casos excepcionales y no los aplique a la mayor parte de las tuplas de la relación.

Utilizar el espacio eficientemente y evitar concatenaciones son los dos criterios principales que determinan si incluir las columnas que pueden tener valores NULL en una relación o tener una relación separada para esas columnas (con las columnas clave apropiadas). Por ejemplo, si sólo el 10 por ciento de los empleados tienen oficinas individuales, no es razón suficiente para la inclusión de un atributo NúmeroOficina en la relación EMPLEADO; en lugar de ello, se puede crear una relación OFICINAS_EMP(S(DniEmpleado, NúmeroOficina)) que incluya las tuplas de los empleados con oficinas individuales.

10.1.4 Generación de tuplas falsas

Considere los dos esquemas de relación EMP_LOCS y EMP_PROY1 de la Figura 10.5(a), la cual puede usarse en lugar de la relación simple EMP_PROY de la Figura 10.3(b). Una tupla en EMP_LOCS significa que el empleado cuyo nombre es NombreE trabaja en *algún proyecto* cuya localización es UbicaciónProyecto. Una tupla EMP_PROY1 se refiere al hecho de que el empleado cuyo Documento Nacional de Identidad es Dni trabaja un número de Horas por semana en el proyecto cuyo nombre, número y ubicación son NombreProyecto, NumProyecto y UbicaciónProyecto. La Figura 10.5(b) muestra el estado de relación de EMP_LOCS y EMP_PROY1 correspondiente a la relación EMP_PROY de la Figura 10.4, la cual se obtiene aplicando la operación PROYECCIÓN (π) adecuada a EMP_PROY [ignore por ahora las líneas discontinuas de la Figura 10.5(b)].

Supongamos que utilizamos EMP_PROY1 y EMP_LOCS como relaciones base en lugar de EMP_PROY. Esto produce un diseño de esquema incorrecto algo peculiar porque no podemos recuperar la información original de EMP_PROY desde EMP_PROY1 y EMP_LOCS. Si intentamos llevar a cabo una operación CONCATENACIÓN NATURAL en estas relaciones, el resultado produce muchas más tuplas que las existentes en el conjunto original de EMP_PROY. En la Figura 10.6, sólo se muestra la aplicación de la concatenación a las tuplas que están *por encima* de las líneas discontinuas de la Figura 10.5(b) (para reducir el tamaño de la relación resultante). Las tuplas adicionales que no se encontraban en EMP_PROY reciben el nombre de **tuplas falsas** (*spurious tuples*) porque representa información falsa que no es válida. Las tuplas falsas están marcadas con asteriscos (*) en la Figura 10.6.

No es aconsejable descomponer EMP_PROY en EMP_LOCS y EMP_PROY1 porque cuando deshacemos la CONCATENACIÓN usando una CONCATENACIÓN NATURAL, no obtenemos la información original correcta. Esto es así porque, en este caso, UbicaciónProyecto es el atributo que relaciona EMP_LOCS y EMP_PROY1, y no es ni una clave principal ni una *foreign key* en EMP_LOCS o EMP_PROY1. Ahora estamos en condiciones de definir otra directriz de diseño.

Directriz 4

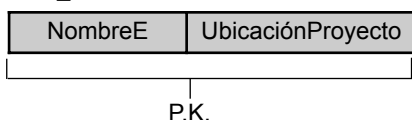
Diseñar los esquemas de relación de forma que puedan concatenarse con condiciones de igualdad en los atributos que son parejas de clave principal y *foreign key* de forma que se garantice que no se van a generar tuplas falsas. Evite las relaciones que contienen atributos coincidentes que no son combinaciones de *foreign key* y clave principal porque la concatenación de estos atributos puede producir tuplas falsas.

Esta directriz informal debe ser, obviamente, redefinida de una manera más adecuada. En el Capítulo 11 trataremos una condición formal llamada propiedad de reunión no aditiva que garantiza que ciertas concatenaciones no producen tuplas falsas.

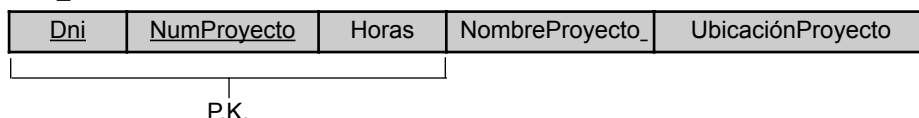
Figura 10.5. Diseño particularmente pobre de la relación EMP_PROY de la Figura 10.3(b). (a) Los dos esquemas de relación EMP_LOCS y EMP_PROY1. (b) El resultado de proyectar la extensión de EMP_PROY de la Figura 10.4 a las relaciones EMP_LOCS y EMP_PROY1.

(a)

EMP_LOCS



EMP_PROY1



(b)

EMP_LOCS

NombreE	Ubicación-Proyecto
Pérez Pérez, José	Valencia
Pérez Pérez, José	Surgarland
Ojeda Ordóñez, Fernando.	Madrid
Oliva Avezuela, Aurora	Valencia
Oliva Avezuela, Aurora	Surgarland
Campos Sastre, Alberto	Surgarland
Campos Sastre, Alberto	Madrid
Campos Sastre, Alberto	Gijón
Jiménez Celaya, Alicia	Gijón
Pajares Morera, Luis	Gijón
Sainz Oreja, Juana	Gijón
Sainz Oreja, Juana	Madrid
Ochoa Paredes, Eduardo	Madrid

EMP_PROY1

Dni	NumProyecto	Horas	NombreProyecto	Ubicación-Proyecto
123456789	1	32.5	ProductoX	Valencia
123456789	2	7.5	ProductoY	Sevilla
666884444	3	40.0	ProductoZ	Madrid
453453453	1	20.0	ProductoX	Valencia
453453453	2	20.0	ProductoY	Sevilla
333445555	2	10.0	ProductoY	Sevilla
333445555	3	10.0	ProductoZ	Madrid
333445555	10	10.0	Computación	Gijón
333445555	20	10.0	Reorganización	Madrid
999887777	30	30.0	Comunicaciones	Gijón
999887777	10	10.0	Computación	Gijón
987987987	10	35.0	Computación	Gijón
987987987	30	5.0	Comunicaciones	Gijón
987654321	30	20.0	Comunicaciones	Gijón
987654321	20	15.0	Reorganización	Madrid
888665555	20	NULL	Reorganización	Madrid

10.1.5 Resumen y explicación acerca de las directrices de diseño

En las Secciones de la 10.1.1 a la 10.1.4, hemos visto situaciones que provocan esquemas de relación problemáticos, y hemos propuesto unas directrices informales para definir un buen diseño relacional. Los problemas que hemos apuntado, que pueden detectarse sin la intervención de herramientas de análisis adicionales, son los siguientes:

- Anomalías que causan trabajo redundante durante la inserción y modificación de una relación, y que pueden causar pérdidas accidentales de información durante el borrado de la misma.
- Desaprovechamiento del espacio de almacenamiento debido a valores NULL y la dificultad de llevar a cabo operaciones de selección, agregación y concatenación debido a estos valores.

Figura 10.6. Resultado de aplicar una CONCATENACIÓN NATURAL a las tuplas que se encuentran por encima de las líneas discontinuas en EMP_PROY1 y EMP_LOCS de la Figura 10.5. Las tuplas falsas generadas aparecen marcadas con asteriscos.

	Dni	NumProyecto	Horas	NombreProyecto	UbicaciónProyecto	NombreE
	123456789	1	32.5	ProductoX	Valencia	Pérez Pérez, José
*	123456789	1	32.5	ProductoX	Valencia	Oliva Avezuela, Aurora
	123456789	2	7.5	ProductoY	Sevilla	Pérez Pérez, José
*	123456789	2	7.5	ProductoY	Sevilla	Oliva Avezuela, Aurora
*	123456789	2	7.5	ProductoY	Sevilla	Campos Sastre, Alberto
	666884444	3	40.0	ProductoZ	Madrid	Ojeda Ordóñez, Fernando.
*	666884444	3	40.0	ProductoZ	Madrid	Campos Sastre, Alberto
*	453453453	1	20.0	ProductoX	Valencia	Pérez Pérez, José
	453453453	1	20.0	ProductoX	Valencia	Oliva Avezuela, Aurora
*	453453453	2	20.0	ProductoY	Sevilla	Pérez Pérez, José
	453453453	2	20.0	ProductoY	Sevilla	Oliva Avezuela, Aurora
*	453453453	2	20.0	ProductoY	Sevilla	Campos Sastre, Alberto
*	333445555	2	10.0	ProductoY	Sevilla	Pérez Pérez, José
*	333445555	2	10.0	ProductoY	Sevilla	Oliva Avezuela, Aurora
	333445555	2	10.0	ProductoY	Sevilla	Campos Sastre, Alberto
*	333445555	3	10.0	ProductoZ	Madrid	Ojeda Ordóñez, Fernando.
	333445555	3	10.0	ProductoZ	Madrid	Campos Sastre, Alberto
	333445555	10	10.0	Computación	Gijón	Campos Sastre, Alberto
*	333445555	20	10.0	Reorganización	Madrid	Ojeda Ordóñez, Fernando.
	333445555	20	10.0	Reorganización	Madrid	Campos Sastre, Alberto

*
*
*

- Generación de datos incorrectos y falsos durante las concatenaciones en relaciones base incorrectamente relacionadas.

En el resto de este capítulo vamos a presentar conceptos y teorías formales que pueden utilizarse para definir de forma más precisa la *idoneidad* y la *mala calidad* de un esquema de relación *individual*. En primer lugar comentaremos la dependencia funcional como una herramienta de análisis. A continuación especificaremos las tres formas normales y la BCNF (Forma normal de Boyce-Codd, *Boyce-Codd Normal Form*) para un esquema de relación. En el Capítulo 11, definiremos formas normales adicionales que están basadas en dependencias de tipos de datos adicionales llamadas dependencias multivalor y dependencias de concatenación.

10.2 Dependencias funcionales

El concepto básico más importante en la teoría de diseño de un esquema relacional es el de una dependencia funcional. En esta sección definiremos formalmente el concepto, mientras que en la Sección 10.3 veremos cómo usarlo para definir formas normales para los esquemas de relación.

10.2.1 Definición de dependencia funcional

Una dependencia funcional es una restricción que se establece entre dos conjuntos de atributos de la base de datos. Supongamos que nuestro esquema de base de datos relacional tiene n atributos $A_1, A_2, \dots, A_n$; pense-

mos que la base de datos completa está descrita por un único esquema de relación **universal** $R = \{A_1, A_2, \dots, A_n\}$.⁷ No sugerimos que vamos a almacenar la base de datos como una única tabla universal; usamos este concepto sólo en el desarrollo de la teoría formal de las dependencias de datos.⁸

Definición. Una dependencia funcional, denotada por $X \rightarrow Y$, entre dos conjuntos de atributos X e Y que son subconjuntos de R , especifica una *restricción* en las posibles tuplas que pueden formar un estado de relación r de R . La restricción dice que dos tuplas t_1 y t_2 en r que cumplen que $t_1[X] = t_2[X]$, deben cumplir también que $t_1[Y] = t_2[Y]$.

Esto significa que los valores del componente Y de una tupla de r dependen de, o *están determinados por*, los valores del componente X ; alternatively, los valores del componente X de una tupla únicamente (o **funcionalmente**) *determinan* los valores del componente Y . Decimos también que existe una dependencia funcional de X hacia Y , o que Y es **funcionalmente dependiente** de X . La abreviatura de dependencia funcional es **DF**, o **FD** o **f.d.** (del inglés, *functional dependency*). El conjunto de atributos X recibe el nombre de **lado izquierdo** de la DF, mientras que Y es el **lado derecho**.

Por tanto, X determina funcionalmente Y si para toda instancia r del esquema de relación R , no es posible que r tenga dos tuplas que coincidan en los atributos de X y no lo hagan en los atributos de Y . Observe lo siguiente:

- Si una restricción de R indica que no puede haber más de una tupla con un valor X concreto en cualquier instancia de relación $r(R)$, es decir, que X es una **clave candidata** de R , se cumple que $X \rightarrow Y$ para cualquier subconjunto de atributos Y de R [ya que la restricción de clave implica que dos tuplas en cualquier estado legal $r(R)$ no tendrán el mismo valor de X].
- Si $X \rightarrow Y$ en R , esto no supone que $Y \rightarrow X$ en R .

Una dependencia funcional es una propiedad de la **semántica** o **significado de los atributos**. Los diseñadores de la base de datos utilizarán su comprensión de la semántica de los atributos de R (esto es, cómo se relacionan unos con otros) para especificar las dependencias funcionales que deben mantenerse en *todos* los estados de relación (extensiones) r de R . Siempre que la semántica de dos conjuntos de atributos de R indique que debe mantenerse una dependencia funcional, la especificamos como una restricción. Las extensiones de relación $r(R)$ que satisfacen la restricción de dependencia funcional reciben el nombre de **estados de relación legales** (o **extensiones legales**) de R . Por tanto, el uso fundamental de las dependencias funcionales es describir más en profundidad un esquema de relación R especificando restricciones de sus atributos que *siempre deben cumplirse*. Ciertas DF pueden especificarse sin hacer referencia a una relación específica. Por ejemplo, $\{\text{Provincia}, \text{NumPermisoConducir}\} \rightarrow \text{Dni}$ debe mantenerse para cualquier adulto que viva en España. También es posible que ciertas dependencias funcionales puedan dejar de existir en el mundo real si cambia la relación. Por ejemplo, en Estados Unidos la DF $\text{CódigoPostal} \rightarrow \text{CodÁrea}$ se utiliza como una relación entre los códigos postales y los códigos de los números telefónicos, pero con la proliferación de los códigos de área telefónica ya no es tan cierta.

Considere el esquema de relación EMP_PROY de la Figura 10.3(b); desde el punto de vista de la semántica de los atributos sabemos que deben mantenerse las siguientes dependencias funcionales:

- a. $\text{Dni} \rightarrow \text{NombreE}$
- b. $\text{NumProyecto} \rightarrow \{\text{NombreProyecto}, \text{UbicaciónProyecto}\}$
- c. $\{\text{Dni}, \text{NúmeroDpto}\} \rightarrow \text{Horas}$

⁷ Este concepto de una relación universal es importante cuando se explican los algoritmos para el diseño de una base de datos relacional en el Capítulo 11.

⁸ Esta presunción implica que cada atributo de la base de datos debe tener un nombre diferente. En el Capítulo 5 prefijamos nombres de atributo derivados de nombres de relación para lograr la unicidad siempre que los atributos de distintas relaciones tuvieran el mismo nombre.

Figura 10.7. Un estado de relación IMPARTIR con una *posible* dependencia funcional TEXTO $\rightarrow$ CURSO. Sin embargo, PROFESOR $\rightarrow$ CURSO no es posible.

IMPARTIR

Profesor	Curso	Texto
Smith	Estructuras de datos	Bartram
Smith	Administración de datos	Martin
Hall	Compiladores	Hoffman
Brown	Estructuras de datos	Horowitz

Estas dependencias funcionales especifican que (a) el valor del Documento Nacional de Identidad de un empleado (Dni) determina de forma inequívoca su nombre (NombreE), (b) el valor de un número de proyecto (NumProyecto) determina de forma única el nombre del mismo (NombreProyecto) y su ubicación (UbicaciónProyecto), y (c) una combinación de valores Dni y NumProyecto determina el número de horas semanales que el empleado ha trabajado en el proyecto (Horas). Alternativamente, decimos que NombreE está determinado funcionalmente por (o es funcionalmente dependiente de) Dni, o que *dado un Dni concreto, conocemos el valor de NombreE*, etc.

Una dependencia funcional es una *propiedad del esquema de relación R*, y no un estado de relación legal particular *r* de *R*. Por consiguiente, una DF *no puede* ser inferida automáticamente a partir de una extensión de relación *r*, sino que alguien que conozca la semántica de los atributos de *R* debe definirla explícitamente. Por ejemplo, la Figura 10.7 muestra un estado particular del esquema de relación IMPARTIR. Aunque a primera vista pudiéramos pensar que Texto $\rightarrow$ Curso, no podemos confirmarlo a menos que sepamos que se cumple *para todos los estados legales posibles* de IMPARTIR. Sin embargo, basta con demostrar un *único ejemplo en contra* para desautorizar una dependencia funcional. Por ejemplo, ya que ‘Smith’ enseña tanto ‘Estructura de datos’ como ‘Administración de datos’, podemos concluir que Profesor *no* determina funcionalmente a Curso.

La Figura 10.3 muestra una **notación diagramática** para visualizar las DFs: cada una de ellas aparece como una línea horizontal. Los atributos del lado izquierdo de la DF están conectados por líneas verticales a la línea que representa la DF, mientras que los del lado derecho lo están a los atributos mediante flechas que apuntan hacia ellos [véanse las Figuras 10.3(a) y 10.3(b)].

10.2.2 Reglas de inferencia para las dependencias funcionales

Decimos que *F* es el conjunto de dependencias funcionales especificadas en un esquema de relación *R*. Habitualmente, el diseñador del esquema especifica las dependencias funcionales que son *semánticamente obvias*; sin embargo, es habitual que otras muchas dependencias funcionales se encuentren en *todas* las instancias de relación legales entre los conjuntos de atributos que pueden derivarse y satisfacen las dependencias de *F*. Esas otras dependencias pueden *inferirse* o *deducirse* de las DF de *F*. En la vida real, es imposible especificar todas las dependencias funcionales posibles para una situación concreta. Por ejemplo, si cada departamento tiene un director, de manera que NúmeroDpto determina de forma única DniDirector (NúmeroDpto $\rightarrow$ DniDirector), y un director tiene un único número de teléfono TeléfonoDirector (DniDirector $\rightarrow$ TeléfonoDirector), entonces ambas dependencias juntas suponen que NúmeroDpto $\rightarrow$ TeléfonoDirector. Esto es una DF inferida y *no* tiene que declararse explícitamente. Por tanto, formalmente es útil definir un concepto llamado *clausura* (*closure*) que incluye todas las posibles dependencias que pueden inferirse de un conjunto *F* dado.

Definición. Formalmente, el conjunto de todas las dependencias que incluyen *F*, junto con las dependencias que pueden inferirse de *F*, reciben el nombre de **clausuras** de *F*; está designada mediante F^+ .

Por ejemplo, suponga que especificamos el siguiente conjunto *F* de dependencias funcionales obvias en el esquema de relación de la Figura 10.3(a):

$$F = \{ \text{Dni} \rightarrow \{ \text{NombreE}, \text{FechaNac}, \text{Dirección}, \text{NúmeroDpto} \}, \\ \text{NúmeroDpto} \rightarrow \{ \text{NombreDpto}, \text{DniDirector} \} \}$$

Las siguientes son algunas de las dependencias funcionales adicionales que se pueden *inferir* de F :

$$\begin{aligned} \text{Dni} &\rightarrow \{ \text{NombreDpto}, \text{DniDirector} \} \\ \text{Dni} &\rightarrow \text{Dni} \\ \text{NúmeroDpto} &\rightarrow \text{NombreDpto} \end{aligned}$$

Una DF $X \rightarrow Y$ es **inferida** de un conjunto de dependencias F especificado en R si $X \rightarrow Y$ se cumple en *todo* estado de relación legal r de R ; es decir, siempre que r satisfaga todas las dependencias en F , $X \rightarrow Y$ también se cumple en r . La clausura F^+ de F es el conjunto de todas las dependencias funcionales que pueden inferirse de F . Para determinar una manera sistemática de inferir dependencias, debemos descubrir un conjunto de **reglas de inferencia** que puedan usarse para deducir nuevas dependencias a partir de un conjunto de dependencias concreto. A continuación vamos a considerar algunas de estas reglas de inferencia. Usamos la notación $F \models X \rightarrow Y$ para indicar que la dependencia funcional $X \rightarrow Y$ se infiere del conjunto de dependencias funcionales F .

En la siguiente explicación, usaremos una notación abreviada para hablar de las dependencias funcionales. Por conveniencia, concatenamos las variables de atributo y eliminamos las comas. Por tanto, la DF $\{X, Y\} \rightarrow Z$ se expresa de forma abreviada como $XY \rightarrow Z$, mientras que la DF $\{X, Y, Z\} \rightarrow \{U, V\}$ se indica como $XYZ \rightarrow UV$. Las reglas de la RI1 a la RI6 son reglas de inferencia bien conocidas para las dependencias funcionales:

- RI1 (regla reflexiva)⁹: Si $X \supseteq Y$, entonces $X \rightarrow Y$.
- RI2 (regla de aumento)¹⁰: $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.
- RI3 (regla transitiva): $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.
- RI4 (regla de descomposición, o proyectiva): $\{X \rightarrow YZ\} \models X \rightarrow Y$.
- RI5 (regla de unión, o aditiva): $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$.
- RI6 (regla pseudotransitiva): $\{X \rightarrow Y, WY \rightarrow Z\} \models WX \rightarrow Z$.

La regla reflexiva (RI1) especifica que un conjunto de atributos siempre se determina a sí mismo o cualquiera de sus subconjuntos, lo que es obvio. Ya que la RI1 genera dependencias que siempre son verdaderas, éstas reciben el nombre de *triviales*. Formalmente, una dependencia funcional $X \rightarrow Y$ es **trivial** si $X \supseteq Y$; en cualquier otro caso es **no trivial**. La regla de aumento (RI2) dice que añadir el mismo conjunto de atributos a ambos lados de una dependencia genera otra dependencia válida. Según la RI3, las dependencias funcionales son transitivas. La regla de descomposición (RI4) especifica que podemos eliminar atributos del lado derecho de una dependencia; si aplicamos esta regla repetidamente podemos descomponer la DF $X \rightarrow \{A_1, A_2, \dots, A_n\}$ en el conjunto de dependencias $\{X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n\}$. La regla de unión (RI5) nos permite realizar lo contrario: podemos combinar un conjunto de dependencias $\{X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n\}$ en una única DF $X \rightarrow \{A_1, A_2, \dots, A_n\}$.

Una *nota preventiva* acerca del uso de estas reglas. Aunque $X \rightarrow A$ y $X \rightarrow B$ implican $X \rightarrow AB$ por la regla de unión antes comentada, $X \rightarrow A$ e $Y \rightarrow B$ no implican que $XY \rightarrow AB$. Además, $XY \rightarrow A$ no implica necesariamente ni $X \rightarrow A$ ni $Y \rightarrow A$.

Cada una de las reglas de inferencia anteriores puede probarse a partir de la definición de dependencia funcional, bien por comprobación directa o bien **por contradicción**. Una comprobación por contradicción asume

⁹ La regla reflexiva puede expresarse también como $X \rightarrow X$, es decir, cualquier conjunto de atributos se determina, funcionalmente, a sí mismo.

¹⁰ La regla de aumento puede definirse también como $\{X \rightarrow Y\} \models XZ \rightarrow YZ$, es decir, incrementar los atributos del lado izquierdo de una DF produce otra DF correcta.

que la regla no se cumple y muestra que ésta no es posible. Vamos a demostrar ahora que las tres primeras reglas son válidas. La segunda comprobación es por contradicción.

Comprobación de la RI1. Suponga que $X \supseteq Y$ y que dos tuplas t_1 y t_2 existen en alguna instancia de relación r de R como $t_1[X] = t_2[X]$. Por tanto, $t_1[Y] = t_2[Y]$ porque $X \supseteq Y$; por tanto, $X \rightarrow Y$ debe cumplirse en r .

Comprobación de la RI2 por contradicción. Asumimos que $X \rightarrow Y$ se cumple en una instancia de relación r de R , pero no así $XZ \rightarrow YZ$. Entonces, deben existir dos tuplas t_1 y t_2 en r tales que (1) $t_1[X] = t_2[X]$, (2) $t_1[Y] = t_2[Y]$, (3) $t_1[XZ] \neq t_2[XZ]$ y (4) $t_1[YZ] \neq t_2[YZ]$. Esto no es posible porque desde los puntos (1) y (3) deducimos el (5) $t_1[Z] = t_2[Z]$, y desde los puntos (2) y (5) deducimos el (6) $t_1[YZ] = t_2[YZ]$, contradiciendo el (4).

Comprobación de la RI3. Asumimos que (1) $X \rightarrow Y$ y (2) $Y \rightarrow Z$ se cumplen en una relación r . Entonces, por cada dos tuplas t_1 y t_2 en r tales que $t_1[X] = t_2[X]$, debemos tener (3) $t_1[Y] = t_2[Y]$, desde la asunción del punto (1); por consiguiente, también debemos tener un punto (4) $t_1[Z] = t_2[Z]$, desde el (3) y asumiendo el (2); por tanto, $X \rightarrow Z$ debe cumplirse en r .

Usando argumentos de comprobación similares, podemos probar las reglas de inferencia de la RI4 a la RI6 y cualquier otra regla de inferencia válida. Sin embargo, una forma más simple de demostrar que una regla de inferencia es válida para las dependencias funcionales es probarla usando algunas de las que ya hemos visto que lo son. Por ejemplo, podemos comprobar las reglas RI4 a RI6 usando las reglas RI1, RI2 y RI3 de la siguiente forma.

Comprobación de RI4 (usando RI1, RI2 y RI3).

1. $X \rightarrow YZ$ (dada).
2. $YZ \rightarrow Y$ (usando RI1 y sabiendo que $YZ \supseteq Y$).
3. $X \rightarrow Y$ (usando RI3 en 1 y 2).

Comprobación de RI5 (usando RI1, RI2 y RI3).

1. $X \rightarrow Y$ (dada).
2. $X \rightarrow Z$ (dada).
3. $X \rightarrow XY$ (usando RI2 en 1 aumentando con X ; observe que $XX = X$).
4. $XY \rightarrow YZ$ (usando RI2 en 2 aumentando con Y).
5. $X \rightarrow YZ$ (usando RI3 en 3 y 4).

Comprobación de RI6 (usando las RI1, RI2 y RI3).

1. $X \rightarrow Y$ (dada).
2. $WY \rightarrow Z$ (dada).
3. $WX \rightarrow WY$ (usando RI2 en 1 aumentando con W).
4. $WX \rightarrow Z$ (usando RI3 en 3 y 2).

Armstrong(1974) demostró que las reglas de inferencia de la RI1 a la RI3 son sólidas y completas. Por **sólida** queremos decir que, dado un conjunto de dependencias funcionales F especificado en un esquema de relación R , cualquier dependencia que podamos inferir de F usando RI1, RI2 y RI3 se cumple en cada estado de relación r de R que *satisfaga* las dependencias de F . Por **completa** decimos que, usando las tres primeras reglas repetidamente para inferir dependencias hasta que ya no se pueda determinar ninguna más, se genera el conjunto completo de *todas las dependencias posibles* que pueden inferirse a partir de F . En otras palabras, el conjunto de dependencias F^+ , que hemos llamado **clausura** de F , pueden determinarse a partir de F usando

sólo las reglas de inferencia de la RI1 a la RI3. Estas tres reglas se conocen como **reglas de inferencia de Armstrong**.¹¹

Habitualmente, los diseñadores de bases de datos especifican, en primer lugar, el conjunto de dependencias funcionales F que pueden determinarse fácilmente a partir de la semántica de los atributos de R ; por tanto, se usan las reglas RI1, RI2 y RI3 para inferir dependencias funcionales adicionales que también se almacenarán en R . Una forma semántica para determinar estas dependencias funcionales adicionales es determinar, en primer lugar, cada conjunto de atributos X que aparece en la parte izquierda de alguna dependencia funcional de F para, a continuación, determinar el conjunto de *todos los atributos* que son dependientes en X .

Definición. Para cada conjunto de atributos X como éste, determinamos el conjunto X^+ de atributos que están funcionalmente determinados por X basados en F ; X^+ recibe el nombre de **clausura de X bajo F** . Puede usarse el Algoritmo 10.1 para calcular X^+ .

Algoritmo 10.1. Determinación de X^+ , la clausura de X bajo F :

```

 $X^+ := X$ ;
  repetir
    antigua $X^+ := X^+$ ;
    por cada dependencia funcional  $Y \rightarrow Z$  en  $F$  ejecutar
      si  $X^+ \supseteq Y$  entonces  $X^+ := X^+ \cup Z$ ;
  hasta que  $(X^+ = \text{antigua}X^+)$ ;

```

El Algoritmo 10.1 empieza asignado a X^+ todos los atributos de X . Según RI1, sabemos que todos esos atributos son funcionalmente dependientes de X . Usando las reglas de inferencia RI3 y RI4, añadimos atributos a X^+ , usando cada dependencia funcional en F . Continuamos por todas las dependencias de F (el bucle *repetir*) hasta que no se añadan más atributos a X^+ *durante un ciclo completo* (del bucle *por cada*) a través de las dependencias de F . Por ejemplo, consideremos el esquema de relación EMP_PROY de la Figura 10.3(b); según la semántica de los atributos, especificamos el siguiente conjunto F de dependencias funcionales que deben cumplirse en EMP_PROY:

$$F = \{ \text{Dni} \rightarrow \text{NombreE}, \\ \text{NumProyecto} \rightarrow \{ \text{NombreProyecto}, \text{UbicaciónProyecto} \}, \\ \{ \text{Dni}, \text{NumProyecto} \} \rightarrow \text{Horas} \}$$

Usando el Algoritmo 10.1, calculamos los siguientes conjuntos de clausura con respecto a F :

$$\begin{aligned} \{ \text{Dni} \}^+ &= \{ \text{Dni}, \text{NombreE} \} \\ \{ \text{NumProyecto} \}^+ &= \{ \text{NumProyecto}, \text{NombreProyecto}, \text{UbicaciónProyecto} \} \\ \{ \text{Dni}, \text{NumProyecto} \}^+ &= \{ \text{Dni}, \text{NumProyecto}, \text{NombreE}, \text{NombreProyecto}, \text{UbicaciónProyecto}, \text{Horas} \} \end{aligned}$$

Intuitivamente, el conjunto de atributos del lado derecho de cada una de las líneas anteriores representa a todos los atributos que son funcionalmente dependientes del conjunto de atributos del lado izquierdo en base al conjunto F dado.

10.2.3 Equivalencia de los conjuntos de dependencias funcionales

En esta sección vamos a tratar la equivalencia de dos conjuntos de dependencias funcionales. En primer lugar vamos a ver algunas definiciones preliminares.

¹¹ En la actualidad se conocen como **axiomas de Armstrong**. En un sentido estrictamente matemático, los *axiomas* (hechos dados) son las dependencias funcionales de F , ya que asumimos que son correctos, mientras que las reglas RI1, RI2 y RI3 son las *reglas de inferencia* para determinar nuevas dependencias funcionales (nuevos hechos).

Definición. Un conjunto de dependencias funcionales F se dice que **cubre** a otro conjunto de dependencias funcionales E si toda DF de E está también en F^+ , es decir, si toda dependencia de E puede ser inferida a partir de F ; alternatively, podemos decir que E está **cubierto por** F .

Definición. Dos conjuntos de dependencias funcionales E y F son **equivalentes** si $E^+ = F^+$. Por consiguiente, equivalencia significa que cada DF de E puede inferirse a partir de F , y viceversa; es decir, E es equivalente a F si se cumplen las condiciones E cubre a F y F cubre a E .

Podemos determinar si F cubre a E calculando X^+ respecto a F para cada DF $X \rightarrow Y$ en E , y comprobando después si X^+ incluye todos los atributos de Y . Si es el caso para *toda* DF en E , entonces F cubre a E . Determinamos si E y F son equivalentes comprobando que E cubre a F y que F cubre a E .

10.2.4 Conjuntos mínimos de dependencias funcionales

Informalmente, una **cobertura mínima** de un conjunto de dependencias funcionales E es un conjunto de dependencias funcionales F que satisface la propiedad de que cada dependencia de E está en la clausura F^+ de F . Además, esta propiedad se pierde si se elimina cualquier dependencia del conjunto F ; F no debe tener redundancias, y las dependencias de F están en una forma estándar. Podemos definir formalmente que un conjunto de dependencias funcionales F es **mínimo** si satisface las siguientes condiciones:

1. Toda dependencia en F tiene un único atributo en su lado derecho.
2. No podemos reemplazar ninguna dependencia $X \rightarrow A$ de F por otra dependencia $Y \rightarrow A$, donde Y es un subconjunto propio de X , y seguir teniendo un conjunto de dependencias equivalente a F .
3. No podemos eliminar ninguna dependencia de F y seguir teniendo un conjunto de dependencias equivalente a F .

Podemos concebir un conjunto mínimo de dependencias como un conjunto de dependencias de una *forma estándar* o *canónica* y *sin redundancias*. La condición 1 sólo representa cada dependencia en forma canónica con un único atributo en el lado derecho.¹² Las condiciones 2 y 3 garantizan que no habrá redundancia en las dependencias ya sea por tener atributos redundantes en el lado izquierdo de una dependencia (condición 2) o por tener una dependencia que puede inferirse a partir del renombrado de las DF en F (condición 3).

Definición. Una cobertura mínima de un conjunto de dependencias funcionales E es un conjunto mínimo de dependencias (en forma canónica estándar y sin redundancia) equivalente a E . Con el Algoritmo 10.2, siempre podemos buscar, *al menos, una* cobertura mínima F por cada conjunto de dependencias E .

Si hay varios conjuntos de DF cualificados como coberturas mínimas de E por la definición anterior, es costumbre utilizar criterios adicionales para minimizar. Por ejemplo, podemos elegir el conjunto mínimo con el *menor número de dependencias* o con la *longitud total* más pequeña (la longitud total de un conjunto de dependencias se calcula concatenando las dependencias y tratándolas como una cadena de caracteres larga).

Algoritmo 10.2. Localizar una cobertura mínima F para un conjunto de dependencias funcionales E .

1. Establecer $F := E$.
2. Reemplazar cada dependencia funcional $X \rightarrow \{A_1, A_2, \dots, A_n\}$ en F por las n dependencias funcionales $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$.
3. Por cada dependencia funcional $X \rightarrow A$ en F
 por cada atributo B que es un elemento de X
 si $\{ \{F - \{X \rightarrow A\} \} \cup \{ (X - \{B\}) \rightarrow A \} \}$ es equivalente a F ,
 entonces reemplazar $X \rightarrow A$ por $(X - \{B\}) \rightarrow A$ en F .

¹² Es una forma estándar de simplificar las condiciones y los algoritmos que garantizan la no existencia de redundancia en F . Con la regla de inferencia R14, podemos convertir una dependencia simple con múltiples atributos en su parte derecha en un conjunto de dependencias con atributos simples en la parte derecha.

4. Por cada dependencia funcional $X \rightarrow A$ sobrante en F
 si $\{F - \{X \rightarrow A\}\}$ es equivalente a F ,
 entonces eliminar $X \rightarrow A$ de F .

Para ilustrar el algoritmo anterior, podemos utilizar lo siguiente:

Partiendo del siguiente conjunto de DF, $E : \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$, tenemos que localizar su cobertura mínima.

- Todas las dependencias anteriores están en forma canónica; ya hemos completado el paso 1 del Algoritmo 10.2 y podemos proceder con el paso 2. En este paso necesitamos determinar si $AB \rightarrow D$ tiene algún atributo redundante en el lado izquierdo, es decir, ¿puede sustituirse por $B \rightarrow D$ o $A \rightarrow D$?
- Ya que $B \rightarrow A$, aumentando con B en ambos lados (RI2), tenemos que $BB \rightarrow AB$, o $B \rightarrow AB$ (i). Sin embargo, $AB \rightarrow D$ como se ha dado (ii).
- Por la regla transitiva (RI3), obtenemos de (i) y (ii), $B \rightarrow D$. Por consiguiente $AB \rightarrow D$ podría sustituirse por $B \rightarrow D$.
- Ahora tenemos un conjunto equivalente al E original, el $E' : \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$. Ya no son posibles más reducciones en el paso 2, ya que todas las DF tienen un único atributo en el lado izquierdo.
- En el paso 3 buscamos una DF redundante en E' . Usando la regla transitiva en $B \rightarrow D$ y $D \rightarrow A$, derivamos $B \rightarrow A$. Por tanto, $B \rightarrow A$ es redundante en E' y puede eliminarse.
- Así pues, la cobertura mínima de E es $\{B \rightarrow D, D \rightarrow A\}$.

En el Capítulo 11 veremos cómo pueden sintetizarse las relaciones de un conjunto de dependencias E localizando primero la cobertura mínima F para E .

10.3 Formas normales basadas en claves principales

Una vez estudiadas las dependencias funcionales y algunas de sus propiedades, estamos preparados para usarlas a la hora de especificar algunos aspectos de la semántica de un esquema de relaciones. Asumimos que contamos con un conjunto de dependencias funcionales para cada relación, y que cada una de estas relaciones dispone de una clave principal; esta información combinada con las verificaciones (condiciones) de las formas normales conduce al *proceso de normalización* del diseño del esquema relacional. Los proyectos de diseño relacional más prácticos siguen una de estas aproximaciones:

- Realizar un diseño de esquema conceptual usando un modelo conceptual como el ER o el EER y asignar el diseño conceptual a un conjunto de relaciones.
- Diseñar las relaciones en base a un conocimiento externo derivado de una implementación existente de ficheros o formularios o informes.

Siguiendo alguno de estos métodos, es útil evaluar las relaciones de idoneidad y descomponerlas todo lo necesario hasta obtener formas normales elevadas usando la teoría de normalización presentada en este capítulo y en el siguiente. En esta sección nos centraremos en las tres primeras formas normales de un esquema de relaciones y en la intuición que se esconde tras ellas, y comentaremos el modo en que fueron desarrolladas históricamente. En la Sección 10.4 veremos las definiciones más generales de estas formas normales, las cuales tienen en cuenta todas las claves candidatas de una relación en lugar de considerar sólo la clave principal.

Empezaremos tratando de manera informal las formas normales y la motivación que se esconde tras su desarrollo, sin olvidarnos de revisar algunas de las definiciones propuestas en el Capítulo 5 y que son necesarias aquí. A continuación trataremos la primera forma normal (1FN) en la Sección 10.3.4, y presentaremos las definiciones de la segunda (2FN) y tercera (3FN) formas normales, que están basadas en claves principales, en las Secciones 10.3.5 y 10.3.6, respectivamente.

10.3.1 Normalización de relaciones

El proceso de normalización, tal y como fue propuesto en un principio por Codd (1972a), hace pasar un esquema de relación por una serie de comprobaciones para *certificar* que satisface una determinada **forma normal**. El proceso, que sigue un método descendente evaluando cada relación contra el criterio de las formas normales y descomponiendo las relaciones según sea necesario, puede considerarse como un *diseño relacional por análisis*. Inicialmente, Codd propuso tres formas normales: la primera, la segunda y la tercera. Una definición más estricta de la 3FN, llamada BCNF (Forma normal Boyce-Codd, *Boyce-Codd Normal Form*) fue propuesta posteriormente por Boyce y Codd. Todas estas formas normales estaban basadas en una única herramienta analítica: las dependencias funcionales entre los atributos de una relación. Más adelante se propusieron una cuarta (4FN) y una quinta (5FN) formas normales basadas en los conceptos de dependencias multivalor y dependencias de concatenación, respectivamente; ambas se explican en el Capítulo 11. Al final de dicho capítulo, comentaremos también el modo en que las relaciones 3FN pueden sintetizarse a partir de un conjunto de DFs dado. Este acercamiento recibe el nombre de *diseño relacional por síntesis*.

La **normalización de datos** puede considerarse como un proceso de análisis de un esquema de relación, basado en sus DF y sus claves principales, para obtener las propiedades deseables de (1) minimizar la redundancia y (2) minimizar las anomalías de inserción, borrado y actualización comentadas en la Sección 10.1.2. Los esquemas de relación no satisfactorios que no cumplen ciertas condiciones (las **pruebas de forma normal**) se decomponen en esquemas de relación más pequeños que cumplen esas pruebas y que, por consiguiente, cuentan con las propiedades deseables. De este modo, el procedimiento de normalización ofrece a los diseñadores de bases de datos lo siguiente:

- Un marco formal para el análisis de los esquemas de relación basado en sus claves y en las dependencias funcionales entre sus atributos.
- Una serie de pruebas de forma normal que pueden efectuarse sobre esquemas de relación individuales, de modo que la base de datos relacional pueda **normalizarse** hasta el grado deseado.

Definición. La **forma normal** de una relación hace referencia a la forma normal más alta que cumple, e indica por tanto el grado al que ha sido normalizada.

Las formas normales, consideradas *aisladas* de otros factores, no garantizan un buen diseño de base de datos. Generalmente, no basta con comprobar por separado que cada esquema de relación de la base de datos está, digamos, en BCNF o en 3FN. En lugar de ello, el proceso de normalización por descomposición debe confirmar también la existencia de las propiedades adicionales que los esquemas relacionales, en conjunto, deben poseer. Dos de estas propiedades son las siguientes:

- La **propiedad de reunión sin pérdida** o **reunión no aditiva**, que garantiza que no se presentará el problema de las tuplas falsas comentado en la Sección 10.1.4, respecto a los esquema de relación creados después de la descomposición.
- La **propiedad de conservación de las dependencias**, que asegura que todas las dependencias funcionales están representadas en alguna relación individual resultante tras la descomposición.

La propiedad de reunión no aditiva es extremadamente crítica y **debe cumplirse a cualquier precio**, mientras que la de conservación de las dependencias, aunque deseable, puede sacrificarse a veces, tal y como veremos en la Sección 11.1.2. Aplazaremos al Capítulo 11 la presentación de los conceptos formales y las técnicas que garantizan las dos propiedades anteriores.

10.3.2 Uso práctico de las formas normales

Los proyectos de diseño más prácticos obtienen información de diseños de bases de datos previos, de modelos heredados o de ficheros existentes. La normalización se lleva a la práctica de forma que los diseños resultantes sean de máxima calidad y cumplan las propiedades deseables antes comentadas. Aunque se han

definido varias formas normales superiores, como la 4FN y la 5FF que veremos en el Capítulo 11, su utilidad práctica es cuestionable cuando las restricciones en las que se basan son difíciles de comprender o detectar por parte de los diseñadores de la base de datos y son los usuarios los que deben descubrirlas. Así, el diseño de base de datos, tal y como se realiza en la actualidad en la industria, presta especial atención a la normalización hasta la 3FN, la BCNF o la 4FN.

Otro punto que conviene resaltar es que los diseñadores de bases de datos *no necesitan* normalizar hasta la forma normal más alta posible. Por razones de rendimiento, las relaciones podrían dejarse en un estado de normalización menor, como el 2FN, como se ha comentado al final de la Sección 10.1.2.

Definición. El proceso para almacenar la concatenación de relaciones de forma normal superiores como relación base (que se encuentra en una forma normal inferior) recibe el nombre de **desnormalización**.

10.3.3 Definiciones de claves y atributos que participan en las claves

Antes de avanzar más, vamos a revisar de nuevo las definiciones de las claves de un esquema de relación mostradas en el Capítulo 5.

Definición. Una **superclave** de un esquema de relación $R = \{A_1, A_2, \dots, A_n\}$ es un conjunto de atributos $S \subseteq R$ con la propiedad de que no habrá un par de tuplas t_1 y t_2 en ningún estado de relación permitido r de R tal que $t_1[S] = t_2[S]$. Una **clave** K es una superclave con la propiedad adicional de que la eliminación de cualquier atributo de K provocará que K deje de ser una superclave.

La diferencia entre una clave y una superclave es que la primera tiene que ser mínima, es decir, si tenemos una clave $K = \{A_1, A_2, \dots, A_k\}$ de R , entonces $K - \{A_i\}$ no es una clave de R para ningún A_i , $1 \leq i \leq k$. En la Figura 10.1, $\{\text{Dni}\}$ es una clave de EMPLEADO, mientras que $\{\text{Dni}\}$, $\{\text{Dni}, \text{NombreE}\}$, $\{\text{Dni}, \text{NombreE}, \text{FechaNac}\}$ y cualquier otro conjunto de atributos que incluya Dni , son superclaves.

Si un esquema de relación tiene más de una clave, cada una de ellas se denomina **clave candidata**. Una de ellas se elige *arbitrariamente* como **clave principal**, mientras que el resto son claves secundarias. Todo esquema de relación debe contar con una clave principal. En la Figura 10.1, $\{\text{Dni}\}$ es la única clave candidata de EMPLEADO, por lo que también será la clave principal.

Definición. Un atributo del esquema de relación R recibe el nombre de **atributo primo** de R si es miembro de *alguna de las claves candidatas* de R . Un atributo es **no primo** si no es miembro de ninguna clave candidata.

En la Figura 10.1, tanto Dni como NúmeroDpto son atributos primos de TRABAJA_EN , mientras que el resto de atributos de TRABAJA_EN no lo son.

Ahora vamos a presentar las tres primeras formas normales: 1FN, 2FN y 3FN. Todas ellas fueron propuestas por Codd (1972a) como una secuencia para alcanzar el estado deseable de relaciones 3FN tras pasar por los estados intermedios 1FN y 2FN en caso de ser necesario. Como veremos, la 2FN y la 3FN atacan diferentes problemas. Sin embargo, por motivos históricos, es habitual seguir las en ese orden; así pues, por definición, una relación 3FN *también satisface* la 2FN.

10.3.4 Primera forma normal

La **primera forma normal (1FN)** está considerada como una parte de la definición formal de una relación en el modelo relacional básico;¹³ históricamente, fue definida para prohibir los atributos multivalor, los

¹³ Esta condición desaparece en el modelo relacional anidado y en los ORDBMS (*Sistemas de objetos relacionales, Object-Relational Systems*), los cuales permiten relaciones no normalizadas (consulte el Capítulo 22).

atributos compuestos y sus combinaciones. Afirmar que el dominio de un atributo sólo debe incluir valores *atómicos* (simples, indivisibles) y que el valor de cualquier atributo en una tupla debe ser un *valor simple* del dominio de ese atributo. Por tanto, 1FN prohíbe tener un conjunto de valores, una tupla de valores o una combinación de ambos como valor de un atributo para una *tupla individual*. En otras palabras, 1FN prohíbe las *relaciones dentro de las relaciones* o las *relaciones como valores de atributo dentro de las tuplas*. Los únicos valores de atributo permitidos por 1FN son los **atómicos** (o **indivisibles**).

Considere el esquema de relación DEPARTAMENTO de la Figura 10.1, cuya clave principal es NúmeroDpto, y suponga que lo ampliamos incluyendo el atributo UbicacionesDpto [véase la Figura 10.8(a)]. Asumimos que cada departamento puede tener *un número de* localizaciones. En la Figura 10.8 podemos ver el esquema DEPARTAMENTO y un estado de relación de ejemplo. Como puede apreciarse, esto no está en la 1FN porque UbicacionesDpto no es un atributo atómico, como queda demostrado por la primera tupla de la Figura 10.8(b). Hay dos formas de ver el atributo UbicacionesDpto:

- El dominio de UbicacionesDpto contiene valores atómicos, pero algunas tuplas pueden tener un conjunto de estos valores. En este caso, UbicacionesDpto no es funcionalmente dependiente de la clave principal NúmeroDpto.
- El dominio de UbicacionesDpto contiene conjuntos de valores y, por tanto, no es atómico. En este caso, NúmeroDpto $\rightarrow$ UbicacionesDpto porque cada conjunto está considerado como un miembro único de un dominio de atributo.¹⁴

En cualquier caso, la relación DEPARTAMENTO de la Figura 10.8 no está en 1FN; de hecho, no está calificada ni como relación según nuestra definición de relación de la Sección 5.1. Existen tres formas principales de alcanzar la primera forma normal para una relación como ésta:

1. Eliminar el atributo UbicacionesDpto que viola la 1FN y colocarlo en una relación aparte LOCALIZACIONES_DPTO junto con la clave principal NúmeroDpto de DEPARTAMENTO. La clave principal de esta relación es la combinación {NúmeroDpto, UbicaciónDpto}, como puede verse en la Figura 10.2. En LOCALIZACIONES_DPTO existe una tupla distinta por *cada localización* de un departamento. Esto descompone la no relación 1FN en dos relaciones 1FN.
2. Expandir la clave de forma que exista una tupla separada en la relación DEPARTAMENTO original por cada localización de un DEPARTAMENTO, como puede verse en la Figura 10.8(c). En este caso, la clave principal resulta de la combinación {NúmeroDpto, UbicaciónDpto}. Esta solución tiene la desventaja de introducir *redundancia* en la relación.
3. Si se conoce un *número máximo de valores* para el atributo (por ejemplo, si se sabe que *pueden existir, al menos, tres localizaciones* para un departamento), sustituir el atributo UbicacionesDpto por tres atributos atómicos: UbicaciónDpto1, UbicaciónDpto2 y UbicaciónDpto3. Esta solución tiene el inconveniente de introducir *valores NULL* en el caso de que algún departamento tenga menos de tres localizaciones. Introduce además una semántica confusa acerca de la ordenación entre los valores de localización, lo que no era lo que originalmente se pretendía. La consulta sobre este atributo se hace más complicada; por ejemplo, considere cómo escribir la siguiente consulta: *enumere los departamentos que tienen 'Valencia' como una de sus localizaciones*.

De las tres posibles soluciones, la primera es la que se considera como la mejor porque no introduce redundancia y es completamente general, sin estar limitada por un número máximo de valores. De hecho, si elegimos la segunda solución, el proceso posterior de normalización nos conducirá a la primera solución.

La primera forma normal también inhabilita los atributos multivalor que son compuestos. Reciben el nombre de **relaciones anidadas** porque cada tupla puede tener una relación *dentro de ella*. La Figura 10.9 muestra a

¹⁴ En este caso, podemos considerar que el dominio de UbicacionesDpto es el **conjunto potencia** del conjunto de localizaciones individuales, es decir, el dominio está compuesto por todos los posibles subconjuntos del conjunto de ubicaciones individuales.

Figura 10.8. Normalización en 1FN. (a) Un esquema de relación que no está en 1FN. (b) Ejemplo de un estado de relación DEPARTAMENTO. (c) Versión 1FN de la misma relación con redundancia.

(a)

DEPARTAMENTO

NombreDpto	NúmeroDpto	DniDirector	UbicacionesDpto

(b)

DEPARTAMENTO

NombreDpto	NúmeroDpto	DniDirector	UbicacionesDpto
Investigación	5	333445555	{Valencia, Sevilla, Madrid}
Administración	4	987654321	{Gijón}
Sede central	1	888665555	{Madrid}

(c)

DEPARTAMENTO

NombreDpto	NúmeroDpto	DniDirector	UbicaciónDpto
Investigación	5	333445555	Valencia
Investigación	5	333445555	Sevilla
Investigación	5	333445555	Madrid
Administración	4	987654321	Gijón
Sede central	1	888665555	Madrid

qué podría parecerse la relación EMP_PROY en caso de permitirse la anidación. Cada tupla representa una entidad empleado, y una relación PROYS(NumProyecto, Horas) *dentro de cada tupla* implica los proyectos de cada empleado y las horas semanales dedicadas a cada proyecto. El esquema de esta relación EMP_PROY puede representarse de la siguiente manera:

EMP_PROY(Dni, NombreE, {PROYS(NumProyecto, Horas)})

Las llaves { } identifican el atributo PROYS como de tipo multivalor, y enumeramos los atributos componente que forman PROYS entre paréntesis (). Las últimas tendencias para el soporte de objetos complejos (consulte el Capítulo 20) y datos XML (consulte el Capítulo 27) intentan permitir y formalizar relaciones anidadas dentro de sistemas de bases de datos relacionales, las cuales fueron prohibidas en principio por la 1FN.

Observe que Dni es la clave principal de la relación EMP_PROY en las Figuras 10.9(a) y (b), mientras que NumProyecto es la clave **parcial** de la relación anidada, esto es, dentro de cada tupla, la relación anidada debe contar con valores únicos de NumProyecto. Para normalizar esto en 1FN, llevamos los atributos de la relación anidada a una nueva relación y *propagamos la clave principal*; la clave principal de la nueva relación combinará la clave parcial con la clave principal de la relación original. La descomposición y la propagación de la clave principal producen los esquemas EMP_PROY1 y EMP_PROY2 [véase la Figura 10.9(c)].

Este procedimiento puede aplicarse recursivamente a una relación con varios niveles de anidamiento para **desanidar** la relación en un conjunto de relaciones 1FN. Esto resulta útil para convertir un esquema de relación no normalizado con varios niveles de anidamiento en relaciones 1FN. La existencia de más de un atributo multivalor en una relación debe ser manipulado con cuidado. Como ejemplo, considere la siguiente relación no 1FN:

PERSONA (Dn#, {NumPermisoConducir#} , {Teléfono#})

Figura 10.9. Normalización de relaciones anidadas en 1FN. (a) Esquema de la relación EMP_PROY con un atributo de *relación anidada* PROYS. (b) Ejemplo de extensión de la relación EMP_PROY mostrando relaciones anidadas dentro de cada tupla. (c) Descomposición de EMP_PROY en las relaciones EMP_PROY1 y EMP_PROY2 por la propagación de la clave principal.

(a)

EMP_PROY		Proyectos	
Dni	NombreE	NumProyecto	Horas

(b)

Dni	NombreE	NumProyecto	Horas
123456789	Pérez Pérez, José	1	32.5
		2	7.5
666884444	Ojeda Ordóñez, Fernando.	3	40.0
453453453	Oliva Avezuela, Aurora	1	20.0
		2	20.0
333445555	Campos Sastre, Alberto	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Pajares Morera, Luis	10	35.0
		30	5.0
987654321	Sainz Oreja, Juana	20	20.0
		30	15.0
888665555	Ochoa Paredes, Eduardo	20	NULL

(c)

EMP_PROY1

<u>Dni</u>	NombreE
------------	---------

EMP_PROY2

<u>Dni</u>	<u>NumProyecto</u>	Horas
------------	--------------------	-------

Esta relación representa el hecho de que una persona tiene varios vehículos y teléfonos. Si se sigue la segunda opción antes comentada, obtendremos una relación repleta de claves:

PERSONA_EN_1FN (Dn#, NumPermisoConducir#, Teléfono#)

Para evitar introducir relaciones extrañas entre el NumPermisoConducir# y Teléfono#, están representadas todas las posibles combinaciones de valores por cada Dn#, dando lugar a redundancias. Esto conduce a los problemas manipulados por las dependencias multivalor y 4FN, de las que hablaremos en el Capítulo 11. La forma correcta de tratar con los dos atributos multivalor de PERSONA es descomponerlos en dos relaciones separadas, usando la primera estrategia comentada anteriormente: P1(Dn#, NumPermisoConducir#) y P2(Dn#, Teléfono#).

10.3.5 Segunda forma normal

La **segunda forma normal (2FN)** está basada en el concepto de *dependencia funcional total*. Una dependencia funcional $X \rightarrow Y$ es **total** si la eliminación de cualquier atributo A de X implica que la dependencia deje de ser válida, es decir, para cualquier atributo $A \in X$, $(X - \{A\})$ no determina funcionalmente a Y . Una dependencia funcional $X \rightarrow Y$ es **parcial** si al eliminarse algún atributo $A \in X$ de X la dependencia sigue siendo válida, es decir, para algún $A \in X$, $(X - \{A\}) \rightarrow Y$. En la Figura 10.3(b), $\{\text{Dni}, \text{NumProyecto}\} \rightarrow \text{Horas}$ es una dependencia completa (ni $\text{Dni} \rightarrow \text{Horas}$ ni $\text{NumProyecto} \rightarrow \text{Horas}$ son válidas). Sin embargo, la dependencia $\{\text{Dni}, \text{NumProyecto}\} \rightarrow \text{NombreE}$ es parcial porque se cumple $\text{Dni} \rightarrow \text{NombreE}$.

Definición. Un esquema de relación R está en 2FN si todo atributo no primo A en R es *completa y funcionalmente dependiente* de la clave principal de R .

La comprobación para 2FN implica la verificación de las dependencias funcionales cuyos atributos del lado izquierdo forman parte de la clave principal. Si ésta contiene un único atributo, no es necesario aplicar la verificación. La relación EMP_PROY de la Figura 10.3(b) está en 1FN pero no en 2FN. El atributo no primo NombreE viola la 2FN debido a DF2, lo mismo que hacen los atributos no primos NombreProyecto y UbicaciónProyecto debido a DF3. Las dependencias funcionales DF2 y DF3 hacen que NombreE, NombreProyecto y UbicaciónProyecto sean parcialmente dependientes de la clave principal $\{\text{Dni}, \text{NumProyecto}\}$ de EMP_PROY, violando la comprobación de 2FN.

Si un esquema de relación está en 2FN, puede ser *normalizado en segundo lugar* o *normalizado 2FN* en un número de relaciones 2FN en las que los atributos no primos sólo están asociados con la parte de la clave principal de la que son completa y funcionalmente dependientes. Por consiguiente, las dependencias funcionales FD1, FD2 y FD3 de la Figura 10.3(b) inducen a la descomposición de EMP_PROY en los tres esquemas de relación EP1, EP2 y EP3 mostrados en la Figura 10.10(a), cada uno de los cuales está en 2FN.

10.3.6 Tercera forma normal

La **tercera forma normal (3FN)** se basa en el concepto de *dependencia transitiva*. Una dependencia funcional $X \rightarrow Y$ en un esquema de relación R es una **dependencia transitiva** si existe un conjunto de atributos Z que ni es clave candidata ni un subconjunto de ninguna clave de R ,¹⁵ y se cumple tanto $X \rightarrow Z$ como $Z \rightarrow Y$. La dependencia $\text{Dni} \rightarrow \text{DniDirector}$ es transitiva a través de NúmeroDpto en EMP_DEPT en la Figura 10.3(a) porque se cumplen las dependencias $\text{Dni} \rightarrow \text{NúmeroDpto}$ y $\text{NúmeroDpto} \rightarrow \text{DniDirector}$ y NúmeroDpto no es una clave por sí misma ni un subconjunto de la clave de EMP_DEPT. Intuitivamente, podemos ver que la dependencia de DniDirector en NúmeroDpto no es deseable en EMP_DEPT ya que NúmeroDpto no es una clave de EMP_DEPT.

Definición. Según la definición original de Codd, un esquema de relación R está en **3FN** si satisface 2FN y ningún atributo no primo de R es transitivamente dependiente en la clave principal.

El esquema de relación EMP_DEPT de la Figura 10.3(a) está en 2FN, ya que no existen dependencias no parciales en una clave. Sin embargo, EMP_DEPT no está en 3FN debido a la dependencia transitiva de DniDirector (y también de NombreDpto) en Dni a través de NúmeroDpto. Podemos normalizar EMP_DEPT descomponiéndola en los dos esquemas de relación 3FN ED1 y ED2 mostrados en la Figura 10.10(b). Intuitivamente, vemos que ED1 y ED2 representan entidades independientes de empleados y departamentos. Una operación CONCATENACIÓN NATURAL en ED1 y ED2 recuperará la relación EMP_DEPT original sin generar tuplas falsas.

Intuitivamente podemos ver que cualquier dependencia funcional en la que el lado izquierdo es parte de la clave principal (subconjunto propio), o cualquier dependencia funcional en la que el lado izquierdo es un

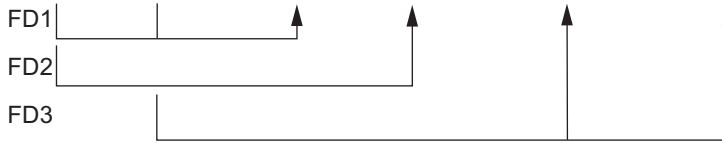
¹⁵ Ésta es la definición general de una dependencia transitiva. Ya que en esta sección sólo nos interesan las claves principales, permitimos las dependencias transitivas donde X es la clave principal pero Z podría ser (un subconjunto de) una clave candidata.

Figura 10.10. Normalización en 2FN y 3FN. (a) Normalizando EMP_PROY en relaciones 2FN. (b) Normalizando EMP_DEPT en relaciones 3FN.

(a)

EMP_PROY

<u>Dni</u>	<u>NumProyecto</u>	Horas	NombreE	NombreProyecto	UbicaciónProyecto
------------	--------------------	-------	---------	----------------	-------------------



Normalización 2NF

EP1

<u>Dni</u>	<u>NumProyecto</u>	Horas
------------	--------------------	-------



EP2

<u>Dni</u>	NombreE
------------	---------



EP3

<u>NumProyecto</u>	NombreProyecto	UbicaciónProyecto
--------------------	----------------	-------------------



(b)

EMP_DEPT

NombreE	<u>Dni</u>	FechaNac	Dirección	NúmeroDpto	NombreDpto	DniDirector
---------	------------	----------	-----------	------------	------------	-------------



Normalización 3NF

ED1

NombreE	<u>Dni</u>	FechaNac	Dirección	NúmeroDpto
---------	------------	----------	-----------	------------



ED2

<u>NúmeroDpto</u>	NombreDpto	DniDirector
-------------------	------------	-------------



atributo no clave, implica una DF *problemática*. La normalización 2FN y 3FN elimina estas DFs descomponiendo la relación original en nuevas relaciones. En términos del proceso de normalización, no es necesario quitar las dependencias parciales antes que las dependencias transitivas, pero históricamente, la 3FN se ha definido con la presunción de que una relación se ha verificado antes para 2FN que para 3FN. La Tabla 10.1 resume de manera informal las tres formas normales basándose en sus claves principales, las pruebas empleadas en cada caso y el *remedio* correspondiente, o la normalización realizada para conseguir la forma normal.

10.4 Definiciones generales de la segunda y tercera formas normales

En general, diseñaremos nuestro esquema de relaciones de forma que no tengan ni dependencias parciales ni transitivas porque causan las anomalías de actualización comentadas en la Sección 10.1.2. Los pasos para la normalización en relaciones 3FN que hemos visto hasta ahora imposibilitan las dependencias parciales y

Tabla 10.1. Resumen de las formas normales en función a sus claves principales y la normalización correspondiente.

Forma normal	Prueba	Remedio (normalización)
Primera (1FN)	La relación no debe tener atributos multivalor o relaciones anidadas.	Generar nuevas relaciones para cada atributo multivalor o relación anidada.
Segunda (2FN)	Para relaciones en las que la clave principal contiene varios atributos, un atributo no clave debe ser funcionalmente dependiente en una parte de la clave principal.	Descomponer y configurar una nueva relación por cada clave parcial con su(s) atributo(s) dependiente(s). Asegurarse de mantener una relación con la clave principal original y cualquier atributo que sea completa y funcionalmente dependiente de ella.
Tercera (3FN)	La relación no debe tener un atributo no clave que esté funcionalmente determinado por otro atributo no clave (o por un conjunto de atributos no clave). Esto es, debe ser una dependencia transitiva de un atributo no clave de la clave principal.	Descomponer y configurar una relación que incluya el(los) atributo(s) no clave que determine(n) funcionalmente otro(s) atributo(s) no clave.

transitivas en la *clave principal*. Estas definiciones, sin embargo, no tienen en cuenta otras claves candidatas de una relación, en caso de que existan. En esta sección vamos a ver unas definiciones más generales de las formas 2FN y 3FN que sí consideran *todas* esas claves candidatas. Observe que esto no afecta a la definición de la 1FN ya que es independiente de claves y dependencias funcionales. Un **atributo primo** es, de forma general, un atributo que forma parte de *cualquier clave candidata*. Ahora consideraremos las dependencias funcionales y transitivas *respecto a todas las claves candidatas* de una relación.

10.4.1 Definición general de la segunda forma normal

Definición. Un esquema de relación R está en **segunda forma normal (2FN)** si cada atributo no primo A en R no es parcialmente dependiente de *ninguna* clave de R .¹⁶

La prueba para 2FN implica la verificación de las dependencias funcionales cuyos atributos de la izquierda forman *parte de* la clave principal. Si la clave principal contiene un único atributo, no es preciso aplicar la comprobación a todo. Considere el esquema de relación PARCELAS de la Figura 10.11(a), que describe los terrenos en venta en distintas provincias. Suponga que existen dos claves candidatas: IdPropiedad y $\{\text{NombreMunicipio}, \text{NúmeroParcela}\}$, es decir, los números de parcela son únicos en cada provincia, mientras que los IdPropiedad son únicos para toda una provincia.

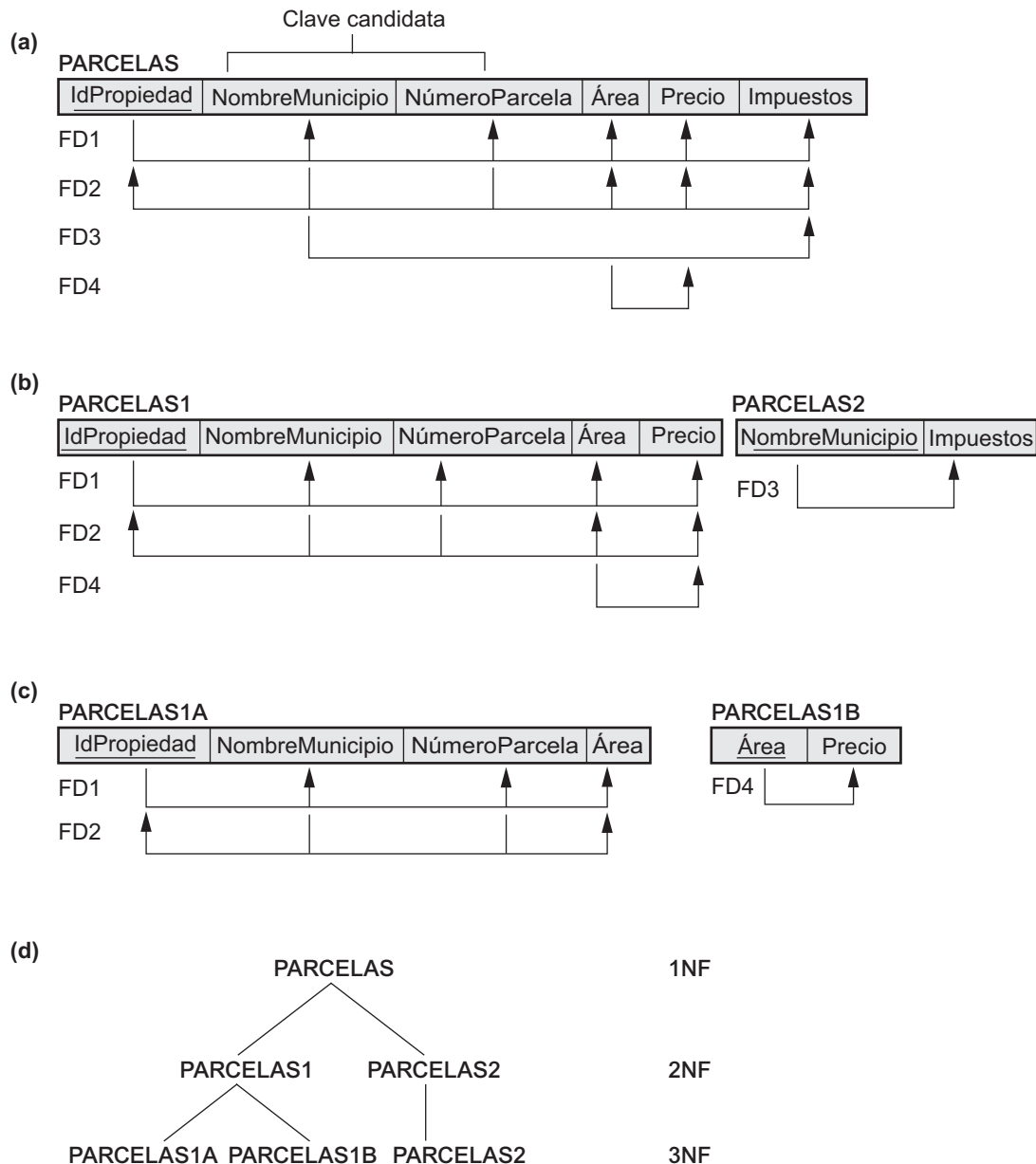
En base a las dos claves candidatas IdPropiedad y $\{\text{NombreMunicipio}, \text{NúmeroParcela}\}$, sabemos que se cumplen las dependencias funcionales FD1 y FD2 de la Figura 10.11(a). Elegimos IdPropiedad como clave principal, razón por la que aparece subrayada en la Figura 10.11(a), aunque no es preciso tomar ninguna consideración especial con esta clave en relación con la otra clave candidata. Supongamos que también se cumplen las dos siguientes dependencias funcionales en PARCELAS:

FD3: $\text{NombreProvincia} \rightarrow \text{Impuestos}$

FD4: $\text{Área} \rightarrow \text{Precio}$

¹⁶ Esta definición puede redefinirse de la siguiente forma: un esquema de relación R está en 2FN si cada atributo no primo A en R es completa y funcionalmente dependiente en *cada* clave de R .

Figura 10.11. Normalización en 2FN y 3FN. (a) La relación PARCELAS con sus dependencias funcionales de la FD1 a la FD4. (b) Descomposición en las relaciones 2FN PARCELAS1 y PARCELAS2. (c) Descomposición de PARCELAS1 en las relaciones 3FN PARCELAS1A y PARCELAS1B. (d) Resumen de la normalización progresiva de PARCELAS.



La dependencia FD3 dice que los impuestos son fijos para cada municipio (no varían de una parcela a otra en el mismo municipio), mientras que FD4 dice que el precio de una parcela lo determina su área sin tener en cuenta el municipio en el que se encuentre (asumimos que éste es el precio de la parcela por motivos fiscales). El esquema de relación PARCELAS viola la definición general de 2FN porque Impuestos es parcialmente dependiente de la clave candidata {NombreMunicipio, NúmeroParcela}, debido a FD3. Para normalizar

PARCELAS a 2FN, la descomponemos en las dos relaciones PARCELAS1 y PARCELAS2, mostradas en la Figura 10.11(b). Construimos PARCELAS1 eliminando de PARCELAS el atributo Impuestos que viola 2FN y colocándolo junto a NombreProvincia (el lado izquierdo de FD3 que provoca la dependencia parcial) en otra relación PARCELAS2. Tanto PARCELAS1 como PARCELAS2 están en 2FN. Observe que FD4 no viola 2FN y persiste en PARCELAS1.

10.4.2 Definición general de la tercera forma normal

Definición. Un esquema de relación R está en **tercera forma normal (3FN)** si, siempre que una dependencia funcional *no trivial* $X \rightarrow A$ se cumple en R , ya sea (a) X una superclave de R , o (b) A un atributo primo de R .

Según esta definición, PARCELAS2 (Figura 10.11(b)) está en 3FN. Sin embargo, FD4 de PARCELAS1 viola 3FN porque Área no es una superclave y Precio no es un atributo primo en PARCELAS1. Para normalizar PARCELAS1 a 3FN, la descomponemos en los esquemas de relación PARCELAS1A y PARCELAS1B de la Figura 10.11(c). Construimos PARCELAS1A eliminando el atributo Precio de PARCELAS1, que viola 3FN, y colocándolo junto con Área (el lado izquierdo de FD4 que provoca la dependencia transitiva) en otra relación PARCELAS1B. Tanto PARCELAS1A como PARCELAS1B están en 3FN.

Dos son los puntos a destacar en este ejemplo y en la definición general de la 3FN:

- PARCELAS1 viola 3FN porque Precio depende transitivamente de cada una de las claves candidatas de PARCELAS1 a través del atributo no primo Área.
- Esta definición general puede aplicarse *directamente* para comprobar si un esquema de relación está en 3FN; esto no implica pasar primero por 2FN. Si aplicamos la definición anterior de 3FN a PARCELAS con las dependencias FD1 a FD4, encontramos que *tanto* FD3 como FD4 violan 3FN. Por consiguiente, podríamos descomponer PARCELAS en PARCELAS1A, PARCELAS1B y PARCELAS2 directamente. Por consiguiente, las dependencias transitiva y parcial que violan 3FN pueden eliminarse *en cualquier orden*.

10.4.3 Interpretando la definición general de la tercera forma normal

Un esquema de relación R viola la definición general de 3FN si una dependencia funcional $X \rightarrow A$ se cumple en R y viola las *dos* condiciones (a) y (b) de la 3FN. La violación del apartado (b) significa que A es un atributo no primo, mientras que la vulneración del (a) implica que X no es una superclave de ninguna clave de R ; por consiguiente, X podría ser no primo o ser un subconjunto propio de una clave de R . Si X es no primo, lo que tenemos es una dependencia transitiva que viola 3FN, mientras que si X es un subconjunto propio de una clave de R , lo que aparece es una dependencia parcial que viola 3FN (y también 2FN). Por tanto, podemos declarar una **definición alternativa general de 3FN** de la siguiente manera:

Definición alternativa. Un esquema de relación R está en 3FN si cada atributo no primo de R cumple las siguientes condiciones:

- Es completa y funcionalmente dependiente de cada clave de R .
- No depende transitivamente de cada clave de R .

10.5 Forma normal de Boyce-Codd

La BCNF (**Forma normal de Boyce-Codd**, *Boyce-Codd Normal Form*) se propuso como una forma más simple de la 3FN, aunque es más estricta que ésta. Es decir, toda relación que esté en BCNF lo está también

en 3FN; sin embargo, una relación 3FN *no está necesariamente* en BCNF. Intuitivamente, podemos ver la necesidad de una forma normal más estricta que la 3FN si volvemos al esquema de relación PARCELAS de la Figura 10.11(a), la cual tiene cuatro dependencias funcionales: de la FD1 a la FD4. Supongamos que tenemos cientos de parcelas en la relación, pero que sólo están en dos municipios: Getafe y Alcorcón. Supongamos también que el tamaño de las parcelas de Alcorcón es de sólo 0,5; 0,6; 0,7; 0,8; 0,9 y 1 hectárea, mientras que las de Getafe están restringidas a 1,1 y 2 hectáreas. En una situación como ésta deberemos contar con una dependencia funcional adicional FD5: Área $\rightarrow$ NombreMunicipio. Si queremos añadirla al resto de dependencias, el esquema de relación PARCELAS1A seguirá estando en 3FN porque NombreMunicipio es un atributo primo. El área de una parcela que determina el municipio, según se especifica en la FD5, puede representarse mediante 16 tuplas en una relación separada $R(\text{Área}, \text{NombreMunicipio})$, ya que sólo existen 16 posibles valores de Área. Esta representación reduce la redundancia de tener que repetir la misma información en los miles de tuplas PARCELAS1A. La BCNF es una *forma normal más estricta* que prohibiría PARCELAS1A y sugeriría la necesidad de descomponerla.

Definición. Un esquema de relación R está en **BCNF** si siempre que una dependencia funcional *no trivial* $X \rightarrow A$ se cumple en R , entonces X es una superclave de R .

La definición formal de BCNF difiere ligeramente de la de 3FN. La única diferencia entre ellas es la condición (b) de 3FN, la cual permite que A sea primo, lo que no se consiente en BCNF. En nuestro ejemplo, FD5 viola BCNF en PARCELAS1A porque Área no es una superclave de PARCELAS1A. Observe que FD5 satisface la 3FN en PARCELAS1A porque NombreMunicipio es un atributo primo (condición b), pero esta condición no existe en la definición de BCNF. Podemos descomponer PARCELAS1A en las dos relaciones BCNF PARCELAS1AX y PARCELAS1AY mostradas en la Figura 10.12(a). Esta descomposición pierde la dependencia funcional FD2 porque sus atributos no coexisten en la misma relación tras la descomposición.

En la práctica, casi todos los esquemas de relación que están en 3FN lo están también en BCNF. Sólo si se cumple $X \rightarrow A$ en un esquema de relación R , no siendo X una superclave y siendo A un atributo primo,

Figura 10.12. Forma normal de Boyce-Codd. (a) Normalización BCNF de PARCELAS1A en la que se pierde la dependencia funcional FD2 en la descomposición. (b) Una relación esquemática con FDs; está en 3FN pero no en BCNF.

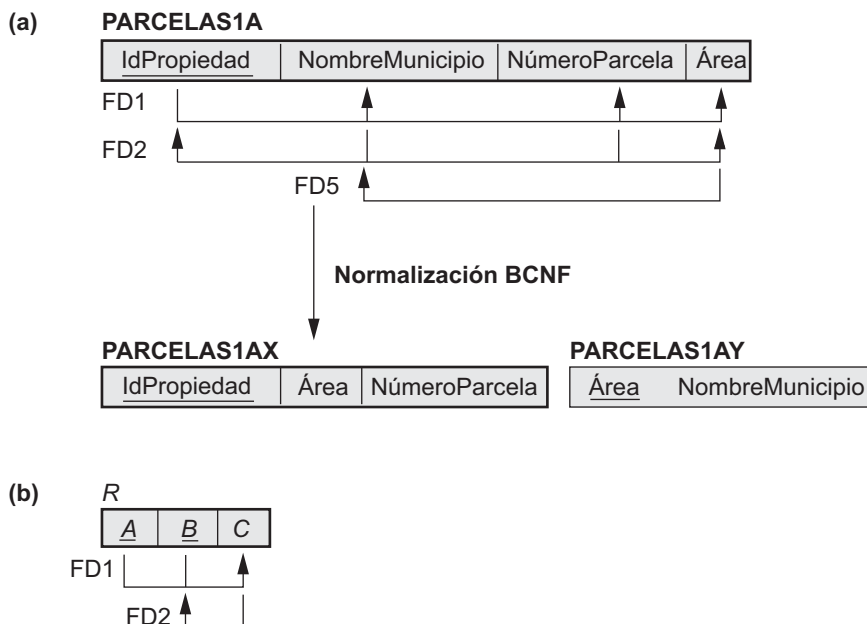


Figura 10.13. Una relación ENSEÑAR que está en 3FN pero no en BCNF.

Estudiante	Curso	Profesor
Campos	Bases de datos	Marcos
Pérez	Bases de datos	María
Pérez	Sistemas operativos	Amanda
Pérez	Teoría	Sergio
Ochoa	Bases de datos	Marcos
Ochoa	Sistemas operativos	Aurora
Morera	Bases de datos	Eduardo
Celaya	Bases de datos	María
Campos	Sistemas operativos	Amanda

estará en 3FN pero no en BCNF. El esquema de relación R mostrado en la Figura 10.12(b) ilustra el caso general de una relación como ésta. De forma ideal, el diseño de una base de datos relacional debe afanarse por cumplir la BCNF o la 3FN en cada esquema de relación. Alcanzar sólo la normalización 1FN o 2FN no se considera adecuado, ya que fueron desarrolladas históricamente como una pasarela hacia la 3FN y la BCNF.

La Figura 10.13 es otro ejemplo que muestra una relación ENSEÑAR con las siguientes dependencias:

FD1: $\{\text{Estudiante}, \text{Curso}\} \rightarrow \text{Profesor}$

FD2:¹⁷ $\text{Profesor} \rightarrow \text{Curso}$

Observe que $\{\text{Estudiante}, \text{Curso}\}$ es una clave candidata para esta relación y que las dependencias mostradas siguen el patrón de la Figura 10.12(b), con Estudiante como A , Curso como B y Profesor como C . Por consiguiente, esta relación está en 3FN pero no en BCNF. La descomposición de este esquema de relación en dos esquemas no es muy correcta porque puede dividirse en uno de los tres siguientes pares:

1. $\{\text{Estudiante}, \text{Profesor}\}$ y $\{\text{Estudiante}, \text{Curso}\}$.
2. $\{\text{Curso}, \text{Profesor}\}$ y $\{\text{Curso}, \text{Estudiante}\}$.
3. $\{\text{Profesor}, \text{Curso}\}$ y $\{\text{Profesor}, \text{Estudiante}\}$.

Las tres descomposiciones *pierden* la dependencia funcional FD1. De ellas, la *descomposición deseable* es la número 3 porque no generará tuplas falsas tras una concatenación.

Una prueba para determinar si una descomposición es no aditiva (sin pérdida) se explica en la Sección 11.1.4 bajo la Propiedad NJB. En general, una relación que no está en BCNF debería descomponerse para cumplir esta propiedad, a la vez que se renuncia a preservar todas las dependencias funcionales en las relaciones resultantes, como ocurre en este ejemplo. El Algoritmo 11.3 hace esto y debería emplearse antes que la descomposición 3 de ENSEÑAR, la cual produce dos relaciones en BCNF como éstas:

$(\text{Profesor}, \text{Curso})$ y $(\text{Profesor}, \text{Estudiante})$

Observe que si designamos $(\text{Profesor}, \text{Estudiante})$ como clave principal de la relación ENSEÑAR, la DF $\text{Profesor} \rightarrow \text{Curso}$ provoca una dependencia parcial (no completamente funcional) de Curso en una parte de esta clave. Esta DF podría eliminarse como parte de una segunda normalización generando exactamente las

¹⁷ Esta dependencia significa que *cada profesor que enseña un curso* es una restricción para esta aplicación.

dos mismas relaciones. Esto es un ejemplo de cómo alcanzar el mismo diseño BCNF a través de rutas de normalización alternativas.

10.6 Resumen

En este Capítulo hemos tratado varios de los peligros a los que nos podemos enfrentar a la hora de diseñar bases de datos relacionales usando argumentos intuitivos. Identificamos de manera informal algunas de las medidas para indicar si un esquema de relación es *idóneo* o *inadecuado*, y facilitamos algunas líneas maestras informales para un buen diseño. Estas directrices están basadas en la realización de un cuidadoso diseño conceptual en el modelo ER y EER, siguiendo correctamente el procedimiento de asignación del Capítulo 7 para asociar entidades y relaciones. Una ejecución correcta de estas directrices, y la ausencia de redundancia, evitará las anomalías en la inserción/borrado/actualización, y la generación de datos falsos. Recomendamos limitar los valores NULL que causan problemas durante las operaciones SELECCIÓN, CONCATENACIÓN y de agregación. A continuación presentamos algunos conceptos formales que nos permiten realizar diseños relacionales de forma descendente mediante el análisis individual de las relaciones. Definimos este proceso de diseño mediante el análisis y la descomposición introduciendo el proceso de normalización.

Definimos el concepto de dependencia funcional y comentamos algunas de sus propiedades. Las dependencias funcionales especifican restricciones semánticas entre los atributos de un esquema de relación. Mostramos cómo se pueden inferir dependencias adicionales de un conjunto de dependencias funcionales usando reglas de inferencia. Definimos los conceptos de clausura y cubierta en relación con las dependencias funcionales. Después mostramos qué es la cubierta mínima de un conjunto de dependencias y ofrecemos un algoritmo para calcularla. También enseñamos el modo de verificar si dos conjuntos de dependencias funcionales son equivalentes.

A continuación describimos el proceso de normalización para lograr buenos diseños mediante la verificación de las relaciones. Ofrecimos un tratamiento de normalización sucesiva basado en una clave principal predefinida en cada relación, y después “aflojamos” este requisito y mostramos una serie de definiciones más generales de la segunda (2FN) y tercera (3FN) formas normales que tienen en cuenta todas las claves candidatas de una relación. Presentamos ejemplos que ilustran cómo, utilizando la definición general de la 3FN, una relación concreta puede analizarse y descomponerse para, eventualmente, producir un conjunto de relaciones en 3FN.

Finalmente, presentamos la BCNF (Forma normal de Boyce-Codd, *Boyce-Codd Normal Form*) y explicamos que se trata de una forma más estricta que la 3FN. Ilustramos también cómo la descomposición de una relación que no está en BCNF debe realizarse considerando los requisitos de descomposición no aditiva. El Capítulo 11 presenta los algoritmos de síntesis y descomposición para un diseño de bases de datos relacional basado en dependencias funcionales. En relación con la descomposición, abordamos los conceptos de reunión no aditiva (sin pérdida) y *conservación de las dependencias*, los cuales están implementados por alguno de estos algoritmos. Otros temas incluidos en el Capítulo 11 son las dependencias multivalor, las de concatenación y la cuarta y quinta formas normales, las cuales consideran estas dependencias.

Preguntas de repaso

- 10.1. Argumente la semántica de atributo como una medida informal de la idoneidad de un esquema de relación.
- 10.2. Comente las anomalías de inserción, borrado y modificación. ¿Por qué están consideradas como malas? Ilustre sus comentarios con ejemplos.
- 10.3. ¿Por qué deben evitarse en la medida de lo posible los valores NULL en una relación? Comente el problema de las tuplas falsas y cómo pueden prevenirse.
- 10.4. Enuncie las directrices informales para un esquema de relación que hemos comentado. Ilustre cómo la violación de estas líneas maestras podría ser dañina.

- 10.5. ¿Qué es una dependencia funcional? ¿Cuáles son las posibles fuentes de información que definen las dependencias funcionales que se cumplen entre los atributos de un esquema de relación?
- 10.6. ¿Por qué no podemos inferir automáticamente una dependencia funcional de un estado de relación particular?
- 10.7. ¿Cuál es el papel de las reglas de inferencia de Armstrong (las que van de la RI1 a la RI3) en el desarrollo de la teoría del diseño relacional?
- 10.8. ¿Qué implican la integridad y la solidez de las reglas de inferencia de Armstrong?
- 10.9. ¿Cuál es el significado de la clausura de un conjunto de dependencias funcionales? Ilústrelolo con un ejemplo.
- 10.10. ¿Cuándo son equivalentes dos dependencias funcionales? ¿Cómo podemos determinar esta equivalencia?
- 10.11. ¿Qué es un conjunto mínimo de dependencias funcionales? ¿Debe tener cada conjunto de dependencias un conjunto equivalente mínimo? ¿Es siempre único?
- 10.12. ¿A qué hace referencia el término *relación no normalizada*? ¿Cómo se desarrollaron históricamente las formas normales desde la primera hasta la de Boyce-Codd?
- 10.13. Defina las tres primeras formas normales cuando sólo se consideran las claves principales. ¿En qué difieren las definiciones generales de la 2FN y la 3FN, las cuales consideran todas las claves de una relación, de las que sólo consideran las claves principales?
- 10.14. ¿Qué dependencias no deseables se evitan cuando una relación está en 2FN?
- 10.15. ¿Qué dependencias no deseables se evitan cuando una relación está en 3FN?
- 10.16. Defina la forma normal de Boyce-Codd. ¿En qué difiere de la 3FN? ¿Por qué se la considera una forma más estricta de la 3FN?

Ejercicios

- 10.17. Suponga que tenemos los siguientes requisitos para la base de datos de una universidad que se utiliza para controlar los certificados de estudios de los estudiantes:
 - a. La universidad controla, por cada estudiante, su NombreEstudiante, su NúmeroEstudiante, su Dni, su DirecciónActualEstudiante y su TeléfonoActualEstudiante, su DirecciónPermanenteEstudiante y su TeléfonoPermanenteEstudiante, su FechaNac, su Sexo, su Curso ('primer año', 'segundo año', . . . , 'graduado') y su Especialidad. Tanto el Dni como el NúmeroEstudiante tienen valores únicos para cada estudiante.
 - b. Cada departamento está descrito mediante un NombreDpto, un CódigoDpto, un NúmeroOficina, un TeléfonoOficina y un Colegio. Tanto el nombre como el código tienen valores únicos para cada departamento.
 - c. Cada curso tienen un NombreCurso, una DescripciónCurso, un NúmeroCurso, el NúmeroHorasSemestre, el Nivel y el DepartamentoImparte. El número de curso es único por cada uno de ellos.
 - d. Cada Sección tiene un profesor (NombreProfesor), un Semestre, un Año, un CursoSección y un NumSección. El número de sección diferencia cada una de las secciones del mismo curso que se imparten durante el mismo semestre/año; sus valores son 1, 2, 3, . . . , hasta alcanzar el número de secciones impartidas durante cada semestre.
 - e. Un registro de nota hace referencia a un Estudiante (Dni), una sección particular y una Nota.
 Diseñar un esquema de base de datos relacional para esta aplicación. Primero, muestre todas las dependencias funcionales que deben cumplirse entre los atributos. A continuación, diseñe el esquema de relaciones para la base de datos que están en 3FN o BCNF. Especifique los atributos clave

de cada relación. Anote cualquier requisito no especificado y tome las decisiones necesarias para suministrar la especificación completa.

- 10.18.** Confirme o rechace las siguientes reglas de inferencia para dependencias funcionales. Puede realizarse una comprobación mediante un argumento de prueba o usando las reglas de inferencia de la RI1 a la RI3. Los rechazos deben llevarse a cabo probando una instancia de relación que satisfaga las condiciones y dependencias funcionales en el lado izquierdo de la regla de inferencia pero que no satisfaga las dependencias del lado derecho.
- $\{W \rightarrow Y, X \rightarrow Z\} \models \{WX \rightarrow Y\}$
 - $\{X \rightarrow Y\} \text{ y } Y \supseteq Z \models \{X \rightarrow Z\}$
 - $\{X \rightarrow Y, X \rightarrow W, WY \rightarrow Z\} \models \{X \rightarrow Z\}$
 - $\{XY \rightarrow Z, Y \rightarrow W\} \models \{XW \rightarrow Z\}$
 - $\{X \rightarrow Z, Y \rightarrow Z\} \models \{X \rightarrow Y\}$
 - $\{X \rightarrow Y, XY \rightarrow Z\} \models \{X \rightarrow Z\}$
 - $\{X \rightarrow Y, Z \rightarrow W\} \models \{XZ \rightarrow YW\}$
 - $\{XY \rightarrow Z, Z \rightarrow X\} \models \{Z \rightarrow Y\}$
 - $\{X \rightarrow Y, Y \rightarrow Z\} \models \{X \rightarrow YZ\}$
 - $\{XY \rightarrow Z, Z \rightarrow W\} \models \{X \rightarrow W\}$
- 10.19.** Considere los dos siguientes conjuntos de dependencias funcionales: $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ y $G = \{A \rightarrow CD, E \rightarrow AH\}$. Compruebe si son equivalentes.
- 10.20.** Considere el esquema de relación EMP_DEPT de la Figura 10.3(a) y el siguiente conjunto G de dependencias funcionales en EMP_DEPT: $G = \{\text{Dni} \rightarrow \{\text{NombreE}, \text{FechaNac}, \text{Dirección}, \text{NúmeroDpto}\}, \text{NúmeroDpto} \rightarrow \{\text{NombreDpto}, \text{DniDirector}\}\}$. Calcule las clausuras $\{\text{Dni}\}^+$ y $\{\text{NúmeroDpto}\}^+$ con respecto a G .
- 10.21.** ¿Es mínimo el conjunto de dependencias funcionales G del Ejercicio 10.20? En caso negativo, intente localizar uno que sea equivalente a G y demuestre que ambos lo son.
- 10.22.** ¿Qué anomalías de actualización ocurren en las relaciones EMP_PROY y EMP_DEPT de las Figuras 10.3 y 10.4?
- 10.23.** ¿En qué forma normal está el esquema de relación PARCELAS de la Figura 10.11(a) con respecto a las interpretaciones que *sólo tienen en cuenta la clave principal*? ¿Estaría en la misma forma normal si se utilizaran sus definiciones generales?
- 10.24.** Demuestre que cualquier esquema de relación con dos atributos está en BCNF.
- 10.25.** ¿Por qué pueden producirse dos tuplas falsas en el resultado de la concatenación de las relaciones EMP_PROY1 y EMP_LOCS de la Figura 10.5 (el resultado se muestra en la Figura 10.6)?
- 10.26.** Considere la relación universal $R = \{A, B, C, D, E, F, G, H, I, J\}$ y el conjunto de dependencias funcionales $F = \{\{A, B\} \rightarrow \{C\}, \{A\} \rightarrow \{D, E\}, \{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}, \{D\} \rightarrow \{I, J\}\}$. ¿Cuál es la clave para R ? Descomponga R en relaciones 2FN y después en 3FN.
- 10.27.** Repita el Ejercicio 10.26 para el siguiente conjunto diferente de dependencias funcionales $G = \{\{A, B\} \rightarrow \{C\}, \{B, D\} \rightarrow \{E, F\}, \{A, D\} \rightarrow \{G, H\}, \{A\} \rightarrow \{I\}, \{H\} \rightarrow \{J\}\}$.
- 10.28.** Considere la relación dada en la tabla de la página siguiente:
- Dada la extensión previa (estado), ¿cuáles de las siguientes dependencias *podrían cumplirse* en la relación anterior? Si la dependencia no se cumple, explique por qué *especificando las tuplas que provocan la violación*.
 - $A \rightarrow B$, ii. $B \rightarrow C$, iii. $C \rightarrow B$, iv. $B \rightarrow A$, v. $C \rightarrow A$

A	B	C	TUPLA#
10	b1	c1	#1
10	b2	c2	#2
11	b4	c1	#3
12	b3	c4	#4
13	b1	c1	#5
14	b3	c4	#6

- b. ¿Tiene la relación anterior alguna clave candidata? En caso afirmativo, ¿cuál es? En caso negativo, ¿por qué no la tiene?

10.29. Considere una relación $R(A, B, C, D, E)$ con las siguientes dependencias:

$$AB \rightarrow C, CD \rightarrow E, DE \rightarrow B$$

¿Es AB una clave candidata de esta relación? En caso negativo, ¿lo es ABD ? Razone su respuesta.

10.30. Considere la relación R , que tiene atributos que guardan programaciones de cursos y secciones en una universidad; $R = \{\text{NúmeroCurso}, \text{NumSección}, \text{DeptOfertante}, \text{HorasCrédito}, \text{NivelCurso}, \text{DniProfesor}, \text{Semestre}, \text{Año}, \text{HorasDía}, \text{NúmeroSala}, \text{NúmeroDeEstudiantes}\}$. Supongamos que en R se mantienen las siguientes dependencias funcionales:

$$\{\text{NúmeroCurso}\} \rightarrow \{\text{DeptOfertante}, \text{HorasCrédito}, \text{NivelCurso}\}$$

$$\{\text{NúmeroCurso}, \text{NumSección}, \text{Semestre}, \text{Año}\} \rightarrow \{\text{HorasDía}, \text{NúmeroSala}, \text{NúmeroDeEstudiantes}, \text{DniProfesor}\}$$

$$\{\text{NúmeroSala}, \text{HorasDía}, \text{Semestre}, \text{Año}\} \rightarrow \{\text{DniProfesor}, \text{NúmeroCurso}, \text{NumSección}\}$$

Intente determinar qué conjuntos de atributos forman las claves de R . ¿Cómo se podría normalizar esta relación?

10.31. Considere las siguientes relaciones para una aplicación de base de datos para el procesamiento de pedidos de la empresa ABC, Inc.

PEDIDO (NúmeroPedido, FechaPedido, NúmeroCliente, CosteTotal)

LÍNEA_PEDIDO(NúmeroPedido, CódigoObjeto, CantidadSolicitada, PrecioTotal, PorcentajeDto)

Asumimos que cada línea de pedido tiene un descuento diferente. El PrecioTotal se refiere a una línea, FechaPedido es la fecha en la que el pedido se realizó y el CosteTotal es el coste total del mismo. Si aplicamos una concatenación natural en la relaciones LÍNEA_PEDIDO y PEDIDO, ¿qué aspecto tendría el esquema de relación resultante? ¿Cuál sería su clave? Muestre las DF resultantes de esta relación. ¿Está en 2FN? ¿Está en 3FN? Razone sus respuestas.

10.32. Considere la siguiente relación:

VENTA_COCHE(NúmeroCoche, FechaVenta, NúmeroVendedor, PorcentajeComisión, Descuento)

Asumimos que un coche lo pueden vender varios vendedores, por lo que $\{\text{NúmeroCoche}, \text{NúmeroVendedor}\}$ es la clave principal. Las siguientes son otras dependencias adicionales

$$\text{FechaVenta} \rightarrow \text{Descuento}$$

$$\text{NúmeroVendedor} \rightarrow \text{PorcentajeComisión}$$

Basándonos en la clave principal anterior, ¿está la relación en 1FN, 2FN o 3FN? Razone su respuesta. ¿Cómo podría normalizarla completamente?

Considere la siguiente relación para registrar los libros publicados:

LIBRO(TítuloLibro, Autor, TipoLibro, ListaPrecio, AfiliaciónAutor, Editorial)

AfiliaciónAutor hace referencia a la afiliación del autor. Suponga que existen las siguientes dependencias:

TítuloLibro $\rightarrow$ Editorial, TipoLibro

TipoLibro $\rightarrow$ ListaPrecio

Autor $\rightarrow$ AfiliaciónAutor

- a. ¿En qué forma normal está la relación? Razone su respuesta.
 - b. Aplique una normalización hasta que la relación no pueda descomponerse más. Explique las razones que se esconden tras cada descomposición.
- 10.34.** Este ejercicio le pide que convierta sentencias de empresa en dependencias. Considere la relación DISCO (NúmeroSerie, Fabricante, Modelo, Versión, Capacidad, Distribuidor). Cada tupla de esta relación contiene información acerca de un disco con un NúmeroSerie único, fabricado por un fabricante, que tiene un modelo particular, distribuido con un número de versión concreto, que tiene una determinada capacidad y que es vendido por un distribuidor concreto. Por ejemplo, la tupla Disco ('1978619', 'WesternDigital', 'A2235X', '765234', 500, 'CompUSA') especifica que WesternDigital fabricó un disco con el número de serie 1978619, el modelo A2235X y la versión 765234; su capacidad es de 500GB y lo vende CompUSA.
- Escriba cada una de las siguientes dependencias como una DF:
- a. El fabricante y el número de serie identifican el disco de forma única.
 - b. Un modelo está registrado por un fabricante y, por consiguiente, no puede utilizarlo ningún otro.
 - c. Todos los discos de una versión particular son del mismo modelo.
 - d. Todos los discos de un determinado modelo de un fabricante particular tienen exactamente la misma capacidad.
- 10.35.** Demuestre que $AB \rightarrow D$ está en la clausura de:

$$\{AB \rightarrow C, CE \rightarrow D, A \rightarrow E\}$$

- 10.36.** Considere la relación siguiente:

R (NúmeroDoctor, NúmeroPaciente, Fecha, Diagnóstico, CódigoTratamiento, Coste)

En la relación anterior, cada tupla describe la visita de un paciente a un médico junto con el código del tratamiento diagnosticado y un coste diario. Asumimos que el diagnóstico está determinado (únicamente) para cada paciente por un médico. Asumimos también que cada código de tratamiento tiene un coste fijo (independientemente del paciente). ¿Esta relación está en 2FN? Justifique su respuesta y descompóngala si fuera necesario. A continuación, argumente si es necesaria una normalización a 3FN y, en caso afirmativo, realícela.

- 10.37.** Considere la siguiente relación:

VENTA_COCHE (IdCoche, Extra, PrecioExtra, FechaVenta, PrecioDescontado)

Esta relación hace referencia a los extras instalados en un coche (por ejemplo, el control de velocidad) y que se vendieron a un distribuidor, y la lista y los precios descontados de los extras.

Si IdCoche $\rightarrow$ FechaVenta y Extra $\rightarrow$ PrecioExtra

e IdCoche, Extra $\rightarrow$ PrecioDescontado,

argumente usando la definición generalizada de la 3FN que esta relación no está en 3FN. A continuación, y desde sus conocimientos de la 2FN, indique por qué tampoco está en 2FN.

- 10.38.** Considere una versión descompuesta de la relación:

PRECIO_ACTUAL_EXTRA (IdCoche, Extra, PrecioDescontado)

COCHE(IdCoche, FechaVenta)

EXTRA(Extra, PrecioExtra)

Usando el algoritmo de comprobación de descomposición sin pérdida (Algoritmo 11.1), determine si esta descomposición es realmente sin pérdidas.

Ejercicios de práctica

Nota. Los siguientes ejercicios utilizan el sistema DBD (Diseñador de base de datos, *Data Base Designer*) descrito en el manual de prácticas. El esquema relacional R y el conjunto de dependencias funcionales F tienen que codificarse como listas. Como ejemplo, R y F para el problema 10.26 están codificados de este modo:

$R = [a, b, c, d, e, f, g, h, i, j]$

$F = [[[a, b], [c]],$

$[[a], [d, e]],$

$[[b], [f]],$

$[[f], [g, h]],$

$[[d], [i, j]]]$

Ya que DBD está implementado en Prolog, el uso de términos en mayúsculas está reservado a las variables del lenguaje y, por consiguiente, las constantes en minúsculas se emplean para codificar los atributos. Para obtener más detalles acerca del uso del sistema DBD, consulte el manual de prácticas.

10.39. Usando el sistema DBD, verifique sus respuestas a los siguientes ejercicios:

- a. 10.19
- b. 10.20
- c. 10.21
- d. 10.26 (sólo 3FN)
- e. 10.27
- f. 10.29
- g. 10.30

Bibliografía seleccionada

Las dependencias funcionales fueron presentadas originalmente por Codd (1970). Las definiciones originales de la primera, segunda y tercera formas normales fueron definidas en Codd (1972a), donde puede encontrarse también un comentario acerca de las anomalías de actualización. La forma normal de Boyce-Codd fue definida en Codd (1974). La definición alternativa de la tercera forma normal fue dada en Ullman (1988), así como la definición de BCNF que hemos mostrado aquí. Ullman (1988), Maier (1983) y Atzeni y De Antonellis (1993) contienen muchos de los teoremas y demostraciones referentes a las dependencias funcionales.

Armstrong (1974) muestra la solidez y la integridad de las reglas de inferencia de la RI1 a la RI3. El Capítulo 11 contiene referencias adicionales sobre la teoría del diseño relacional.

CAPÍTULO 13

Almacenamiento en disco, estructuras básicas de ficheros y dispersión

Las bases de datos se almacenan físicamente como ficheros de registros, normalmente en discos magnéticos. Este capítulo y el siguiente hablan de la organización de las bases de datos en el almacenamiento y de las técnicas para acceder a ellas de forma eficaz mediante diversos algoritmos, algunos de los cuales requieren estructuras auxiliares de datos denominadas índices. Empezamos en la Sección 13.1 haciendo una introducción de los conceptos de las jerarquías de almacenamiento y de cómo se utilizan en los sistemas de bases de datos. La Sección 13.2 está dedicada a una descripción de los dispositivos de almacenamiento en disco magnético y sus características, y también describe brevemente los dispositivos de almacenamiento de cinta magnética. Después de explicar las diferentes tecnologías de almacenamiento, centramos nuestra atención en los métodos de organización de los datos en el disco. La Sección 13.3 explica la técnica del doble búfer, que se utiliza para acelerar la recuperación de varios bloques de disco. En la Sección 13.4 hablamos de las distintas formas de formatear y almacenar registros de fichero en disco. En la Sección 13.5 explicamos los distintos tipos de operaciones que se aplican normalmente a los registros de los ficheros. Presentamos los tres métodos principales para organizar los registros de un fichero en un disco: registros desordenados en la Sección 13.6; registros ordenados en la Sección 13.7; y registros dispersos en la Sección 13.8. La Sección 13.9 explica brevemente los ficheros de registros mixtos y otros métodos de organización de registros, como los árboles B. Son particularmente interesantes para almacenar las bases de datos orientadas a objetos, que explicamos posteriormente en los Capítulos 20 y 21. La Sección 13.10 describe RAID (Matriz redundante de discos baratos [o independientes], *Redundant Array of Inexpensive [o Independent] Disks*), una arquitectura de sistema de almacenamiento de datos que se utiliza normalmente en empresas grandes para mejorar la fiabilidad y el rendimiento. Por último, en la Sección 13.11 describimos dos desarrollos recientes que han visto la luz en el área de los sistemas de almacenamiento: las redes de área de almacenamiento (SAN, *storage area networks*) y el almacenamiento conectado a la red (NAS, *network attached storage*). En el Capítulo 14 explicamos técnicas para crear estructuras auxiliares de datos, denominadas índices, que aceleran la búsqueda y la recuperación de registros. Estas técnicas implican el almacenamiento de datos auxiliares, denominados ficheros de índice, además de los propios registros del fichero.

El lector puede ojear los Capítulos 13 y 14, o incluso omitirlos si ya ha estudiado las organizaciones de ficheros. El material aquí explicado, en particular las Secciones 13.1 a 13.8, es necesario para comprender los Capítulos 15 y 16, que están dedicados al procesamiento y la optimización de las consultas.

13.1 Introducción

La colección de datos que constituye una base de datos computerizada debe almacenarse físicamente en un **medio de almacenamiento** de un computador. El software DBMS puede recuperar, actualizar y procesar después estos datos cuando lo necesite. Los medios de almacenamiento forman una *jerarquía de almacenamiento* que incluye dos categorías principales:

- **Almacenamiento principal o primario.** Esta categoría incluye los medios de almacenamiento en los que la CPU (unidad central de procesamiento) puede operar, como la memoria principal del computador y las memorias caché, más pequeñas pero más rápidas. El almacenamiento principal normalmente proporciona un acceso rápido a los datos, pero tiene una capacidad de almacenamiento limitada.
- **Almacenamientos secundario y terciario.** Esta categoría incluye los discos magnéticos, los discos ópticos y las cintas. Las unidades de disco duro se clasifican como almacenamiento secundario, mientras que los medios removibles o extraíbles están considerados como almacenamiento terciario. Estos dispositivos normalmente tienen gran capacidad, cuestan poco y proporcionan un acceso más lento a los datos que los dispositivos de almacenamiento principales. La CPU no puede procesar directamente los datos almacenados en un almacenamiento secundario o terciario; primero deben copiarse en el almacenamiento principal.

En la Sección 13.1.1 ofrecemos una visión general de los distintos dispositivos de almacenamiento que se utilizan como almacenamiento principal y secundario, y en la Sección 13.1.2 veremos cómo se manipulan las bases de datos en la jerarquía de almacenamiento.

13.1.1 Jerarquías de memoria y dispositivos de almacenamiento

En un computador moderno, los datos residen y se transportan por una jerarquía de medios de almacenamiento. La memoria de alta velocidad es la más cara y, por tanto, está disponible con una menor capacidad. La memoria de velocidad más baja es el almacenamiento en cinta *offline*, que está esencialmente disponible con capacidades de almacenamiento indefinido.

En el extremo más caro del *nivel de almacenamiento principal* tenemos la **memoria caché**, que es una RAM (memoria de acceso aleatorio) estática. La memoria caché la utiliza normalmente la CPU para acelerar la ejecución de los programas. El siguiente nivel del almacenamiento principal es la DRAM (RAM dinámica), que proporciona el área de trabajo principal para que la CPU almacene programas y datos. Popularmente se denomina **memoria principal**. La ventaja de la DRAM es su bajo coste, que sigue bajando; el inconveniente es su volatilidad¹ y baja velocidad en comparación con la RAM estática. En el *nivel de almacenamiento secundario y terciario*, la jerarquía incluye los discos magnéticos, así como el **almacenamiento en masa** en forma de dispositivos CD-ROM (Disco compacto—memoria de sólo lectura) y DVD y, finalmente, las cintas en el extremo más caro de la jerarquía. La **capacidad de almacenamiento** se mide en kilobytes (Kbyte o 1.000 bytes), megabytes (MB o 1 millón de bytes), gigabytes (GB o mil millones de bytes), e incluso terabytes (1.000 GB).

Los programas residen y se ejecutan en la DRAM. Por lo general, las bases de datos permanentes más grandes residen en el almacenamiento secundario, y las porciones de la base de datos se leen y se escriben en los búferes de la memoria principal según se necesitan. Ahora que los computadores personales y las estaciones de trabajo tienen cientos de megabytes de datos en la DRAM, es posible cargar gran parte de la base de datos en la memoria principal. Es común que un servidor tenga de 8 a 16 GB de memoria RAM. En algunos casos, la memoria principal puede albergar la base de datos entera (con una copia de seguridad en un disco magnético), lo que lleva a las **bases de datos en memoria principal**; son particularmente útiles en las aplicaciones

¹ La memoria volátil normalmente pierde su contenido en caso de una interrupción del suministro eléctrico, algo que no ocurre con la memoria no volátil.

en tiempo real que requieren unos tiempos de respuesta extremadamente rápidos. Un ejemplo son las aplicaciones de conmutación telefónica, que almacenan bases de datos que contienen información de enrutamiento y de línea en la memoria principal.

Entre la DRAM y el almacenamiento en disco magnético encontramos otra forma de memoria, la **memoria flash**, muy popular actualmente debido principalmente a que no es volátil. Las memorias flash son de alta densidad y alto rendimiento que utilizan la tecnología EEPROM (Memoria de sólo lectura programable y borrrable eléctricamente, *Electrically Erasable Programmable Read-Only Memory*). La ventaja de la memoria flash es su alta velocidad de acceso; el inconveniente es que debe borrarse y escribirse un bloque entero simultáneamente.² Las tarjetas de memoria flash se presentan como medio de almacenamiento de datos en dispositivos cuya capacidad va desde unos pocos megabytes hasta unos cuantos gigabytes. Dichas tarjetas se utilizan en cámaras, reproductores de MP3, accesorios de almacenamiento USB, etcétera.

Los discos CD-ROM almacenan los datos ópticamente y los leen mediante un láser. Los CD-ROMs contienen datos pregrabados que no se pueden sobrescribir. Los discos WORM (Escribir una vez y leer muchas veces, *Write-Once-Read-Many*) son una forma de almacenamiento óptico que se utiliza para archivar datos; permiten que los datos se escriban una vez y se lean un número indefinido de veces sin posibilidad de borrarlos. Permiten almacenar aproximadamente medio giga de datos por disco.³ Las **memorias jukebox ópticas** utilizan un array de bandejas CD-ROM, que se van cargando en las unidades bajo demanda. Aunque los jukeboxes ópticos tienen capacidades que rondan cientos de gigabytes, sus tiempos de recuperación se mueven en cientos de milisegundos, un poco más lentos que los discos magnéticos. Este tipo de almacenamiento está en declive debido a la rápida bajada de precios y el incremento de la capacidad de los discos magnéticos. El DVD (disco de vídeo digital) es un estándar reciente en discos ópticos, que permite una capacidad de almacenamiento de 4,5 a 15 GB por disco. La mayoría de las unidades de disco de los computadores personales leen discos CD-ROM y DVD.

Por último, las **cintas magnéticas** se utilizan para archivar y para el almacenamiento de las copias de seguridad de los datos. Los **jukeboxes de cinta** (que contienen un banco de cintas que se catalogan y pueden cargarse automáticamente en las unidades de cinta) se están empezando a popularizar como **almacenamiento terciario** para almacenar terabytes de datos. Por ejemplo, el sistema EOS (Satélite de observación de la Tierra, *Earth Observation Satellite*) de la NASA almacena de este modo las bases de datos archivadas.

Ya es normal que las bases de datos de muchas empresas grandes rondan el terabyte de tamaño. El término **base de datos muy grande** ya no puede definirse con precisión, porque las capacidades de almacenamiento de los discos siguen creciendo y los costes reduciéndose. Muy pronto, este término puede quedar reservado para las bases de datos que contienen decenas de terabytes.

13.1.2 Almacenamiento de bases de datos

Las bases de datos normalmente almacenan grandes cantidades de datos que pueden persistir durante largos periodos de tiempo. Durante ese periodo, se acceden y procesan los datos repetidamente. Esto contrasta con la noción de estructuras de datos *transitorias* que persisten únicamente durante el tiempo limitado que corresponde a la ejecución del programa. La mayoría de las bases de datos se almacenan de forma permanente (o *persistentemente*) en almacenamiento secundario de disco magnético, por las siguientes razones:

- Generalmente, las bases de datos son demasiado grandes para entrar completamente en la memoria principal.

² Por ejemplo, la INTEL DD28F032SA es una memoria flash con capacidad para 32 megabits con una velocidad de acceso de 70 nanosegundos, y una tasa de transferencia de escritura de 430 KB/segundo.

³ Sus velocidades rotacionales son más bajas (aproximadamente 400 rpm), lo que arroja unos retardos por latencia más altos y unas velocidades de transferencia bajas (aproximadamente 100 a 200 KB/segundo).

- Las circunstancias que provocan la pérdida permanente de datos almacenados son menos frecuentes con el almacenamiento secundario que con el principal. Por tanto, nos referimos al disco (y a otros dispositivos de almacenamiento secundario) como **almacenamiento no volátil**, mientras que la memoria principal se denomina, a menudo, **almacenamiento volátil**.
- El coste de almacenamiento por unidad de datos es un orden de magnitud inferior para el almacenamiento secundario en disco que para el almacenamiento principal.

Es probable que algunas de las tecnologías más nuevas (como los discos ópticos, los DVDs y los jukeboxes de cinta) ofrezcan alternativas viables al uso de discos magnéticos. En el futuro, las bases de datos pueden residir, por tanto, en diferentes niveles de la jerarquía de memoria de los descritos en la Sección 13.1.1. No obstante, se prevé que los discos magnéticos sigan siendo el medio de almacenamiento principal para las bases de datos grandes en los próximos años. Por consiguiente, es importante estudiar y entender las propiedades y características de los discos magnéticos y la forma en que los ficheros de datos pueden organizarse en el disco, a fin de diseñar bases de datos eficaces con un rendimiento aceptable.

Las cintas magnéticas se utilizan con frecuencia como medio de almacenamiento para hacer copias de seguridad de las bases de datos, porque el almacenamiento en cinta es incluso menos costoso que el almacenamiento en disco. No obstante, el acceso a los datos almacenados en una cinta es bastante lento. Los datos almacenados en las cintas están **offline**; es decir, para que los datos estén disponibles, antes se necesita alguna intervención por parte de un operador (o un dispositivo de carga automática) para cargar una cinta. Por el contrario, los discos son dispositivos **online** a los que se puede acceder directamente en cualquier momento.

Las técnicas utilizadas para almacenar grandes cantidades de datos estructurados en disco son importantes para los diseñadores de bases de datos, el DBA y los encargados de implantar un DBMS. Los diseñadores de bases de datos y el DBA deben conocer las ventajas y los inconvenientes de cada técnica de almacenamiento cuando se dispongan a diseñar, implementar y operar una base de datos en un DBMS concreto. Normalmente, el DBMS tiene varias opciones para organizar los datos. El proceso de **diseño físico de la base de datos** implica elegir las técnicas de organización de datos particulares que mejor se adapten a los requisitos de la aplicación. Los encargados de implementar el sistema DBMS deben estudiar las técnicas de organización de datos para poder implementarlas eficazmente y, de este modo, ofrecer al DBA y los usuarios del DBMS las opciones suficientes.

Las aplicaciones típicas de bases de datos sólo necesitan procesar una pequeña porción de la base de datos en cada momento. Siempre que se necesita una determinada porción de datos, debe buscarse en el disco, copiarse en la memoria principal para su procesamiento y, después, reescribirse en el disco si los datos han cambiado. Los datos almacenados en el disco se organizan como **ficheros de registros**. Un registro es una colección de valores de datos que pueden interpretarse como hechos relacionados con las entidades, sus atributos y sus relaciones. Los registros deben almacenarse en disco de un modo que haga posible su localización de la forma más eficaz cuando sean necesarios.

Hay varias **organizaciones principales de ficheros**, que determinan la *colocación física* de los registros del fichero en el disco y, por tanto, *cómo se puede acceder a ellos*. Un *fichero heap* (o fichero desordenado) coloca los registros en el disco sin un orden particular, añadiendo los registros nuevos al final del fichero, mientras que un *fichero ordenado* (o *fichero secuencial*) mantiene ordenados los registros por el valor de un campo particular (denominado clave de ordenación). Un *fichero disperso* utiliza una función de dispersión (*hash*) aplicada a un campo concreto (denominado clave de dispersión) para determinar la ubicación de un registro en el disco. Otras organizaciones de ficheros, como los árboles B, utilizan estructuras en árbol. Explicaremos las principales organizaciones de ficheros en las Secciones 13.6 a 13.9. Una **organización secundaria o estructura de acceso auxiliar** permite un acceso eficaz a los registros de un fichero basándose en campos alternos a los utilizados para la organización principal del fichero. La mayoría de ellos son índices y los explicaremos en el Capítulo 14.

13.2 Dispositivos de almacenamiento secundario

En esta sección describimos algunas de las características de los dispositivos de almacenamiento de disco magnético y de cinta magnética. Los lectores que ya hayan estudiado estos dispositivos, pueden echar un vistazo simplemente a la sección.

13.2.1 Descripción del hardware de los dispositivos de disco

Los discos magnéticos se utilizan para almacenar grandes cantidades de datos. La unidad de datos más básica es el **bit** de información. Al magnetizar un área del disco de determinadas formas, podemos hacer que represente un valor de bit de 0 (cero) o 1 (uno). Para codificar la información, los bits se agrupan en **bytes** (o **caracteres**). El tamaño de los bytes es normalmente de 4 a 8 bits, en función del computador y del dispositivo. Asumimos que un carácter se almacena en un byte, y utilizamos indistintamente los términos *byte* y *carácter*. La **capacidad** de un disco es el número de bytes que puede almacenar, que normalmente es muy grande. Los disquetes que se utilizaban con los microcomputadores almacenaban de 400 KB a 1,5 MB; los discos duros de los microcomputadores almacenan desde varios cientos de MB hasta decenas de GB; y los grandes paquetes de discos que se utilizan en los servidores y los *mainframes* tienen capacidades de cientos de GB. La capacidad de los discos sigue creciendo a medida que mejora la tecnología.

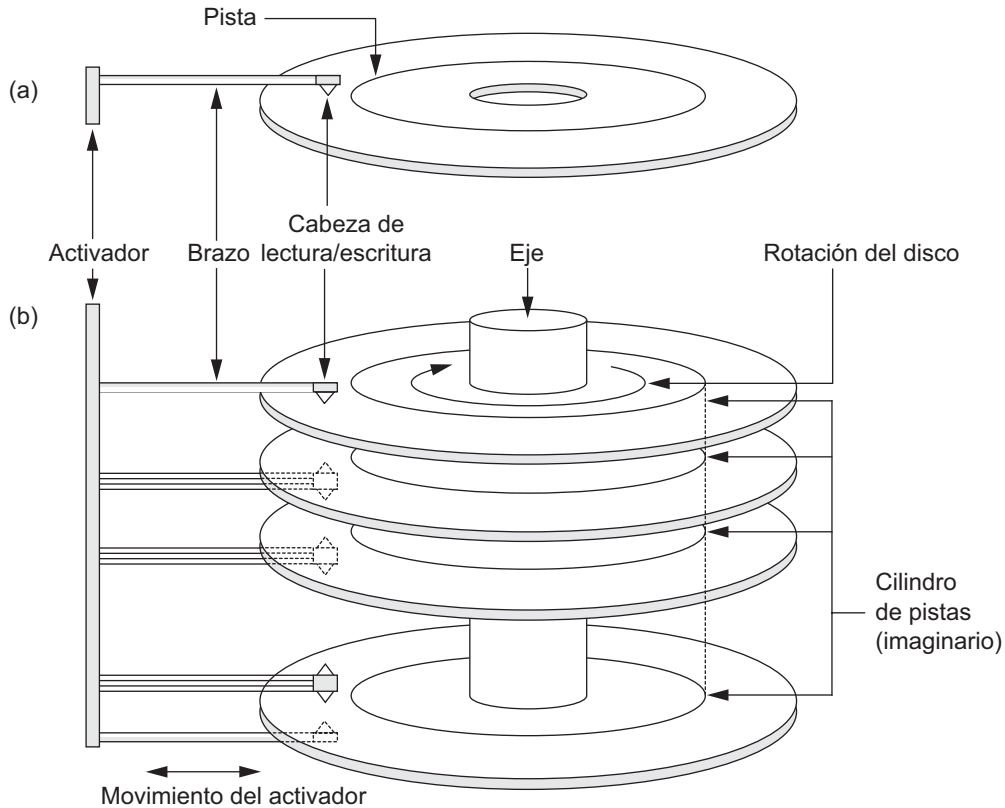
Cualquiera que sea su capacidad, todos los discos están fabricados con un material magnético al que se le da forma de disco circular, como muestra la Figura 13.1(a), que va protegido por una carcasa plástica o acrílica. Un disco es de **una cara** si sólo almacena información en una de sus superficies, y de **doble cara** si utiliza las dos superficies. Para aumentar la capacidad de almacenamiento, los discos se ensamblan como **paquetes de discos** (véase la Figura 13.1[b]), que pueden incluir muchos discos y, por tanto, muchas superficies. La información se almacena en la superficie del disco en círculos concéntricos de *poca anchura*,⁴ y cada uno de un diámetro distinto. Cada uno de esos círculos es una **pista**. En los paquetes de discos, las pistas de las distintas superficies que tienen el mismo diámetro reciben el nombre de **cilindro**, debido a la forma que tendrían si se conectaran en el espacio. El concepto de cilindro es importante porque los datos almacenados en un mismo cilindro se pueden recuperar mucho más rápidamente que si estuvieran distribuidos por diferentes cilindros.

El número de pistas de un disco varía de unos cuantos centenares a unos cuantos miles, y la capacidad de cada pista normalmente varía de unas decenas de Kbytes a 150 Kbytes. Como una pista normalmente contiene una gran cantidad de información, está dividida en bloques más pequeños o **sectores**. La división de una pista en sectores está codificada en la superficie del disco y no se puede cambiar. Un tipo de organización de sectores (véase la Figura 13.2[a]), toma una porción de una pista que subtiende un ángulo fijo al centro de un sector. Son posibles otras organizaciones de los sectores, una de las cuales tiene los sectores subtendidos al centro en ángulos más pequeños a medida que nos alejamos, lo que mantiene una densidad uniforme de grabación (véase la Figura 13.2[b]). Una técnica denominada ZBR (*Zone Bit Recording*) permite que un rango de cilindros tenga la misma cantidad de sectores por arco. Por ejemplo, los cilindros 0–99 pueden tener un sector por pista, los cilindros 100–199 pueden tener dos por pista, etcétera. No todos los discos tienen sus pistas divididas en sectores.

La división de una pista en **bloques de disco** (o **páginas**) es establecida por el sistema operativo durante el **formateo** (o **inicialización**) del disco. El tamaño del bloque se fija durante la inicialización y no puede cambiarse dinámicamente. El tamaño típico de un bloque de disco oscila entre 512 y 8.192 bytes. A menudo, los discos que tienen los sectores codificados subdividen los sectores en bloques durante la inicialización. Los bloques están separados por **huecos entre bloques** de un tamaño fijo, que incluyen información de control codificada de forma especial y que se escribe durante la inicialización del disco. Esta información se utiliza

⁴ En algunos discos, los círculos están ahora conectados como en una especie de espiral continua.

Figura 13.1. (a) Un disco de una cara con hardware de lectura/escritura. (b) Un paquete de discos con hardware de lectura/escritura.



para determinar el bloque de la pista que va a continuación de cada hueco. La Tabla 13.1 representa las especificaciones de un disco típico.

Hay una mejora continua en la capacidad de almacenamiento y las velocidades de transferencia asociadas con los discos; además, su precio baja progresivamente (actualmente, un megabyte de almacenamiento de disco cuesta una fracción de dólar).

Un disco es un dispositivo direccionable de *acceso aleatorio*. La transferencia de datos entre la memoria principal y el disco tiene lugar en unidades de bloques de disco. La **dirección de hardware** de un bloque (una combinación de número de cilindro, número de pista [número de superficie dentro del cilindro en el que se encuentra la pista] y número de bloque [dentro de la pista]) se suministra al hardware de E/S del disco. En muchas unidades de disco modernas, el controlador de la unidad de disco mapea automáticamente un número denominado LBA (Dirección de bloque lógica, *Logical Block Address*), que es un número entre 0 y n (asumiendo que la capacidad total del disco es $n + 1$ bloques), al bloque correcto. También se proporciona la dirección de un **búfer** (un área reservada contigua en el almacenamiento principal que almacena un bloque). Para un comando de **lectura**, el bloque del disco se copia en el búfer, mientras que para un comando de **escritura**, el contenido del búfer se copia en el bloque de disco. En ocasiones, varios bloques contiguos, denominados **clúster**, se pueden transferir como una unidad. En este caso, el tamaño del búfer se ajusta para que coincida con el número de bytes del clúster.

El mecanismo hardware actual que lee o escribe un bloque es la **cabeza de lectura/escritura** del disco, que es parte de un sistema denominado **unidad de disco**. En la unidad de disco se monta un disco o un paquete de discos, que giran gracias a un motor incluido en la unidad. Una cabeza de lectura/escritura incluye un

Figura 13.2. Diferentes organizaciones de sectores en un disco. (a) Sectores subtendidos en un ángulo fijo. (b) Sectores que mantienen una densidad de grabación uniforme.

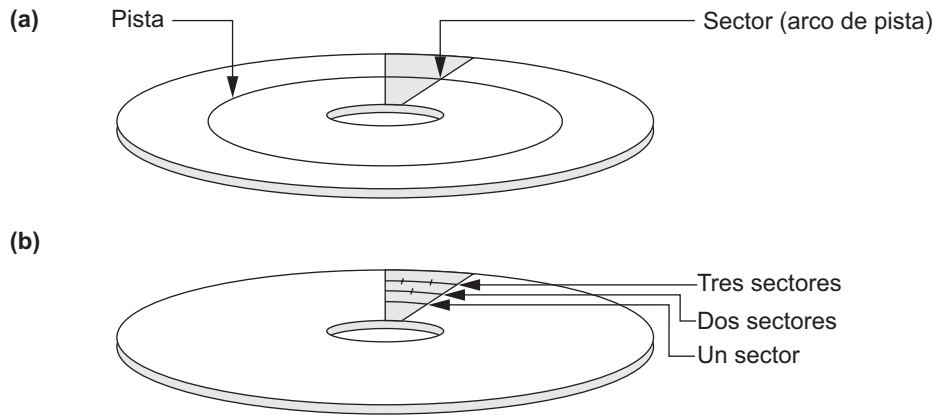


Tabla 13.1. Especificaciones de los discos Cheetah de alta gama de Seagate.

Descripción	Cheetah 10K.6	Cheetah 10K.7
Número de modelo	ST3146807LC	ST3300007LW
Factor de forma (anchura)	3,5 pulgadas	3,5 pulgadas
Factor de forma (altura)	1 pulgada	1 pulgada
Altura	25,4 mm.	25,4 mm.
Anchura	101,6 mm.	101,6 mm.
Longitud	146,05 mm.	146,05 mm.
Peso	0,73 Kg.	0,726 kg.
Capacidad/Interfaz		
Capacidad formateado	146,8 Gbytes	300 Gbytes
Tipo de interfaz	80-pin	68-pin
Configuración		
Número de discos (físicos)	4	4
Número de cabezas (físicas)	8	8
Número de cilindros	49.854	90.774
Total de pistas		726.192
Bytes por sector	512	512
Densidad superficial	36.000 Mb/pulg.cuadrada	
Densidad de pista	64.000 pistas/pulgada	105.000 pistas/pulgada
Densidad de grabación	570.000 bits/pulgada	658.000 bits/pulgada
Bytes/pista (promedio)		556
Rendimiento		
Velocidades de transferencia		
Velocidad de transferencia interna (mín.)	475 Mb/seg.	472 Mb/seg.
Velocidad de transferencia interna (máx.)	840 Mb/seg.	944 Mb/seg.
Velocidad de transferencia int. formateado (mín.)	43 MB/seg.	59 MB/seg.
Velocidad de transferencia int. formateado (máx.)	78 MB/seg.	118 MB/seg.
Velocidad de transferencia de E/S externa (máx.)	320 MB/seg.	320 MB/seg.
Promedio de velocidad de transferencia formateado	59,9 MB/seg.	59,5 MB/seg.

Tabla 13.1. Especificaciones de los discos Cheetah de alta gama de Seagate. (*Continuación*)

Tiempos de búsqueda	Cheetah 10K.6	Cheetah 10K.7
Tiempo medio de búsqueda (lectura)	4,7 ms. (típica)	4,7 ms. (típica)
Tiempo medio de búsqueda (escritura)	5,2 ms. (típica)	5,3 ms. (típica)
Pista a pista, búsqueda, lectura	0,3 ms. (típica)	0,2 ms. (típica)
Pista a pista, búsqueda, escritura	0,5 ms. (típica)	0,5 ms. (típica)
Buscar en el disco entero, lectura		9,5 ms. (típica)
Buscar en el disco entero, escritura		10,3 ms. (típica)
Latencia media	2,99 ms.	3 mseg.
Otros		
Tamaño predeterminado del búfer (caché)	8.000 KB	8.192 KB
Velocidad de giro	10.000 RPM	10.000 RPM
Tiempo desde el encendido hasta estar listo		25 segundos
Requisitos eléctricos		
Corriente		
Corriente normal (12VDC +/- 5%)	0,95 amperios	1,09 amperios
Corriente típica (5VDC +/- 5%)	0,9 amperios	0,68 amperios
Potencia en modo de bajo consumo (typ)	10,6 vatios	10,14 vatios
Fiabilidad		
Promedio de tiempo entre fallos (MTBF)	1.200.000 horas	1.400.000 horas
Errores de lectura recuperables	10 por 1012 bits	10 por 1012 bits en lectura
Errores de lectura no recuperables	10 por 1015 bits	10 por 1015 bits en lectura
Errores de búsqueda	10 por 108 bits	
Vida de servicio	5 año(s)	5 año(s)
Periodo de garantía limitado	5 año(s)	5 año(s)

(Cortesía de Seagate Technology)

componente electrónico conectado a un **brazo mecánico**. Los paquetes de discos con varias superficies están controlados por varias cabezas de lectura/escritura, una por cada superficie (véase la Figura 13.1[b]). Todos los brazos están conectados a un **activador**, conectado a otro motor eléctrico, que mueve las cabezas de lectura/escritura al unísono y las coloca con precisión sobre el cilindro o las pistas especificadas en una dirección de bloque.

Las unidades de disco de los discos duros hacen girar el paquete de discos continuamente, a una velocidad constante (normalmente, entre 5400 y 15.000 rpm). En el caso de un disquete, la unidad de disco empieza a girar el disco siempre que se inicia una solicitud de lectura o escritura, y la rotación cesa tan pronto como se completa la transferencia de datos. Una vez que la cabeza de lectura/escritura está posicionada en la pista correcta y el bloque especificado en la dirección del bloque ubicado bajo dicha cabeza, el componente electrónico de la cabeza se activa para transferir los datos. Algunas unidades de disco tienen cabezas de lectura/escritura fijas, tantas como pistas. Estas unidades se conocen como **discos de cabeza fija**, mientras que las unidades de disco con un activador se denominan **discos de cabeza móvil**. En las primeras, una pista o cilindro se selecciona electrónicamente cambiando a la cabeza de lectura/escritura adecuada, en lugar de recurrir a un movimiento mecánico real; en consecuencia, son mucho más rápidas. No obstante, el coste de las cabezas de lectura/escritura adicionales es muy alto, por lo que estos discos no se utilizan comúnmente.

El **controlador de disco**, normalmente incrustado en la unidad de disco, controla ésta y su interacción con el sistema. Una de las interfaces estándar que actualmente se utilizan para las unidades de disco de los PCs y las estaciones de trabajo es SCSI (Interfaz de almacenamiento para equipos pequeños, *Small Computer Storage Interface*). El controlador acepta comandos de E/S de alto nivel y toma la acción apropiada para posicionar el

brazo y provoca la acción de lectura/escritura. Para transferir un bloque de disco, dada su dirección, el controlador de disco primero debe colocar mecánicamente la cabeza de lectura/escritura en la pista correcta. El tiempo requerido para ello es el **tiempo de búsqueda**. Los tiempos de búsqueda típicos son de 7 a 10 msecs en los sobremesa y de 3 a 8 msecs en los servidores. Siguiendo a esto, hay otro retardo, denominado **retardo rotacional** o **latencia**, que se produce mientras el principio del bloque deseado gira hasta su posición bajo la cabeza de lectura/escritura. Este retardo depende de las rpm del disco. Por ejemplo, a 15.000 rpm, el tiempo por rotación es de 4 mseg y el retardo rotacional medio es el tiempo por media revolución, o 2 mseg. Por último, aún se necesita algo más de tiempo para transferir los datos; es lo que se conoce como **tiempo de transferencia del bloque**. Por tanto, el tiempo total necesario para localizar y transferir un bloque arbitrario, dada su dirección, es la suma del tiempo de búsqueda, el retardo rotacional y el tiempo de transferencia del bloque. El tiempo de búsqueda y el retardo rotacional son normalmente mucho más grandes que el tiempo de transferencia del bloque. Para que la transferencia de varios bloques sea más eficaz, es común transferir varios bloques consecutivos de la misma pista o cilindro. Esto elimina el tiempo de búsqueda y el retardo rotacional de todos los bloques menos del primero, lo que puede ahorrar un tiempo sustancial al transferir numerosos bloques contiguos. Normalmente, el fabricante del disco proporciona una **velocidad de transferencia en masa** para calcular el tiempo requerido para transferir bloques consecutivos. El Apéndice B contiene una explicación de éstos y de otros parámetros.

El tiempo necesario para localizar y transferir un bloque de disco está en el orden de los milisegundos, normalmente de 9 a 60 msecs. En el caso de bloques contiguos, la localización del primer bloque lleva aproximadamente de 9 a 60 msecs, pero la transferencia de los bloques subsiguientes puede tomar sólo de 0,4 a 2 msecs cada uno. Muchas técnicas de búsqueda se benefician de la recuperación consecutiva de bloques cuando se buscan datos en el disco. En cualquier caso, un tiempo de transferencia en el orden de milisegundos se considera bastante alto en comparación con el tiempo requerido para procesar los datos en la memoria principal por parte de las CPUs actuales. Por tanto, la localización de los datos en el disco es un *importante cuello de botella* en las aplicaciones de bases de datos. Las estructuras de ficheros las explicamos aquí y en el Capítulo 14 intentaremos *minimizar el número de transferencias de bloques* necesario para localizar y transferir los datos necesarios desde el disco a la memoria principal.

13.2.2 Dispositivos de almacenamiento en cinta magnética

Los discos son dispositivos de almacenamiento secundario de **acceso aleatorio** porque es posible acceder aleatoriamente a un bloque arbitrario del disco una vez especificada su dirección. Las cintas magnéticas son dispositivos de acceso secuencial; para acceder al bloque n -ésimo de una cinta, primero tenemos que escanear los $n - 1$ bloques precedentes. Los datos se almacenan en bobinas de cinta magnética de alta capacidad, parecidas a las cintas de audio o de vídeo. Para leer datos de, o escribir datos en, una **bobina de cinta** se necesita una unidad de cinta. Normalmente, cada grupo de bits que forma un byte se almacena a través de la cinta, y los bytes se almacenan consecutivamente en la cinta.

Se utiliza una cabeza de lectura/escritura para leer o escribir datos en la cinta. Los registros de datos también se almacenan en la cinta en bloques (aunque los bloques pueden ser sustancialmente más grandes que los de los discos, y los huecos entre los bloques también son bastante grandes). Con las densidades de cinta típicas de 1.600 a 6.250 bytes por pulgada, un hueco entre bloques típico⁵ de 0,6 pulgadas corresponde a un espacio de almacenamiento desaprovechado de 960 a 3750 bytes. Es costumbre agrupar muchos registros juntos en un bloque para una mejor utilización del espacio.

La principal característica de una cinta es su requisito de acceder a los bloques de datos en **orden secuencial**. Para obtener un bloque de en medio de una bobina de cinta, se monta la cinta y se escanea hasta que el bloque requerido se coloca bajo la cabeza de lectura/escritura. Por esta razón, el acceso a la cinta puede ser lento

⁵ Denominados *huecos entre registros* en la terminología de las cintas.

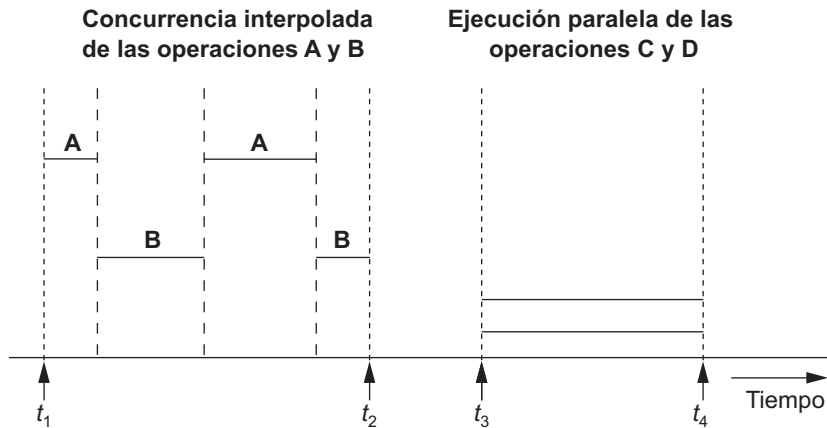
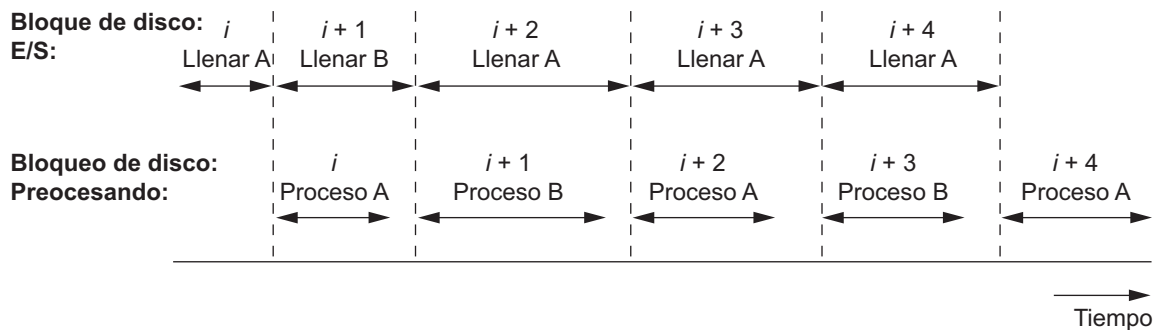
y las cintas no se utilizan para almacenar datos online, excepto para algunas aplicaciones especializadas. Sin embargo, las cintas sirven para una función muy importante, la **copia de seguridad** de la base de datos. Una razón para hacer una copia de seguridad es mantener copias de los ficheros en disco por si se pierden datos como consecuencia de un fallo del disco, que puede ocurrir si la cabeza de lectura/escritura toca la superficie del disco debido a un mal funcionamiento mecánico. Por esta razón, los ficheros en disco se copian periódicamente en una cinta. En muchas aplicaciones online críticas, como los sistemas de reserva en aerolíneas, y al objeto de evitar tiempos de inactividad, se utilizan sistemas de réplica para mantener tres conjuntos de discos idénticos (dos en funcionamiento online y uno como copia de seguridad). Aquí, los discos offline se convierten en dispositivos de copia de seguridad. Los tres giran para poder alternarse, por si uno de ellos falla. Las cintas también se pueden utilizar para almacenar los ficheros de bases de datos excesivamente grandes. Los ficheros de bases de datos que rara vez se utilizan o están desactualizados, pero requieren que se guarde un registro histórico, pueden **archivarse** en cinta. Recientemente, las cintas magnéticas de 8 mm más pequeñas (parecidas a las que se utilizan en las videocámaras) que permiten almacenar hasta 50 GB, así como los cartuchos de exploración helicoidal de 4 mm y los CDs y DVDs grabables, se han popularizado como medios para hacer una copia de seguridad de los ficheros de datos almacenados en los PCs y las estaciones de trabajo. También se utilizan para almacenar imágenes y librerías del sistema. Una tarea primordial es hacer una copia de seguridad de las bases de datos de la empresa para que no se pierda información. Actualmente, las librerías de cintas con slots para varios cientos de cartuchos se utilizan con las Cintas lineales digitales y superdigitales (DLTs y SDLTs) que tienen capacidades de cientos de gigabytes y que graban los datos en pistas lineales. Los brazos robotizados se utilizan para escribir en varios cartuchos en paralelo utilizando varias unidades de cinta y software de etiquetado automático para identificar los cartuchos de copia de seguridad. Un ejemplo de librería gigante es el modelo L5500 de Storage Technology, que puede almacenar hasta 13,2 petabytes (petabyte = 1000 TB), con un rendimiento de 55TB/hora. Dejamos la explicación de la tecnología RAID de almacenamiento en disco y de las redes de área de almacenamiento y del almacenamiento conectado a la red para el final del capítulo.

13.3 Almacenamiento de bloques en el búfer

Cuando es necesario transferir varios bloques desde el disco a la memoria principal y se conocen todas las direcciones de bloque, se pueden reservar varios búferes en la memoria principal para acelerar la transferencia. Mientras se está leyendo o escribiendo en un búfer, la CPU puede procesar datos en otro búfer porque hay un procesador (controlador) de E/S de disco independiente que, una vez iniciado, puede proseguir con la transferencia de un bloque de datos entre la memoria y el disco independientemente de, y en paralelo, al procesamiento de la CPU.

La Figura 13.3 ilustra cómo se pueden llevar a cabo dos procesos en paralelo. Los procesos A y B están ejecutándose **al mismo tiempo** de forma **interpolada**, mientras que los procesos C y D se están ejecutando **al mismo tiempo pero en paralelo**. Cuando una CPU controla varios procesos, la ejecución en paralelo no es posible. Sin embargo, los procesos todavía se pueden ejecutar al mismo tiempo de forma interpolada. El almacenamiento en búfer es más útil cuando los procesos se pueden ejecutar al mismo tiempo y en paralelo, porque un procesador de E/S de disco independiente esté disponible, o porque la CPU tenga varios procesadores.

La Figura 13.4 ilustra cómo se pueden efectuar en paralelo la lectura y el procesamiento, cuando el tiempo necesario para procesar un bloque de disco en la memoria es inferior al tiempo necesario para leer el siguiente bloque y llenar un búfer. La CPU puede iniciar el procesamiento de un bloque una vez completada su transferencia a la memoria principal; al mismo tiempo, el procesador de E/S de disco puede estar leyendo y transfiriendo el siguiente bloque a un búfer diferente. Esta técnica se denomina **doble búfer** y también puede utilizarse para leer un flujo continuo de bloques del disco a la memoria. El doble búfer permite la lectura o escritura continuas de datos en bloques de disco consecutivos, lo que elimina el tiempo de búsqueda y el retardo rotacional de la transferencia de todos los bloques, excepto del primero. Además, los datos quedan listos para su procesamiento, lo que reduce el tiempo de espera en los programas.

Figura 13.3. Concurrency interpolada frente a ejecución en paralelo.**Figura 13.4.** Uso de dos búferes, A y B, para leer desde el disco.

13.4 Ubicación de los registros de fichero en disco

En esta sección definimos los conceptos de registro, tipos de registros y fichero. Después, explicamos las técnicas para colocar los registros de un fichero en el disco.

13.4.1 Registros y tipos de registros

Los datos normalmente se almacenan en forma de **registros**. Un registro consta de una colección de **valores** o **elementos** de datos relacionados, donde cada valor está formado por uno o más bytes y corresponde a un **campo** concreto del registro. Los registros normalmente describen entidades y sus atributos. Por ejemplo, un registro EMPLEADO representa una entidad empleado, y el valor de cada campo del registro especifica algún atributo de ese empleado, como su Nombre, FechaNac, Sueldo o Supervisor. Una colección de nombres de campos y sus correspondientes tipos de datos constituyen un **tipo de registro** o una definición de **formato de registro**. Un **tipo de datos**, asociado a cada campo, especifica los tipos de valores que un campo puede tomar.

El tipo de datos de un campo normalmente es uno de los tipos de datos estándar que se utilizan en programación. Nos referimos a los tipos numéricos (entero, entero largo o de coma flotante), cadenas de caracteres (de longitud fija o variable), booleanos (que sólo pueden tener los valores 0 y 1, o TRUE y FALSE) y, en ocasio-

nes, tipos de datos de **fecha** y **hora** especialmente codificados. Un entero puede requerir 4 bytes, un entero largo 8 bytes, un número real 4 bytes, un booleano 1 byte, una fecha 10 bytes (asumiendo el formato AAAA-MM-DD), y una cadena de longitud fija de k caracteres k bytes. Las cadenas de longitud variable pueden requerir tantos bytes como caracteres tenga el valor del campo. Por ejemplo, un tipo de registro EMPLEADO puede definirse (utilizando la notación del lenguaje de programación C) como la siguiente estructura:

```
struct empleado{
    char nombre[30];
    char dni[9];
    int sueldo;
    int codtrabajo;
    char departamento[20];
} ;
```

En las aplicaciones de bases de datos recientes, puede darse la necesidad de almacenar elementos de datos consistentes en grandes objetos desestructurados, que representan imágenes, flujos de vídeo o audio digitalizados, o texto libre. Son los denominados **BLOBS** (Objetos binarios grandes, *Binary Large Objects*). Un elemento de datos BLOB normalmente se almacena independientemente de su registro en un almacén de bloques de disco, y en el registro se incluye un puntero al BLOB.

13.4.2 Ficheros, registros de longitud fija y registros de longitud variable

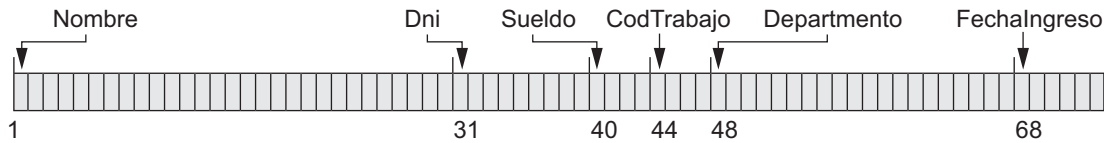
Un **fichero** es una *secuencia* de registros. En muchos casos, todos los registros de un fichero son del mismo tipo de registro. Si cada registro del fichero tiene exactamente el mismo tamaño (en bytes), se dice que el fichero está compuesto por **registros de longitud fija**. Si en el fichero hay registros que tienen tamaños diferentes, se dice que el fichero está compuesto por **registros de longitud variable**. Un fichero puede tener registros de longitud variable por varias razones:

- Los registros del fichero son del mismo tipo de registro, pero uno o más de los campos tienen un tamaño variable (**campos de longitud variable**). Por ejemplo, el campo Nombre de EMPLEADO puede ser un campo de longitud variable.
- Los registros del fichero son del mismo tipo de registro, pero uno o más de los campos pueden tener varios valores para registros individuales; un campo así se denomina **campo repetitivo** y un grupo de valores para el campo normalmente se conoce como **grupo repetitivo**.
- Los registros del fichero son del mismo tipo, pero uno o más de los campos son **opcionales**; es decir, pueden tener valores para algunos de los registros del fichero (**campos opcionales**).
- El fichero contiene registros de *diferentes tipos de registro* y, por tanto, de tamaño variable (**fichero mixto**). Esto suele ocurrir si se agruparon (se colocaron juntos) registros relacionados de tipos diferentes en bloques de disco; por ejemplo, los registros de INFORME_CALIF de un estudiante concreto pueden estar colocados a continuación del registro de ese ESTUDIANTE.

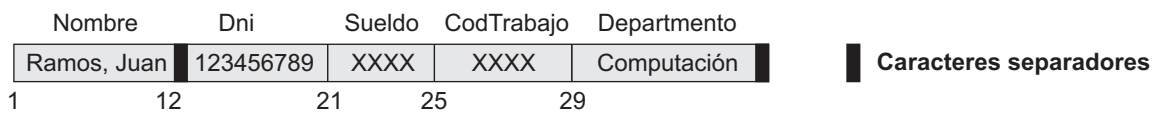
Los registros EMPLEADO de longitud fija de la Figura 13.5(a) tienen un tamaño de registro de 71 bytes. Cada registro tiene los mismos campos, y las longitudes de campo son fijas, por lo que el sistema puede identificar la posición del byte inicial de cada campo relativa a la posición inicial del registro. Esto facilita la localización de valores de campo por parte de los programas que acceden a dichos ficheros. Es posible representar un fichero que lógicamente debe tener registros de longitud variable como un fichero de registros de longitud fija. Por ejemplo, en el caso de los campos opcionales, podríamos incluir *todos los campos en cada registro del fichero*, pero almacenando un valor NULL especial si no existe un valor para esos campos. En un campo repetitivo, podríamos asignar tantos espacios en cada registro como el *número máximo de valores* que el campo puede tomar. En cualquier caso, el espacio se desaprovecha cuando ciertos registros no tienen especificados

Figura 13.5. Tres formatos de almacenamiento de un registro. (a) Un registro de longitud fija con seis campos y un tamaño de 71 bytes. (b) Un registro con dos campos de longitud variable y tres campos de longitud fija. (c) Un registro de campo variable con tres tipos de caracteres separadores.

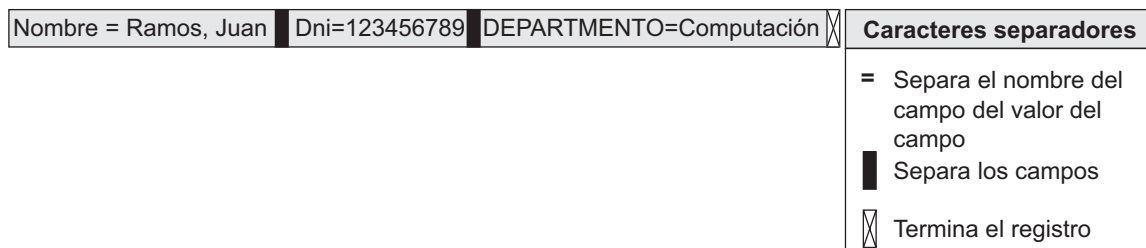
(a)



(b)



(c)



valores para todos los espacios físicos. Ahora vamos a considerar otras opciones para formatear los registros de un fichero de registros de longitud variable.

Para los *campos de longitud variable*, cada registro tiene un valor para cada campo, pero no conocemos la longitud exacta de algunos valores de campo. Al objeto de determinar dentro de un registro en particular los bytes que representan a cada campo, podemos utilizar caracteres **separadores** especiales (por ejemplo, ? o % o \$), que no aparecen en ningún valor de campo, para finalizar los campos de longitud variable (véase la Figura 13.5[b]), o podemos almacenar la longitud en bytes del campo en el registro, por delante del valor del campo.

Un fichero de registros con *campos opcionales* puede formatearse de varias formas. Si el número total de campos del registro es grande, pero el número de campos que realmente aparecen en un registro típico es pequeño, podemos incluir en cada registro una secuencia de pares <nombre-del-campo, valor-del-campo> en lugar de únicamente los valores de campo. En la Figura 13.7(c) se utilizan tres tipos de caracteres separadores, aunque podemos utilizar el mismo separador para los dos primeros propósitos: separar el nombre del campo del valor del campo, y separar un campo del siguiente. Una opción más práctica es asignar un pequeño código de **tipo de campo** (por ejemplo, número entero) a cada campo e incluir en cada registro una secuencia de pares <tipo-de-campo, valor-del-campo>, en lugar de los pares <nombre-del-campo, valor-del-campo>.

Un *campo repetitivo* necesita un carácter separador para separar los valores repetitivos del campo y otro separador para indicar la terminación del campo. Por último, en el caso de un fichero que incluye *registros de diferentes tipos*, cada registro va precedido por un indicador de **tipo de registro**. Con razón, los programas que

procesan ficheros de registros de longitud variable (que normalmente son parte del sistema de ficheros y, por tanto, están ocultos a los programadores típicos) tienen que ser más complejos que los que procesan registros de longitud fija, en los que se conocen la posición de inicio y el tamaño de cada campo, que son fijos.⁶

13.4.3 Bloqueo de registros y registros extendidos frente a no extendidos

Los registros de un fichero deben asignarse a bloques de disco porque un bloque es la *unidad de transferencia de datos* entre el disco y la memoria. Cuando el tamaño del bloque es mayor que el tamaño del registro, un bloque puede contener varios registros, aunque algunos ficheros pueden tener registros que por su tamaño no entran en un bloque. Suponga que el tamaño de bloque es de B bytes. En el caso de un fichero de registros con una longitud fija de R bytes, siendo $B \geq R$, podemos encajar $bfr = \lfloor B/R \rfloor$ registros por bloque, donde $\lfloor (x) \rfloor$ (función suelo) redondea por abajo el número x a un entero. El valor bfr se conoce como **factor de bloqueo** del fichero. En general, R no divide B con exactitud, por lo que en cada bloque tenemos algo de espacio inutilizado igual a:

$$B - (bfr * R) \text{ bytes}$$

Para utilizar este espacio desaprovechado, podemos almacenar parte de un registro en un bloque y el resto en otro. Un **puntero** al final del primer bloque apunta al bloque que contiene el resto del registro, en caso de que no sean bloques consecutivos en el disco. Esta organización se denomina **extendida** porque los registros pueden abarcar más de un bloque. Siempre que un registro es más grande que un bloque, *debemos* utilizar una organización extendida. Si los registros no tienen permitido sobrepasar los límites de un bloque, se dice que la organización **no es extendida**. Ésta se utiliza con los registros de longitud fija cuando $B > R$, porque hace que cada registro empiece en una ubicación conocida del bloque, simplificándose el procesamiento del registro. En el caso de los registros de longitud variable, se puede utilizar cualquiera de las dos organizaciones, extendida o no extendida. Si el registro medio es grande, es recomendable utilizar la organización extendida para reducir la pérdida de espacio en cada bloque. La Figura 13.6 ilustra la organización extendida frente a la no extendida. Para los registros de longitud variable utilizando la organización extendida, cada bloque puede almacenar una cantidad diferente de registros. En este caso, el factor de bloqueo bfr representa la cantidad media de registros por bloque de ese fichero. Podemos utilizar bfr para calcular el número de bloques b necesarios para un fichero de r registros:

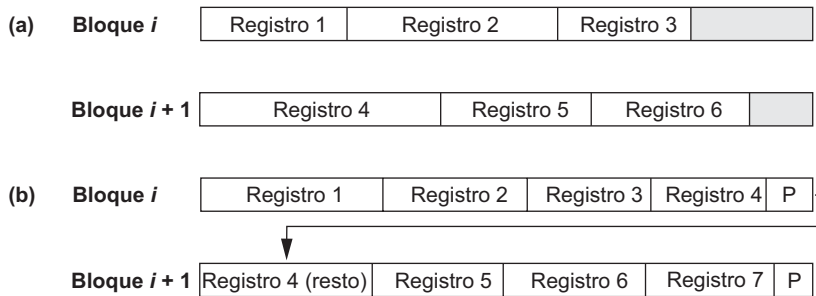
$$b = \lceil (r/bfr) \rceil \text{ bloques}$$

donde $\lceil (x) \rceil$ (función techo) redondea el valor x al siguiente entero.

13.4.4 Asignación de bloques de fichero en un disco

Hay varias técnicas estándar para asignar los bloques de un fichero en disco. En una **asignación continua**, los bloques del fichero se asignan a bloques de disco consecutivos. De este modo, la lectura del fichero entero es mucho más rápida mediante el búfer doble, pero se hace más complejo expandir el fichero. En una **asignación enlazada**, cada bloque del fichero contiene un puntero al siguiente bloque del fichero. Esto hace más sencilla la expansión del fichero, pero más lenta la lectura del fichero entero. Una combinación de las dos asigna **clústeres** de bloques de disco consecutivos, y los clústeres se enlazan. En ocasiones, a los clústeres se les denomina **segmentos de fichero** o **extensiones**. Otra posibilidad es utilizar una **asignación indexada**, en la que uno o más **bloques de índice** contienen punteros a los bloques del fichero. También se utilizan combinaciones de estas técnicas.

⁶ También son posibles otros esquemas para representar los registros de longitud variable.

Figura 13.6. Tipos de organización de registros. (a) No extendida. (b) Extendida.

13.4.5 Cabeceras de fichero

Una **cabecera de fichero** o **descriptor de fichero** contiene información sobre un fichero que los programas del sistema necesitan para acceder a los registros. La cabecera incluye información para determinar las direcciones de disco de los bloques del fichero y las descripciones de formato del registro (que pueden incluir longitudes de campo y orden de los campos dentro del registro en el caso de los registros no extendidos de longitud fija, y códigos de tipo, caracteres separadores y códigos de tipo de registro para los registros de longitud variable).

Para buscar un registro en el disco, se copian uno o más bloques en los búferes de la memoria principal. Los programas buscan después el registro o registros deseados en los búferes, utilizando la información de la cabecera del fichero. Si no se conoce la dirección del bloque que contiene el registro deseado, los programas de búsqueda deben realizar una **búsqueda lineal** a través de los bloques del fichero. Los bloques del fichero se van copiando en un búfer y se va buscando hasta localizar el registro o hasta que se ha buscado sin éxito en todos los bloques del fichero. Si el fichero es muy grande, esto puede llevar mucho tiempo. El objetivo de una buena organización del fichero es localizar el bloque que contiene el registro deseado con una mínima cantidad de transferencias de bloques.

13.5 Operaciones sobre ficheros

Las operaciones sobre ficheros se pueden agrupar en **operaciones de recuperación** y **operaciones de actualización**. Las primeras no cambian ningún dato del fichero, puesto que únicamente localizan ciertos registros para que los valores de sus campos se puedan examinar o procesar. Las segundas modifican el fichero mediante la inserción o la eliminación de registros, o modificando los valores de los campos. En cualquier caso, tenemos que **seleccionar** uno o más registros para la recuperación, la eliminación o la modificación, basándonos en una **condición de selección** (o **condición de filtrado**), que especifica los criterios que el(los) registro(s) debe(n) satisfacer.

Consideremos que el fichero EMPLEADO tiene los campos Nombre, Dni, Sueldo, CodTrabajo y Departamento. Una **condición de selección sencilla** puede implicar una comparación de igualdad sobre el valor de algún campo: por ejemplo, (Dni = '123456789') o (Departamento = 'Investigación'). Las condiciones más complejas pueden implicar otros tipos de operadores de comparación, como $>$ o $\geq$; por ejemplo, (Sueldo $\geq$ 30000). El caso general es tener una expresión booleana en los campos del fichero como condición de selección.

Cuando varios registros del fichero satisfacen una condición de búsqueda, el primer registro localizado (respecto a la secuencia física de los registros del fichero) es designado como **registro actual**. Las siguientes operaciones de búsqueda comienzan a partir de este registro y localizan el siguiente registro que satisface la condición.

Las operaciones de localización y acceso a los registros de un fichero varían de un sistema a otro. Presentamos a continuación un conjunto de operaciones representativas. Normalmente, los programas de alto nivel, como los programas DBMS, acceden a los registros utilizando estos comandos, por lo que a veces nos referimos a **variables de programa** en las siguientes descripciones:

- **Open (abrir).** Prepara el fichero para la lectura o escritura. Asigna los búferes apropiados (normalmente, al menos dos) para albergar los bloques del fichero, y recupera su cabecera. Establece el puntero del fichero al principio del mismo.
- **Reset (reiniciar).** Hace que el puntero de un fichero abierto apunte al principio del fichero.
- **Find (o Locate) (buscar).** Busca el primer registro que satisface una condición de búsqueda. Transfiere el bloque que contiene ese registro a un búfer de la memoria principal (si todavía no está en el búfer). El puntero del fichero apunta al registro del búfer, que se convierte en el *registro actual*. En ocasiones, se utilizan diferentes verbos para indicar si el registro localizado será recuperado o actualizado.
- **Read (o Get) (leer u obtener).** Copia el registro actual desde el búfer a una variable de programa del programa de usuario. Este comando también puede hacer avanzar el puntero del registro actual al siguiente registro del fichero, lo que puede hacer necesario leer del disco el siguiente bloque del fichero.
- **FindNext (buscar siguiente).** Busca el siguiente registro que satisface la condición de búsqueda. Transfiere el bloque en el que se encuentra el registro a un búfer de la memoria principal (si todavía no se encuentra aquí). El registro se almacena en el búfer y se convierte en el registro actual.
- **Delete (borrar).** Borra el registro actual y (finalmente) actualiza el fichero en disco para reflejar el borrado.
- **Modify (modificar).** Modifica los valores de algunos campos del registro actual y (finalmente) actualiza el fichero en disco para reflejar la modificación.
- **Insert (insertar).** Inserta un registro nuevo en el fichero localizando el bloque donde se insertará el registro, transfiriendo ese bloque a un búfer de la memoria principal (si todavía no se encuentra allí), escribiendo el registro en el búfer y (finalmente) escribiendo el búfer en el disco para reflejar la inserción.
- **Close (cerrar).** Completa el acceso al fichero liberando los búferes y ejecutando cualquier otra operación de limpieza necesaria.

Las operaciones anteriores (excepto las de abrir y cerrar) se conocen como operaciones de **un registro por vez**, porque cada operación se aplica a un solo registro. Es posible aglutinar las operaciones Find, FindNext y Read en una sola operación, Scan, cuya descripción es la siguiente:

- **Scan.** Si el fichero simplemente se ha abierto o reiniciado, *Scan* devuelve el primer registro; en caso contrario, devuelve el siguiente registro. Si con la operación especificamos una condición, el registro devuelto es el primero o el siguiente que satisface esa condición.

En los sistemas de bases de datos, se pueden aplicar **operaciones de grupo** al fichero, como, por ejemplo, las siguientes:

- **FindAll (buscar todo).** Localiza *todos* los registros del fichero que satisfacen una condición de búsqueda.
- **Find (o Locate) n (buscar n).** Busca el primer registro que satisface una condición de búsqueda y, después, continúa localizando los $n - 1$ siguientes registros que satisfacen la misma condición. Transfiere los bloques que contienen los n registros a un búfer de la memoria principal (si todavía no están allí).

- **FindOrdered (buscar ordenados).** Recupera todos los registros del fichero en un orden específico.
- **Reorganize (reorganizar).** Inicia el proceso de reorganización. Como veremos, algunas organizaciones de ficheros requieren una reorganización periódica. Un ejemplo puede ser reordenar los registros del fichero ordenándolos por un campo específico.

En este punto, conviene observar la diferencia entre los términos **organización del fichero** y **método de acceso**. El primero se refiere a la organización de los datos de un fichero en registros, bloques y estructuras de acceso; esto incluye la forma en que los registros y los bloques se colocan e interconectan en el medio de almacenamiento. Un método de acceso, por el contrario, proporciona un grupo de operaciones (como las enumeradas anteriormente) que se pueden aplicar a un fichero. En general, es posible aplicar varios métodos de acceso a una organización de fichero. Algunos, no obstante, sólo se pueden aplicar a ficheros organizados de determinadas formas. Por ejemplo, no podemos aplicar un método de acceso indexado a un fichero que no tiene un índice (consulte el Capítulo 14).

Normalmente, esperamos utilizar unas condiciones de búsqueda más que otras. Algunos ficheros pueden ser **estáticos**, pues es raro ejecutar sobre ellos operaciones de actualización; otros, los denominados archivos **dinámicos**, pueden cambiar más a menudo pues se les están aplicando operaciones de actualización constantemente. Una organización de fichero satisfactoria debe ejecutar tan eficazmente como sea posible las operaciones que esperamos aplicar frecuentemente al fichero. Por ejemplo, el fichero EMPLEADO de la Figura 13.5(a) almacena los registros de los empleados actuales de la empresa. Esperamos insertar registros (cuando se contratan empleados), eliminar registros (cuando los empleados abandonan la empresa) y modificar registros (por ejemplo, cuando se modifica el sueldo o el puesto de trabajo de un empleado). La eliminación o modificación de un registro requiere una condición de selección para identificar un registro o un conjunto de registros concreto. La recuperación de uno o más registros también requiere una condición de selección.

Si los usuarios prevén aplicar una condición de búsqueda basada principalmente en el DNI, el diseñador debe elegir una organización de fichero que facilite la localización de un registro dado su valor de Dni. Esto puede implicar que los registros se ordenen físicamente por el valor de Dni o que se deba crear un índice con los valores de Dni (consulte el Capítulo 14). Vamos a suponer que una segunda aplicación utiliza el fichero para generar las nóminas de los empleados, y que requiere que esas nóminas estén agrupadas por departamento. Para esta aplicación, es recomendable almacenar contiguamente los registros de todos los empleados que tienen el mismo valor de departamento, agrupándolos en bloques y, quizá, ordenándolos por su nombre dentro de cada departamento. No obstante, esta distribución entra en conflicto con la ordenación de los registros por sus valores de DNI. Si las dos aplicaciones son importantes, el diseñador debe elegir una organización que permita llevar a cabo eficazmente ambas operaciones. Por desgracia, en muchos casos no puede haber una organización que permita la implementación eficaz de todas las operaciones necesarias. En estos casos, hay que tener en cuenta la importancia esperada y la mezcla de las operaciones de recuperación y actualización.

En las siguientes secciones y en el Capítulo 14 explicamos los métodos para organizar los registros de un fichero en el disco. Para crear los métodos de acceso se utilizan diversas técnicas generales, como la ordenación, la dispersión y la indexación. Además, con muchas de las organizaciones de ficheros funcionan diversas técnicas generales de manipulación de inserciones y borrados.

13.6 Ficheros de registros desordenados (ficheros heap)

Es el tipo de organización más sencillo y básico, según el cual los registros se guardan en el fichero en el mismo orden en que se insertan; es decir, los registros se insertan al final del fichero. Esta organización se conoce como **fichero heap o pila**.⁷ Se utiliza a menudo con rutas de acceso adicionales, como los índices

⁷ En ocasiones, a esta organización se la conoce como **fichero secuencial**.

secundarios que explicamos en el Capítulo 14. También se utiliza para recopilar y almacenar registros de datos para un uso futuro.

La inserción de un registro nuevo es muy *eficaz*. El último bloque de disco del fichero se copia en el búfer, se añade el registro nuevo y se **reescribe** el bloque de nuevo en el disco. En la cabecera del fichero se guarda la dirección del último bloque del fichero. Sin embargo, la búsqueda de un registro utilizando cualquier condición de búsqueda implica una **búsqueda lineal**, bloque a bloque, por todo el fichero (un procedimiento muy costoso). Si sólo un registro satisface la condición de búsqueda, entonces, por término medio, un programa leerá en memoria y buscará en la mitad de los bloques antes de encontrar el registro. En el caso de un fichero de b bloques, esto requiere buscar de media en $(b/2)$ bloques. Si ningún registro satisface la condición de búsqueda, el programa debe buscar en los b bloques del fichero.

Para eliminar un registro, un programa primero debe buscar su bloque, copiar el bloque en un búfer, eliminar el registro del búfer y, por último, **reescribir el bloque** en el disco. Esto deja espacio inutilizado en el bloque de disco. La eliminación de una gran cantidad de registros de este modo supone un derroche de espacio de almacenamiento. Otra técnica que se utiliza para borrar registros es contar con un byte o un bit extra, denominado **marcador de borrado**, en cada registro. Un registro se borra estableciendo el marcador de borrado a un determinado valor. Un valor diferente del marcador indica un registro válido (no borrado). Los programas de búsqueda sólo consideran los registros válidos cuando realizan sus búsquedas. Estas dos técnicas de borrado requieren una **reorganización** periódica del fichero para reclamar el espacio inutilizado correspondiente a los registros borrados. Durante la reorganización, se accede consecutivamente a los bloques del fichero, y los registros se empaquetan eliminando los registros borrados. Tras la reorganización, los bloques se llenan de nuevo en toda su capacidad. Otra posibilidad es utilizar el espacio de los registros borrados al insertar registros nuevos, aunque esto requiere una contabilidad adicional para hacer un seguimiento de las ubicaciones vacías.

Podemos utilizar la organización extendida o la no extendida para un fichero desordenado, y podemos utilizarla con registros de longitud fija o de longitud variable. La modificación de un registro de longitud variable puede requerir el borrado del registro antiguo y la inserción del registro modificado, porque puede ser que este último no encaje en su antiguo espacio en disco.

Para leer todos los registros en orden por los valores de algún campo, debemos crear una copia ordenada del fichero. La ordenación es una operación costosa si el fichero es grande, y para ella se utilizan técnicas especiales de **ordenación externa** (consulte el Capítulo 15).

En un fichero desordenado de *registros de longitud fija* que utiliza *bloques extendidos* y la *asignación continua*, es inmediato el acceso a cualquier registro por su **posición** en el fichero. Si los registros del fichero están numerados como $0, 1, 2, \dots, r - 1$ y los registros de cada bloque están numerados como $0, 1, \dots, bfr - 1$, donde bfr es el factor de bloque, entonces el registro i -ésimo del fichero se encuentra en el bloque $\lfloor (i/bfr) \rfloor$ y es el registro número $(i \bmod bfr)$ de ese bloque. Este tipo de fichero suele recibir el nombre de **fichero relativo** o **directo** porque es fácil acceder directamente a los registros por sus posiciones relativas. El acceso a un registro por su posición no ayuda a localizar un registro basándose en una condición de búsqueda; sin embargo, facilita la construcción de rutas de acceso en el fichero, como los índices que explicamos en el Capítulo 14.

13.7 Ficheros de registros ordenados (ficheros ordenados)

Los registros de un fichero se pueden ordenar físicamente en el disco en función de los valores de uno de sus campos, denominado **campo de ordenación**. Esto conduce a un **fichero ordenado** o **secuencial**.⁸ Si el campo

⁸ El término *fichero secuencial* también se ha utilizado para referirse a los ficheros desordenados.

Figura 13.7. Algunos bloques de un fichero ordenado (secuencial) de registros EMPLEADO con Nombre como campo clave de ordenación.

	Nombre	Dni	FechaNac	Trabajo	Sueldo	Sexo
Bloque 1	Aaron, Ed					
	Abbott, Diane					
	⋮					
	Acosta, Marc					
Bloque 2	Adams, John					
	Adams, Robin					
	⋮					
	Akers, Jan					
Bloque 3	Alexander, Ed					
	Alfred, Bob					
	⋮					
	Allen, Sam					
Bloque 4	Allen, Troy					
	Anders, Keith					
	⋮					
	Anderson, Rob					
Bloque 5	Anderson, Zach					
	Angeli, Joe					
	⋮					
	Archer, Sue					
Bloque 6	Arnold, Mack					
	Arnold, Steven					
	⋮					
	Atkins, Timothy					
Bloque n-1	⋮					
	Wong, James					
	Wood, Donald					
	⋮					
Bloque n	Woods, Manny					
	Wright, Pam					
	Wyatt, Charles					
	⋮					
	Zimmer, Byron					

de ordenación también es un **campo clave** (un campo que garantiza un valor exclusivo en cada registro) del fichero, entonces el campo se denomina **clave de ordenación** del fichero. La Figura 13.7 muestra un fichero ordenado con Nombre como campo clave de ordenación (asumiendo que los empleados tienen nombres distintos).

Los registros ordenados tienen algunas ventajas sobre los ficheros desordenados. En primer lugar, la lectura de los registros en el orden marcado por los valores de la clave de ordenación es extremadamente eficaz porque no se necesita una ordenación. En segundo lugar, encontrar el siguiente registro al actual según el orden de la clave de ordenación normalmente no requiere acceder a bloques adicionales porque el siguiente registro se encuentra en el mismo bloque que el actual (a menos que el registro actual sea el último del bloque). En tercer lugar, el uso de una condición de búsqueda basándose en el valor de un campo clave de ordenación ofrece un acceso más rápido cuando se utiliza la técnica de búsqueda binaria, que constituye una mejora respecto a las búsquedas lineales, aunque no se utiliza habitualmente para los ficheros de disco.

Una **búsqueda binaria** en ficheros de disco puede realizarse en los bloques en lugar de en los registros. Vamos a suponer que el fichero tiene b bloques numerados como 1, 2,..., b ; los registros están ordenados ascendentemente por el valor de su campo clave de ordenación; y estamos buscando un registro cuyo campo clave de ordenación tenga el valor K . Asumiendo que las direcciones de disco de los bloques del fichero están disponibles en la cabecera del fichero, la búsqueda binaria puede describirse con el Algoritmo 13.1. Una búsqueda binaria normalmente accede a $\log_2(b)$ bloques, se encuentre o no el registro [una mejora respecto a las búsquedas lineales, en las que, por término medio, se accede a $(b/2)$ bloques cuando se encuentra el registro y a b bloques cuando el registro no se encuentra].

Algoritmo 13.1. Búsqueda binaria en una clave de ordenación de un fichero de disco.

```

 $l \leftarrow 1; u \leftarrow b$ ; (*  $b$  es el número de bloques del fichero *)
while ( $u \geq l$ ) do
    begin  $i \leftarrow (l + u) \text{ div } 2$ ;
        leer el bloque  $i$  del fichero en el búfer;
        if  $K < (\text{valor del campo clave de ordenación del primer registro en el bloque } i)$ 
            then  $u \leftarrow i - 1$ 
        else if  $K > (\text{valor del campo clave de ordenación del último registro en el bloque } i)$ 
            then  $l \leftarrow i + 1$ 
        else if el registro con el campo clave de ordenación =  $K$  está en el búfer
            then goto encontrado
        else goto noencontrado;
    end;
goto noencontrado;

```

Un criterio de búsqueda que implica las condiciones $>$, $<$, $\geq$ y $\leq$, es muy eficaz, porque la ordenación física de los registros significa que todos los registros que satisfacen la condición están contiguos en el fichero. Por ejemplo, y en referencia a la Figura 13.9, si el criterio de búsqueda es ($\text{Nombre} < 'G'$), donde $<$ significa *alfabéticamente anterior*, los registros que satisfacen el criterio de búsqueda son los que se encuentran entre el principio del fichero y el primer registro cuyo campo Nombre tiene un valor que empieza por la letra 'G'.

La ordenación no ofrece ninguna ventaja para el acceso aleatorio u ordenado de los registros basándose en los valores de otros *campos no ordenados* del fichero. En estos casos, realizamos una búsqueda lineal para el acceso aleatorio. Para acceder a los registros en orden basándose en un campo desordenado, es necesario crear otra copia ordenada (con un orden diferente) del fichero.

La inserción y eliminación de registros son operaciones costosas para un archivo ordenado, porque los registros deben permanecer físicamente ordenados. Para insertar un registro debemos encontrar su posición correcta en el fichero, basándonos en el valor de su campo de ordenación, y después hacer espacio en el fichero para insertar el registro en esa posición. Si se trata de un fichero grande, esta operación puede consumir mucho tiempo porque, por término medio, han de moverse la mitad de los registros del fichero para hacer espacio al nuevo registro. Esto significa que deben leerse y reescribirse la mitad de los bloques del fichero después de

haber movido los bloques entre sí. En la eliminación de un registro, el problema es menos grave si se utilizan marcadores de borrado y una reorganización periódica.

Una opción para que la inserción sea más eficaz es reservar algo de espacio sin usar en cada bloque de cara a los registros nuevos. Sin embargo, una vez que se agota este espacio, resurge el problema original. Otro método que se utiliza con frecuencia es crear un fichero temporal desordenado denominado **fichero de desbordamiento** (*overflow*) o de **transacciones**. Con esta técnica, el fichero ordenado actual se denomina **fichero principal** o **maestro**. Los registros nuevos se insertan al final del fichero de desbordamiento, en lugar de hacerlo en su posición correcta en el fichero principal. Periódicamente, el fichero de desbordamiento se ordena y mezcla con el maestro durante la reorganización del fichero. La inserción es muy eficaz, pero a costa de aumentar la complejidad del algoritmo de búsqueda. La búsqueda en el fichero de desbordamiento debe hacerse con una búsqueda lineal si, después de la búsqueda binaria, no se encuentra el registro en el fichero principal. En las aplicaciones que no requieren la mayoría de la información actualizada pueden ignorarse los registros de desbordamiento durante una búsqueda.

La modificación del valor de un campo depende de dos factores: la condición de búsqueda para localizar el registro y el campo que se va a modificar. Si la condición de búsqueda involucra al campo clave de ordenación, podemos localizar el registro utilizando una búsqueda binaria; en caso contrario, debemos hacer una búsqueda lineal. Un campo no ordenado se puede modificar cambiando el registro y reescribiendo en la misma ubicación física del disco (si los registros son de longitud fija). La modificación del campo de ordenación significa que el registro puede cambiar su posición en el fichero. Esto requiere la eliminación del registro antiguo, seguida por la inserción del registro modificado.

La lectura de los registros del fichero en el orden especificado por el campo de ordenación es bastante eficaz si ignoramos los registros desbordados, puesto que con el doble búfer podemos leer los bloques consecutivamente. Para incluir los registros desbordados, debemos mezclarlos en sus posiciones correctas; en este caso, primero tenemos que reorganizar el fichero y después leer sus bloques secuencialmente. Para reorganizar el fichero, primero ordenamos los registros del fichero de desbordamiento, y después los mezclamos con el fichero maestro. Los registros marcados para su borrado se eliminan durante la reorganización.

La Tabla 13.2 resume el tiempo de acceso medio a los bloques para encontrar un registro específico en un fichero que tiene b bloques.

Los ficheros ordenados raramente se utilizan en las aplicaciones de bases de datos, a menos que se utilice una ruta de acceso adicional, denominada **índice principal**; el resultado es un **fichero indexado-secuencial**. Esto mejora el tiempo de acceso aleatorio al campo clave de ordenación. En el Capítulo 14 explicamos los índices. Si el atributo de ordenación no es una clave, el fichero se denomina **fichero agrupado**.

13.8 Técnicas de dispersión

Otro tipo de organización de ficheros está basado en la dispersión, que proporciona un acceso muy rápido a los registros bajo ciertas condiciones de búsqueda. Esta organización se denomina normalmente **fichero disperso** o fichero *hash*.⁹ La condición de búsqueda debe ser una condición de igualdad sobre un solo campo, denominado **campo de dispersión** o campo *hash*. En la mayoría de los casos, el campo de dispersión también es un campo clave del fichero, en cuyo caso se denomina **clave de dispersión** (o clave *hash*). La idea que hay detrás de la dispersión es proporcionar una función h , denominada **función de dispersión** (o *hash*) o **función de aleatorización**, que se aplica al valor del campo de dispersión de un registro y produce la *dirección* del bloque de disco en el que está almacenado el registro. Una búsqueda del registro dentro del bloque puede llevarse a cabo en un búfer de la memoria principal. En la mayoría de los registros, sólo necesitamos acceder a un bloque para recuperar el registro.

⁹ Un fichero disperso también recibe el nombre de *fichero directo*.

Tabla 13.2. Tiempos de acceso medio para un fichero de b bloques bajo organizaciones de fichero básicas.

Tipo de organización	Método de acceso/búsqueda	Bloques promedio para acceder a un registro concreto
<i>Heap</i> (desordenado)	Exploración secuencial (búsqueda lineal)	$b/2$
Ordenado	Exploración secuencial	$b/2$
Ordenado	Búsqueda binaria	$\log_2 b$

La dispersión también se utiliza como una estructura de búsqueda interna dentro de un programa, siempre que se acceda a un grupo de registros exclusivamente utilizando el valor de un campo. En la Sección 13.8.1 describimos el uso de la dispersión para los ficheros internos; después, en la Sección 13.8.2, mostramos cómo se modifica para almacenar ficheros externos en disco. En la Sección 13.8.3 explicamos las técnicas para extender la dispersión a los ficheros de crecimiento dinámico.

13.8.1 Dispersión interna

En los ficheros internos, la dispersión normalmente se implementa como una **tabla de dispersión** mediante el uso de un array de registros. Vamos a suponer que el índice del array va de 0 a $M - 1$ (véase la Figura 13.8[a]); por tanto, tenemos M **slots** cuyas direcciones corresponden a los índices del array. Elegimos una función de dispersión que transforma el valor del campo de dispersión en un entero entre 0 y $M - 1$. Una función de dispersión común es la función $h(K) = K \bmod M$, que devuelve el resto del valor entero de un campo de dispersión K después de dividirlo por M ; este valor se utiliza después para la dirección del registro.

Los valores no enteros de un campo de dispersión se pueden transformar en enteros antes de aplicar la función mod. En las cadenas de caracteres, podemos utilizar en la transformación los códigos numéricos (ASCII) asociados a los caracteres (por ejemplo, multiplicar estos códigos). Para un campo de dispersión cuyo tipo de datos es una cadena de 20 caracteres, es posible utilizar el Algoritmo 13.2(a) para calcular la dirección de dispersión. Asumimos que la función devuelve el código numérico de un carácter y que tenemos un valor de campo de dispersión K de tipo K : *array* [1..20] *of char* (en PASCAL) o *char* $K[20]$ (en C).

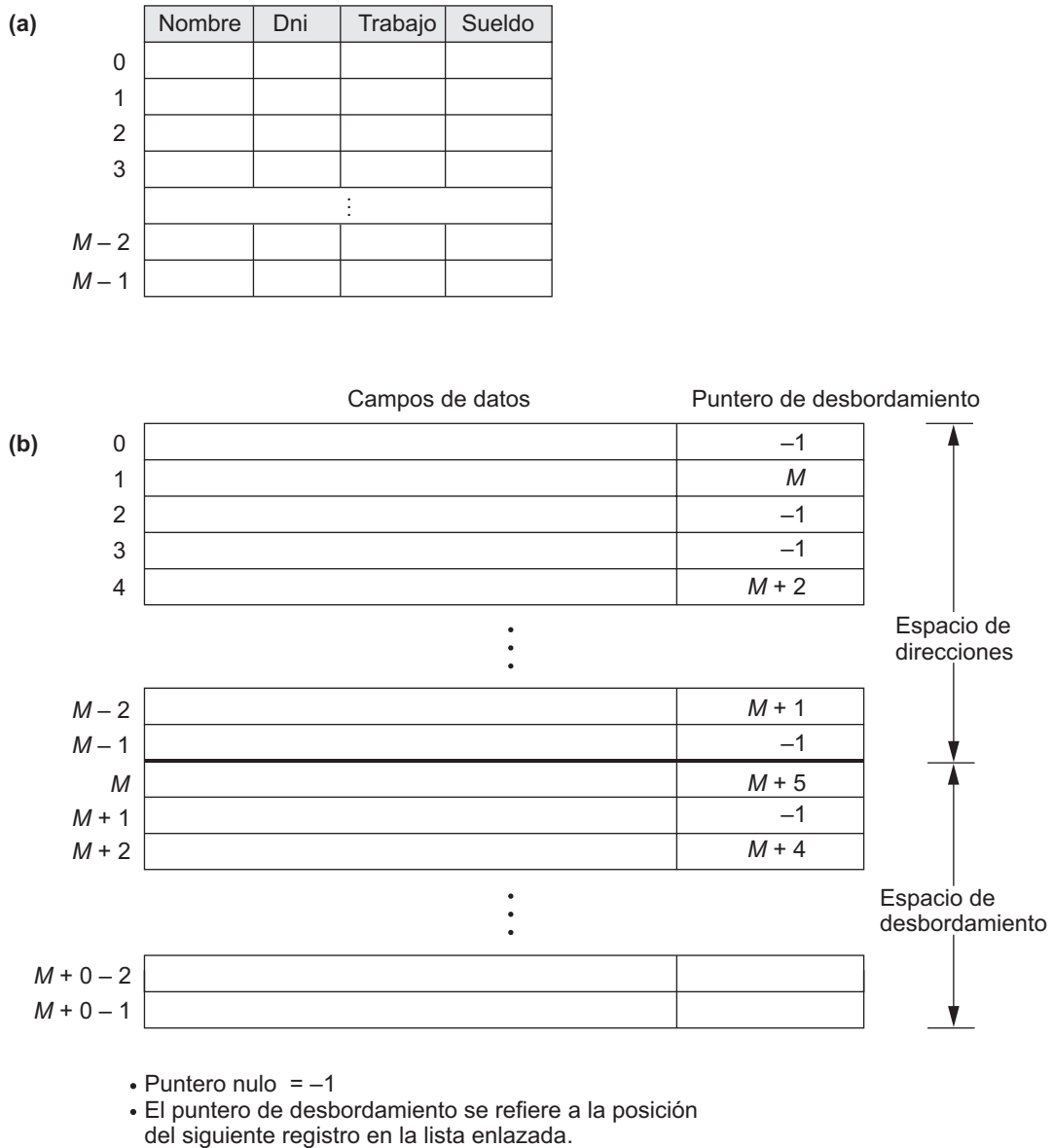
Algoritmo 13.2. Dos algoritmos de dispersión simples. (a) Aplicación de la función de dispersión mod a una cadena de caracteres K . (b) Resolución de la colisión por direccionamiento abierto.

```

(a) temp  $\leftarrow 1$ ;
    for  $i \leftarrow 1$  to 20 do temp  $\leftarrow$  temp * code( $K[i]$ ) mod  $M$ ;
    dirección_dispersión  $\leftarrow$  temp mod  $M$ ;
(b)  $i \leftarrow$  dirección_dispersión( $K$ );  $a \leftarrow i$ ;
    if ubicación  $i$  está ocupada
    then begin  $i \leftarrow (i + 1) \bmod M$ ;
        while ( $i \neq a$ ) y la ubicación  $i$  esté ocupada
        do  $i \leftarrow (i + 1) \bmod M$ ;
        if ( $i = a$ ) then todas las posiciones están llenas
        else nueva_dirección_dispersión  $\leftarrow i$ ;
    end;
```

También se pueden utilizar otras funciones de dispersión. Una técnica, denominada **reversible**, implica la aplicación de una función aritmética como una *suma* o una función lógica como un *or exclusivo* a diferentes porciones del valor del campo de dispersión para calcular la dirección de dispersión. Otra técnica implica

Figura 13.8. Estructuras de datos de dispersión interna. (a) Array de M posiciones para su uso en la dispersión interna. (b) Resolución de la colisión mediante encadenación de registros.



tomar algunos dígitos del valor del campo de dispersión (por ejemplo, los dígitos tercero, quinto y octavo) para formar la dirección de dispersión.¹⁰ El problema con la mayoría de las funciones de dispersión es que no garantizan que valores distintos se dispersen a direcciones distintas, porque el **espacio del campo de dispersión** (el número de valores posibles que un campo de dispersión puede tomar) es normalmente mucho más grande que el **espacio de direcciones** (el número de direcciones disponibles para los registros). La función de dispersión mapea el espacio del campo de dispersión al espacio de direcciones.

¹⁰ Una explicación detallada de las funciones de dispersión queda fuera del objetivo de nuestra presentación.

Una **colisión** se produce cuando el valor del campo de dispersión de un registro que se está insertando se dispersa a una dirección que ya contiene un registro diferente. En esta situación, debemos insertar el registro nuevo en alguna otra posición, puesto que su dirección de dispersión está ocupada. El proceso de encontrar otra posición se denomina **resolución de colisiones**. Hay varios métodos para resolver una colisión:

- **Direccionamiento abierto.** A partir de la posición ocupada especificada por la dirección de dispersión, el programa comprueba las posiciones subsiguientes en orden hasta encontrar una posición sin utilizar (vacía). El Algoritmo 13.2(b) puede utilizarse con este propósito.
- **Encadenamiento.** Para este método, se conservan varias ubicaciones de dispersión, normalmente extendiendo el array con algunas posiciones de desbordamiento. Adicionalmente, se añade un campo puntero a cada ubicación de registro. Una colisión se resuelve colocando el registro nuevo en una ubicación de desbordamiento sin utilizar y estableciendo el puntero de la ubicación de la dirección de dispersión ocupada a la dirección de esa ubicación de desbordamiento. Se conserva entonces una lista enlazada de registros de desbordamiento por cada dirección de dispersión, como se muestra en la Figura 13.8(b).
- **Dispersión múltiple.** El programa aplica una segunda función de dispersión si la primera desemboca en una colisión. Si se produce otra colisión, el programa utiliza el desbordamiento abierto o aplica una tercera función de dispersión y utiliza después el direccionamiento abierto si es necesario.

Todo método de resolución de colisiones requiere sus propios algoritmos para insertar, recuperar y borrar registros. Los algoritmos de encadenamiento son los más sencillos. Los de borrado para el direccionamiento abierto son bastante difíciles. Los libros sobre estructuras de datos explican más en profundidad los algoritmos de dispersión interna.

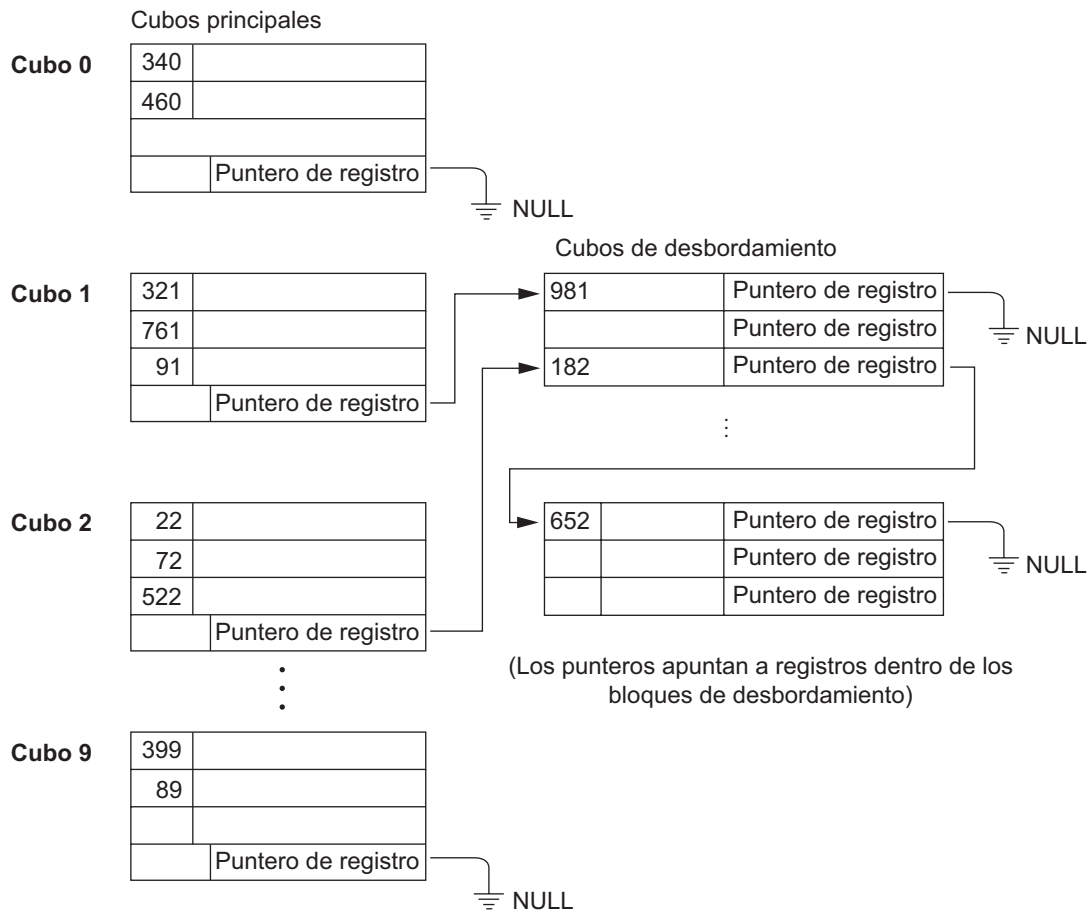
El objetivo de una buena función de dispersión es distribuir uniformemente los registros sobre el espacio de direcciones para minimizar las colisiones sin dejar demasiadas ubicaciones sin utilizar. Los estudios de simulación y análisis han mostrado que normalmente es mejor mantener una tabla de dispersión entre el 70 y el 90 por ciento llena, para que el número de colisiones sea bajo y no tengamos que malgastar demasiado espacio. Por tanto, si esperamos almacenar r registros en la tabla, debemos elegir M ubicaciones para el espacio de direcciones de tal modo que (r/M) esté entre 0,7 y 0,9. También puede ser útil elegir un número primo para M , ya que se ha demostrado que esto distribuye mejor las direcciones de dispersión sobre el espacio de direcciones cuando se utiliza la función de dispersión mod. Otras funciones de dispersión pueden requerir que M sea una potencia de 2.

13.8.2 Dispersión externa para los ficheros de disco

La dispersión para los ficheros de disco se denomina **dispersión externa**. Para ajustar las características del almacenamiento en disco, el espacio de direcciones de destino se compone de **cubos**, cada uno de los cuales almacena varios registros. Un cubo puede ser un bloque de disco o un grupo de bloques contiguos. La función de dispersión mapea una clave a un número de cubo relativo, en lugar de asignar una dirección de bloque absoluta al cubo. Una tabla almacenada en la cabecera del fichero convierte el número de cubo en la correspondiente dirección de bloque del disco (véase la Figura 13.9).

El problema de las colisiones es menos grave con los cubos, porque cuantos más registros encajen en un cubo más posible será que se dispersen al mismo cubo sin causar problemas. Sin embargo, debemos prever que el cubo puede estar lleno y que un registro nuevo se esté dispersando a ese cubo. Podemos utilizar una variación del encadenamiento según la cual se mantiene un puntero en cada cubo apuntando a una lista enlazada de registros de desbordamiento para el cubo (véase la Figura 13.10). Los punteros de la lista enlazada deben ser **punteros de registro**, que incluyen tanto una dirección de bloque como una posición relativa de registro dentro de bloque.

Figura 13.10. Manipulación del desbordamiento de cubos por encadenamiento.



igualdad en el campo de dispersión, podemos localizar el registro eficazmente utilizando la función de dispersión; en caso contrario, debemos realizar una búsqueda lineal. Un campo que no es de dispersión se puede modificar cambiando el registro y reescribiéndolo en el mismo cubo. La modificación del campo de dispersión significa que el registro se puede mover a otro cubo, lo que requiere la eliminación del registro antiguo seguida de la inserción del registro modificado.

13.8.3 Técnicas de dispersión que permiten la expansión dinámica del fichero

Un importante inconveniente del esquema de dispersión *estática* que acabamos de ver es que el espacio de direcciones de dispersión es fijo. Por tanto, resulta difícil expandir o contraer dinámicamente el fichero. Los esquemas descritos en esta sección intentan remediar esta situación. El primer esquema (la dispersión extensible) almacena una estructura de acceso además del fichero, por lo que se parece a la indexación (consulte el Capítulo 14). La principal diferencia es que la estructura de acceso está basada en los valores resultantes después de aplicar la función de dispersión al campo de búsqueda. En la indexación, la estructura de acceso está basada en los valores del propio campo de búsqueda. La segunda técnica, denominada dispersión lineal, no requiere estructuras de acceso adicionales.

Estos esquemas de dispersión tienen la ventaja de que el resultado de aplicar una función de dispersión es un entero positivo y que puede representarse como un número binario. La estructura de acceso se fundamenta en la **representación binaria** del resultado de la función de dispersión, que es un cadena de **bits**; lo llamamos **valor de dispersión** de un registro. Los registros se distribuyen entre cubos basándose en los valores de los *primeros bits* de sus valores de dispersión.

Dispersión extensible. En esta dispersión, se conserva un tipo de directorio (un array de 2^d direcciones de cubos), donde d se conoce como **profundidad global** del directorio. El valor entero correspondiente a los d primeros (orden superior) bits de un valor de dispersión se utiliza como índice del array para determinar una entrada del directorio, y la dirección de esa entrada determina el cubo en el que se almacenan los registros correspondientes.

Sin embargo, no hay que tener un cubo distinto para cada una de las 2^d ubicaciones de directorio. Varias ubicaciones de directorio con los mismos d' primeros bits para sus valores de dispersión pueden contener la misma dirección de cubo si todos los registros que se dispersan a estas ubicaciones encajan en un solo cubo. Una **profundidad local d'** (almacenada con cada cubo) especifica el número de bits en los que está basado el contenido del cubo. La Figura 13.13 muestra un directorio con una profundidad local $d = 3$.

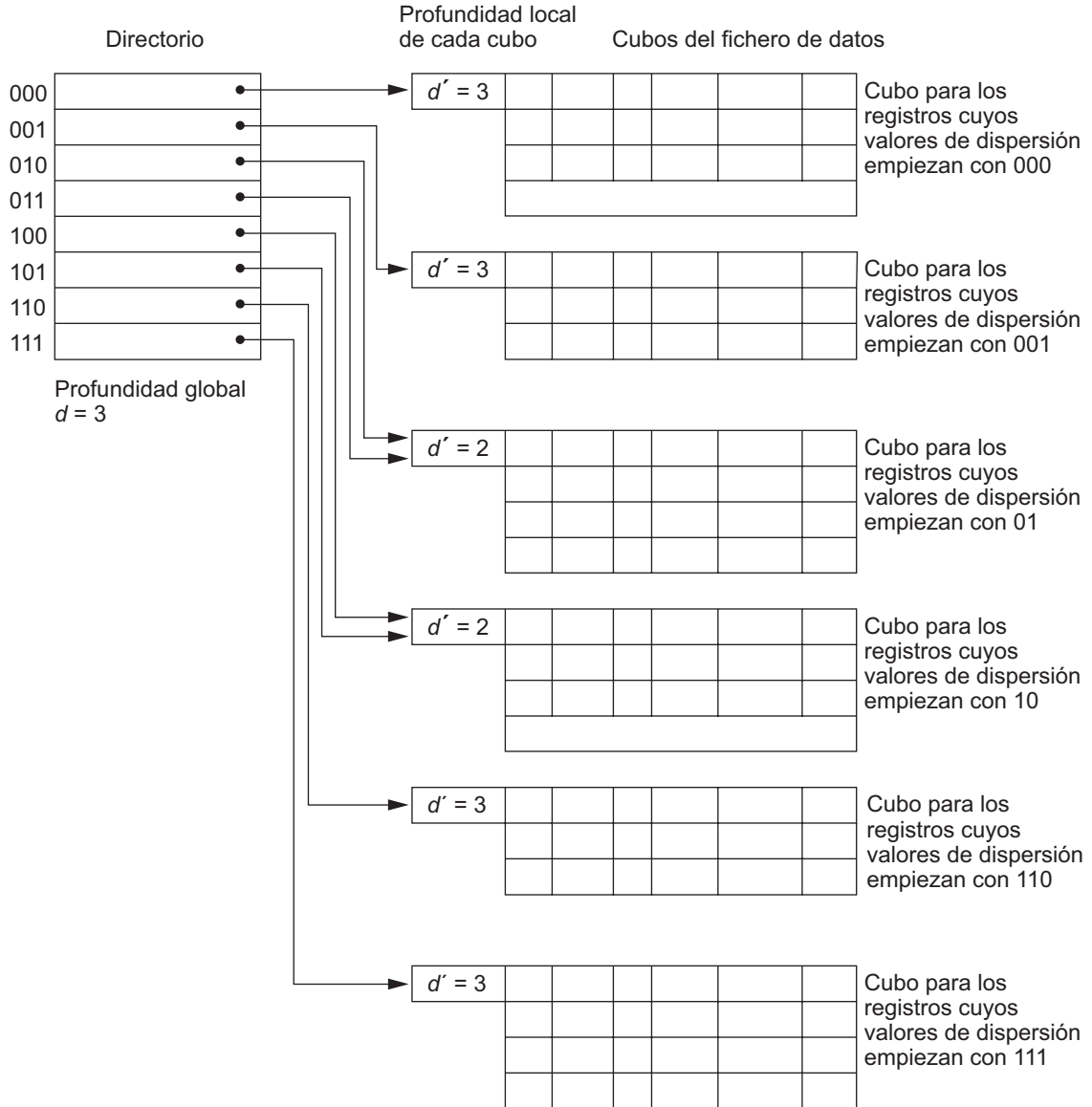
El valor de d se puede incrementar o reducir de uno en uno, doblando o reduciendo a la mitad de este modo el número de entradas del array de directorio. La opción de la duplicación se necesita si un cubo, cuya profundidad local d' es igual a la profundidad global d , desborda. La reducción a la mitad se da si $d > d'$ para todos los cubos después de producirse algunos borrados. La mayoría de las recuperaciones de registros requieren dos accesos de bloque: uno al directorio y otro al cubo.

Para ilustrar la división del cubo, suponga que la inserción de un registro nuevo provoca el desbordamiento en el cubo cuyos valores de dispersión empiezan con 01 (el tercer cubo de la Figura 13.13). Los registros se distribuirán entre dos cubos: el primero contiene todos los registros cuyos valores de dispersión empiezan por 010, y el segundo todos los que sus valores de dispersión empiezan con 011. Ahora, las dos ubicaciones de directorio para 010 y 011 apuntan a los dos cubos nuevos y distintos. Antes de la división, apuntaban al mismo cubo. La profundidad local d' de los dos cubos nuevos es 3, uno más que la profundidad local del cubo antiguo.

Si un cubo que se desborda y se divide tenía una profundidad local d' igual a la profundidad global d del directorio, entonces el tamaño de este último debe duplicarse ahora para que podamos utilizar un bit adicional a fin de distinguir los dos cubos nuevos. Por ejemplo, si el cubo de los registros cuyos valores de dispersión empiezan con 111 de la Figura 13.11 se desborda, los dos cubos nuevos necesitan un directorio con una profundidad global de $d = 4$, porque los dos cubos aparecen ahora etiquetados como 1110 y 1111, y por tanto sus profundidades locales son 4. El tamaño del directorio se ha doblado, y cada una de las ubicaciones originales del directorio también se ha dividido en dos ubicaciones, y ambas tienen el mismo valor de puntero que la ubicación original.

La principal ventaja de la dispersión extensible es que el rendimiento del fichero no se degrada al crecer, al contrario de lo que ocurre con la dispersión externa estática, con la que aumentan las colisiones y el encadenamiento correspondiente provoca accesos adicionales. Además, en la dispersión extensible no se asigna espacio para un crecimiento futuro, sino que se asignan dinámicamente cubos adicionales a medida que se necesitan. La sobrecarga de espacio para la tabla de directorio es insignificante. El tamaño máximo del directorio es 2^k , donde k es el número de bits del valor de dispersión. Otra ventaja es que la división provoca una menor reorganización en la mayoría de los casos, por lo que únicamente los registros de un cubo se redistribuyen a los dos cubos nuevos. La reorganización más costosa en cuanto a tiempo es la de duplicar el directorio (o dividirlo en dos). Un inconveniente es que antes de acceder a los propios cubos, debe buscarse el directorio, lo que da lugar a dos accesos de bloque en lugar de uno, como ocurre en la dispersión estática. Esta penalización en el rendimiento se considera menor y, por tanto, se considera que este esquema es muy recomendable para los ficheros dinámicos.

Figura 13.11. Estructura del esquema de dispersión extensible.



Dispersión lineal. La idea que hay detrás de la dispersión lineal es permitir el aumento y la reducción dinámica del número de cubos de un fichero de dispersión *sin* necesidad de un directorio. Supongamos que el fichero empieza con M cubos numerados como $0, 1, \dots, M-1$ y utiliza la función de dispersión $\text{mod } h(K) = K \text{ mod } M$; esta función de dispersión se denomina función de dispersión lineal h_i . Todavía se necesita el desbordamiento debido a las colisiones y puede manipularse manteniendo cadenas de desbordamiento individuales para cada cubo. Sin embargo, cuando una colisión lleva a un registro desbordado en *cualquier* cubo del fichero, el *primer* cubo del fichero (el cubo 0) se divide en dos cubos: el cubo 0 original y un nuevo cubo M al final del fichero. Los registros que se encontraban originalmente en el cubo 0 se distribuyen entre los dos cubos basándose en una función de dispersión diferente, $h_{i+1}(K) = K \text{ mod } 2M$. Una propiedad importante de las dos

funciones de dispersión h_i y h_{i+1} es que cualquier registro que se disperse al cubo 0 basándose en h_i se dispersará al cubo 0 o al cubo M basándose en h_{i+1} ; esto es necesario para que funcione la dispersión lineal.

Cuando colisiones posteriores llevan al desbordamiento de registros, se dividen cubos adicionales en orden *lineal*, 1, 2, 3,... Si se producen bastantes desbordamientos, todos los cubos del fichero original, 0, 1,..., $M-1$, se habrán dividido, por lo que el fichero tendrá ahora $2M$ cubos en lugar de M , y todos los cubos utilizarán la función de dispersión h_{i+1} . Por tanto, los registros desbordados se redistribuyen en cubos normales, utilizando la función h_{i+1} a través de una *división retardada* de sus cubos. No hay ningún directorio; sólo se necesita un valor n (que inicialmente se establece a 0 y se incrementa en una unidad siempre que se produce una división) para determinar los cubos que se han dividido. Para recuperar un registro con el valor de clave de dispersión K , primero aplicamos la función h_i a K ; si $h_i(K) < n$, entonces aplicamos la función h_{i+1} sobre K porque el cubo ya está dividido. Inicialmente, $n = 0$, lo que indica que la función h_i se aplica a todos los cubos; n aumenta linealmente a medida que se dividen los cubos.

Cuando $n = M$ después de haberse incrementado, significa que todos los cubos originales se han dividido y que la función de dispersión h_{i+1} se ha aplicado a todos los registros del fichero. En este punto, n se reinicia a 0 (cero), y cualquier colisión nueva que provoque un desbordamiento conduce al uso de una nueva función de dispersión, $h_{i+2}(K) = K \bmod 4M$. En general, se utiliza una secuencia de funciones de dispersión $h_{i+j}(K) = K \bmod (2^j M)$, donde $j = 0, 1, 2, \dots$; se necesita una nueva función de dispersión h_{i+j+1} siempre que todos los cubos 0, 1,..., $(2^j M)-1$ se hayan dividido y n se haya reiniciado a 0. El Algoritmo 13.3 muestra la búsqueda de un registro con un valor de clave de dispersión de K .

La división puede controlarse monitorizando el factor de carga del fichero, en lugar de aplicar una división siempre que se produzca un desbordamiento. En general, el **factor de carga del fichero** l se puede definir como $l = r/(bfr * N)$, donde r es el número actual de registros del fichero, bfr es el número máximo de registros que pueden encajar en un cubo, y N es el número actual de cubos del fichero. Los cubos que se han dividido también se pueden recombinar si el factor de carga del fichero cae por debajo de un determinado umbral. Los bloques se combinan linealmente y N se reduce en consecuencia. La carga del fichero se puede utilizar para lanzar tanto divisiones como combinaciones; de este modo, la carga del fichero puede mantenerse dentro del rango deseado. Las divisiones se pueden lanzar cuando la carga excede un determinado umbral (por ejemplo, 0,9) y las combinaciones se pueden lanzar cuando la carga cae por debajo de otro umbral (por ejemplo, 0,7).

Algoritmo 13.3. Procedimiento de búsqueda para la dispersión lineal.

```

if  $n = 0$ 
  then  $m \leftarrow h_j(K)$  (*  $m$  es el valor de dispersión del registro con la clave de dispersión  $K$  *)
else begin
   $m \leftarrow h_j(K)$ ;
  if  $m < n$  then  $m \leftarrow h_{j+1}(K)$ 
end;
  buscar el cubo cuyo valor de dispersión es  $m$  (y su desbordamiento, si lo hay);

```

13.9 Otras organizaciones principales de ficheros

13.9.1 Ficheros de registros mezclados

Las organizaciones de ficheros que hemos estudiado hasta ahora asumen que todos los registros de un fichero en particular son del mismo tipo de registro. Los registros pueden ser de EMPLEADOS, PROYECTOS, ESTUDIANTES o DEPARTAMENTOS, pero cada fichero contiene registros de un solo tipo. En la mayoría de las aplicaciones de bases de datos nos encontramos con situaciones en las que varios tipos de entidades se

interrelacionan de distintas formas, como vimos en el Capítulo 3. Las relaciones entre registros de varios ficheros se pueden representar mediante **campos de conexión**.¹¹ Por ejemplo, un registro ESTUDIANTE puede tener un campo de conexión DptoEspecialidad cuyo valor proporciona el nombre del DEPARTAMENTO en el que el estudiante se está especializando. Este campo DptoEspecialidad se *refiere* a una entidad DEPARTAMENTO, que debe estar representada por un registro propio en el fichero DEPARTAMENTO. Si queremos recuperar los valores de campo de dos registros relacionados, primero debemos recuperar uno de los registros. Después, podemos utilizar el valor de su campo de conexión para recuperar el registro relacionado del otro fichero. Por tanto, las relaciones se implementan mediante **referencias de campo lógicas** entre los registros de ficheros distintos.

Las organizaciones de ficheros en los DBMSs de objetos, así como los sistemas heredados (por ejemplo, los DBMSs jerárquicos y de red), a menudo implementan las relaciones entre los registros como **relaciones físicas** materializadas por la contigüidad física (o agrupamiento) de los registros relacionados o por los punteros físicos. Estas organizaciones de fichero normalmente asignan un **área** del disco para albergar registros de más de un tipo, para que esos registros de tipos distintos puedan **agruparse físicamente** en el disco. Si se espera utilizar con frecuencia una relación particular, la implementación de una relación física puede aumentar la eficacia del sistema al recuperar registros relacionados. Por ejemplo, si es frecuente la consulta para recuperar un registro DEPARTAMENTO y los registros de todas las especialidades de los ESTUDIANTES de ese departamento, sería deseable colocar cada registro de DEPARTAMENTO y su grupo de registros de ESTUDIANTE contiguamente en disco en un fichero mixto. En los DBMSs de objetos se utiliza el concepto de **agrupamiento físico** de tipos de objetos para almacenar juntos los objetos relacionados en un fichero mixto.

Para distinguir los registros de un fichero mixto, cada registro tiene (además de los valores de sus campos) un campo con el **tipo de registro**. Normalmente es el primer campo de cada registro y lo utiliza el software del sistema para determinar el tipo del registro que está a punto de procesar. Con la información del catálogo, el DBMS puede determinar los campos de ese tipo de registro y sus tamaños, a fin de interpretar los valores del registro.

13.9.2 Árboles B y otras estructuras de datos como organización primaria

Se pueden utilizar otras estructuras de datos como organizaciones primarias de ficheros. Por ejemplo, si tanto el tamaño del registro como el número de registros de un fichero son pequeños, algunos DBMSs ofrecen la opción de una estructura de árbol B como organización primaria de fichero. En la Sección 14.3.1 describiremos los árboles B cuando expliquemos el uso de la estructura de datos de árbol B para la indexación. En general, cualquier estructura de datos que pueda adaptarse a las características de los dispositivos de disco puede utilizarse como organización primaria de fichero para la ubicación de registros en el disco.

13.10 Paralelismo del acceso al disco mediante la tecnología RAID

Con el crecimiento exponencial del rendimiento y la capacidad de los dispositivos semiconductores y de las memorias, aparecen continuamente microprocesadores más rápidos con memorias primarias más y más grandes. Para igualar este crecimiento, es natural esperar que la tecnología del almacenamiento secundario también avance a la par que la tecnología de los procesadores en cuanto a rendimiento y fiabilidad.

Una importante ventaja de la tecnología de almacenamiento secundario está representada por el desarrollo de **RAID**, que originariamente significaba **Matriz redundante de discos baratos** (*Redundant Arrays of*

¹¹ El concepto de *foreign keys* o claves externas en el modelo relacional (Capítulo 5) y las referencias entre objetos en los modelos orientados a objetos (Capítulo 20) son ejemplos de campos de conexión.

Inexpensive Disks). Últimamente, la *I* de RAID ha cambiado su significado por el de “independientes”. La idea tras RAID recibió un respaldo muy positivo por parte de la industria y se ha convertido en un conjunto elaborado de arquitecturas RAID alternativas (RAID de niveles 0 a 6). Destacaremos las principales características de esta tecnología.

El objetivo principal de RAID es igualar las amplias diferencias de rendimiento y velocidad entre los discos y la memoria y los microprocesadores.¹² Mientras que la capacidad de la RAM se cuadruplica cada dos a tres años, los *tiempos de acceso* al disco se mejoran menos de un 10% por año, y las *velocidades de transferencia* al disco se mejoran apenas un 20% al año. Las *capacidades* de los discos están mejorando por encima del 50% anual, pero las mejoras en la velocidad y el tiempo de acceso son de una magnitud mucho más pequeña. La Tabla 13.3 compara la tecnología de los discos con una diferencia de diez años, así como las tasas de mejora.

Existe una segunda disparidad cualitativa entre la capacidad de los microprocesadores especiales que satisfacen las necesidades de las aplicaciones que deben procesar vídeo, audio, imagen y datos especiales (consulte los Capítulos 24 y 29 si desea más información sobre estas aplicaciones), con la correspondiente carencia de un acceso rápido a los conjuntos más grandes de datos compartidos.

La solución natural es una matriz grande de pequeños discos independientes que actúan como un solo disco lógico de alto rendimiento. Se utiliza un concepto denominado **segmentación (*striping*) de datos**, que utiliza el *paralelismo* para mejorar el rendimiento del disco. La segmentación de datos distribuye los datos de forma transparente por varios discos para hacerlos aparecer como un solo disco más grande y rápido. La Figura 13.12 muestra un fichero segmentado o distribuido por cuatro discos. La segmentación mejora el rendimiento de E/S global al permitir varias operaciones de E/S en paralelo, lo que proporciona unas altas velocidades de transferencia global. La segmentación de los datos también acomete el equilibrado de la carga entre los discos. Además, al almacenar información redundante en los discos mediante la paridad o algún otro código de corrección de errores se puede mejorar la fiabilidad. En las Secciones 13.3.1 y 13.3.2, explicamos cómo RAID logra los importantes objetivos de mejorar la fiabilidad y el rendimiento. La Sección 13.3.3 explica las organizaciones RAID.

Tabla 13.3. Tendencias en la tecnología del disco.

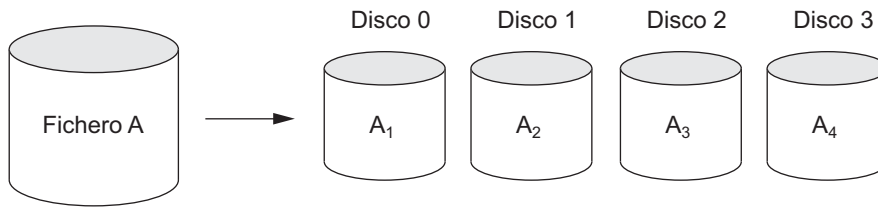
	Valores en 1993*	Tasa histórica de mejora anual (%)*	Valores en 2003**
Densidad superficial	50–150 Mb/pulgada cuadrada	27	36 Gb/pulgada cuadrada
Densidad lineal	40.000–60.000 bits/pulgada	13	570 Kb/pulgada
Densidad entre pistas	1.500–3.000 pistas/pulgada	10	64,000 pistas/pulgada
Capacidad (factor de forma de 3,5 pulgadas)	100–2.000 MB	27	146 GB
Velocidad de transferencia	3–4 MB/s	22	43–78 MB/seg
Tiempo de búsqueda	7–20 ms	8	3,5–6 ms

*Fuente: De Chen, Lee, Gibson, Katz y Patterson (1994), *ACM Computing Surveys*, Vol. 26, No. 2 (junio de 1994).

Reproducido con permiso.

**Fuente: Unidades de disco duro IBM Ultrastar 36XP y 18ZX.

¹² Gordon Bell predijo que sería aproximadamente el 40 por ciento por año entre 1974 y 1984, y hoy se supone que excede el 50 por ciento anual.

Figura 13.12. Segmentación de datos. El Fichero A está dividido en cuatro discos.

13.10.1 Mejora de la fiabilidad con RAID

Para un array de n discos, la probabilidad de un fallo es n veces superior que para un solo disco. Por tanto, si asumimos que el MTTF (tiempo medio entre fallos) de una unidad de disco es de 200.000 horas o aproximadamente 22,8 años (los tiempos normales llegan al millón de horas), el de un banco de 100 unidades de disco es sólo de 2.000 horas u 83,3 días. Guardar una sola copia de los datos en una matriz semejante de discos provocará una significativa pérdida de fiabilidad. Una solución obvia es emplear la redundancia de datos a fin de poder tolerar los fallos de disco. Los inconvenientes son muchos: operaciones de E/S adicionales para escribir, cálculos extra para mantener la redundancia y recuperarse de los errores, y capacidad de disco adicional para almacenar la información redundante.

Una técnica para introducir la redundancia se denomina **espejo** o **reflejo** (*mirroring*) o **shadowing**. Los datos se escriben redundantemente en dos discos físicos idénticos que se tratan como un solo disco lógico. Cuando se leen los datos, se pueden recuperar del disco con retardos de cola, búsqueda y rotación más cortos. Si falla un disco, se utiliza el otro hasta que el primero es reparado. Supongamos que el tiempo medio de reparación es de 24 horas, entonces el tiempo medio de que se pierdan datos en un sistema de disco en espejo que utiliza 100 discos con un MTTF de 200.000 horas cada uno es de $(200.000)^2 / (2 * 24) = 8,33 * 10^8$ horas, que son 95.028 años.¹³ Los discos en espejo también duplican la velocidad a la que se manipulan las solicitudes de lectura, puesto que la lectura puede ir a cualquier disco. Sin embargo, la velocidad de transferencia de cada lectura sigue siendo la misma que para un solo disco.

Otra solución al problema de la fiabilidad es almacenar información extra que normalmente no es necesaria, pero que puede utilizarse para reconstruir la información perdida en caso de un fallo en el disco. La incorporación de la redundancia debe tener en consideración dos problemas: seleccionar una técnica para calcular la información redundante, y seleccionar un método de distribución de la información redundante por la matriz de discos. El primer problema se resuelve utilizando códigos de corrección de errores que implican el uso de bits de paridad, o códigos especializados como los códigos Hamming. Si optamos por el esquema de paridad, podemos tener un disco redundante con la suma de todos los datos de los otros discos. Cuando un disco falla, la información perdida puede reconstruirse con un proceso parecido a la substracción.

Para resolver el segundo problema, los dos métodos principales consisten en almacenar información redundante en un pequeño número de discos o distribuirla uniformemente por todos los discos. Este último método ofrece un mejor equilibrado de la carga. Los diferentes niveles de RAID eligen una combinación de estas opciones para implementar la redundancia y mejorar la fiabilidad.

13.10.2 Mejora del rendimiento con RAID

Las matrices de discos emplean la técnica de la segmentación de los datos para lograr unas velocidades de transferencia más altas. Los datos sólo se pueden leer o escribir de bloque en bloque, por lo que una transferencia típica contiene de 512 a 8.192 bytes. La segmentación del disco puede aplicarse con una granularidad

¹³ Las fórmulas para calcular el MTTF aparecen en Chen y otros (1994).

más final dividiendo un byte de datos en bits y diseminando los bits por diferentes discos. De este modo, la **segmentación de datos a nivel de bit** consiste en dividir un byte de datos y escribir el bit j en el disco número j . Con bytes de 8 bits, podemos considerar que ocho discos físicos son un disco lógico, con una velocidad de transferencia de datos aumentada ocho veces. Cada disco participa en cada solicitud de E/S y la cantidad total de datos leída por solicitud es casi ocho veces superior. La segmentación a nivel de bit puede generalizarse a un número de discos que sea múltiplo o factor de ocho. Así, en una matriz de cuatro discos, el bit n va al disco $(n \bmod 4)$.

La granularidad de la intercalación o interpolación de datos puede ser superior a un bit; por ejemplo, los bloques de un fichero se pueden segmentar por varios discos, dando lugar a la **segmentación a nivel de bloque**. La Figura 13.12 muestra la segmentación de datos a nivel de bloque asumiendo que el fichero de datos está contenido en cuatro bloques. Con esta segmentación, varias solicitudes independientes que acceden a bloques sencillos (solicitudes pequeñas) pueden ser servidas en paralelo por discos separados, reduciéndose de este modo el tiempo de espera en cola de las solicitudes de E/S. Las solicitudes que acceden a varios bloques (solicitudes grandes) se pueden hacer en paralelo, lo que reduce su tiempo de respuesta. En general, cuantos más discos haya en una matriz, mayor es la posibilidad de mejorar el rendimiento. Sin embargo, y asumiendo fallos independientes, una matriz de 100 discos tiene una fiabilidad colectiva que es $1/100$ de la fiabilidad de un solo disco. Por tanto, son necesarios la redundancia mediante códigos de corrección de errores y discos reflejados (en espejo) para mejorar la fiabilidad, junto con un rendimiento más alto.

13.10.3 Organizaciones y niveles de RAID

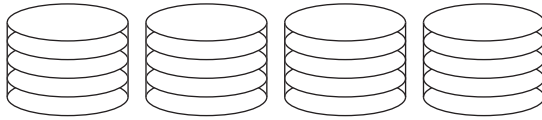
Las diferentes organizaciones de RAID se definieron basándose en diferentes combinaciones de los dos factores de granularidad del interpolado de datos (segmentación) y del patrón utilizado para calcular la información redundante. En la propuesta inicial aparecían 5 niveles de RAID (del 1 al 5), y más tarde se añadieron los niveles 0 y 6.

El nivel 0 de RAID utiliza la segmentación de datos, no tiene datos redundantes y, por tanto, ofrece un mejor rendimiento de escritura porque las actualizaciones no tienen que duplicarse. Sin embargo, su rendimiento en la lectura no es tan bueno como en el nivel 1 de RAID, que utiliza discos reflejados. En este último, es posible mejorar el rendimiento planificando una solicitud de lectura del disco con un retardo de búsqueda y rotacional más corto de lo esperado. El nivel 2 de RAID utiliza una redundancia al estilo de la memoria utilizando códigos Hamming, que contienen bits de paridad para distintos subconjuntos superpuestos de componentes. Por tanto, en una versión particular de este nivel, tres discos redundantes serían suficientes para cuatro discos originales, mientras que con los discos en espejo (como en el nivel 1) se necesitarían cuatro discos. El nivel 2 incluye la detección de errores y la corrección, aunque normalmente no se requiere la detección porque los discos dañados quedan identificados.

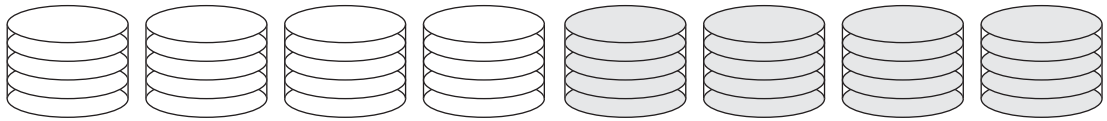
El nivel 3 de RAID utiliza un disco de paridad sencilla que cuenta con el controlador de disco para determinar el disco que ha fallado. Los niveles 4 y 5 utilizan la segmentación de datos a nivel de bloque; el nivel 5 distribuye los datos y la información de paridad por todos los discos. Por último, el nivel 6 de RAID aplica lo que se conoce como esquema de redundancia $P + Q$ que utiliza los códigos Reed-Soloman para protegerse contra el fallo de hasta dos discos, utilizando dos discos redundantes. En la Figura 13.13 se ilustran los siete niveles de RAID (0 a 6).

Con el nivel 1 de RAID la reconstrucción es sencilla en caso de que falle un disco. Los otros niveles necesitan leer varios discos para poder reconstruir un disco dañado. El nivel 1 se utiliza para aplicaciones críticas, como el almacenamiento de los registros de transacciones. Los niveles 3 y 5 son los preferidos para el almacenamiento de grandes volúmenes, mientras que el nivel 3 proporciona unas velocidades de transferencia más altas. Los niveles RAID más utilizados actualmente son el 0 (con segmentación), el 1 (con espejo) y el 5 con una unidad adicional para la paridad. Los diseñadores de una configuración RAID para una aplicación mixta dada deben confrontar muchas decisiones de diseño, como el nivel de RAID, el número de discos, la elección

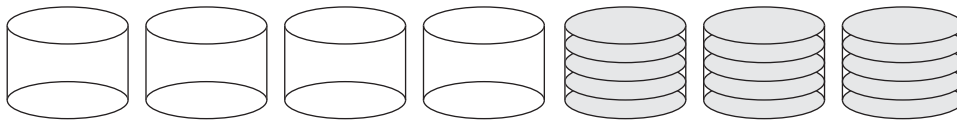
Figura 13.13. Niveles de RAID. Información extraída de Chen, Lee, Gibson, Katz y Patterson (1994), ACM Computing Survey, Vol. 26, No. 2 (junio de 1994). Reproducido con permiso.



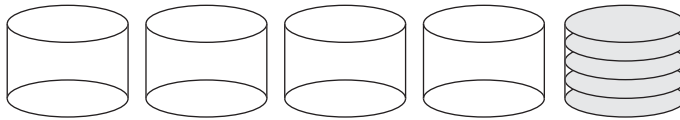
No redundante (RAID de nivel 0)



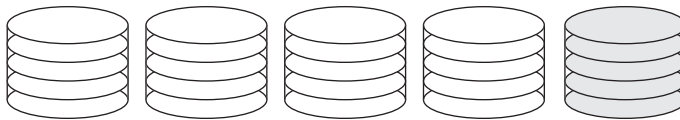
En espejo (RAID de nivel 1)



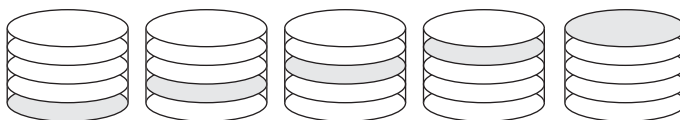
ECC de estilo memoria (RAID de nivel 2)



Paridad con interpolación de bits (RAID de nivel 3)



Paridad con interpolación de bloque (RAID de nivel 4)



Paridad distribuida con bloques interpolados (RAID de nivel 5)



Redundancia P+Q (RAID de nivel 6)

de los esquemas de paridad, y el agrupamiento de discos para la segmentación a nivel de bloque. Se han realizado estudios detallados del rendimiento sobre pequeñas cantidades de lecturas y escrituras (en referencia a solicitudes de E/S para una unidad de segmentación) y grandes cantidades de lecturas y escrituras

(en referencia a solicitudes de E/S para una unidad de segmentación de cada disco en un grupo de corrección de errores).

13.11 Nuevos sistemas de almacenamiento

En esta sección describimos dos desarrollos recientes en los sistemas de almacenamiento que se han convertido en parte integral de la mayoría de arquitecturas de los sistemas de información empresariales.

13.11.1 Redes de área de almacenamiento

Con el rápido crecimiento del comercio electrónico, los sistemas de Planificación de recursos empresariales (ERP, *Enterprise Resource Planning*) que integran datos de una aplicación a través de varias empresas, y almacenes de datos que almacenan información agregada histórica (consulte el Capítulo 27), la demanda de almacenamiento ha crecido sustancialmente. Las empresas actuales orientadas a Internet se han visto en la necesidad de moverse de un centro de datos fijo y estático orientado a las operaciones, a una infraestructura más flexible y dinámica para sus requisitos de procesamiento de información. El coste total de administración de todos los datos crece tan rápidamente que en muchos casos el coste de administrar el almacenamiento conectado a un servidor excede el coste del propio servidor. Además, el coste de adquisición del almacenamiento sólo es una pequeña fracción (normalmente, sólo del 10 al 15% del coste global de administración del almacenamiento). Muchos usuarios de sistemas RAID no pueden utilizar eficazmente la capacidad porque tienen que estar conectados de un modo fijo a uno o más servidores. Por consiguiente, las empresas grandes se están moviendo a un concepto denominado **Redes de área de almacenamiento (SAN, Storage Area Networks)**. En una SAN, los periféricos de almacenamiento online están configurados como nodos en una red de alta velocidad y se pueden conectar y desconectar de los servidores con mucha flexibilidad. Varias empresas han surgido como proveedores SAN y proporcionan sus propias topologías patentadas. Estas redes permiten que los sistemas de almacenamiento se coloquen a mayor distancia de los servidores y proporcionan diferentes opciones de rendimiento y conectividad. Las aplicaciones de administración del almacenamiento existentes se pueden portar a configuraciones de SAN mediante redes de canal de fibra (Fiber Channel) que encapsulan el protocolo SCSI heredado. En consecuencia, los dispositivos conectados a la SAN aparecen como dispositivos SCSI.

Las alternativas arquitectónicas actuales para SAN incluyen lo siguiente: conexiones punto a punto entre servidores y sistemas de almacenamiento a través de un canal de fibra, el uso de un *switch* de canal de fibra para conectar varios sistemas RAID, librerías de cinta, etcétera a los servidores, y el uso de *hubs* y *switches* de canal de fibra para conectar los servidores y los sistemas de almacenamiento en diferentes configuraciones. Las empresas pueden moverse lentamente de unas topologías más sencillas a otras más complejas añadiendo servidores y dispositivos de almacenamiento según lo necesiten. No ofreceremos aquí más detalles porque varían mucho de unos proveedores de SAN a otros. Las principales ventajas exigibles son las siguientes:

- Conectividad “varios a varios” flexible entre los servidores y los dispositivos de almacenamiento utilizando *hubs* y *switches* de canal de fibra.
- Hasta 10 kilómetros de separación entre un servidor y un sistema de almacenamiento utilizando los cables de fibra óptica apropiados.
- Buenas capacidades de aislamiento que permitan añadir sin complicaciones nuevos periféricos y servidores.

Las SANs están creciendo muy rápidamente, pero todavía se enfrentan a muchos problemas, como la combinación de opciones de almacenamiento de diferentes fabricantes y el desarrollo de estándares para el software y el hardware de administración del almacenamiento. La mayoría de las principales empresas están evaluando SAN como una opción viable para el almacenamiento de bases de datos.

13.11.2 Almacenamiento conectado a la red

Con el fenomenal crecimiento de los datos digitales, en particular los generados por las aplicaciones multimedia y otras aplicaciones empresariales, la necesidad de soluciones de almacenamiento de alto rendimiento y bajo coste se ha convertido en algo extremadamente importante. Los dispositivos **NAS (Almacenamiento conectado a la red, Network-Attached Storage)** se encuentran entre los últimos dispositivos de almacenamiento que se están utilizando con este propósito. Estos dispositivos son, de hecho, servidores que no proporcionan ninguno de los servicios de servidor comunes, sino que simplemente permiten la adición de almacenamiento para compartir ficheros. Los dispositivos NAS permiten añadir vastas cantidades de espacio de almacenamiento en disco duro a una red y hacen que ese espacio esté disponible para varios servidores sin tener que pararlos para realizar el mantenimiento y las actualizaciones. Los dispositivos NAS pueden residir en cualquier parte de una red de área local (LAN) y pueden combinarse en diferentes configuraciones. Un único dispositivo hardware, a menudo denominado **caja NAS** o **cabecera NAS**, actúa como interfaz entre el sistema NAS y los clientes de la red. Estos dispositivos NAS no requieren monitor, teclado o ratón. Es posible conectar una o más unidades de disco o cinta a varios sistemas NAS para aumentar la capacidad total. Los clientes se conectan a la cabecera NAS, en lugar de hacerlo a los dispositivos de almacenamiento individuales. Un NAS puede almacenar cualquier dato que aparezca en forma de ficheros, como buzones de correo electrónico, contenido web, copias de seguridad remotas del sistema, etcétera. En este sentido, los dispositivos NAS se están implantando como un sustituto de los servidores de ficheros tradicionales.

Los sistemas NAS se esfuerzan por gozar de un funcionamiento fiable y una administración sencilla. Incluyen características integradas como la autenticación segura o el envío automático de alertas por email en caso de un error en el dispositivo. Los dispositivos NAS se están ofreciendo con un alto grado de escalabilidad, fiabilidad, flexibilidad y rendimiento. Normalmente soportan los niveles 0, 1 y 5 de RAID. Las SANs tradicionales difieren de los NASs en varias cosas. En concreto, las SANs a menudo utilizan Fiber Channel en lugar de Ethernet, y una SAN a menudo incorpora varios dispositivos de red o *puntos finales* en una LAN autocontenida o *privada*, mientras que un NAS depende de dispositivos individuales conectados directamente a una LAN pública existente. Mientras que los servidores de ficheros Windows, UNIX y NetWare demandan cada uno el soporte de un protocolo específico en el lado del cliente, los sistemas NAS exigen una independencia mayor del sistema operativo por parte de los clientes.

13.12 Resumen

Empezamos este capítulo explicando las características de las jerarquías de la memoria y después nos centramos en los dispositivos de almacenamiento secundario. En particular, nos centramos en los discos magnéticos porque son los que más se utilizan para almacenar los ficheros de bases de datos online.

Los datos se almacenan en los bloques de los discos; el acceso a un bloque de un disco es costoso debido al tiempo de búsqueda, el retardo rotacional y el tiempo de transferencia del bloque. Para reducir el tiempo medio de acceso a un bloque, se puede utilizar el doble búfer para acceder a bloques de disco consecutivos. En el Apéndice B explicamos otros parámetros relacionados con un disco. Presentamos las diferentes formas de almacenar los registros de un fichero en un disco. Los registros de un fichero se agrupan en bloques del disco y pueden ser de longitud fija o variable, extendida o no extendida, y del mismo tipo de registro o de una mezcla de tipos. Explicamos la cabecera de un fichero, que describe el formato del registro y alberga las direcciones de disco de los bloques del fichero. El software del sistema utiliza la información de esta cabecera para acceder a los registros del fichero.

Después, presentamos un conjunto de comandos típicos para acceder a los registros individuales de un fichero, y explicamos el concepto de registro actual de un fichero. También explicamos cómo las condiciones de búsqueda de registros complejas se pueden transformar en condiciones de búsqueda sencillas que se utilizan para localizar registros en un fichero.

A continuación explicamos las tres principales organizaciones de un fichero: desordenado, ordenado y disperso. Los ficheros desordenados requieren una búsqueda lineal para localizar registros, pero la inserción de un registro es muy sencilla. Explicamos el problema del borrado y el uso de los marcadores de borrado.

Los ficheros ordenados reducen el tiempo necesario para leer registros en el orden estipulado por el campo de ordenación. El tiempo necesario para realizar la búsqueda de un registro arbitrario, dado el valor de su campo clave de ordenación, también se reduce si se utiliza una búsqueda binaria. Sin embargo, el mantenimiento de un orden en los registros hace que la inserción sea muy costosa; por eso explicamos la técnica de utilizar un fichero de desbordamiento desordenado para reducir el coste que supone insertar un registro. Los registros de desbordamiento se mezclan periódicamente con el fichero maestro durante la reorganización del fichero.

La dispersión proporciona un acceso muy rápido a un registro arbitrario de un fichero, dado el valor de su clave de dispersión. El método más adecuado para la dispersión externa es la técnica de los cubos, en la que a cada cubo le corresponden uno o más bloques contiguos. Las colisiones provocadas por un desbordamiento del cubo se manipulan con el encadenamiento. El acceso a cualquier campo no disperso es lento. Explicamos dos técnicas de dispersión para los ficheros cuya cantidad de registros crece o disminuye dinámicamente: dispersión extensible y lineal.

Explicamos brevemente otras posibilidades para la reorganización primaria de un fichero, como los árboles B y los ficheros de registros mixtos, que implementan relaciones físicas entre registros de diferentes tipos como parte de la estructura de almacenamiento. Por último, repasamos los avances más recientes representados por la tecnología RAID.

Preguntas de repaso

- 13.1. ¿Cuál es la diferencia entre almacenamiento primario y secundario?
- 13.2. ¿Por qué se utilizan discos, y no cintas, para almacenar los ficheros de bases de datos online?
- 13.3. Defina los siguientes términos: *disco*, *paquete de discos*, *pista*, *bloque*, *cilindro*, *sector*, *hueco entre bloques*, *cabeza de lectura/escritura*.
- 13.4. Explique el proceso de inicialización de un disco.
- 13.5. Explique el mecanismo que se utiliza para leer los datos o escribir datos en el disco.
- 13.6. ¿Cuáles son los componentes de la dirección de un bloque del disco?
- 13.7. ¿Por qué resulta costoso acceder a un bloque de un disco? Explique los distintos tiempos implicados en el acceso a un bloque del disco.
- 13.8. ¿Cómo mejora el tiempo de acceso a un bloque utilizando el doble búfer?
- 13.9. ¿Cuáles son las razones que llevan a tener registros de longitud variable? ¿Qué tipos de caracteres separadores se necesitan para cada uno?
- 13.10. Explique las técnicas de asignación de bloques de fichero en el disco.
- 13.11. ¿Cuál es la diferencia entre una organización de fichero y un método de acceso?
- 13.12. ¿Cuál es la diferencia entre un fichero estático y uno dinámico?
- 13.13. ¿Cuáles son las operaciones típicas de un registro por vez para acceder a un fichero? ¿Cuáles de ellas dependen del registro actual del fichero?
- 13.14. Explique las técnicas para borrar un registro.
- 13.15. Explique las ventajas y los inconvenientes de utilizar (a) un fichero desordenado, (b) un fichero ordenado, y (c) un fichero disperso estático con cubos y encadenamiento. ¿Qué operaciones se pueden llevar a cabo eficazmente en cada una de estas organizaciones, y qué operaciones resultan costosas?
- 13.16. Explique las técnicas para permitir que un fichero de dispersión crezca o disminuya dinámicamente. ¿Cuáles son las ventajas y los inconvenientes de cada una de ellas?

- 13.17.** ¿Para qué se utilizan los ficheros mixtos? ¿Cuáles son los otros tipos de organizaciones primarias de fichero?
- 13.18.** Describa la incompatibilidad entre el procesador y las tecnologías de disco.
- 13.19.** ¿Cuáles son los principales objetivos de la tecnología RAID? ¿Cómo se consiguen?
- 13.20.** ¿Cómo ayuda un disco reflejado a mejorar la fiabilidad? Ofrezca un ejemplo cuantitativo.
- 13.21.** ¿Qué técnicas se utilizan para mejorar el rendimiento de los discos en RAID?
- 13.22.** ¿Qué caracteriza los niveles en la organización RAID?

Ejercicios

- 13.23.** Piense en un disco con las siguientes características (no son parámetros de una unidad de disco específica): tamaño de bloque $B = 512$ bytes; tamaño del hueco entre bloques $G = 128$ bytes; número de bloques por pista = 20; número de pistas por superficie = 400. Un paquete de discos consta de 15 discos de doble cara.
- ¿Cuál es la capacidad total de una pista, y cuál es su capacidad útil (excluyendo los huecos entre bloques)?
 - ¿Cuántos cilindros tiene?
 - ¿Cuál es la capacidad total y la capacidad útil de un cilindro?
 - ¿Cuál es la capacidad total y la capacidad útil de un paquete de discos?
 - Suponga que la unidad de disco gira el paquete de discos a una velocidad de 2.400 rpm (revoluciones por minuto); ¿cuál es la velocidad de transferencia (tr) en bytes/mseg y el tiempo de transferencia de un bloque (btt) en mseg? ¿Cuál es el retardo rotacional medio (rd) en mseg? ¿Cuál es la velocidad de transferencia en masa? (Consulte el Apéndice B.)
 - Suponga que el tiempo de búsqueda medio es de 30 mseg. ¿Cuánto tiempo tarda (por término medio) en mseg en localizar y transferir un solo bloque, dada la dirección de ese bloque?
 - Calcule el tiempo medio que tardaría en transferir 20 bloques aleatorios, y compárelo con el tiempo que tardaría en transferir 20 bloques consecutivos utilizando el doble búfer para ahorrar tiempo de búsqueda y retardo rotacional.
- 13.24.** Un fichero tiene $r = 20.000$ registros ESTUDIANTE de *longitud fija*. Cada registro tiene los siguientes campos: Nombre (30 bytes), Dni (9 bytes), Direcc (40 bytes), Tlf (9 bytes), FechaNac (8bytes), Sexo (1 byte), CodDptoEspecialidad (4 bytes), CodDptoEspecSec (4 bytes), CodClase (4bytes, entero) y ProgramaGrado (3 bytes). Se utiliza un byte adicional como marcador de borrado. El fichero está almacenado en un disco cuyos parámetros son los facilitados en el Ejercicio 13.23.
- Calcule el tamaño de registro R en bytes.
 - Calcule el factor de bloqueo bfr y el número de bloques del fichero b , asumiendo una organización no extendida.
 - Calcule el tiempo medio que se tardaría en encontrar un registro realizando una búsqueda lineal sobre el fichero si (i) los bloques del fichero están almacenados contiguamente y se utiliza el doble búfer; (ii) los bloques del fichero no están almacenados contiguamente.
 - Asuma que el fichero está ordenado por Dni; con una búsqueda binaria, calcule el tiempo que se tardaría en encontrar un registro dado el valor de su Dni.
- 13.25.** Suponga que sólo el 80% de los registros de ESTUDIANTE del Ejercicio 13.24 tienen un valor para Tlf, el 85% para CodDptoEspecialidad, el 15% para CodDptoEspecSec y el 90% para ProgramaGrado; y suponga que utilizamos un fichero de registros de longitud variable. Cada registro tiene 1 byte como *tipo de campo* por cada campo del registro, más 1 byte del marcador de borrado y 1 byte como marcador de fin de registro. Suponga además que utilizamos una organización de

registros extendida, donde cada bloque tiene un puntero de 5 bytes apuntando al siguiente bloque (este espacio no se utiliza para el almacenamiento del registro).

- a. Calcule la longitud media del registro R en bytes.
 - b. Calcule el número de bloques necesarios para el fichero.
- 13.26.** Suponga que una unidad de disco tiene los siguientes parámetros: tiempo de búsqueda $s = 20$ mseg; retardo rotacional $rd = 10$ mseg; tiempo de transferencia de un bloque $btt = 1$ mseg; tamaño de bloque $B = 2.400$ bytes; tamaño de hueco entre bloques $G = 600$ bytes. Un fichero EMPLEADO tiene los siguientes campos: Dni, 9 bytes; Nombre, 20 bytes; Apellidos, 20 bytes; Inicial, 1 byte; FechaNac, 10 bytes; Direcc, 35 bytes; Tlf, 12 bytes; DniSupervisor, 9 bytes; Departamento, 4 bytes; CodTrabajo, 4 bytes; *marcador de borrado*, 1 byte. El fichero EMPLEADO tiene $r = 30.000$ registros, formato de longitud fija, y bloqueo no extendido. Escriba las fórmulas apropiadas y calcule los siguientes valores para este fichero EMPLEADO:
- a. Tamaño de registro R (incluyendo el marcador de borrado), factor de bloqueo bfr , y número de bloques de disco b .
 - b. Calcule el espacio desperdiciado en cada bloque de disco debido a la organización no extendida.
 - c. Calcule la velocidad de transferencia tr y la velocidad de transferencia en masa btr de esta unidad de disco (consulte el Apéndice B si desea conocer las definiciones de tr y btr).
 - d. Calcule la media de *accesos a bloques* necesarios para buscar un registro arbitrario en el fichero, utilizando una búsqueda lineal.
 - e. Calcule en mseg el *tiempo medio* necesario para buscar un registro arbitrario en el fichero, utilizando la búsqueda lineal, si los bloques del fichero están almacenados en bloques de disco consecutivos y utilizando el doble búfer.
 - f. Calcule en mseg el *tiempo medio* necesario para buscar un registro arbitrario en el fichero, utilizando la búsqueda lineal, si los bloques del fichero no están almacenados en bloques de disco consecutivos.
 - g. Asuma que los registros están ordenados mediante algún campo clave. Calcule el *promedio de accesos a bloques* y el *tiempo medio* necesario para buscar un registro arbitrario en el disco utilizando una búsqueda binaria.
- 13.27.** Un fichero REPUESTOS con NumRep como clave de dispersión incluye registros con los siguientes valores de NumRep: 2369, 3760, 4692, 4871, 5659, 1821, 1074, 7115, 1620, 2428, 3943, 4750, 6975, 4981, 9208. El fichero utiliza ocho cubos, numerados de 0 a 7. Cada cubo es un bloque de disco y almacena dos registros. Cargue estos registros en el fichero en el orden dado, utilizando la función de dispersión $h(K) = K \bmod 8$. Calcule la cantidad media de accesos a bloque para una recuperación aleatoria sobre NumRep.
- 13.28.** Cargue los registros del Ejercicio 13.27 en ficheros de dispersión expansibles basándose en la dispersión extensible. Muestre la estructura del directorio en cada paso, así como las profundidades global y local. Utilice la función de dispersión $h(K) = K \bmod 128$.
- 13.29.** Cargue los registros del Ejercicio 13.27 en un fichero de dispersión expansible, utilizando la dispersión lineal. Empiece con un solo bloque de disco, utilizando la función de dispersión $h_0 = K \bmod 2^0$, y muestre cómo crece el fichero y cómo las funciones de dispersión cambian a medida que se insertan registros. Asuma que los bloques se dividen siempre que se produce un desbordamiento, y muestre el valor de n en cada etapa.
- 13.30.** Compare los comandos de fichero de la Sección 13.6 con los disponibles en un método de acceso a ficheros con el que esté familiarizado.
- 13.31.** Suponga que tenemos un fichero desordenado de registros de longitud fija que utiliza una organización de registros no extendida. Diseñe unos algoritmos para insertar, borrar y modificar un registro del fichero. Especifique las suposiciones que asuma.

- 13.32.** Suponga que tenemos un fichero ordenado de registros de longitud fija y un fichero de desbordamiento desordenado para manipular la inserción. Los dos ficheros utilizan registros no extendidos. Diseñe unos algoritmos para insertar, borrar y modificar un registro del fichero y para reorganizar el fichero. Especifique las suposiciones que asuma.
- 13.33.** ¿Puede pensar en otras técnicas diferentes a usar un fichero de desbordamiento desordenado que puedan utilizarse para realizar inserciones en un fichero ordenado de un modo más eficaz?
- 13.34.** Suponga que tenemos un fichero de dispersión de registros de longitud fija, y suponga también que el desbordamiento lo manipulamos con el encadenamiento. Diseñe unos algoritmos para insertar, borrar y modificar un registro del fichero. Especifique las suposiciones que asuma.
- 13.35.** Escriba el código de un programa para acceder a los campos individuales de los registros bajo cada una de las siguientes circunstancias. En cada caso, enuncie las suposiciones que está haciendo en relación a punteros, caracteres de separación, etcétera. Determine el tipo de información necesaria en la cabecera del fichero para que su código sea general en cada caso.
- Registros de longitud fija con bloqueo no extendido.
 - Registros de longitud fija con bloqueo extendido.
 - Registros de longitud variable con campos de longitud variable y bloqueo extendido.
 - Registros de longitud variable con repetición de grupos y bloqueo extendido.
 - Registros de longitud variable con campos opcionales y bloqueo extendido.
 - Registros de longitud variable que permite los tres casos indicados en las partes c, d, y e.

Bibliografía seleccionada

Wiederhold (1983) tiene una explicación y un análisis detallados de los dispositivos de almacenamiento secundario y de las organizaciones de ficheros. Los discos ópticos se describen en Berg y Roth (1989) y se analizan en Ford y Christodoulakis (1991). La memoria flash se explica en Dippert y Levy (1993). Ruemmler y Wilkes (1994) presenta un estudio de la tecnología de disco magnético. La mayoría de los libros sobre bases de datos incluyen explicaciones del material aquí presentado. Casi todos los libros sobre estructuras de datos, incluyendo Knuth (1973), explican la dispersión estática más en detalle; Knuth tiene una explicación completa de las funciones de dispersión y de las técnicas de resolución de colisiones, así como de la comparación de su rendimiento. Entre los libros sobre estructuras de datos citamos Claybrook (1983), Smith y Barnes (1987) y Salzberg (1988); explican organizaciones adicionales de ficheros, como los ficheros estructurados en árbol, y tienen algoritmos detallados para las operaciones sobre ficheros. Algunas obras sobre las organizaciones de los ficheros son Miller (1987) y Livadas (1989). Salzberg y otros (1990) describe un algoritmo de ordenación externa distribuida. Las organizaciones de ficheros con un alto grado de tolerancia a los fallos se describe en Bitton and Gray (1988) y en Gray y otros (1990). La segmentación de disco se propone en Salem y García Molina (1986). El primer ensayo sobre las matrices redundantes de discos baratos (RAID) lo realizó Patterson y otros (1988). Chen y Patterson (1990) y el excelente estudio sobre RAID realizado por Chen y otros (1994) son otras referencias. Grochowski y Hoyt (1996) explica las tendencias futuras en lo que se refiere a las unidades de disco. En Chen y otros (1994) aparecen varias fórmulas para la arquitectura RAID.

Morris (1968) es un antiguo ensayo sobre la dispersión. La dispersión extensible se describe en Fagin y otros (1979). La dispersión lineal se describe en Litwin (1980). La dispersión dinámica, que no hemos explicado en detalle, se propuso en Larson (1978). Se han propuesto muchas variaciones de la dispersión extensible y lineal; a modo de ejemplo, consulte Cesarini y Soda (1991), Du y Tong (1991), y Hachem y Berra (1992).

En los sitios de los fabricantes puede encontrar detalles sobre los dispositivos de almacenamiento (por ejemplo, <http://www.seagate.com>, <http://www.ibm.com>, <http://www.ecm.com>, <http://www.hp.com>, y <http://www.storagetek.com>). IBM dispone de un centro de investigación de tecnología de almacenamiento en IBM Almaden (<http://www.almaden.ibm.com/sst/>).

CAPÍTULO 14

Estructuras de indexación para los ficheros

En este capítulo asumimos que ya existe un fichero con alguna de las organizaciones primarias: desordenado, ordenado o disperso (consulte el Capítulo 13). En este capítulo vamos a describir unas estructuras de acceso auxiliares denominadas índices, que se utilizan para acelerar la recuperación de registros en respuesta a ciertas condiciones de búsqueda. Las estructuras de índice normalmente proporcionan rutas de acceso, que ofrecen formas alternativas de acceder a los registros sin que se vea afectada la ubicación física de los registros en el disco. Permiten un acceso eficaz a los registros basándose en la indexación de los campos que se utilizan para construir el índice. Básicamente, es posible utilizar *cualquier campo* del fichero para crear un índice y se pueden construir *varios índices* con diferentes campos en el mismo fichero. También son posibles varios tipos de índices; cada uno de ellos utiliza una estructura de datos en particular para acelerar la búsqueda. Para encontrar uno o varios registros del fichero basándose en ciertos criterios de selección de un campo indexado, primero hay que acceder al índice, que apunta a uno o más bloques del fichero donde están almacenados los registros requeridos. Los tipos de índices predominantes están basados en los ficheros ordenados (índices de un nivel) y en estructuras de datos en forma de árbol (índices multinivel, árboles B^+). Los índices también pueden construirse basándose en la dispersión o en otras estructuras de datos de búsqueda.

En la Sección 14.1 describimos los diferentes tipos de índices ordenados de un nivel: primario, secundario y agrupado (o de agrupamiento). Si vemos un índice de un nivel como un fichero ordenado, podemos desarrollar índices adicionales para él, lo que nos lleva al concepto de índices multinivel. En esta idea está basado el popular esquema de indexación ISAM (Método de acceso secuencial indexado, *Indexed Sequential Access Method*). En la Sección 14.2 explicamos los índices multinivel. En la Sección 14.3 describimos los árboles B y B^+ , que son estructuras de datos que se utilizan con frecuencia en los DBMSs para implementar índices multinivel que cambian dinámicamente. Los árboles B^+ se han convertido en una estructura predeterminada comúnmente aceptada en la mayoría de los DBMSs relacionales para la generación de índices bajo demanda. La Sección 14.4 está dedicada a las distintas alternativas de acceso a datos en base a una combinación de varias claves. En la Sección 14.5 explicamos cómo podemos utilizar otras estructuras de datos (por ejemplo, la dispersión) para construir índices. También ofrecemos una breve introducción al concepto de los índices lógicos, que dotan de un nivel adicional de indirección a los índices físicos, ya que les permite ser flexibles y extensibles en su organización. La Sección 14.6 resume el capítulo.

14.1 Tipos de índices ordenados de un nivel

La idea tras la estructura de acceso de un índice ordenado es parecida a la que hay tras el índice de un libro, que enumera al final de la obra los términos importantes ordenados alfabéticamente, junto con las páginas en las que aparecen. Podemos buscar en un índice en busca de una lista de *direcciones* (números de páginas en este caso) y utilizar esas direcciones para localizar un término en el libro, *buscando* en las páginas especificadas. La alternativa, de no indicarse lo contrario, sería desplazarse lentamente por todo el libro, palabra por palabra, hasta encontrar el término en el que estuviéramos interesados; sería como hacer una búsqueda lineal en un fichero. Por supuesto, la mayoría de los libros tienen información adicional, como títulos de capítulo y de sección, que nos ayuda a encontrar el término sin tener que buscar por todo el libro. Sin embargo, el índice es la única indicación exacta del lugar del libro donde se encuentra cada término.

En un fichero con una estructura de registro dada compuesta por varios campos (o atributos), normalmente se define una estructura de índice con un solo campo del fichero, que se conoce como **campo de indexación** (o **atributo de indexación**).¹ Normalmente, el índice almacena todos los valores del campo de índice, junto con una lista de punteros a todos los bloques de disco que contienen los registros con ese valor de campo. Los valores del índice están ordenados, de modo que podemos hacer una búsqueda binaria en el índice. El fichero de índice es mucho más pequeño que el fichero de datos, por lo que la búsqueda en el índice mediante una búsqueda binaria es razonablemente eficaz. La indexación multinivel (consulte la Sección 14.2) anula la necesidad de una búsqueda binaria a expensas de crear índices al propio índice.

Hay varios tipos de índices ordenados. Un **índice principal** o **primario** se especifica en el *campo clave de ordenación* de un fichero ordenado de registros. Como recordará de la Sección 13.7, se utiliza un campo clave de ordenación para *ordenar físicamente* los registros del fichero en el disco, y cada registro tiene un *valor único* para ese campo. Si el campo de ordenación no es un campo clave (es decir, si varios registros del fichero tienen el mismo valor para el campo de ordenación), podemos utilizar otro tipo de índice, denominado **índice agrupado**. Un fichero puede tener como máximo un campo de ordenación física, por lo que puede tener a lo sumo un índice principal o un índice agrupado, *pero no ambos*. Un tercer tipo de índice, el **índice secundario**, se puede especificar sobre cualquier campo no ordenado de un fichero. Un fichero puede tener varios índices secundarios además de su método de acceso principal. Explicamos estos tipos de índices de un nivel en las siguientes tres subsecciones.

14.1.1 Índices principales

Un **índice principal** es un fichero ordenado cuyos registros son de longitud fija con dos campos. El primer campo es del mismo tipo de datos que el campo clave de ordenación (denominado **clave principal**) del fichero de datos, y el segundo campo es un puntero a un bloque del disco (una dirección de bloque). En el fichero índice hay una **entrada de índice** (o **registro de índice**) por cada *bloque* del fichero de datos. Cada entrada del índice tiene dos valores: el valor del campo clave principal para el *primer* registro de un bloque, y un puntero a ese bloque. Nos referiremos a los dos valores de la entrada *i* del índice como $\langle K(i), P(i) \rangle$.

Para crear un índice principal en el fichero ordenado de la Figura 13.7, utilizamos el campo Nombre como clave principal, porque es el campo clave de ordenación del fichero (asumiendo que cada valor de Nombre es único). Cada entrada del índice tiene un valor Nombre y un puntero. Las tres primeras entradas del índice son las siguientes:

$\langle K(1) = (\text{Aaron}, \text{Ed}), P(1) = \text{dirección de bloque 1} \rangle$

$\langle K(2) = (\text{Adams}, \text{John}), P(2) = \text{dirección de bloque 2} \rangle$

$\langle K(3) = (\text{Alexander}, \text{Ed}), P(3) = \text{dirección de bloque 3} \rangle$

¹ En este capítulo utilizamos indistintamente los términos *campo* y *atributo*.

La Figura 14.1 ilustra este índice principal. El número total de entradas del índice coincide con el *número de bloques de disco* del fichero de datos ordenado. El primer registro de cada bloque del fichero de datos es el **registro ancla** del bloque, o simplemente **ancla de bloque**.²

Los índices se pueden clasificar como densos o escasos. Un **índice denso** tiene una entrada de índice por *cada valor de la clave de búsqueda* (y, por tanto, cada registro) del fichero de datos. Un **índice escaso** (o **no denso**) sólo tiene entradas para algunos de los valores de búsqueda. Por consiguiente, un índice principal es un índice escaso, porque incluye una entrada por cada bloque de disco del fichero de datos y las claves de su registro ancla, en lugar de una entrada por cada valor de búsqueda (es decir, por cada registro).

El fichero de índice para un índice principal necesita menos bloques que el fichero de datos, por dos razones. En primer lugar, hay *menos entradas de índice* que registros en el fichero de datos. En segundo lugar, cada entrada del índice tiene normalmente un *tamaño más pequeño* que un registro de datos, porque sólo tiene dos campos; en consecuencia, en un bloque entran más entradas de índice que registros de datos. Por tanto, una búsqueda binaria en el fichero de índice requiere menos accesos a bloques que una búsqueda binaria en el fichero de datos. En referencia a la Tabla 13.2, observe que la búsqueda binaria en un fichero de datos ordenado requiere $\log_2 b$ accesos a bloques. Pero si el fichero de índice principal contiene b_i bloques, la localización de un registro con un valor de clave de búsqueda requiere una búsqueda binaria de ese índice y acceder al bloque que contiene ese registro: un total de $\log_2 b_i + 1$ accesos.

Un registro cuyo valor de la clave principal es K consiste en el bloque cuya dirección es $P(i)$, donde $K(i) \leq K < K(i+1)$. El bloque número i del fichero de datos contiene tales registros debido a la ordenación física de los registros del fichero por el campo clave principal. Para recuperar un registro, dado el valor K de su campo clave principal, hacemos una búsqueda binaria en el fichero índice para encontrar la entrada de índice i apropiada, y después recuperar el bloque del fichero de datos cuya dirección es $P(i)$.³ El Ejemplo 1 ilustra el ahorro en accesos a bloques que se logra cuando se utiliza un índice principal para buscar un registro.

Ejemplo 1. Supongamos que tenemos un fichero ordenado con $r = 30.000$ registros almacenados en un disco cuyo tamaño de bloque es $B = 1.024$ bytes. Los registros del fichero son de tamaño fijo y no están extendidos, con una longitud de registro $R = 100$ bytes. El factor de bloqueo para el fichero sería $bfr = \lfloor (B/R) \rfloor = \lfloor (1.024/100) \rfloor = 10$ registros por bloque. El número de bloques necesarios para el fichero es $b = \lceil (r/bfr) \rceil = \lceil (30.000/10) \rceil = 3.000$ bloques. Una búsqueda binaria en el fichero de datos necesitaría aproximadamente $\lceil \log_2 b \rceil = \lceil \log_2 3.000 \rceil = 12$ accesos a bloques.

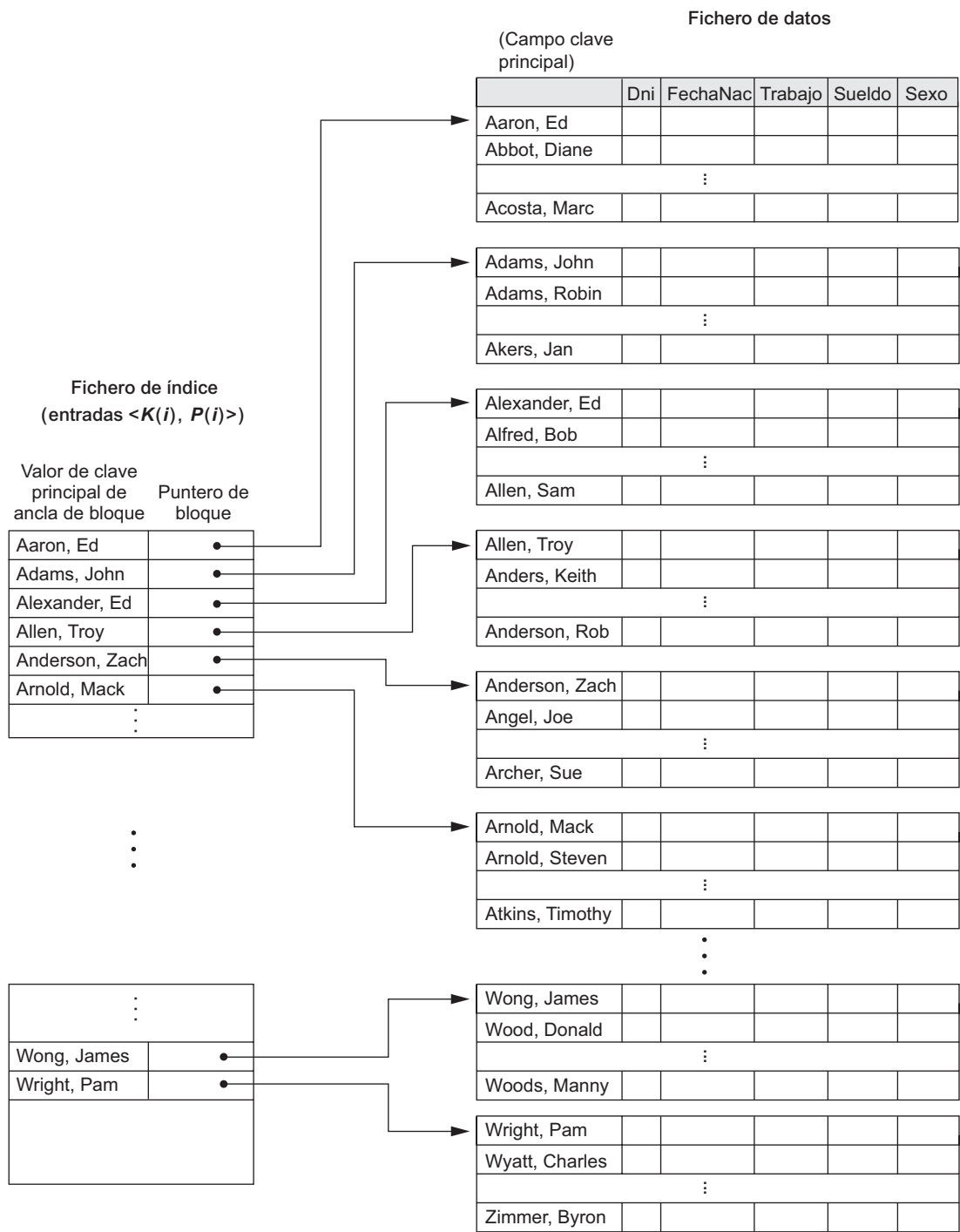
Ahora, suponga que el campo clave de ordenación del fichero tiene una longitud de $V = 9$ bytes, una longitud de puntero de bloque de $P = 6$ bytes, y que hemos construido un índice principal para el fichero. El tamaño de cada entrada del índice es $R_i = (9 + 6) = 15$ bytes, por lo que el factor de bloqueo para el índice es $bfr_i = \lfloor (B/R_i) \rfloor = \lfloor (1.024/15) \rfloor = 68$ entradas por bloque. El número total de entradas del índice r_i es igual al número de bloques del fichero de datos, que son 3.000. El número de bloques del índice es, por tanto, $b_i = \lceil (r_i/bfr_i) \rceil = \lceil (3.000/68) \rceil = 45$ bloques. Para realizar una búsqueda binaria en el fichero índice necesitaríamos $\lceil \log_2 b_i \rceil = \lceil \log_2 45 \rceil = 6$ accesos a bloques. Para buscar un registro utilizando el índice, necesitamos un acceso a bloque adicional al fichero de datos para un total de $6 + 1 = 7$ accesos a bloques (una mejora sobre la búsqueda binaria en el fichero de datos, que requería 12 accesos a bloques).

La inserción y el borrado de registros supone un gran problema con un índice principal (como con cualquier fichero ordenado). Con un índice principal, el problema se complica porque si queremos insertar un registro en su posición correcta dentro del fichero de datos, no sólo tenemos que mover los registros para hacer sitio al registro nuevo, sino que también tenemos que cambiar algunas entradas del índice, puesto que al mover los

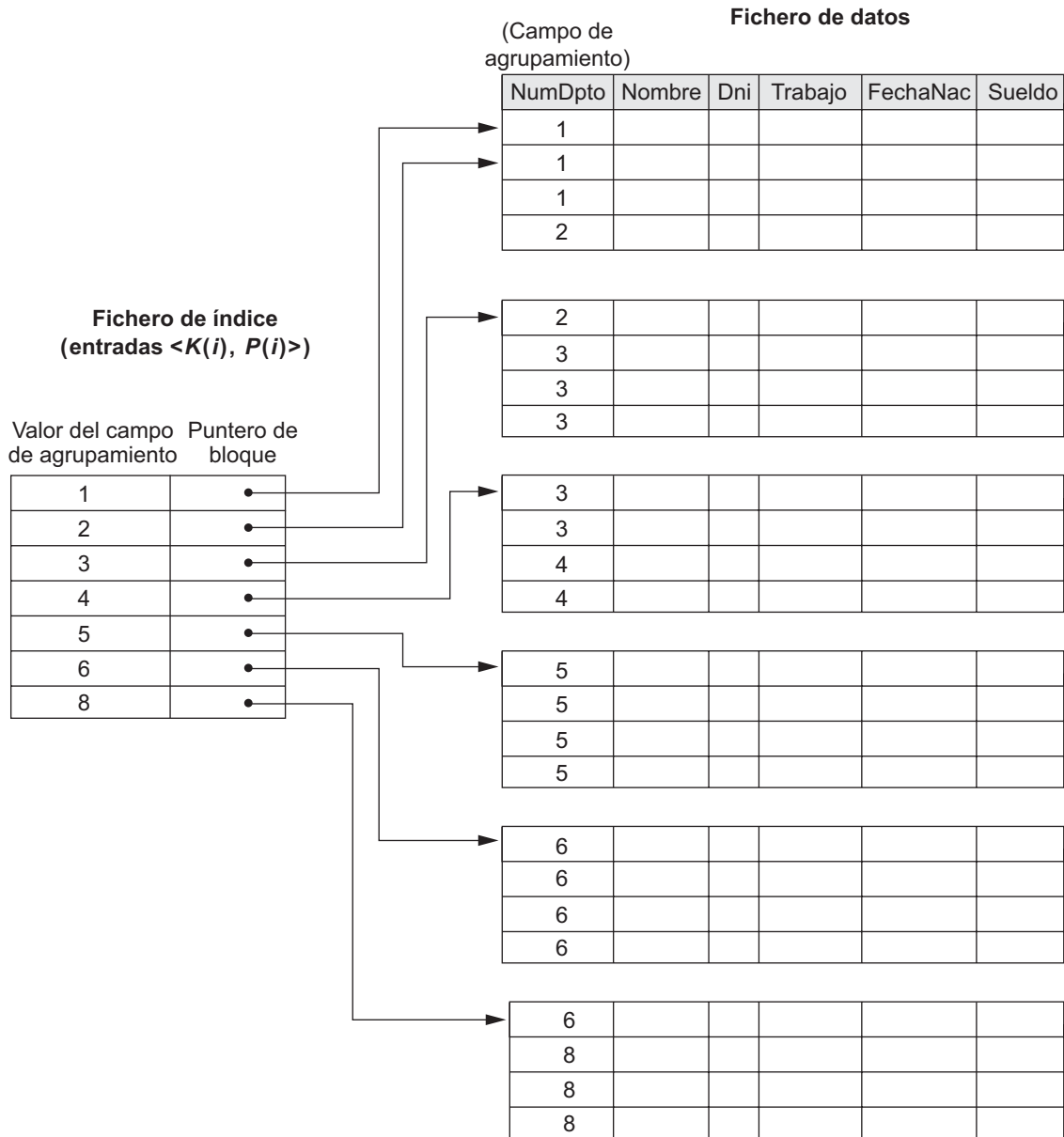
² Podemos utilizar un esquema parecido al descrito aquí, con el último registro de cada bloque (en lugar del primero) como ancla de bloque. Esto mejora ligeramente la eficacia del algoritmo de búsqueda.

³ Observe que la fórmula anterior no sería correcta si el fichero de datos estuviera ordenado por un campo no clave; en tal caso, el mismo valor de índice en el ancla de bloque podría repetirse en los últimos registros del bloque anterior.

Figura 14.1. El índice principal en el campo clave de ordenación del fichero de la Figura 13.7.



registros modificaremos los registros ancla de algunos bloques. El uso de un fichero de desbordamiento desordenado (consulte la Sección 13.7), puede minimizar este problema. Otra posibilidad es utilizar una lista de enlaces de registros desbordados por cada bloque del fichero de datos. Se parece al método de tratar con los

Figura 14.2. Un índice agrupado en el campo no clave de ordenación NumDpto de un fichero EMPLEADO.

registros desbordados con dispersión descrito en la Sección 13.8.2. Los registros de cada bloque y su lista enlazada de desbordamiento se pueden ordenar para mejorar el tiempo de recuperación. El borrado de un registro se realiza con marcadores de borrado.

14.1.2 Índices agrupados

Si los registros del fichero están ordenados físicamente por un campo no clave (es decir, un campo que no tiene un valor distinto para cada registro), este campo se denomina **campo agrupado**. Podemos crear un tipo de índice diferente, denominado **índice agrupado**, para acelerar la recuperación de los registros que tienen el

mismo valor para el campo agrupado. Esto difiere de un índice principal, que requiere que el campo de ordenación del fichero de datos tenga un *valor distinto* para cada registro. Un índice agrupado también es un fichero ordenado con dos campos; el primero es del mismo tipo que el campo agrupado del fichero de datos, y el segundo es un puntero a un bloque. En el índice agrupado hay una entrada por cada *valor distinto* del campo agrupado, que contiene el valor y un puntero al *primer bloque* del fichero de datos que tiene un registro con ese valor para su campo agrupado. La Figura 14.2 muestra un ejemplo. La inserción y el borrado de un registro todavía provoca problemas, porque los registros de datos están ordenados físicamente. Para aliviar el problema de la inserción, se suele reservar un bloque entero (o un grupo de bloques contiguos) para *cada valor* del campo agrupado; todos los registros con ese valor se colocan en el bloque (o grupo de bloques). Esto hace que la inserción y el borrado sean relativamente directos. La Figura 14.3 muestra este esquema.

Un índice agrupado es otro ejemplo de índice no denso porque tiene una entrada por cada *valor distinto* del campo de indexación, que no es una clave por definición y, por tanto, tiene valores duplicados en lugar de un valor único por cada registro del fichero. Hay alguna similitud entre las Figuras 14.1 a 14.3, por un lado, y la Figura 13.11, por otro. Un índice es algo parecido a las estructuras de directorio utilizadas para la dispersión extensible que explicamos en la Sección 13.8.3. Ambas estructuras se exploran para encontrar un puntero al bloque de datos que contiene el registro deseado. Una diferencia importante es que una búsqueda en un índice utiliza los valores del propio campo de búsqueda, mientras que una búsqueda en un directorio disperso utiliza el valor de dispersión que se calcula aplicando la función de dispersión al campo de búsqueda.

14.1.3 Índices secundarios

Un **índice secundario** proporciona un medio secundario de acceso a un fichero para el que ya existe algún acceso principal. El índice secundario puede ser un campo que es una clave candidata y tiene un valor único en cada registro, o puede ser una “no clave” con valores duplicados. El índice es un fichero ordenado con dos campos. El primero es del mismo tipo de datos que algún *campo no ordenado* del fichero de datos que es un **campo de indexación**. El segundo campo es un puntero de *bloque* o un puntero de *registro*. Puede haber muchos índices secundarios (y, por tanto, campos de indexación) para el mismo fichero.

Primero consideramos que la estructura de acceso de un índice secundario en un campo clave tiene un *valor distinto* por cada registro. Dicho campo recibe a veces el nombre de **clave secundaria**. En este caso, hay una entrada de índice por *cada registro* del fichero de datos, que contiene el valor de la clave secundaria para el registro y un puntero al bloque en el que está almacenado el registro o al propio registro. Por tanto, dicho índice es **denso**.

De nuevo nos referimos a los dos valores de la entrada de índice i como $\langle K(i), P(i) \rangle$. Las entradas están **ordenadas** por el valor de $K(i)$, por lo que podemos efectuar una búsqueda binaria. Como los registros del fichero de datos no están físicamente ordenados por los valores del campo clave secundario, *no podemos utilizar* las anclas de bloque. Por eso creamos una entrada de índice por cada registro del fichero de datos, en lugar de hacerlo por cada bloque, como en el caso de un índice principal. La Figura 14.4 ilustra un índice secundario en el que los punteros $P(i)$ de las entradas de índice son *punteros de bloque*, y no punteros de registro. Una vez transferido el bloque adecuado a la memoria principal, puede efectuarse una búsqueda del registro deseado dentro del bloque.

Un índice secundario normalmente necesita más espacio de almacenamiento y un tiempo de búsqueda mayor que un índice principal, debido a su mayor cantidad de entradas. Sin embargo, la *mejora* en el tiempo de búsqueda de un registro arbitrario es mucho mayor para un índice secundario que para un índice principal, puesto que tendríamos que hacer una *búsqueda lineal* en el fichero de datos si no existiera el índice secundario. En el caso de un índice principal, todavía podríamos utilizar una búsqueda binaria en el fichero principal, aun cuando no existiera el índice. El Ejemplo 2 ilustra la mejora en el número de bloques accedidos.

Ejemplo 2. Considere el fichero del Ejemplo 1 con $r = 30.000$ registros de longitud fija y un tamaño $R = 100$ bytes almacenados en un disco con un tamaño de bloque de $B = 1.024$ bytes. El fichero tiene $b = 3.000$

Figura 14.3. Índice agrupado con un grupo de bloques separados por cada grupo de registros que comparten el mismo valor del campo agrupado.

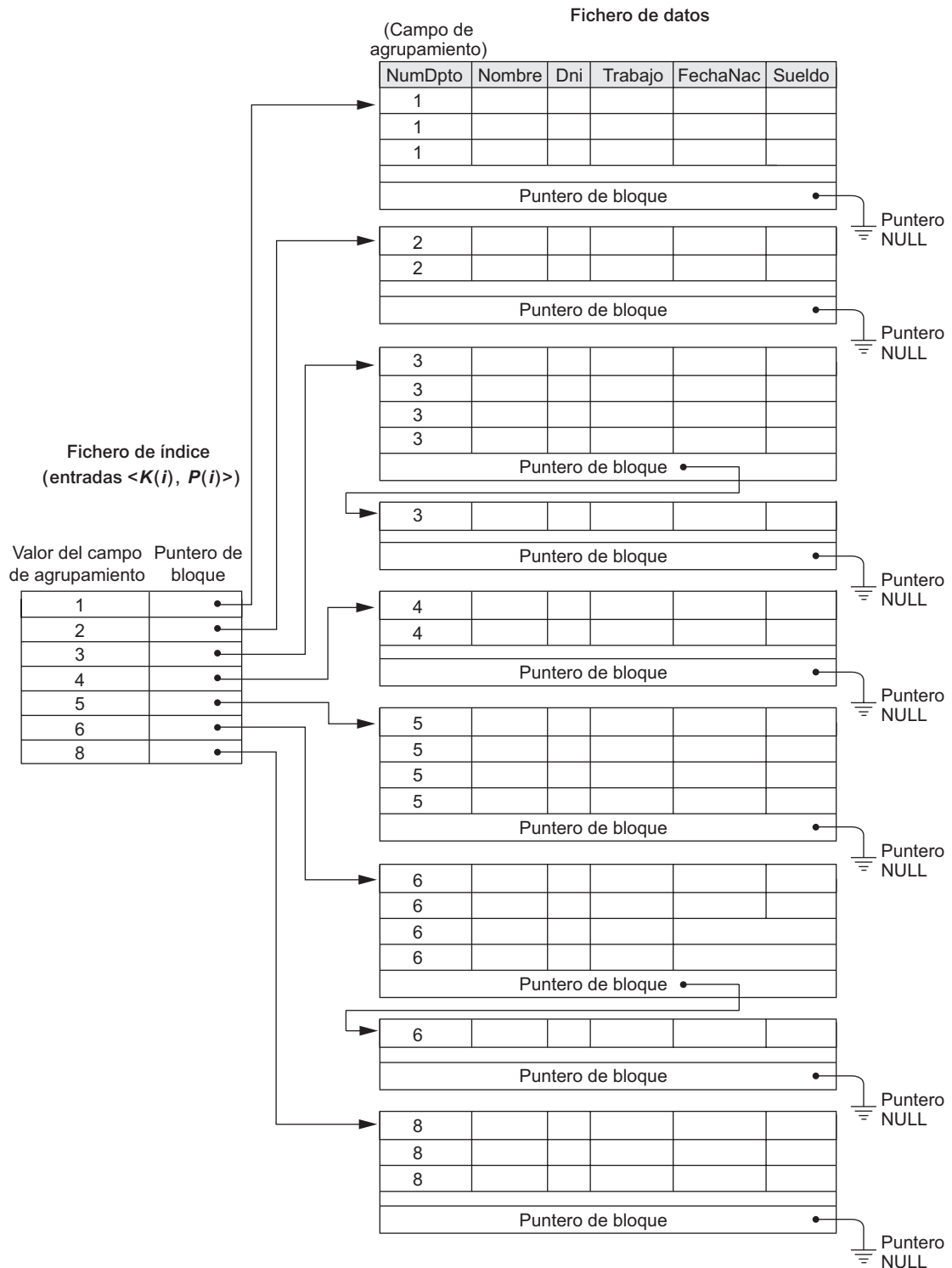
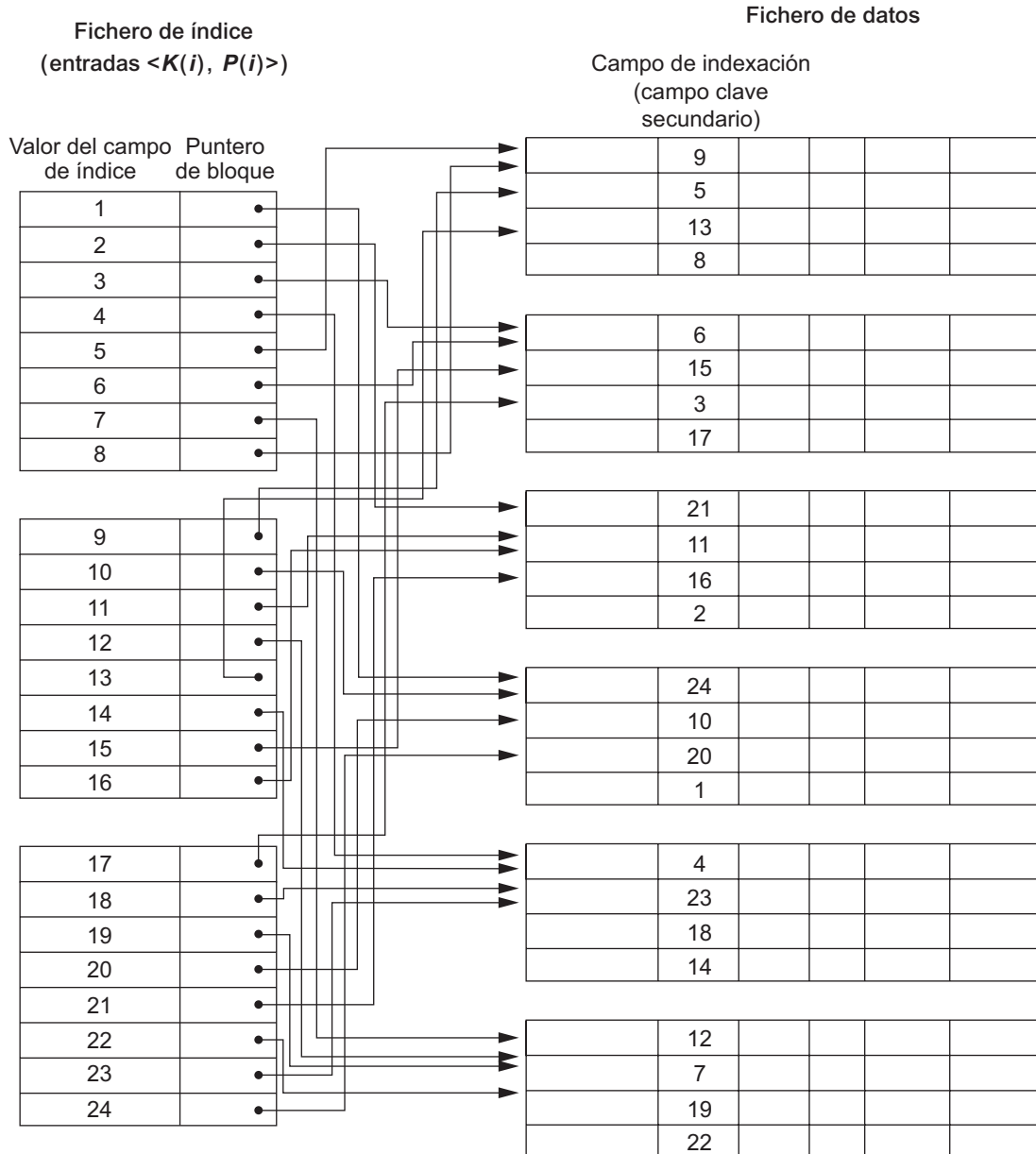


Figura 14.4. Índice secundario denso (con punteros de bloque) en un campo clave desordenado de un fichero.



bloques, según calculamos en el Ejemplo 1. Para efectuar una búsqueda lineal en el fichero, necesitaríamos acceder a $b/2 = 3.000/2 = 1.500$ bloques por término medio. Suponga que creamos un índice secundario en un campo clave desordenado del fichero y que tiene una longitud de $V = 9$ bytes. Como en el Ejemplo 1, un puntero de bloque tiene una longitud $P = 6$ bytes, por lo que cada entrada de índice es de $R_i = (9 + 6) = 15$ bytes, y el factor de bloqueo para el índice es de $bfr_i = \lfloor (B/R_i) \rfloor = \lfloor (1024/15) \rfloor = 68$ entradas por bloque. En un índice secundario denso como éste, la cantidad total de entradas de índice r_i es igual al número de registros del fichero de datos, es decir, 30.000. El número de bloques necesarios para el índice es, por tanto, $b_i = \lceil (r_i / bfr_i) \rceil = \lceil (3.000/68) \rceil = 442$ bloques.

Una búsqueda binaria en este índice secundario necesita acceder a $\lceil (\log_2 b_i) \rceil = \lceil (\log_2 442) \rceil = 9$ bloques. Para buscar un registro utilizando el índice, necesitamos un acceso a bloque adicional al fichero de datos para un total de $9 + 1 = 10$ accesos a bloques (una enorme mejora respecto a los 1.500 accesos a bloques necesarios por término medio para una búsqueda lineal, pero es una cifra ligeramente peor que los siete accesos a bloques requeridos por un índice principal).

También podemos crear un índice secundario en un campo que no sea una clave del fichero. En este caso, varios registros del fichero de datos pueden tener el mismo valor para el campo de indexación. Hay varias opciones para implementar dicho índice:

- Opción 1. Incluir varias entradas de índice con el mismo valor $K(i)$, una por cada registro. Sería un índice denso.
- Opción 2. Tener registros de longitud variable para las entradas del índice, con un campo repetitivo para el puntero. Guardamos una lista de punteros $\langle P(i,1), \dots, P(i,k) \rangle$ en la entrada de índice para $K(i)$ [un puntero a cada bloque que contiene un registro cuyo valor del campo de indexación es igual a $K(i)$]. En cualquiera de estas dos opciones que hemos visto por ahora, es preciso modificar apropiadamente el algoritmo de búsqueda binaria en el índice.
- Opción 3. Es la más utilizada y consiste en mantener las entradas del índice con una longitud fija y disponer de una sola entrada por cada *valor del campo de índice*, pero crear un nivel de indirección extra para manipular los múltiples punteros. En este esquema no denso, el puntero $P(i)$ en la entrada de índice $\langle K(i), P(i) \rangle$ apunta a un *bloque de punteros de registro*; cada puntero de registro de ese bloque apunta a uno de los registros del fichero de datos cuyo campo de indexación tiene el valor $K(i)$. Si algún valor $K(i)$ se repite en demasiados registros, de modo que sus punteros de registro no pueden encajar en un solo bloque de disco, se utiliza un grupo o lista enlazada de bloques. Esta técnica se ilustra en la Figura 14.5. La recuperación a través de un índice requiere uno o más accesos a bloques debido al nivel extra, pero los algoritmos de búsqueda en el índice y (lo que es más importante) de inserción de registros nuevos en el fichero de datos son directos. Además, las recuperaciones en condiciones de selección complejas pueden manipularse haciendo referencia a los punteros de registro, sin tener que recuperar muchos registros innecesarios del fichero (consulte el Ejercicio 14.19).

Un índice secundario proporciona una **ordenación lógica** de los registros por el campo de indexación. Si accedemos a los registros según el orden de las entradas del índice secundario, los obtendremos ordenados por el campo de indexación.

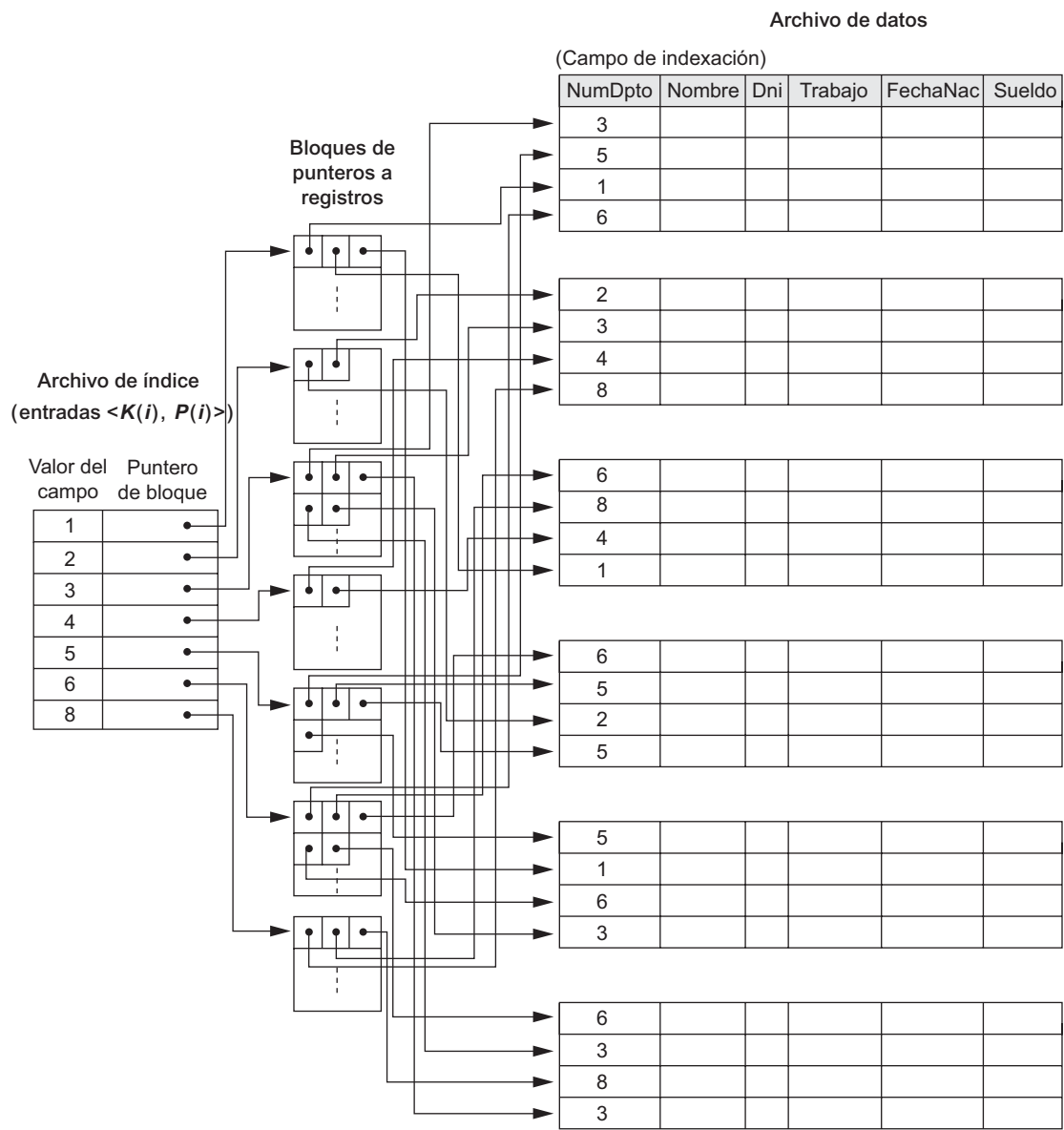
14.1.4 Resumen

Para concluir esta sección, resumimos la explicación de los tipos de índices en dos tablas. La Tabla 14.1 muestra las características del campo de índice de cada tipo de índice ordenado de un solo nivel: principal, agrupado y secundario. La Tabla 14.2 resume las propiedades de cada tipo de índice comparando el número de entradas de índice y especificando los índices que son densos y los que utilizan anclas de bloque del fichero de datos.

Tabla 14.1. Tipos de índices basados en las propiedades del campo de indexación.

	Campo de índice utilizado para ordenar el fichero	Campo de índice no usado para ordenar el fichero
El campo de indexación es una clave	Índice principal	Índice secundario (clave)
El campo de indexación no es una clave	Índice agrupado	Índice secundario (no clave)

Figura 14.5. Índice secundario (con punteros de registro) en un campo que no es índice e implementado utilizando un nivel de indirección, de modo que las entradas del índice son de longitud fija y tienen valores de campo únicos.



14.2 Índices multinivel

Los esquemas de indexación hasta ahora descritos implican un fichero de índice ordenado. Al índice se le aplica una búsqueda binaria para localizar los punteros a un bloque de disco o a un registro (o registros) del fichero que tiene un valor de campo de índice específico. Una búsqueda binaria requiere aproximadamente $(\log_2 b_i)$ accesos a bloques para un índice con b_i bloques, porque cada paso del algoritmo reduce por un factor de 2 la parte del fichero de índice que continuamos explorando. Por eso, tomamos la función log en base 2. La idea

Tabla 14.2. Propiedades de los tipos de índice.

Tipo de índice	Número de entradas de índice (primer nivel)	Denso no denso	Bloque de anclaje en el fichero de datos
Principal	Número de bloques en el fichero de datos	No denso	Sí
Agrupado	Número de valores del campo de índice distintos	No denso	Sí/no ^a
Secundario (clave)	Número de registros del fichero de datos	Denso	No
Secundario (no clave)	Número de registros ^b o número de valores del campo de índice distintos ^c	Denso o no denso	No

^a Sí, si cada valor distinto del campo de ordenación inicia un nuevo bloque; no, en cualquier otro caso.

^b Para la opción 1.

^c Para las opciones 2 y 3.

que hay detrás de un **índice multinivel** es reducir la parte del índice que continuamos explorando por bfr_i , el factor de bloqueo para el índice, que es mayor que 2. Por tanto, el espacio de búsqueda se reduce mucho más rápidamente. El valor bfr_i se denomina **fan-out** del índice multinivel, al que nos referiremos mediante el símbolo **fo**. La búsqueda en un índice multinivel requiere aproximadamente $(\log_{fo} b_i)$ accesos a bloques, que es un número más pequeño que para la búsqueda binaria si el **fan-out** es mayor que 2.

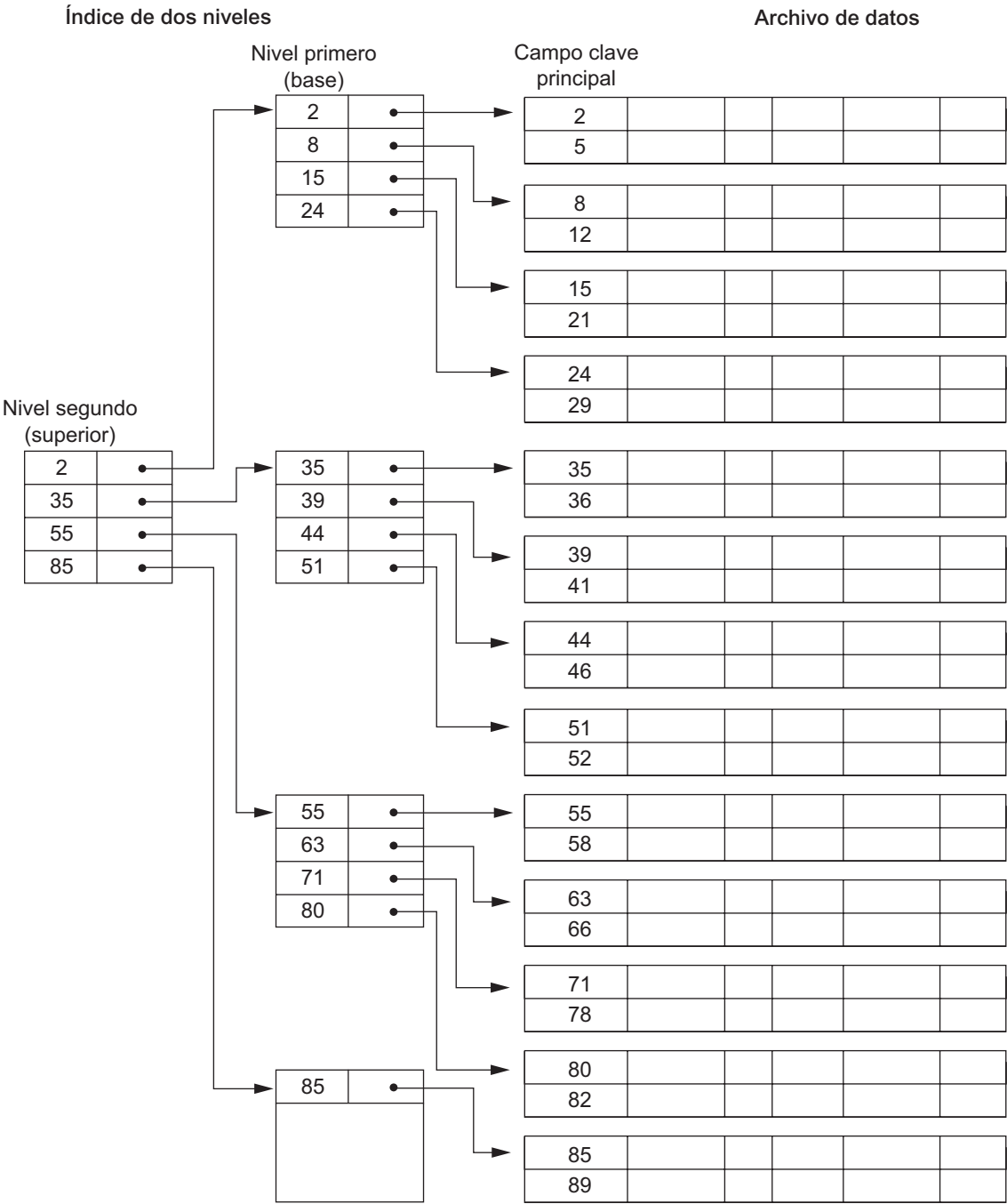
Un índice multinivel considera el fichero de índice, al que ahora nos referiremos como **primer nivel** (o **base**) de un índice multinivel, como un *fichero ordenado* con un *valor distinto* por cada $K(i)$. Por consiguiente, podemos crear un índice principal para el primer nivel; este índice al primer nivel se denomina **segundo nivel** del índice multinivel. Como el segundo nivel es un índice principal, podemos utilizar anclas de bloque de modo que el segundo nivel tenga una sola entrada por *cada bloque* del primer nivel. El factor de bloqueo bfr_i para el segundo nivel (y para todos los niveles subsiguientes) es el mismo que para el índice de primer nivel porque todas las entradas del índice tienen el mismo tamaño; cada una con un valor de campo y una dirección de bloque. Si el primer nivel tiene r_1 entradas, y el factor de bloqueo (que también es el **fan-out**) para el índice es $bfr_i = fo$, entonces el primer nivel necesita $\lceil r_1/fo \rceil$ bloques, que es por consiguiente el número de entradas r_2 necesarias en el segundo nivel del índice.

Podemos repetir este proceso para el segundo nivel. El **tercer nivel**, que es un índice principal para el segundo nivel, tiene una entrada por cada bloque de segundo nivel, por lo que el número de entradas de tercer nivel es $r_3 = \lceil r_2/fo \rceil$. Observe que necesitamos un segundo nivel sólo si el primer nivel necesita más de un bloque de almacenamiento en disco y, de forma parecida, requerimos un tercer nivel sólo si el segundo nivel necesita más de un bloque. Podemos repetir el proceso anterior hasta que todas las entradas del mismo nivel del índice t encajen en un solo bloque. Este bloque en el nivel t se denomina nivel de índice **superior**.⁴ Cada nivel reduce el número de entradas en el nivel previo por un factor de fo (el índice **fan-out**) por lo que podemos utilizar la fórmula $1 \leq (r_1/(fo)^t)$ para calcular t . Por tanto, un índice multinivel con r_1 entradas de primer nivel tendrá aproximadamente t niveles, donde $t = \lceil (\log_{fo}(r_1)) \rceil$.

El esquema multinivel aquí descrito puede utilizarse en cualquier tipo de índice, sea principal, agrupado o secundario (siempre y cuando el índice de primer nivel tenga *valores distintos para $K(i)$ y entradas de longitud fija*). La Figura 14.6 muestra un índice multinivel construido sobre un índice principal. El Ejemplo 3 ilustra la mejora en el número de bloques accedidos cuando se utiliza un índice multinivel para buscar un registro.

⁴ El esquema de numeración aquí utilizado para los niveles del índice es lo contrario a como normalmente se definen los niveles para las estructuras de datos en árbol. En estas últimas, t es como el nivel 0 (cero), $t-1$ es el nivel 1, etcétera.

Figura 14.6. Índice principal de dos niveles que se parece a la organización ISAM.



Ejemplo 3. Supongamos que el índice secundario denso del Ejemplo 2 se convierte en un índice multinivel. Calculamos el factor de bloqueo del índice como $bfr_i = 68$ entradas de índice por bloque, que también es el *fan-out* fo para el índice multinivel; también hemos calculado el número de bloques de primer nivel $b_1 = 442$ bloques. El número de bloques de segundo nivel será $b_2 = \lceil (b_1/fo) \rceil = \lceil (442/68) \rceil = 7$ bloques, y el número

de bloques de tercer nivel será $b_3 = \lceil (b_2/f_0) \rceil = \lceil (7/68) \rceil = 1$ bloque. Por tanto, el tercer nivel es el nivel superior del índice, y $t = 3$. Para acceder a un registro buscando en el índice multinivel, debemos acceder a un bloque en cada nivel más un bloque del fichero de datos, por lo que necesitamos $t + 1 = 3 + 1 = 4$ accesos a bloques. Compare esto con el Ejemplo 2, donde necesitábamos 10 accesos a bloques cuando utilizábamos un índice de un solo nivel y una búsqueda binaria.

Observe que también podríamos tener un índice principal multinivel, que no sería denso. El Ejercicio 14.14(c) ilustra este caso, donde *debemos* acceder al bloque de datos del fichero antes de poder determinar si el registro buscado está en el fichero. En el caso de un índice denso, esto puede determinarse accediendo al primer nivel del índice (sin tener que acceder a un bloque de datos), puesto que hay una entrada de índice por *cada* registro del fichero.

Una organización de fichero bastante común en el procesamiento de datos empresarial es un fichero ordenado con un índice principal multinivel en su campo clave de ordenación. Una organización de este tipo se conoce como **fichero secuencial indexado** y se utilizó en un buen número de los antiguos sistemas IBM.

La inserción se realiza con alguna forma de fichero de desbordamiento que se combina periódicamente con el fichero de datos. El índice se vuelve a crear durante la reorganización del fichero. La organización **ISAM** de IBM incorpora un índice de dos niveles que está estrechamente relacionado con la organización del disco. El primer nivel es un índice de cilindro, que tiene el valor clave de un registro ancla por cada cilindro del paquete de discos y un puntero al índice de pista para el cilindro. El índice de pista tiene el valor clave de un registro de ancla por cada pista del cilindro y un puntero a la pista. Después se puede buscar secuencialmente en la pista el registro o bloque deseado.

El Algoritmo 14.1 esboza el procedimiento de búsqueda de un registro en un fichero de datos que utiliza un índice principal multinivel no denso con t niveles. Nos referimos a la entrada i del nivel j del índice como $\langle K_j(i), P_j(i) \rangle$, y buscamos un registro cuyo valor de clave principal es K . Asumimos que se ignoran los registros desbordados. Si el registro está en el fichero, debe haber alguna entrada a nivel 1 con $K_1(i) \leq K < K_1(i+1)$ y el registro estará en el bloque del fichero de datos cuya dirección es $P_1(i)$. El Ejercicio 14.19 explica la modificación del algoritmo de búsqueda para otros tipos de índices.

Algoritmo 14.1. Buscando un índice principal multinivel no denso con t niveles.

```

 $p \leftarrow$  dirección del bloque de nivel superior del índice;
for  $j \leftarrow t$  step  $-1$  to 1 do
    begin
        leer el bloque de índice (en el nivel de índice número  $j$ ) cuya dirección es  $p$ ;
        buscar bloque  $p$  para la entrada  $i$  tal que  $K_j(i) \leq K < K_j(i+1)$  (if  $K_j(i)$ 
            es la última entrada del bloque, es suficiente satisfacer  $K_j(i) \leq K$ );
         $p \leftarrow P_j(i)$  (* elegir el puntero apropiado en el nivel de índice  $j$  *)
    end;
leer el bloque del fichero de datos cuya dirección es  $p$ ;
buscar en el bloque  $p$  el registro con la clave  $= K$ ;

```

Como hemos visto, un índice multinivel reduce el número de accesos a bloques cuando se busca un registro, dado el valor de su campo de indexación. Todavía nos enfrentamos con los problemas de tratar con las inserciones y los borrados en el índice, porque todos los niveles de índice son *ficheros ordenados físicamente*. Para mantener los beneficios de utilizar la indexación multinivel a la vez que se reducen los problemas de la inserción y la eliminación en el índice, los diseñadores adoptaron un índice multinivel denominado **índice multinivel dinámico** que deja algo de espacio en cada uno de sus bloques para insertar nuevas entradas. Esto se implementa normalmente utilizando estructuras de datos denominadas árboles B y árboles B⁺, que describimos en la siguiente sección.

14.3 Índices multinivel dinámicos utilizando árboles B y B⁺

Los árboles B y B⁺ son casos especiales de estructuras de datos en forma de árbol bien conocidas, de cuya terminología hacemos una introducción. Un **árbol** está formado por **nodos**. Cada nodo del árbol, excepto el nodo especial denominado **raíz**, tiene un nodo **padre** y varios (ninguno o más) nodos **hijo**. El nodo raíz no tiene padre. Un nodo que no tiene ningún nodo hijo se llama nodo **hoja**; un nodo que no es hoja es un nodo **interno**. El **nivel** de un nodo siempre es uno más que el nivel de su padre, siendo cero el nivel del nodo raíz.⁵ Un **subárbol** de un nodo consta de ese nodo y de todos sus nodos **descendientes** (sus nodos hijo, los nodos hijo de sus nodos hijo, etcétera). Una definición recursiva precisa de subárbol es que consta de un nodo n y de los subárboles de todos los nodos hijo de n . La Figura 14.7 ilustra una estructura de datos en árbol. En esta figura, el nodo raíz es A, y sus nodos hijo son B, C y D. Los nodos E, J, C, G, H y K son nodos hoja.

Normalmente, mostramos un árbol con el nodo raíz en la parte superior (véase la Figura 14.7). Una forma de implementar un árbol es con tantos punteros en cada nodo como nodos hijo tiene ese nodo. En algunos casos, también se almacena un puntero padre en cada nodo. Además de los punteros, un nodo normalmente contiene algún tipo de información almacenada. Cuando un índice multinivel se implementa como una estructura en árbol, esa información incluye los valores del campo de indexación del fichero que se utiliza para guiar la búsqueda de un determinado registro.

En la Sección 14.3.1 introducimos los árboles de búsqueda y después explicamos los árboles B, que pueden utilizarse como índices multinivel dinámicos para guiar la búsqueda de los registros en un fichero de datos. Los nodos de un árbol B se mantienen llenos entre un 50% y un 100%, y los punteros a los bloques de datos se almacenan tanto en los nodos internos como en los nodos hoja de la estructura de árbol B. En la Sección 14.3.2 explicamos los árboles B⁺, una variación de los árboles B en los que los punteros a los bloques de datos de un fichero se almacenan únicamente en los nodos hoja, lo que puede llevar a índices de menos niveles y mayor capacidad.

14.3.1 Árboles de búsqueda y árboles B

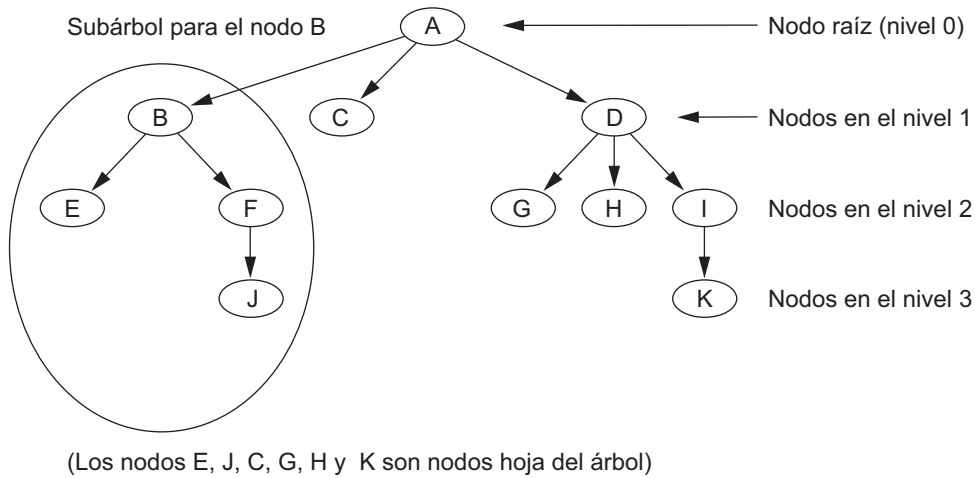
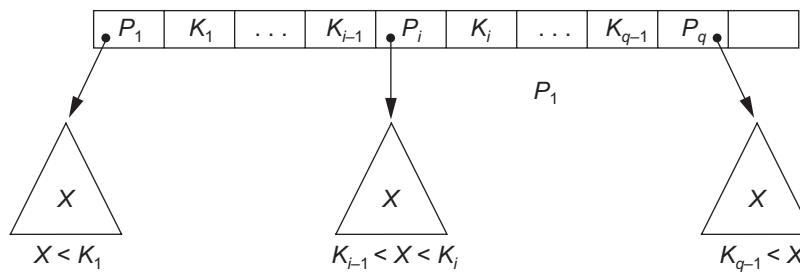
Un árbol de búsqueda es un tipo especial de árbol que sirve para guiar la búsqueda de un registro, dado el valor de uno de sus campos. Los índices multinivel explicados en la Sección 14.2 pueden imaginarse como una variante de un árbol de búsqueda; cada nodo del índice multinivel puede tener hasta f_o punteros y f_o valores clave, donde f_o es el índice *fan-out*. Los valores del campo de índice de cada nodo nos guía hasta el siguiente nodo, hasta que alcanzamos el bloque del fichero de datos que contiene el registro deseado. Siguiendo un puntero, restringimos nuestra búsqueda en cada nivel a un subárbol del árbol de búsqueda e ignoramos todos los nodos que no están en este subárbol.

Árboles de búsqueda. Un árbol de búsqueda es ligeramente distinto a un índice multinivel. Un **árbol de búsqueda** de orden p es un árbol tal que cada nodo contiene, a lo sumo, $p-1$ valores de búsqueda y p punteros en el orden $\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$, donde $q \leq p$; cada P_i es un puntero a un nodo hijo (o puntero NULL o nulo); y cada K_i es un valor de búsqueda proveniente de algún conjunto ordenado de valores. Se supone que todos los valores de búsqueda son únicos.⁶ La Figura 14.8 ilustra un nodo de un árbol de búsqueda. Un árbol de búsqueda debe cumplir, en todo momento, las siguientes restricciones:

1. Dentro de cada nodo, $K_1 < K_2 < \dots < K_{q-1}$

⁵ Esta definición estándar del nivel de un nodo de un árbol, que utilizamos a lo largo de la Sección 14.3, es diferente de la que ofrecimos para los índices multinivel en la Sección 14.2.

⁶ Esta restricción se puede relajar algo. Si el índice se encuentra en un campo que no es clave, pueden existir valores de búsqueda duplicados y pueden modificarse la estructura de nodos y las reglas de navegación del árbol.

Figura 14.7. Estructura de datos en árbol que muestra un árbol desequilibrado.**Figura 14.8.** Nodo de un árbol de búsqueda con punteros a los subárboles que tiene por debajo.

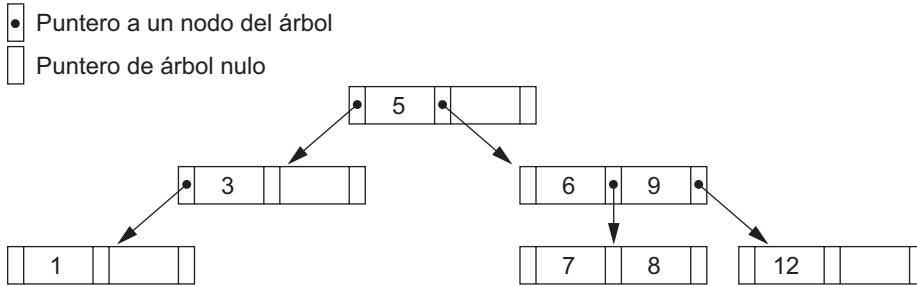
- Para todos los valores de X del subárbol al cual apunta P_i , tenemos que $K_{i-1} < X < K_i$ para $1 < i < q$; $X < K_i$ para $i = 1$; y $K_{i-1} < X$ para $i = q$ (véase la Figura 14.8).

Siempre que busquemos un valor X , seguimos el puntero P_i apropiado, de acuerdo con las fórmulas de la condición 2. La Figura 14.9 ilustra un árbol de búsqueda de orden $p = 3$ y valores de búsqueda enteros. Observe que algunos de los punteros P_i de un nodo pueden ser punteros nulos.

Podemos usar un árbol de búsqueda como mecanismo para buscar registros almacenados en un fichero de disco. Los valores del árbol pueden ser los valores de uno de los campos del registro, el llamado **campo de búsqueda** (que es el mismo que el campo de índice si un índice multinivel guía la búsqueda). Cada valor clave del árbol está asociado a un puntero de registro que tiene ese valor en el fichero de datos. Como alternativa, puede apuntar al bloque de disco que contiene ese registro. El árbol de búsqueda en sí puede almacenarse en disco, asignando cada nodo del árbol a un bloque del disco. Cuando se inserta un registro nuevo, es preciso actualizar el árbol de búsqueda incluyendo en él una entrada con el valor del campo de búsqueda del nuevo registro y un puntero a éste.

Para insertar valores de búsqueda en el árbol y eliminarlos, sin violar las dos restricciones anteriores, se utilizan algoritmos que, por regla general, no garantizan que el árbol de búsqueda esté equilibrado (es decir, que todas las hojas estén al mismo nivel).⁷ El árbol de la Figura 14.7 no está equilibrado porque tiene nodos hoja en los niveles 1, 2 y 3. Es importante mantener equilibrados los árboles de búsqueda porque esto garantiza

⁷ La definición de *equilibrado* es diferente para los árboles binarios. Los árboles binarios equilibrados se conocen como árboles AVL.

Figura 14.9. Árbol de búsqueda de orden $p = 3$.

que no habrá nodos en niveles muy profundos que requieran muchos accesos a bloques durante una búsqueda. Los árboles equilibrados procuran una velocidad de búsqueda uniforme independientemente del valor de la clave de búsqueda. Además, las eliminaciones de registros pueden hacer que queden nodos casi vacíos, con lo que hay un desperdicio de espacio importante y un aumento en el número de niveles. El árbol B hace frente a estos dos problemas especificando restricciones adicionales en el árbol de búsqueda.

Árboles B. El árbol B tiene restricciones adicionales que garantizan que el árbol siempre estará equilibrado y que el espacio desperdiciado por la eliminación, si lo hay, nunca será excesivo. Los algoritmos para insertar y eliminar son más complejos para poder mantener estas restricciones. No obstante, la mayor parte de las inserciones y eliminaciones son procesos simples que se complican sólo en circunstancias especiales (por ejemplo, siempre que intentamos una inserción en un nodo que ya está lleno o una eliminación de un nodo que está a menos de la mitad de su capacidad). De manera más formal, un **árbol B** de **orden p** , utilizado como estructura de acceso según un *campo clave* para buscar registros en un fichero de datos, se puede definir de este modo:

1. Cada nodo interno del árbol B (véase la Figura 14.10[a]) tiene la forma:

$$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle$$

donde $q \leq p$. Cada P_i es un **puntero de árbol** (un puntero a otro nodo del árbol B). Cada Pr_i es un **puntero de datos**⁸ (un puntero al registro cuyo valor del campo clave de búsqueda es igual a K_i [o al bloque del fichero de datos que contiene ese registro]).

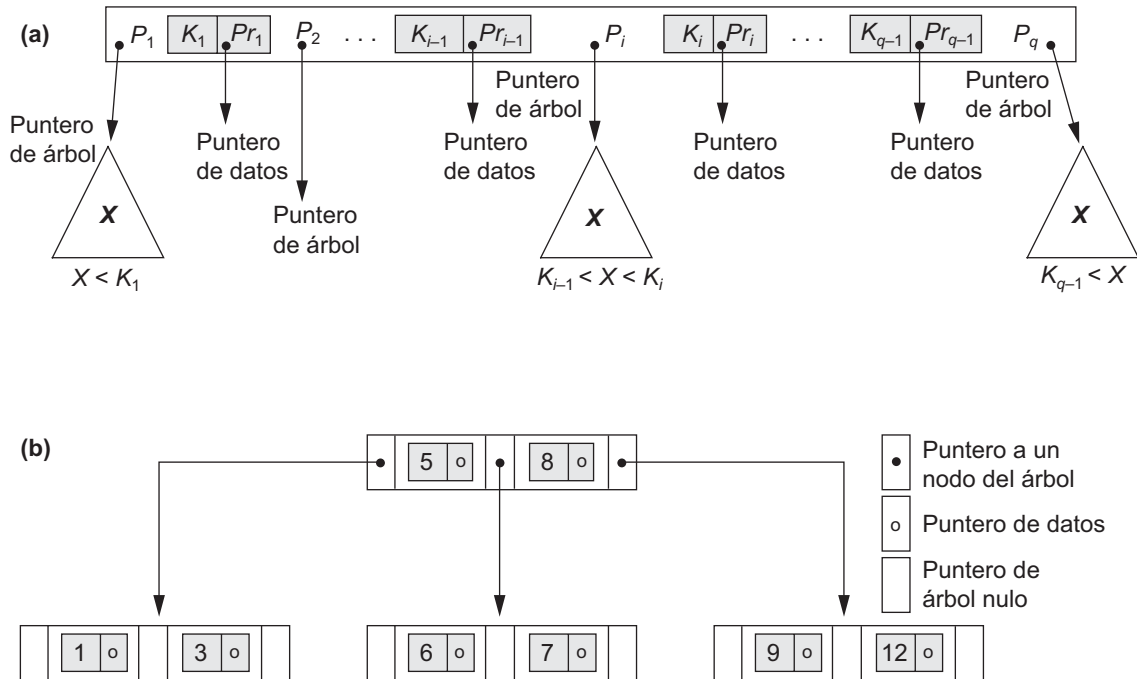
2. Dentro de cada nodo, $K_1 < K_2 < \dots < K_{q-1}$.
3. Para todos los valores del campo clave de búsqueda X del subárbol al que apunta P_i (el subárbol i ; véase la Figura 14.10[a]), tenemos:

$$K_{i-1} < X < K_i \text{ para } 1 < i < q; X < K_i \text{ para } i = 1; \text{ y } K_{i-1} < X \text{ para } i = q.$$

4. Cada nodo tiene a lo sumo p punteros de árbol.
5. Cada nodo, excepto los nodos raíz y hoja, tienen por lo menos $\lceil (p/2) \rceil$ punteros de árbol. El nodo raíz tiene como mínimo dos punteros de árbol, a menos que sea el único nodo del árbol.
6. Un nodo con q punteros de árbol, $q \leq p$, tiene $q - 1$ valores del campo clave de búsqueda (y, por tanto, tiene $q - 1$ punteros de datos).
7. Todos los nodos hoja se encuentran en el mismo nivel. Los nodos hoja tienen la misma estructura que los internos, excepto que todos sus *punteros de árbol* P_i son nulos.

⁸ Un puntero de datos es una dirección de bloque, o una dirección de registro; este último es esencialmente una dirección de bloque y un desplazamiento de registro dentro del bloque.

Figura 14.10. Estructuras de árbol B. (a) Nodo A en un árbol B con $q - 1$ valores de búsqueda. (b) Un árbol B de orden $p = 3$. Los valores se insertaron en este orden: 8, 5, 1, 7, 3, 12, 9, 6.



La Figura 14.10(b) ilustra un árbol B de orden $p = 3$. Observe que todos los valores de búsqueda K del árbol B son únicos porque asumimos que el árbol se utiliza como una estructura de acceso en un campo clave. Si utilizamos un árbol B en un campo que no es clave, debemos cambiar la definición de los punteros de fichero Pr_i para que apunten a un bloque (o grupo de bloques) que contienen los punteros a los registros del fichero. Este nivel adicional de indirección es parecido a la Opción 3, explicada en la Sección 14.1.3, para los índices secundarios.

Un árbol B empieza con un solo nodo raíz (que también es un nodo hoja) en el nivel 0 (cero). Una vez que el nodo raíz está lleno con $p - 1$ valores de clave de búsqueda e intentamos insertar otra entrada en el árbol, el nodo raíz se divide en dos nodos en el nivel 1. En el nodo raíz sólo queda el valor central, y el resto de valores se dividen uniformemente entre los otros dos nodos. Cuando un nodo no raíz está lleno y se inserta en él una entrada nueva, dicho nodo se divide en otros dos al mismo nivel, y la entrada central se mueve al nodo padre junto con dos punteros a los nuevos nodos divididos. Si el nodo padre está lleno, también es dividido. La división puede propagarse hasta llegar al nodo raíz, creando un nuevo nivel si se divide la raíz. Aquí no explicamos en profundidad los algoritmos para los árboles B; en cambio, en la siguiente sección hacemos una introducción de los procedimientos de búsqueda e inserción para los árboles B⁺.

Si la eliminación de un valor provoca que un nodo no llegue a la mitad de su capacidad, es combinado con sus nodos vecinos, algo que también puede propagarse hasta la raíz. Por tanto, la eliminación puede reducir el número de niveles del árbol. Con ayuda de análisis y simulaciones se ha demostrado que, después de numerosas inserciones y eliminaciones aleatorias en un árbol B, los nodos quedan ocupados aproximadamente al 69% de su capacidad cuando se estabiliza el número de valores en el árbol. Esto también es cierto con los árboles B⁺. Si ocurre esto, la división y combinación de nodos sólo se dará raramente, por lo que la inserción y la eliminación son muy eficaces. Si el número de valores aumenta, el árbol se expandirá sin problemas (aunque podría darse la división de nodos, de modo que algunas inserciones tardarían algo más de tiempo). El Ejemplo 4 ilustra el cálculo del orden p de un árbol B almacenado en disco.

Ejemplo 4. Supongamos que el campo de búsqueda tiene una longitud de $V = 9$ bytes, que el tamaño del bloque de disco es $B = 512$ bytes, que un puntero de registro (datos) es $P_r = 7$ bytes, y que un puntero de bloque es $P = 6$ bytes. Cada nodo del árbol B puede tener como máximo p punteros de árbol, $p - 1$ punteros de datos, y $p - 1$ valores de campo clave de búsqueda (véase la Figura 14.10[a]). Deben encajar en un solo bloque de disco si cada nodo del árbol B corresponde a un bloque de disco. Por tanto, tenemos:

$$(p * P) + ((p - 1) * (P_r + V)) \leq B$$

$$(p * 6) + ((p - 1) * (7 + 9)) \leq 512$$

$$(22 * p) \leq 528$$

Podemos elegir que p sea un valor mayor que satisfaga la desigualdad anterior, que da $p = 23$ (no se elige $p = 24$ debido a las razones que exponemos a continuación).

En general, un nodo de un árbol B puede contener información adicional que los algoritmos que manipulan el árbol necesitan, como el número de entradas q en el nodo y un puntero al nodo padre. Por tanto, antes de que hagamos el cálculo precedente para p , debemos reducir el tamaño del bloque para que pueda entrar dicha información. A continuación veremos cómo calcular el número de bloques y niveles para un árbol B.

Ejemplo 5. Supongamos que el campo de búsqueda del Ejemplo 4 es un campo clave desordenado, y que construimos un árbol B en ese campo. Los nodos del árbol B están al 60% de su capacidad. Cada nodo, por término medio, tendrá $p * 0,69 = 23 * 0,69$ o aproximadamente 16 punteros y, por tanto, 15 valores de campo clave de búsqueda. El *fan-out medio* es $fo = 16$. Podemos empezar en la raíz y ver cuántos valores y punteros pueden existir, por término medio, en cada nivel subsiguiente:

Raíz:	1 nodo	15 entradas	16 punteros
Nivel 1:	16 nodos	240 entradas	256 punteros
Nivel 2:	256 nodos	3.840 entradas	4.096 punteros
Nivel 3:	4.096 nodos	61.440 entradas	

En cada nivel calculamos el número de entradas multiplicando el número total de punteros del nivel anterior por 15, la cantidad media de entradas de cada nodo. Por tanto, para un tamaño de bloque, tamaño de puntero y tamaño del campo clave de búsqueda dados, un árbol B de dos niveles almacena $3.840 + 240 + 15 = 4.095$ entradas por término medio; un árbol B de tres entradas alberga 65.535 entradas por término medio.

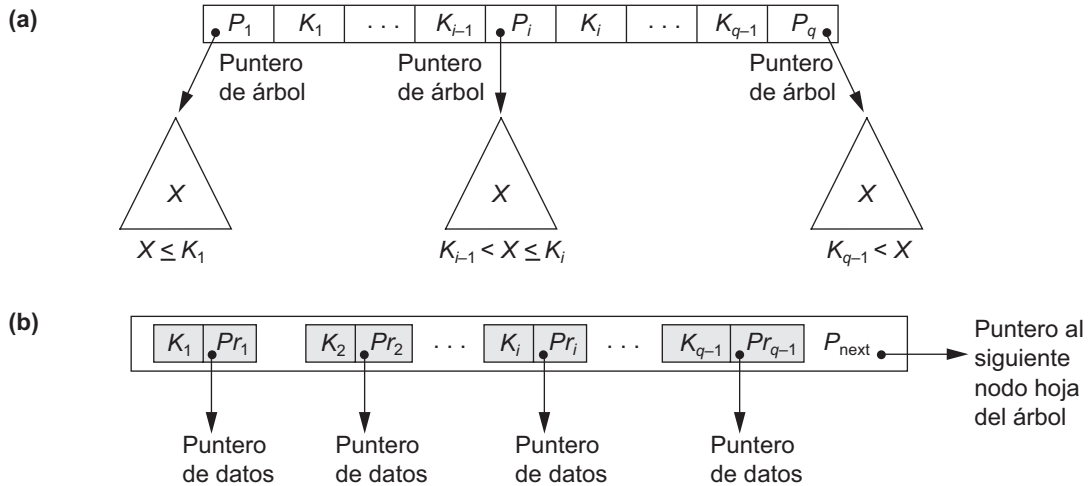
Los árboles B se utilizan a veces como organizaciones de fichero principales. En este caso, dentro de los nodos del árbol B se guardan los registros enteros, en lugar de entradas <clave de búsqueda, puntero de registro>. Esto funciona bien para ficheros con un *número relativamente pequeño de registros* y un *tamaño pequeño de registro*. En caso contrario, el *fan-out* y el número de niveles se hacen demasiado grandes para permitir un acceso eficaz.

En resumen, los árboles B proporcionan una estructura de acceso multinivel que es una estructura de árbol equilibrado en la que cada nodo está, como mínimo, medio lleno. Cada nodo de un árbol B de orden p puede tener como máximo $p - 1$ valores de búsqueda.

14.3.2 Árboles B+

La mayoría de las implementaciones de un índice multinivel dinámico utilizan una variante de la estructura de datos en árbol B denominada **árbol B⁺**. En un árbol B, cada valor del campo de búsqueda aparece una vez en algún nivel del árbol, junto con un puntero de datos. En un árbol B⁺ los punteros a datos se almacenan *sólo en los nodos hoja* del árbol, por lo cual, la estructura de los nodos hoja difiere de la de los nodos internos. Los nodos hoja tienen una entrada por *cada valor* del campo de indexación, junto con un puntero al registro (o al bloque que contiene ese registro) si el campo de búsqueda es un campo clave. En el caso de un campo de búsqueda que no es clave, el puntero apunta a un bloque que contiene punteros a los registros del fichero de datos, creándose un nivel extra de indirección.

Figura 14.11. Nodos de un árbol B⁺. (a) Nodo interno de un árbol B⁺ con $q - 1$ valores de búsqueda. (b) Nodo hoja de un árbol B⁺ con $q - 1$ valores de búsqueda y $q - 1$ punteros de datos.



Los nodos hoja de un árbol B⁺ normalmente están enlazados para ofrecer un acceso ordenado a los registros a través del campo de búsqueda. Estos nodos hoja son parecidos al primer nivel (nivel base) de un índice. Los nodos internos del árbol B⁺ corresponden a los demás niveles de un índice multinivel. Algunos valores del campo de búsqueda de los nodos hoja se *repiten* en los nodos internos del árbol B⁺ con el fin de guiar la búsqueda. La estructura de los *nodos internos* de un árbol B⁺ de orden p (véase la Figura 14.11[a]) es la siguiente:

1. Cada nodo interno tiene esta forma:

$$\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$$

donde $q = p$ y cada P_i es un **puntero de árbol**.

2. Dentro de cada nodo interno, $K_1 < K_2 < \dots < K_{q-1}$.
3. Para todos los valores X del campo de búsqueda en el subárbol al cual apunta P_i , tenemos que $K_{i-1} < X \leq K_i$ para $1 < i < q$; $X \leq K_i$ para $i = 1$; y $K_{i-1} < X$ para $i = q$ (véase la Figura 14.11[a]).⁹
4. Cada nodo interno tiene, a lo sumo, p punteros de árbol.
5. Cada nodo interno, excepto el raíz, tiene al menos $\lceil (p/2) \rceil$ punteros de árbol. El nodo raíz tiene como mínimo dos punteros de árbol si es un nodo interno.
6. Un nodo interno con q punteros, $q \leq p$, tiene $q - 1$ valores del campo de búsqueda.

La estructura de los *nodos hoja* del árbol B⁺ de orden p (véase la Figura 14.11[b]) es la siguiente:

1. Cada nodo hoja tiene la siguiente forma:

$$\langle \langle K_1, Pr_1 \rangle, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_{sig} \rangle$$

donde $q \leq p$, cada Pr_i es un puntero de datos, y P_{sig} apunta al siguiente *nodo hoja* del árbol B⁺.

2. Dentro de cada nodo hoja, $K_1 \leq K_2 \dots, K_{q-1}$, $q \leq p$.
3. Cada Pr_i es un **puntero a datos** que apunta al registro cuyo valor del campo de búsqueda es K_i o a un bloque del fichero que contiene el registro (o a un bloque de punteros de registro que apuntan a los registros cuyo valor del campo de búsqueda es K_i si el campo de búsqueda no es una clave).

⁹ Nuestra definición obedece a Knuth (1973). Un árbol B⁺ puede definirse de forma diferente intercambiando los símbolos $<$ y $\leq$ ($K_{i-1} \leq X < K_i$; $K_{q-1} \leq X$), pero los principios siguen siendo los mismos.

4. Cada nodo hoja tiene como mínimo $\lceil (p/2) \rceil$ valores.
5. Todos los nodos hoja están en el mismo nivel.

Los punteros en los nodos internos son *punteros de árbol* a bloques que son nodos del árbol, mientras que los punteros de los nodos hoja son *punteros de datos* a registros o bloques del fichero (excepto para el puntero P_{sig} , que es un puntero de árbol al siguiente nodo hoja). Empezando por el nodo hoja más a la izquierda, es posible atravesar los nodos hoja como una lista enlazada, utilizando los punteros P_{sig} . Esto ofrece un acceso ordenado a los registros de datos del campo de indexación. También podemos incluir un puntero $P_{anterior}$. Para un árbol B^+ en un campo no clave, se necesita un nivel de indirección adicional parecido al de la Figura 14.5, de tal manera que los punteros P_r son punteros de bloque a bloques que contienen un conjunto de punteros de registro a los registros actuales del fichero de datos, como explicamos en la Opción 3 de la Sección 14.1.3.

Como las entradas de los *nodos internos* de un árbol B^+ incluyen valores de búsqueda y punteros de árbol sin punteros a los datos, es posible empaquetar más entradas en un nodo interno de un árbol B^+ que en un árbol B similar. Por tanto, si el tamaño de bloque (nodo) es el mismo, el orden p será mayor para el árbol B^+ que para el árbol B , como se ilustra en el Ejemplo 6. Esto puede reducir el número de niveles del árbol B^+ , mejorándose así el tiempo de búsqueda. Como las estructuras de los nodos internos y de los nodos hoja de los árboles B^+ son diferentes, su orden p puede ser diferente. Utilizaremos p para denotar el orden de los *nodos internos* y p_{hoja} para denotar el orden de los *nodos hoja*, que definimos como el número máximo de punteros a datos en un nodo hoja.

Ejemplo 6. Para calcular el orden p de un árbol B^+ , vamos a suponer que el campo clave de búsqueda tiene una longitud de $V = 9$ bytes, que el tamaño del bloque es de $B = 512$ bytes, que un puntero de registro es de $Pr = 7$ bytes, y que un puntero de bloque es de $P = 6$ bytes, como en el Ejemplo 4. Un nodo interno del árbol B^+ puede tener hasta p punteros de árbol y $p - 1$ valores del campo de búsqueda; éstos deben encajar en un solo bloque. Por tanto, tenemos:

$$\begin{aligned}(p * P) + ((p - 1) * V) &\leq B \\(p * 6) + ((p - 1) * 9) &\leq 512 \\(15 * p) &\leq 521\end{aligned}$$

Podemos elegir p para que sea el valor más grande que satisfaga la desigualdad anterior, que da $p = 34$. Se trata de un valor mayor que el obtenido para un árbol B (23), lo que ofrece un *fan-out* más grande y más entradas en cada nodo interno de un árbol B^+ que en el correspondiente árbol B . Los nodos hoja del árbol B^+ tendrán el mismo número de valores y punteros, excepto que los punteros son punteros a datos y un puntero “siguiente”. Por tanto, el orden p_{hoja} para los nodos hoja puede calcularse de este modo:

$$\begin{aligned}(p_{hoja} * (Pr + V)) + P &\leq B \\(p_{hoja} * (7 + 9)) + 6 &\leq 512 \\(16 * p_{hoja}) &\leq 506\end{aligned}$$

Resulta que cada nodo hoja puede albergar hasta $p_{hoja} = 31$ combinaciones de puntero valor/datos, asumiendo que los punteros de datos son punteros de registro.

Como con el árbol B , necesitamos información adicional (para implementar los algoritmos de inserción y eliminación) en cada nodo. Esta información puede incluir el tipo de nodo (interno u hoja), el número actual de entradas q en el nodo y punteros a los nodos padre y hermanos. Por tanto, antes de que hagamos los cálculos anteriores para p y p_{hoja} , debemos reducir el tamaño del bloque en la cantidad de espacio necesario para toda esta información. El siguiente ejemplo ilustra cómo podemos calcular el número de entradas en un árbol B^+ .

Ejemplo 7. Supongamos que construimos un árbol B^+ en el campo del Ejemplo 6. Para calcular el número aproximado de entradas del árbol B^+ , asumimos que cada nodo está ocupado al 69%. Por término medio, cada nodo interno tendrá $34 * 0,69$ o aproximadamente 23 punteros, y por tanto 22 valores. Cada nodo hoja, por

término medio, albergará $0,69 * p_{\text{hoja}} = 0,69 * 31$ o aproximadamente 21 punteros a registros de datos. Un árbol B⁺ tendrá por término medio la siguiente cantidad de entradas en cada nivel:

Raíz:	1 nodo	22 entradas	23 punteros
Nivel 1:	23 nodos	506 entradas	529 punteros
Nivel 2:	529 nodos	11.638 entradas	12.167 punteros
Nivel hoja:	12.167 nodos	255.507 punteros a registro de datos	

Para un tamaño de bloque, tamaño de puntero y tamaño de campo de búsqueda dados, un árbol B⁺ de tres niveles alberga hasta 255.507 punteros de registro, por término medio. Compare esto con las 65.535 entradas para el árbol B correspondiente del Ejemplo 5.

Búsqueda, inserción y eliminación con árboles B⁺. El Algoritmo 14.2 muestra el procedimiento utilizando el árbol B⁺ como estructura de acceso para buscar un registro. El Algoritmo 14.3 ilustra el procedimiento para insertar un registro en un fichero con un árbol B⁺ como estructura de acceso. Estos algoritmos asumen la existencia de un campo clave de búsqueda, y deben modificarse apropiadamente para el caso de un árbol B⁺ en un campo no clave. Ilustramos la inserción y la eliminación con un ejemplo.

Algoritmo 14.2. Búsqueda de un registro con un valor K del campo clave de búsqueda, utilizando un árbol B⁺.

```

 $n \leftarrow$  bloque que contiene la raíz del árbol B+;
leer el bloque  $n$ ;
while ( $n$  no sea un nodo hoja del árbol B+) do
    begin
         $q \leftarrow$  número de punteros de árbol en el nodo  $n$ ;
        if  $K \leq n.K_1$  (* $n.K_i$  se refiere al valor del campo de búsqueda  $i$  en el nodo  $n$ *)
            then  $n \leftarrow n.P_1$  (* $n.P_i$  se refiere al puntero de árbol  $i$  en el nodo  $n$ *)
        else if  $K > n.K_{q-1}$ 
            then  $n \leftarrow n.P_q$ 
        else begin
            buscar el nodo  $n$  para una entrada  $i$  tal que  $n.K_{i-1} < K \leq n.K_i$ ;
             $n \leftarrow n.P_i$ 
        end;
    leer el bloque  $n$ 
    end;
buscar en el bloque  $n$  la entrada ( $K_i, Pr_i$ ) con  $K = K_i$ ; (* buscar nodo hoja *)
if encontrado
    then leer el bloque del fichero de datos con la dirección  $Pr_i$  y recuperar el registro
    else el registro con el valor de campo de búsqueda  $K$  no está en el fichero de datos;
```

Algoritmo 14.3. Inserción de un registro con un valor de campo clave de búsqueda K en un árbol B⁺ de orden p .

```

 $n \leftarrow$  bloque que contiene la raíz del árbol B+;
leer el bloque  $n$ ; establecer la pila  $S$  a vacío;
while ( $n$  no sea un nodo hoja del árbol B+) do
    begin
        push la dirección de  $n$  a la pila  $S$ ;
        (*la pila  $S$  alberga los nodos padre necesarios en el caso de división*)
         $q \leftarrow$  número de punteros de árbol en el nodo  $n$ ;
```

```

if  $K \leq n.K_1$  (* $n.K_i$  se refiere al valor del campo de búsqueda  $i$  en el nodo  $n$ *)
  then  $n \leftarrow n.P_1$  (* $n.P_i$  se refiere al puntero de árbol  $i$  en el nodo  $n$ *)
  else if  $K > n.K_{q-1}$ 
    then  $n \leftarrow n.P_q$ 
  else begin
    buscar el nodo  $n$  para una entrada  $i$  tal que  $n.K_{i-1} < K \leq n.K_i$ ;
     $n \leftarrow n.P_i$ 
  end;
  leer el bloque  $n$ 
end;
  buscar en el bloque  $n$  la entrada  $(K, Pr_i)$  con  $K = K_i$ ; (*buscar el nodo hoja  $n$ *)
  if encontrado
    then el registro ya existe en el fichero; no se puede insertar
    else (*insertar la entrada en el árbol  $B^+$  para apuntar al registro*)
      begin
        crear la entrada  $(K, Pr)$  donde  $Pr$  apunta al registro nuevo;
        if el nodo hoja  $n$  no está lleno
          then insertar la entrada  $(K, Pr)$  en la posición correcta en el nodo hoja  $n$ 
          else begin (*el nodo hoja  $n$  está lleno con  $p_{\text{hoja}}$  punteros de registro; se divide*)
            copiar  $n$  en  $temp$  (* $temp$  es un nodo hoja muy grande para almacenar la entrada extra*);
            insertar la entrada  $(K, Pr)$  en la posición correcta de  $temp$ ;
            (* $temp$  alberga ahora  $p_{\text{hoja}} + 1$  entradas en forma de  $(K_i, Pr_i)$ *)
             $nuevo \leftarrow$  un nuevo nodo hoja vacío para el árbol;  $nuevo.P_{\text{sig}} \leftarrow n.P_{\text{sig}}$ ;
             $j \leftarrow \lceil (p_{\text{hoja}} + 1)/2 \rceil$ ;
             $n \leftarrow$  primeras  $j$  entradas de  $temp$  (hasta la entrada  $(K_j, Pr_j)$ );  $n.P_{\text{sig}} \leftarrow nuevo$ ;
             $nuevo \leftarrow$  entradas restantes en  $temp$ ;  $K \leftarrow K_j$ ;
            (*ahora debemos mover  $(K, nuevo)$  e insertar en el nodo padre interno; sin embargo, si el padre está lleno, la división se puede propagar*)
            finalizado  $\leftarrow$  falso;
            repeat
              if la pila  $S$  está vacía
                then (*no hay nodo padre; se crea un nuevo nodo raíz para el árbol*)
                  begin
                     $root \leftarrow$  un nuevo nodo interno vacío para el árbol;
                     $root \leftarrow \langle n, K, nuevo \rangle$ ; finalizado  $\leftarrow$  verdad;
                  end
                else begin
                   $n \leftarrow$  pop pila  $S$ ;
                  if nodo interno  $n$  no está lleno
                    then
                      begin (*el nodo padre no está lleno; no hay división*)
                        insertar  $(K, nuevo)$  en la posición correcta en el nodo interno  $n$ ;
                        finalizado  $\leftarrow$  verdad
                      end
                    else begin (*el nodo interno  $n$  está lleno con  $p$  punteros de árbol; hay división*)
                        copiar  $n$  en  $temp$  (* $temp$  es un nodo interno bastante grande*);
                        insertar  $(K, nuevo)$  en  $temp$  en la posición correcta;
                        (* $temp$  tiene ahora  $p + 1$  punteros de árbol*)

```

```

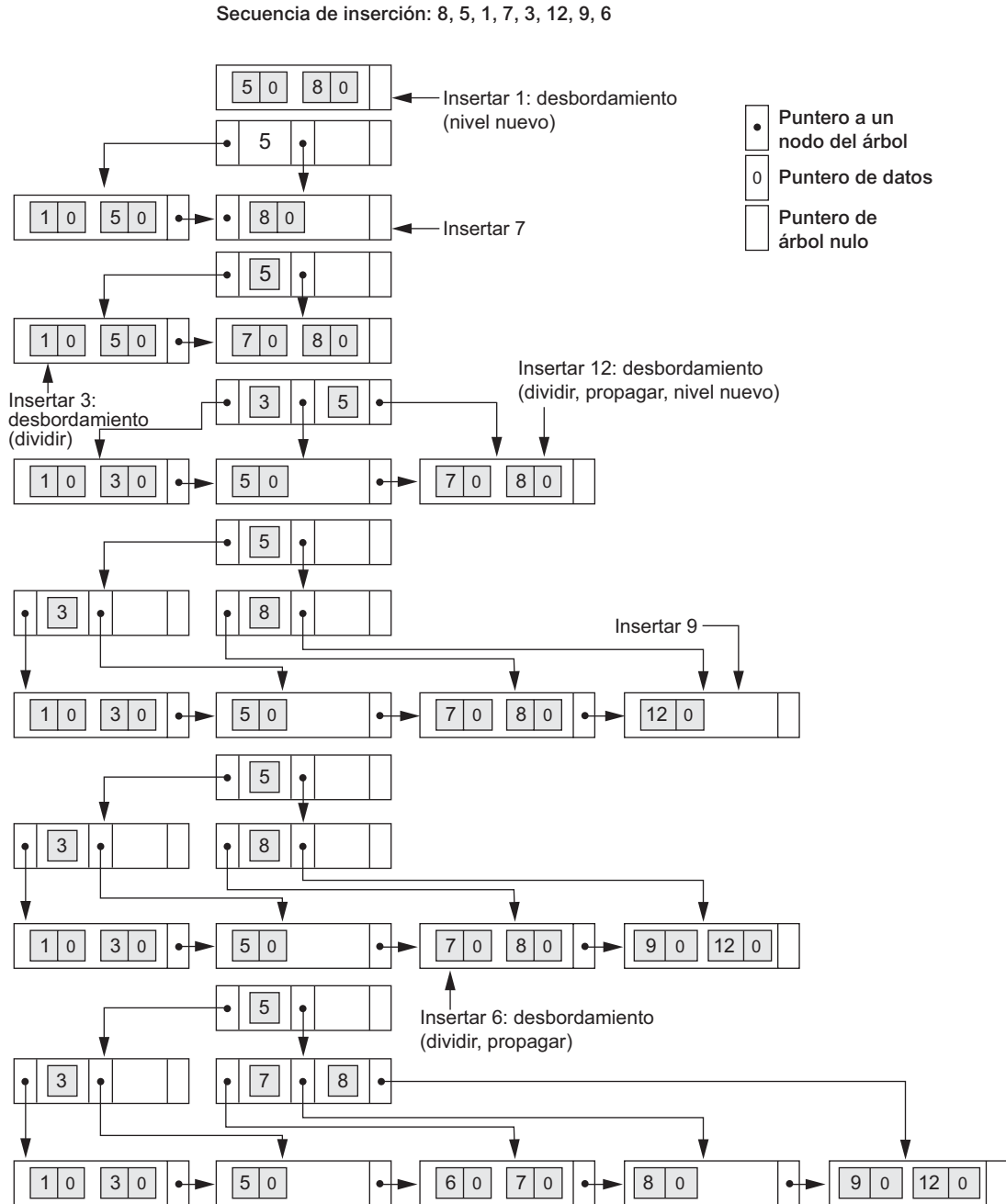
    nuevo ← un nuevo nodo interno vacío para el árbol;
     $j \leftarrow \lceil ((p + 1)/2) \rceil$ ;
     $n \leftarrow$  entradas hasta el puntero de árbol  $P_j$  en  $temp$ ;
    (* $n$  contiene  $\langle P_1, K_1, P_2, K_2, \dots, P_{j-1}, K_{j-1}, P_j \rangle$ *)
    nuevo ← entradas del puntero de árbol  $P_{j+1}$  en  $temp$ ;
    (* $nuevo$  contiene  $\langle P_{j+1}, K_{j+1}, \dots, K_{p-1}, P_p, K_p, P_{p+1} \rangle$ *)
     $K \leftarrow K_j$ 
    (*ahora debemos mover  $(K, nuevo)$  e insertar en el nodo interno
    padre*)
  end
end
until finalizado
end;
end;
```

La Figura 14.12 ilustra la inserción de registros en un árbol B⁺ de orden $p = 3$ y $p_{\text{hoja}} = 2$. En primer lugar, observamos que la raíz es el único nodo del árbol, por lo que también es un nodo hoja. En cuanto se crea más de un nivel, el árbol se divide en nodos internos y nodos hoja. Observe que *cada valor clave debe existir en el nivel hoja*, porque todos los punteros de datos están en el nivel hoja. Sin embargo, en los nodos internos sólo existen algunos valores para guiar la búsqueda. Además, cada valor que aparece en un nodo interno también aparece como *el valor más a la derecha* en el nivel hoja del subárbol apuntado por el puntero de árbol a la izquierda del valor.

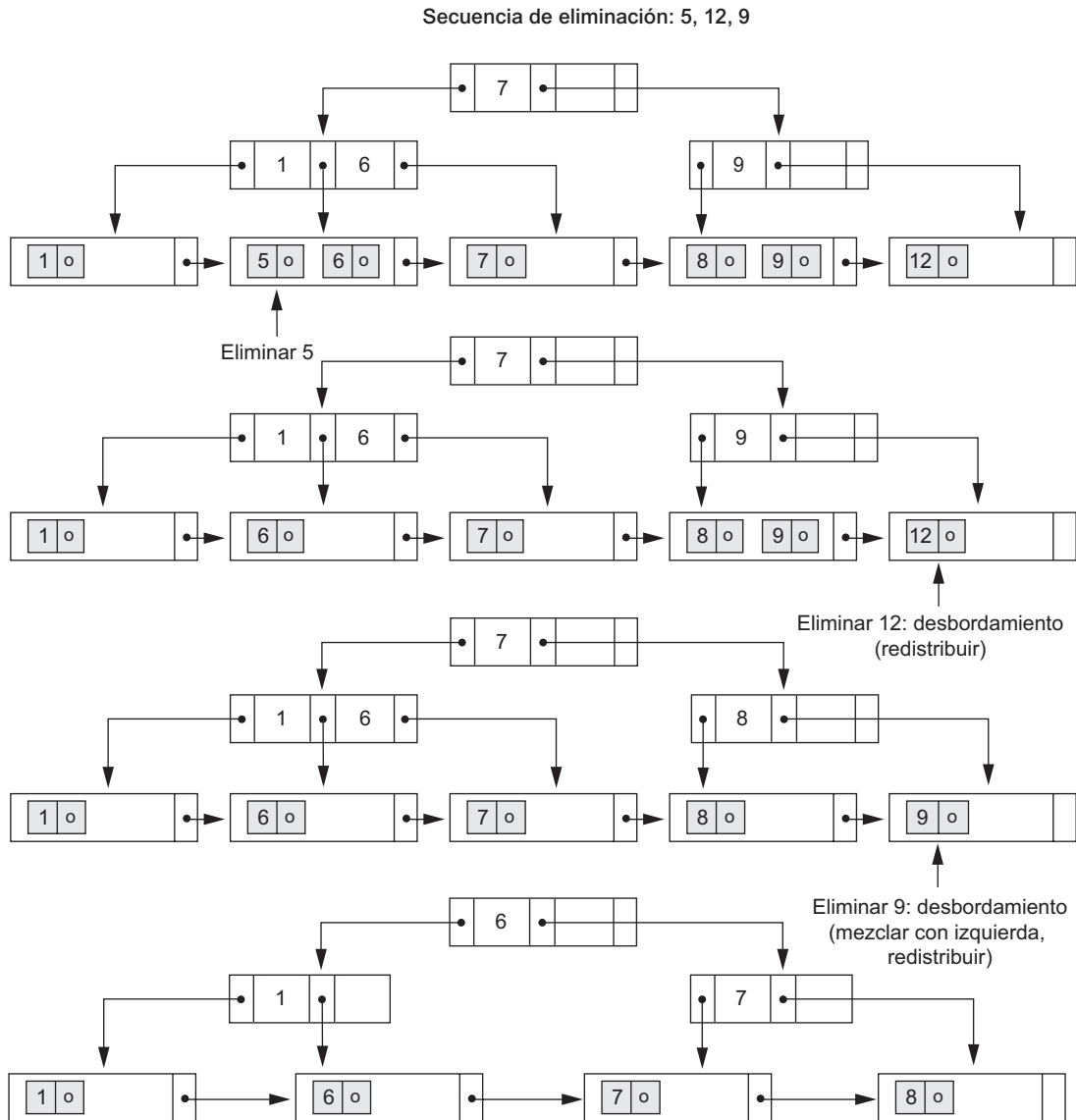
Cuando un *nodo hoja* está lleno y se inserta en él una entrada nueva, el nodo se desborda y debe dividirse. Las primeras $j = \lceil ((p_{\text{hoja}} + 1)/2) \rceil$ entradas del nodo original se mantienen, mientras que el resto se mueve a un nuevo nodo hoja. El valor de búsqueda en la posición j se duplica en el nodo interno padre, y en éste se crea un puntero extra al nodo nuevo. Deben insertarse en el nodo padre en su secuencia correcta. Si el nodo interno padre está lleno, el valor nuevo también provocará su desbordamiento, por lo que deberá dividirse. Las entradas del nodo interno hasta P_j (el puntero de árbol número j después de insertar el valor y el puntero nuevos, donde $j = \lceil ((p + 1)/2) \rceil$) se mantienen, mientras que el valor de búsqueda número j se mueve al padre, no se duplica. Un nuevo nodo interno albergará las entradas desde P_{j+1} hasta el final de las entradas del nodo (consulte el Algoritmo 14.3). Esta división puede propagarse hasta crear un nuevo nodo raíz y, por tanto, un nuevo nivel para el árbol B⁺.

La Figura 14.13 ilustra la eliminación en un árbol B⁺. Cuando se borra una entrada, siempre se elimina del nivel hoja. Si ocurre en un nodo interno, también debe eliminarse de allí. En este último caso, el valor a su izquierda en el nodo hoja debe sustituirse en el nodo interno porque este valor es ahora la entrada más a la derecha en el subárbol. La eliminación puede provocar un **desbordamiento por abajo**, al reducirse el número de entradas del nodo hoja por debajo del mínimo requerido. En este caso, intentamos encontrar un nodo hoja hermano (un nodo hoja situado inmediatamente a la izquierda o a la derecha del nodo desbordado por debajo) y redistribuir las entradas entre el nodo y su **hermano**, de modo que ambos estén rellenos, como mínimo, a la mitad; en caso contrario, el nodo se combina con sus hermanos y se reduce el número de nodos hoja. Un método común es intentar **redistribuir** las entradas con el hermano de la izquierda; si no es posible, se realiza un intento de redistribución con el hermano de la derecha. Si tampoco esto es posible, los tres nodos se combinan en dos nodos hoja. En tal caso, el desbordamiento por debajo se puede propagar a los nodos **internos** porque se necesitan un valor de búsqueda y un puntero de árbol menos. Esto puede propagarse y reducir los niveles del árbol.

La implementación de los algoritmos de inserción y eliminación puede requerir punteros padre y hermano para cada nodo, o el uso de una pila como la del Algoritmo 14.3. Cada nodo también debe incluir el número de entradas que tiene y su tipo (hoja o interno). Otra alternativa es implementar la inserción y la eliminación como procedimientos recursivos.

Figura 14.12. Ejemplo de inserción en un árbol B^+ con $p = 3$ y $p_{\text{hoja}} = 2$.

Variaciones de los árboles B y B^+ . Para concluir esta sección, mencionaremos brevemente algunas variantes de los árboles B y B^+ . En algunos casos, la restricción 5 del árbol B (o del árbol B^+), que requiere que cada nodo esté lleno como mínimo hasta la mitad, puede modificarse para que cada nodo esté lleno a sus dos tercios. En este caso, el árbol B se denomina **árbol B^*** . En general, algunos sistemas permiten al usuario elegir un **factor de relleno** entre 0,5 y 1,0, donde esto último significa que los nodos del árbol B (índice) están completamente llenos. También es posible especificar dos factores de relleno para un árbol B^+ : uno para el

Figura 14.13. Ejemplo de eliminación en un árbol B⁺.

nivel hoja y otro para los nodos internos del árbol. Cuando se construye el índice al principio, los nodos se rellenan hasta aproximadamente los factores de relleno especificados. Recientemente, los investigadores han sugerido relajar el requisito de que un nodo esté lleno hasta la mitad, y en su lugar permiten que un nodo se quede completamente vacío antes de combinarse, simplificándose así el algoritmo de eliminación. Los estudios de simulación muestran que esto no desperdicia demasiado espacio adicional bajo inserciones y eliminaciones aleatoriamente distribuidas.

14.4 Índices en claves múltiples

Hasta ahora, en nuestra explicación hemos asumido que las claves principal y secundaria con las que se accede a los ficheros son atributos sencillos (campos). En muchas solicitudes de recuperación y actualización, son

varios los atributos implicados. Si se utiliza con frecuencia una cierta combinación de atributos, es recomendable configurar una estructura de acceso para ofrecer un acceso eficaz por medio de un valor clave que sea una combinación de esos atributos.

Por ejemplo, piense en un fichero EMPLEADO que contiene los atributos Dno (número de departamento), Edad, Calle, Ciudad, CodPostal, Sueldo y CodExperiencia, con Dni (documento nacional de identidad) como clave. Considere esta consulta: *Listar los empleados del departamento número 4 cuya edad sea 59*. Ni Dno ni Edad son atributos clave, lo que significa que un valor de búsqueda para cualquiera de ellos apuntará a varios registros. Podemos considerar las siguientes estrategia de búsqueda alternativas:

1. Asumiendo que Dno tiene un índice, pero no así Edad, acceder a los registros con $Dno = 4$ utilizando el índice y después seleccionar de entre ellos aquellos registros que satisfacen la condición $Edad = 59$.
2. Como alternativa, si Edad está indexado pero no así Dno, acceder a los registros con $Edad = 59$ utilizando el índice y después seleccionar de entre ellos aquellos registros que satisfacen la condición $Dno = 4$.
3. Si se han creado índices tanto en Dno como en Edad, se pueden utilizar los dos; cada uno ofrece un conjunto de registros o un conjunto de punteros (a bloques o registros). Una intersección de estos conjuntos de registros o punteros proporciona los registros que satisfacen las dos condiciones, los punteros de registro que satisfacen las dos condiciones, o los bloques en los que se encuentran los registros que satisfacen las dos condiciones.

Todas estas alternativas proporcionan al final el resultado correcto. No obstante, si el conjunto de registros que satisface cada condición por separado ($Dno = 4$ o $Edad = 59$) es grande, y sólo unos cuantos registros satisfacen la condición combinada, entonces ninguna de las anteriores es una técnica eficaz para la solicitud de búsqueda dada. Existen varias posibilidades que tratarían la combinación $\langle Dno, Edad \rangle$ o $\langle Edad, Dno \rangle$ como una clave de búsqueda compuesta por varios atributos. Perfilaremos brevemente estas técnicas a continuación. Nos referiremos a las claves que contienen varios atributos como **claves compuestas**.

14.4.1 Índice ordenado por varios atributos

Todo lo explicado hasta ahora todavía es aplicable si creamos un índice en un campo clave de búsqueda que es una combinación de $\langle Dno, Edad \rangle$. En el ejemplo anterior, la clave de búsqueda es un par de valores, $\langle 4, 59 \rangle$. En general, si creamos un índice sobre los atributos $\langle A_1, A_2, \dots, A_n \rangle$, los valores de la clave de búsqueda son tuplas con n valores: $\langle v_1, v_2, \dots, v_n \rangle$.

Una ordenación lexicográfica de los valores de esta tupla establece un orden en esta clave de búsqueda compuesta. Para nuestro ejemplo, todas las claves de departamento para el departamento número 3 preceden a las del departamento número 4. Por tanto, $\langle 3, n \rangle$ precede a $\langle 4, m \rangle$ para cualquier valor de m y n . El orden de clave ascendente para las claves con $Dno = 4$ sería $\langle 4, 18 \rangle$, $\langle 4, 19 \rangle$, $\langle 4, 20 \rangle$, etcétera. La ordenación lexicográfica funciona de forma parecida para ordenar las cadenas de caracteres. Un índice en una clave compuesta por n atributos funciona de forma parecida a cualquier índice de los explicados hasta ahora en este capítulo.

14.4.2 Dispersión partida

La dispersión partida (o subdividida) es una extensión de la dispersión externa estática (Sección 13.8.2) que permite el acceso sobre varias claves. Sólo resulta adecuada para comparaciones de igualdad; las consultas de rango no están soportadas. En la dispersión partida, para una clave compuesta por n componentes, la función de dispersión está diseñada para producir un resultado con n direcciones de dispersión separadas. La dirección de cubo es una concatenación de estas n direcciones. Entonces es posible buscar por la clave de búsqueda compuesta requerida buscando los cubos adecuados que coinciden con las partes de la dirección en la que estamos interesados.

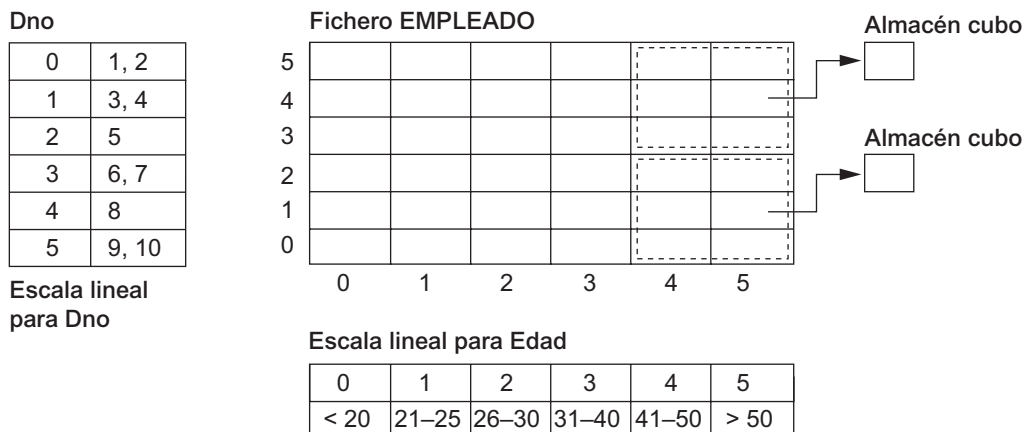
Por ejemplo, considere la clave de búsqueda compuesta <Dno, Edad>. Si Dno y Edad están dispersas en una dirección de 3 bits y 5 bits respectivamente, obtenemos una dirección de cubo de 8 bits. Suponga que Dno = 4 tiene una dirección de dispersión '100' y que Edad = 59 tiene una dirección de dispersión '10101'. Entonces, para buscar el valor de búsqueda combinado, Dno = 4 y Edad = 59, acudimos a la dirección de cubo 100 10101; para buscar todos los empleados con una Edad = 59, se buscarán todos los cubos (ocho de ellos) cuyas direcciones son '000 10101', '001 10101', ..., etcétera. Una ventaja de la dispersión partida es que puede extenderse fácilmente a cualquier número de atributos. Las direcciones de cubo pueden diseñarse para que los bits de orden superior de las direcciones correspondan a los atributos a los que se accede con más frecuencia. Además, no es necesario mantener estructuras de acceso separadas para los atributos individuales. El principal inconveniente de la dispersión partida es que no puede manipular consultas de rango sobre cualquiera de los atributos componente.

14.4.3 Ficheros rejilla

Otra alternativa es reorganizar el fichero EMPLEADO como un fichero rejilla. Si queremos acceder a un fichero con dos claves, como Dno y Edad en nuestro ejemplo, podemos construir un array rejilla con una escala (o dimensión) lineal por cada uno de los atributos de búsqueda. La Figura 14.14 muestra un array de rejilla para el fichero EMPLEADO con una escala lineal para Dno y otra para el atributo Edad. Las escalas se hacen en cierto modo para lograr una distribución uniforme de ese atributo. Así, en nuestro ejemplo, vemos que la escala lineal para Dno tiene Dno = 1, 2 combinados como un valor 0 en la escala, mientras que Dno = 5 corresponde al valor 2 en esa escala. De forma parecida, Edad se divide en su escala de 0 a 5 agrupando edades a fin de distribuir los empleados uniformemente por edad. El array rejilla mostrado para este fichero tiene un total de 36 celdas. Cada celda apunta a alguna dirección de cubo donde se almacenan los registros correspondientes a esa celda. La Figura 14.14 también muestra la asignación de celdas a cubos (sólo parcialmente).

Por tanto, nuestra solicitud de Dno = 4 y Edad = 59 se mapea dentro de la celda (1, 5) correspondiente al array rejilla. Los registros para esta combinación se encontrarán en el cubo correspondiente. Este método es particularmente útil para consultas de rango que se mapearían en un conjunto de celdas correspondiente a un grupo de valores a lo largo de las escalas lineales. Conceptualmente, el concepto de fichero rejilla puede aplicarse a cualquier número de claves de búsqueda. Para n claves de búsqueda, el array rejilla tendría n dimensiones. El array rejilla permite así una partición del fichero a lo largo de las dimensiones de los atributos clave de búsqueda y proporciona un acceso mediante combinaciones de valores a lo largo de esas dimensiones. Los ficheros rejilla funcionan bien al reducir el tiempo de acceso por clave múltiple. Sin embargo, representan un

Figura 14.14. Ejemplo de un array rejilla sobre los atributos Dno y Edad.



gasto de espacio debido a la estructura en rejilla. Además, con los ficheros dinámicos, se añade una reorganización frecuente del fichero al coste de mantenimiento.¹⁰

14.5 Otros tipos de índices

14.5.1 Uso de la dispersión y otras estructuras de datos como índices

También es posible crear estructuras de acceso parecidas a índices basándose en la *dispersión*. Las entradas de índice $\langle K, Pr \rangle$ (o $\langle K, P \rangle$) pueden organizarse como un fichero de dispersión ampliable dinámicamente, utilizando una de las técnicas explicadas en la Sección 13.8.3; la búsqueda de una entrada utiliza el algoritmo de búsqueda dispersa sobre K . Una vez encontrada una entrada, se utiliza el puntero Pr (o P) para encontrar el registro correspondiente en el fichero de datos. También es posible utilizar otras estructuras de búsqueda como índices.

14.5.2 Índices lógicos frente a índices físicos

Hasta ahora, hemos asumido que las entradas de índice $\langle K, Pr \rangle$ (o $\langle K, P \rangle$) siempre incluyen un puntero físico Pr (o P) que especifica la dirección física del registro en el disco, a modo de número de bloque y desplazamiento. Es lo que a veces se denomina índice físico, y tiene el inconveniente de que el puntero debe modificarse si el registro se mueve a otra ubicación dentro del disco. Por ejemplo, suponga que la organización principal de un fichero está basada en la dispersión lineal o extensible; entonces, cada vez que un cubo se divide, algunos registros son asignados a cubos nuevos y, por tanto, cambian sus direcciones físicas. Si había un índice secundario en el fichero, tendrían que buscarse y actualizarse los punteros a esos registros, que es una tarea compleja.

Para remediar esta situación, podemos utilizar una estructura denominada índice lógico, cuyas entradas tienen la forma $\langle K, Kp \rangle$. Cada entrada tiene un valor K para el campo de indexación secundario coincidente con el valor Kp del campo utilizado para la organización primaria del fichero. Buscando en el índice secundario con el valor de K , un programa puede localizar el valor correspondiente de Kp y utilizarlo para acceder al registro a través de la organización primaria del fichero. Los índices lógicos introducen así un nivel adicional de indirección entre la estructura de acceso y los datos. Se utilizan cuando se espera que las direcciones físicas de los registros cambien con frecuencia. El coste de esta indirección es una búsqueda extra basada en la organización primaria del fichero.

14.5.3 Explicación

En muchos sistemas, un índice no es parte integral del fichero de datos, pero puede crearse y descartarse dinámicamente. Es por lo que a menudo se le denomina *estructura de acceso*. Siempre que esperemos acceder con frecuencia a un fichero basándonos en alguna condición de búsqueda que implique un campo en particular, podemos solicitar al DBMS que cree un índice con ese campo. Normalmente, se crea un índice secundario para evitar la ordenación física de los registros en el fichero de datos en disco.

La principal ventaja de los índices secundarios es que (al menos teóricamente) pueden crearse en combinación con *prácticamente cualquier organización primaria de registros*. Por tanto, un índice secundario podría utilizarse para complementar otros métodos de acceso principales, como la ordenación o la dispersión, o incluso podría utilizarse con ficheros mixtos. Para crear un índice secundario de árbol B^+ en algún campo de un

¹⁰ Los algoritmos de inserción/eliminación para los ficheros rejilla pueden encontrarse en Nievergelt (1984).

fichero, debemos pasar por todos los registros del fichero para crear las entradas en el nivel hoja del árbol. Estas entradas se ordenan y rellenan entonces de acuerdo con el factor de relleno especificado; a la vez, se crean los otros niveles del índice. Es más costoso y mucho más duro crear índices primarios o principales e índices agrupados dinámicamente, porque los registros del fichero de datos deben ordenarse físicamente en el disco según el orden del campo de indexación. No obstante, algunos sistemas permiten a los usuarios crear dinámicamente esos índices en sus ficheros ordenando el fichero durante la creación del índice.

Es común utilizar un índice para implementar una *restricción de clave* en un atributo. Mientras se busca en el índice para insertar un nuevo registro, es fácil comprobar al mismo tiempo si otro registro del fichero (y, por tanto, del árbol del índice) tiene el mismo atributo clave que el registro nuevo. Si es así, la inserción puede rechazarse.

Un fichero que tiene un índice secundario en cada uno de sus campos se denomina a veces **fichero totalmente invertido**. Como todos los índices son secundarios, los registros nuevos se insertan al final del fichero; por consiguiente, el propio fichero de datos es un fichero desordenado (*heap*). Los índices normalmente se implementan como árboles B^+ , por lo que se actualizan dinámicamente para reflejar la inserción y la eliminación de registros. Algunos DBMSs comerciales, como ADABAS de Software-AG, utilizan mucho este método.

En la Sección 14.2 nos referimos a la popular organización de fichero de IBM denominada ISAM. Otro método de IBM, **VSAM (Método de acceso de almacenamiento virtual, Virtual Storage Access Method)**, es algo parecido a la estructura de acceso de árbol B^+ .

14.6 Resumen

En este capítulo presentamos las organizaciones de ficheros que implican estructuras de acceso adicionales, denominadas índices, para mejorar la eficacia de la recuperación de los registros de un fichero de datos. Estas estructuras de acceso pueden utilizarse *en combinación con* las organizaciones de fichero primarias explicadas en el Capítulo 13, que se utilizan para organizar los registros del fichero en el disco.

Hemos visto tres tipos de índices ordenados de un solo nivel: primario o principal, agrupado y secundario. Cada índice se especifica en un campo del fichero. Los índices primario y agrupado se construyen sobre el campo de ordenación física de un fichero, mientras que los índices secundarios se especifican sobre los campos desordenados. El campo para un índice principal también debe ser una clave del fichero, mientras que es un campo no clave para un índice agrupado. Un índice de un solo nivel es un fichero ordenado y se explora utilizando una búsqueda binaria. Hemos visto la construcción de índices multinivel para mejorar la eficacia de la búsqueda en un índice.

A continuación, vimos cómo pueden implementarse índices multinivel como árboles B y B^+ , que son estructuras dinámicas que permiten que un índice se expanda o se reduzca dinámicamente. Los nodos (bloques) de estas estructuras de índice se mantienen rellenos entre el 50% y el 100% de su capacidad gracias a los algoritmos de inserción y eliminación. Los nodos se estabilizan finalmente y por término medio cuando su ocupación ronda el 69%, lo que ofrece espacio para realizar inserciones sin tener que reorganizar el índice en la mayoría de los casos. Los árboles B^+ pueden mantener generalmente más entradas en sus nodos internos que los árboles B , por lo que pueden tener menos niveles o albergar menos entradas que el árbol B equivalente.

También hemos ofrecido una visión general de los métodos de acceso por clave múltiple, así como de la construcción de un índice basándonos en estructuras de datos dispersos. Después ofrecimos la introducción del concepto de índice lógico, y lo comparamos con los índices físicos que describimos anteriormente. Por último, explicamos cómo podemos utilizar combinaciones de estas organizaciones. Por ejemplo, los índices secundarios se utilizan con frecuencia con índices mixtos, así como con ficheros desordenados y ordenados. También es posible crear índices secundarios para los ficheros dispersos y los ficheros dispersos dinámicos.

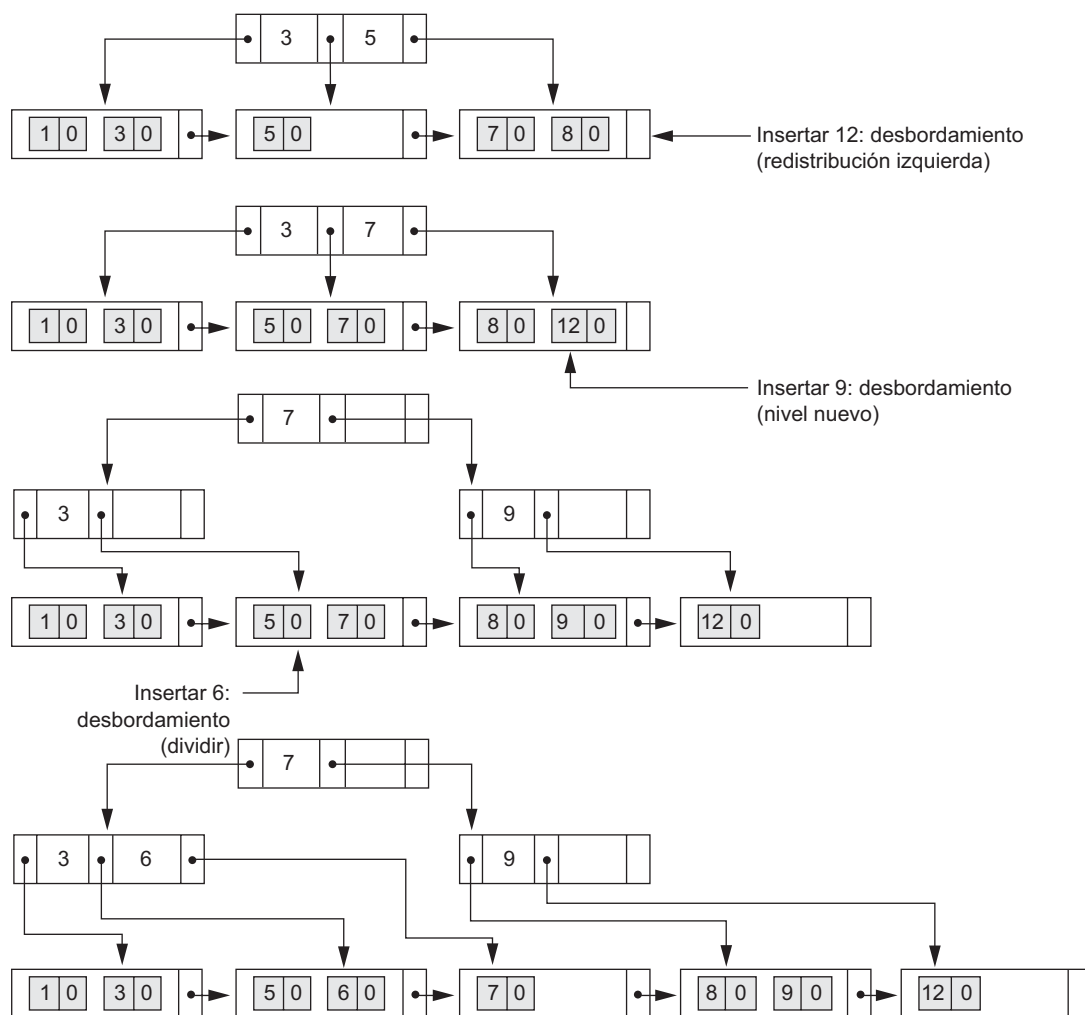
Preguntas de repaso

- 14.1. Defina los siguientes términos: campo de indexación, campo clave principal, campo agrupado, campo clave secundario, ancla de bloque, índice denso e índice no denso (escaso).
- 14.2. ¿Cuáles son las diferencias entre índice principal, secundario y agrupado? ¿Cómo afectan estas diferencias a la forma en que se implementan estos índices? ¿Cuáles son índices densos, y cuáles no?
- 14.3. ¿Por qué en un fichero podemos tener a lo sumo un índice principal o agrupado, pero varios índices secundarios?
- 14.4. ¿Cómo mejora la eficacia de búsqueda de un fichero de índice la indexación multinivel?
- 14.5. ¿Cuál es el orden p de un árbol B ? Describa la estructura de los nodos de un árbol B .
- 14.6. ¿Cuál es el orden p de un árbol B^+ ? Describa la estructura de los nodos internos y hoja de un árbol B^+ .
- 14.7. ¿Cómo difiere un árbol B de un árbol B^+ ? ¿Por qué se prefiere normalmente un árbol B^+ como estructura para acceder a un fichero de datos?
- 14.8. Explique las alternativas que existen para acceder a un fichero basándose en claves de búsqueda múltiples.
- 14.9. ¿Qué es la dispersión partida? ¿Cómo funciona? ¿Cuáles son sus limitaciones?
- 14.10. ¿Qué es un fichero rejilla? ¿Cuáles son sus ventajas y sus inconvenientes?
- 14.11. Muestre un ejemplo de construcción de un array rejilla sobre dos atributos de algún fichero.
- 14.12. ¿Qué es un fichero totalmente invertido? ¿Qué es un fichero secuencial indexado?
- 14.13. ¿Cómo puede utilizarse la dispersión para construir un índice? ¿Cuál es la diferencia entre un índice lógico y un índice físico?

Ejercicios

- 14.14. Considere un disco con un tamaño de bloque de $B = 512$ bytes. Un puntero de bloque tiene una longitud de $P = 6$ bytes, y un puntero de registro tiene una longitud de $P_R = 7$ bytes. Un fichero tiene $r = 30.000$ registros EMPLEADO de *longitud fija*. Cada registro tiene los siguientes campos: Nombre (30 bytes), Dni (9 bytes), CodDpto (9 bytes), Direcc (40 bytes), Tlf (9 bytes), FechaNac (8 bytes), Sexo (1 byte), CodTrabajo (4 bytes), Sueldo (4 bytes, número real). Se utiliza un byte adicional como marcador de eliminación.
 - a. Calcule el tamaño de registro R en bytes.
 - b. Calcule el factor de bloqueo bfr y el número de bloques de fichero b , asumiendo una organización no extendida.
 - c. Suponga que el fichero está *ordenado* por el campo clave Dni y que queremos construir un *índice principal* sobre Dni. Calcule (i) el factor de bloqueo de índice bfr_i (que también es el índice *fan-out* fo); (ii) el número de entradas de índice de primer nivel y el número de bloques de índice de primer nivel; (iii) el número de niveles necesarios si lo convertimos en un índice multinivel; (iv) el número total de bloques que el índice multinivel necesita; y (v) el número de accesos a bloques que se necesitan para buscar y recuperar un registro del fichero (dado el valor de su Dni) utilizando el índice principal.
 - d. Suponga que el fichero *no está ordenado* por el campo clave Dni y queremos construir un *índice secundario* sobre Dni. Repita el ejercicio anterior (parte c) para el índice secundario y compárelo con el índice principal.
 - e. Suponga que el fichero *no está ordenado* por el campo no clave CodDpto y queremos construir un *índice secundario* sobre CodDpto, utilizando la opción 3 de la Sección 14.1.3, con un nivel

- extra de indirección que almacena punteros de registro. Asuma que hay 1.000 valores distintos de CodDpto y que los registros EMPLEADO están uniformemente distribuidos entre estos valores. Calcule (i) el factor de bloqueo de índice bfr_i (que también es el índice *fan-out fo*); (ii) el número de bloques que el nivel de indirección que almacena los punteros de registro necesita; (iii) el número de entradas de índice de primer nivel y el número de bloques de índice de primer nivel; (iv) el número de niveles necesarios si lo convertimos en un índice multinivel; (v) el número total de bloques que el índice multinivel necesita y los bloques utilizados en el nivel extra de indirección; y (vi) el número aproximado de accesos a bloques necesarios para buscar y recuperar todos los registros del fichero que tienen un valor CodDpto determinado, utilizando el índice.
- f. Suponga que el fichero está *ordenado* por el campo no clave CodDpto y queremos construir un *índice agrupado* sobre CodDpto que utiliza anclas de bloque (cada valor nuevo de CodDpto empieza al principio de un bloque nuevo). Asuma que hay 1.000 valores distintos de CodDpto y que los registros EMPLEADO están uniformemente distribuidos entre estos valores. Calcule (i) el factor de bloqueo de índice bfr_i (que también es el índice *fan-out fo*); (ii) el número de entradas de índice de primer nivel y el número total de bloques de índice de primer nivel; (iii) el número de niveles necesarios si lo convertimos en un índice multinivel; (iv) el número total de bloques que el índice multinivel necesita; y (v) el número de accesos a bloques que se necesitan para buscar y recuperar todos los registros del fichero que tienen un valor concreto de CodDpto, utilizando el índice agrupado (asuma que varios bloques en un grupo son contiguos).
- g. Suponga que el fichero *no está ordenado* por el campo clave Dni y que queremos construir una estructura de acceso en árbol B^+ (índice) sobre Dni. Calcule (i) los órdenes p y p_{hoja} del árbol B^+ ; (ii) el número de bloques a nivel hoja necesarios si los bloques están llenos aproximadamente al 69% (redondeado por arriba por conveniencia); (iii) el número de niveles necesarios si los nodos internos están llenos al 69% (redondeado por arriba por conveniencia); (iv) el número total de bloques que el árbol B^+ requiere; y (v) el número de accesos a bloques necesarios para buscar y recuperar un registro del fichero (dado su valor de Dni) utilizando el árbol B^+ .
- h. Repita la parte g, pero para un árbol B en lugar de para un árbol B^+ . Compare los resultados obtenidos con ambos tipos de árbol.
- 14.15.** Un fichero PIEZAS con NumPieza como campo clave incluye registros con los siguientes valores de NumPieza: 23, 65, 37, 60, 46, 92, 48, 71, 56, 59, 18, 21, 10, 74, 78, 15, 16, 20, 24, 28, 39, 43, 47, 50, 69, 75, 8, 49, 33, 38. Suponga que los valores del campo de búsqueda se insertan en el orden dado en un árbol B^+ de orden $p = 4$ y $p_{hoja} = 3$; muestre la expansión del árbol y cuál será su aspecto final.
- 14.16.** Repita el Ejercicio 14.15, pero utilice un árbol B de orden $p = 4$ en lugar de un árbol B^+ .
- 14.17.** Suponga que se borran los siguientes valores del campo de búsqueda, en el orden dado, del árbol B^+ del Ejercicio 14.15; muestre cómo se reducirá el árbol, así como el árbol final. Los valores borrados son 65, 75, 43, 18, 20, 92, 59, 37.
- 14.18.** Repita el Ejercicio 14.17, pero para el árbol B del Ejercicio 14.16.
- 14.19.** El Algoritmo 14.1 esboza el procedimiento de búsqueda en un índice principal multinivel no denso para recuperar un registro del fichero. Adapte el algoritmo para cada uno de los siguientes casos:
- Un índice secundario multinivel sobre un campo no ordenado y no clave de un fichero. Asuma que se utiliza la opción 3 de la Sección 14.1.3, donde un nivel de indirección extra almacena punteros a los registros individuales con el correspondiente valor del campo índice.
 - Un índice secundario multinivel sobre un campo clave no ordenado de un fichero.
 - Un índice agrupado multinivel sobre un campo de ordenación no clave de un fichero.

Figura 14.15. Inserción en un árbol B⁺ con redistribución izquierda.

- 14.20.** Suponga que existen varios índices secundarios en campos no clave de un fichero, implementados con la opción 3 de la Sección 14.1.3; por ejemplo, podríamos tener índices secundarios en los campos CodDpto, CodTrabajo y Sueldo del fichero EMPLEADO del Ejercicio 14.14. Describa una forma eficaz de buscar y recuperar los registros que satisfagan una condición de selección compleja sobre estos campos, como, por ejemplo, (CodDpto = 5 AND CodTrabajo = 12 AND Sueldo = 50.000), utilizando los punteros de registro en el nivel de indirección.
- 14.21.** Adapte para un árbol B los Algoritmos 14.2 y 14.3, que esbozan los procedimientos de búsqueda e inserción en un árbol B⁺.
- 14.22.** Es posible modificar el algoritmo de inserción de un árbol B⁺ para retardar el caso de que tenga que producirse un nivel nuevo verificando una posible *redistribución* de valores a lo largo de los nodos hoja. La Figura 14.15 ilustra cómo se puede hacer esto para nuestro ejemplo de la Figura 14.12; en lugar de dividir el nodo hoja situado más a la izquierda cuando se inserta 12, hacemos una *redistribución a la izquierda* moviendo el 7 al nodo hoja de su izquierda (si hay espacio en este nodo). La Figura 14.15 muestra el aspecto que debe tener el árbol cuando se considera la redistri-

bución. También es posible considerar la *redistribución derecha*. Intente modificar el algoritmo de inserción para un árbol B^+ para tener en cuenta la redistribución.

14.23. Diseñe un algoritmo para la eliminación en un árbol B^+ .

14.24. Repita el Ejercicio 14.23 para un árbol B .

Bibliografía seleccionada

Bayer y McCreight (1972) ofrece una introducción a los árboles B y los algoritmos asociados. Comer (1979) proporciona un excelente estudio de los árboles B y su historia, así como de sus variantes. Knuth (1973) proporciona un análisis detallado de muchas técnicas de búsqueda, incluyendo los árboles B y algunas de sus variantes. Nievergelt (1974) explica el uso de los árboles de búsqueda binaria para la organización de los ficheros. Libros que hablan de las estructuras de los ficheros, como Wirth (1972), Claybrook (1983), Smith y Barnes (1987), Miller (1987) y Salzberg (1988), explican la indexación en detalle y en ellos pueden consultarse los algoritmos de búsqueda, inserción y eliminación para árboles B y B^+ . Larson (1981) analiza los ficheros secuenciales indexados, y Held y Stonebraker (1978) compara los índices multinivel estáticos con los índices dinámicos de árboles B . Lehman y Yao (1981) y Srinivasan y Carey (1991) realizan un análisis más profundo del acceso concurrente a los árboles B . Los libros Wiederhold (1983), Smith y Barnes (1987) y Salzberg (1988), entre otros, explican muchas de las técnicas de búsqueda descritas en este capítulo. Los ficheros rejilla se introducen en Nievergelt (1984). La recuperación por coincidencia parcial, que hace uso de la dispersión partida, se explica en Burkhard (1976, 1979).

Las nuevas técnicas y aplicaciones de los índices y los árboles B^+ se explican en Lanka y Mays (1991), Zobel y otros (1992), y Faloutsos y Jagadish (1992). Mohan y Narang (1992) explican la creación de índices. El rendimiento de diversos algoritmos de árboles B y B^+ se evalúa en Baeza-Yates y Larson (1989) y Johnson y Shasha (1993). La administración de los búferes de cara a los índices se explica en Chan y otros (1992).

CAPÍTULO 23

Seguridad en las bases de datos

En este capítulo se revisan las técnicas para proteger las bases de datos contra diferentes amenazas. También se presentan métodos para proporcionar privilegios de acceso a los usuarios autorizados. La Sección 23.1 muestra una introducción a los temas de seguridad y a las amenazas a las bases de datos y ofrece una revisión de las medidas de control que se tratan en el resto de este capítulo. En la Sección 23.2 se revisan los mecanismos utilizados para conceder y retirar privilegios en los sistemas de bases de datos relacionales y en SQL, mecanismos se conocen a veces como **control de acceso discrecional**. La Sección 23.3 ofrece un repaso a los mecanismos utilizados para reforzar diversos niveles de seguridad, una preocupación reciente en la seguridad de las bases de datos que se conoce como **control de acceso obligatorio**. También presenta la estrategia desarrollada recientemente para el **control de acceso basado en roles** y revisa brevemente el control de acceso XML. La Sección 23.4 hace un breve repaso al problema de la seguridad en las bases de datos estadísticas. La Sección 23.5 presenta los desafíos actuales en materia de seguridad en las bases de datos. La Sección 23.9 hace un resumen del capítulo. Los lectores que estén interesados sólo en mecanismos básicos de seguridad en las bases de datos tendrán suficiente con revisar el contenido de las Secciones 23.1 y 23.2.

23.1 Introducción a los temas de seguridad en las bases de datos

23.1.1 Tipos de seguridad

La seguridad en las bases de datos es un tema muy amplio que comprende muchos conceptos, entre los cuales se incluyen los siguientes:

- Aspectos legales y éticos en relación con el derecho de acceso a determinada información. Ciertos tipos de información están considerados como privados y no pueden ser accedidos legalmente por personas no autorizadas. En los Estados Unidos existen muchas leyes que regulan la privacidad de la información.
- Temas de políticas a nivel gubernamental, institucional o de empresa en relación con los tipos de información que no debería estar disponible públicamente (por ejemplo, la concesión de créditos o los informes médicos personales).

- Temas relativos al sistema, como los *niveles del sistema* en los que se deberían reforzar las distintas funciones de seguridad (por ejemplo, si una función de seguridad debería ser manipulada a nivel físico del hardware, a nivel del sistema operativo o a nivel del DBMS).
- La necesidad dentro de algunas organizaciones de identificar diferentes niveles de seguridad y de clasificar según éstos a los datos y a los usuarios: por ejemplo, alto secreto, secreto, confidencial y no clasificado. Se debe reforzar la política de seguridad de la organización en lo que respecta a las diferentes clasificaciones de los datos.

Amenazas a las bases de datos. Las amenazas a las bases de datos tienen como consecuencia la pérdida o la degradación de todos o de algunos de los siguientes objetivos de seguridad comúnmente aceptados: integridad, disponibilidad y confidencialidad.

- **Pérdida de integridad.** La integridad de la base de datos tiene relación con el requisito a cumplir de que la información se encuentre protegida frente a modificaciones inadecuadas. La modificación de datos incluye la creación, inserción, modificación, cambio del estado de los datos y el borrado. La integridad se pierde si se realizan cambios no autorizados en los datos mediante acciones intencionadas o accidentales. Si no se corrige la pérdida de integridad del sistema o de los datos, el uso continuado de un sistema contaminado o de datos corrompidos podría tener como consecuencia la toma de decisiones inexactas, fraudulentas o erróneas.
- **Pérdida de disponibilidad.** La disponibilidad de la base de datos tiene relación con que los objetos estén disponibles para un usuario humano o para un programa que tenga los derechos correspondientes.
- **Pérdida de confidencialidad.** La confidencialidad de la base de datos tiene relación con la protección de los datos frente al acceso no autorizado. El impacto del acceso no autorizado a la información confidencial puede variar desde la violación de las leyes sobre privacidad de los datos hasta la amenaza a la seguridad nacional. El acceso no autorizado, no previsto o no intencionado podría tener como resultado la pérdida de la confianza en la organización, el que ésta quede en entredicho o que sea objeto de acciones legales en su contra.

Para proteger las bases de datos contra estos tipos de amenazas, es habitual implementar cuatro tipos de medidas de control: control de accesos, control de inferencias, control de flujo y cifrado. En este capítulo revisaremos cada una de ellas.

En un sistema de base de datos multiusuario, el DBMS debe proporcionar técnicas que permitan a determinados usuarios o grupos de usuarios el acceso a partes concretas de una base de datos sin tener acceso al resto de esta última. Esto resulta muy importante cuando una base de datos integrada de gran tamaño va a ser utilizada por muchos usuarios distintos dentro de la misma organización. Por ejemplo, la información confidencial (por ejemplo, los salarios de los empleados o sus evaluaciones de rendimiento) deberían permanecer como confidenciales para la mayoría de los usuarios del sistema de base de datos. Un DBMS incluye, por lo general, un **subsistema de autorizaciones y seguridad en bases de datos** responsable de garantizar la seguridad de partes de una base de datos frente a accesos no autorizados. En este momento, resulta obligado indicar dos tipos de mecanismos de seguridad en bases de datos:

- **Mecanismos de seguridad discrecionales.** Se utilizan para conceder permisos a usuarios, incluyendo la capacidad de acceso a determinados archivos de datos, registros o campos en un modo en concreto (lectura, inserción, borrado o actualización).
- **Mecanismos de seguridad obligatorios.** Se utilizan para reforzar la seguridad a varios niveles mediante la clasificación de los datos y de los usuarios en varias clases de seguridad (o niveles) para después implementar la política de seguridad adecuada a la organización. Por ejemplo, una política de seguridad habitual es permitir a los usuarios de un determinado nivel de clasificación ver sólo los elementos de datos clasificados en el mismo (o menor) nivel de clasificación que el del usuario. Una

extensión a esto es la *seguridad basada en roles*, en la cual se refuerzan las políticas y permisos basándose en el concepto de roles.

Veremos la seguridad discrecional en la Sección 23.2 y la seguridad obligatoria y basada en roles en la Sección 23.3.

23.1.2 Medidas de control

Existen cuatro medidas de control principales que se utilizan para proporcionar seguridad a los datos en las bases de datos. Son las siguientes:

- Control de accesos.
- Control de inferencias.
- Control de flujo.
- Cifrado de datos.

Un problema de seguridad habitual en los sistemas de computadores es prevenir que personas no autorizadas tengan acceso al sistema, bien para obtener información o bien para realizar cambios malintencionados en partes de la base de datos. El mecanismo de seguridad de un DBMS debe incluir medidas para restringir el acceso al sistema de base de datos en su totalidad. Esta función se denomina **control de acceso** y se gestiona mediante la creación de cuentas de usuario y contraseñas para controlar el proceso de entrada al DBMS. Revisaremos las técnicas de control de acceso en la Sección 23.1.3.

Las **bases de datos estadísticas** se utilizan para proporcionar información estadística o resúmenes de valores basados en diversos criterios. Por ejemplo, una base de datos con información estadística de población puede proporcionar datos estadísticos basados en los grupos de edad, niveles de ingresos, dotación de los hogares, niveles de educación y otros criterios. Los usuarios de bases de datos estadísticas, como los institutos de estadística de los gobiernos o las compañías de investigación de mercados, tienen privilegios para acceder a la base de datos para obtener información estadística sobre un determinado grupo de población, pero no para acceder a la información confidencial detallada sobre individuos en particular. La seguridad en las bases de datos estadísticas debe garantizar que la información relativa a los individuos no pueda ser accesible. A veces es posible deducir ciertos datos acerca de los individuos a partir de las consultas relativas únicamente a valores resumen sobre grupos; esto tampoco se debe permitir. Este problema, llamado **seguridad en bases de datos estadísticas**, se revisa brevemente en la Sección 23.4. Las medidas de control correspondientes se denominan **medidas de control de inferencias**.

Otro tema en relación con la seguridad es el control de flujo, que previene que la información fluya de tal manera que llegue a usuarios no autorizados. Se verá en la Sección 23.5. Los canales que resultan ser caminos que permiten que la información fluya implícitamente de modo que se viole la política de seguridad de una organización se denominan **canales ocultos**. En la Sección 23.5.1 veremos brevemente algunos temas relacionados con los canales ocultos.

Una última medida de seguridad es el **cifrado de datos**, que se utiliza para proteger datos confidenciales (como los números de las tarjetas de crédito) que sean transmitidos a través de algún tipo de red de comunicación. El cifrado se puede utilizar también para proporcionar protección adicional a partes confidenciales de la base de datos. Los datos se **codifican** utilizando algún algoritmo de codificación o cifrado. Un usuario no autorizado que acceda a datos codificados tendrá dificultades para descifrarlos, pero a los usuarios autorizados se les proporcionarán algoritmos de descodificación o descifrado (claves) para descifrar los datos. En las aplicaciones militares se han desarrollado técnicas de cifrado en las que es muy difícil descifrar sin conocer la clave. En la Sección 23.6 se hace un breve repaso a las técnicas de cifrado, incluyendo técnicas muy populares, como el cifrado con clave pública, de amplia utilización en la red para posibilitar las transacciones sobre bases de datos, y las firmas digitales, que se utilizan en las comunicaciones personales.

Queda fuera del ámbito de este libro una revisión en profundidad de la seguridad en los sistemas de computadores y en las bases de datos. Aquí sólo haremos un breve repaso a las técnicas de seguridad en las bases de datos. Los lectores más interesados en el tema pueden dirigirse a algunas de las referencias mencionadas en la Bibliografía seleccionada que aparece al final de este capítulo.

23.1.3 La seguridad en bases de datos y el DBA

Según vimos en el Capítulo 1, el administrador de la base de datos (DBA) es el responsable principal de la gestión de un sistema de base de datos. Entre las responsabilidades del DBA se encuentran la concesión y retirada de privilegios a los usuarios que necesitan utilizar el sistema, y la clasificación de usuarios y datos en función de la política de la organización. El DBA dispone de una **cuenta de DBA** en el DBMS, llamada también a veces **cuenta de sistema** o de **superusuario**, que proporciona enormes posibilidades que no están disponibles para las cuentas y los usuarios normales de la base de datos.¹ Entre los comandos privilegiados a nivel del DBA se encuentran los comandos para conceder o retirar privilegios a las cuentas o usuarios individuales o a los grupos de usuarios y para ejecutar los siguientes tipos de acciones:

1. **Creación de cuentas.** Mediante esta acción se crea una nueva cuenta y contraseña para posibilitar el acceso al DBMS a un usuario o grupo de usuarios.
2. **Concesión de privilegios.** Esta acción permite al DBA conceder determinados permisos a determinadas cuentas.
3. **Retirada de privilegios.** Esta acción permite al DBA retirar (cancelar) determinados permisos concedidos previamente a determinadas cuentas.
4. **Asignación del nivel de seguridad.** Esta acción consiste en asignar cuentas de usuario al nivel de clasificación de seguridad adecuado.

El DBA es el responsable de la seguridad general del sistema de bases de datos. La acción 1 de la lista anterior se utiliza para controlar el acceso al DBMS en su totalidad, mientras que las acciones 2 y 3 se utilizan para controlar las autorizaciones *discrecionales* en la base de datos y la acción 4 se utiliza para controlar la autorización *obligatoria*.

23.1.4 Protección de accesos, cuentas de usuario y auditorías de bases de datos

Siempre que una persona o grupo de personas necesitan acceder a un sistema de base de datos, ese individuo o grupo deben solicitar, en primer lugar, una cuenta de usuario. El DBA creará entonces una **cuenta de usuario** y una **contraseña** nuevas para el usuario si existe la necesidad legitimada de su acceso a la base de datos. El usuario deberá **iniciar sesión** en el DBMS introduciendo el número de cuenta y la contraseña cada vez que necesite acceso a la base de datos. El DBMS comprueba que el número de cuenta y la contraseña son válidos; si lo son, se permite que el usuario utilice el DBMS y acceda a la base de datos. También se puede considerar a los programas de aplicación como usuarios y se les podrá solicitar la contraseña.

Resulta sencillo seguir la pista a los usuarios de la base de datos y a sus cuentas y contraseñas mediante la creación de una tabla cifrada o archivo con dos campos: Cuenta de usuario y Contraseña. El DBMS puede realizar el mantenimiento de esta tabla de una forma muy sencilla. Cada vez que se cree una nueva cuenta, se insertará un registro en la tabla. Cuando una cuenta sea cancelada, el registro correspondiente debe ser borrado de la tabla.

¹ Esta cuenta es similar a las cuentas de tipo *root* (*raíz*) o *superuser* (superusuario) que pertenecen a los administradores de sistemas y permite el acceso a comandos restringidos del sistema operativo.

El sistema de base de datos también debe seguir el rastro de todas las operaciones en la base de datos que un usuario determinado realice durante cada **sesión**, que comprende toda la secuencia de interacciones realizadas por ese usuario en la base de datos desde el momento en que inicia la sesión hasta el momento en que finaliza. Cuando un usuario inicia una sesión, el DBMS puede registrar el número de cuenta del usuario y asociarlo al terminal desde el cual el usuario abrió la sesión. Todas las operaciones desde ese terminal se asignarán a la cuenta del usuario hasta que éste cierre la sesión. Es particularmente importante seguir el rastro a las operaciones de actualización que se realicen sobre la base de datos, de modo que el DBA pueda determinar qué usuario provocó el daño en el caso de que la base de datos sea modificada de forma malintencionada.

Para mantener un registro de todas las actualizaciones realizadas en la base de datos y de los usuarios en particular que realizaron cada actualización, podemos modificar el registro de sucesos del sistema (*system log*). Recuerde de los Capítulos 17 y 19 que el **registro (de sucesos) del sistema** incluye una entrada por cada operación realizada sobre la base de datos que se pueda necesitar en caso de fallo transaccional o de caída del sistema. Es posible expandir las entradas del registro de forma que incluyan también en el registro el número de cuenta del usuario y el identificador del terminal desde el que se realizó la operación. Si se sospecha de alguna modificación malintencionada, se realizará una **auditoría de la base de datos**, que consiste en la revisión del archivo de registro para examinar todos los accesos y operaciones realizadas sobre la base de datos durante un determinado período de tiempo. Cuando se encuentre una operación ilegal o no autorizada, el DBA podrá determinar el número de cuenta utilizado para realizar la operación. Las auditorías en la base de datos son particularmente importantes en el caso de bases de datos con información confidencial que sean actualizadas por un gran número de transacciones y usuarios, como una base de datos bancaria actualizada por un gran número de cajeros del banco. A un registro de una base de datos que se utilice principalmente para propósitos de seguridad se le llama, a veces, **registro de auditoría**.

23.2 Control de acceso discrecional basado en la concesión y revocación de privilegios

El método habitual para reforzar el **control de acceso discrecional** en un sistema de base de datos se basa en la concesión y revocación de **privilegios**. Vamos a considerar los privilegios en el contexto de un DBMS relacional. En particular, veremos un sistema de privilegios similar en cierto modo al desarrollado originalmente para el lenguaje SQL (consulte el Capítulo 8). Muchos DBMSs relacionales de la actualidad utilizan alguna variación de esta técnica. La idea principal es incluir declaraciones en el lenguaje de consultas que permitan al DBA y a los usuarios seleccionados conceder y revocar privilegios.

23.2.1 Tipos de privilegios discrecionales

En SQL2, el concepto de **identificador de autorización** se utiliza para referirse, a grandes rasgos, a una cuenta de usuario (o a un grupo de cuentas de usuario). Por simplicidad, utilizaremos las palabras *usuario* o *cuenta* de modo indiferente en lugar de *identificador de autorización*. El DBMS debe proporcionar acceso selectivo a cada relación de la base de datos basándose en cuentas determinadas. También podemos controlar las operaciones; de este modo, la posesión de una cuenta no significa necesariamente que el propietario de la cuenta esté autorizado a toda la funcionalidad que permita el DBMS. Informalmente hablando, existen dos niveles de asignación de privilegios de uso del sistema de base de datos:

- **El nivel de cuenta.** En este nivel, el DBA especifica los privilegios en particular que posee cada cuenta, independientemente de las relaciones existentes en la base de datos.
- **El nivel de relación (o tabla).** En este nivel, el DBA puede controlar el privilegio de acceso a cada relación o vista individual en la base de datos.

Los privilegios a **nivel de cuenta** se aplican a las funcionalidades de la propia cuenta y pueden incluir el privilegio sobre CREATE SCHEMA o CREATE TABLE para crear un esquema o relación base; el privilegio sobre CREATE VIEW; el privilegio sobre ALTER, para realizar cambios en el esquema como la agregación o eliminación de atributos en las relaciones; el privilegio sobre DROP, para borrar relaciones o vistas; el privilegio sobre MODIFY, para insertar, borrar o actualizar tuplas; y el privilegio sobre SELECT, para obtener información de la base de datos usando una consulta SELECT. Observe que estos privilegios de cuenta se aplican a las cuentas en general. Si una cuenta determinada no tiene el privilegio sobre CREATE TABLE, no se podrán crear relaciones desde esa cuenta. Los privilegios a nivel de cuenta *no forman* parte de SQL2; deberán ser definidos por los creadores del DBMS. En versiones anteriores de SQL, existía un privilegio denominado CREATETAB para proporcionar a una cuenta el privilegio para crear tablas (relaciones).

El segundo nivel de privilegios se aplica al **nivel de relación**, bien sean relaciones base o relaciones virtuales (vistas). Estos privilegios *se definen* en SQL2. En lo que veremos a continuación, el término *relación* puede referirse a una relación base o a una vista, a menos que especifiquemos explícitamente una o la otra. Los privilegios a nivel de relación especifican, a nivel de usuario, las relaciones individuales sobre las que se puede aplicar cada tipo de comando. Algunos privilegios también hacen referencia a columnas individuales (atributos) de las relaciones. Los comandos de SQL2 proporcionan privilegios *sólo a nivel de relación y atributos* y, aunque sea un tratamiento bastante general, esto dificulta la creación de cuentas con privilegios limitados. La concesión y revocación de privilegios sigue, por lo general, un modelo de autorizaciones para los privilegios discrecionales conocido como modelo **de matriz de acceso**, en el que las filas de una matriz M representan los sujetos (usuarios, cuentas, programas) y las columnas representan objetos (relaciones, registros, columnas, vistas, operaciones). Cada posición $M(i, j)$ de la matriz representa los tipos de permisos (lectura, escritura, actualización) que posee el sujeto i sobre el objeto j .

Para controlar la concesión y revocación de privilegios de una relación, a cada relación R de una base de datos se le asigna una **cuenta de propietario** que es, por lo general, la cuenta que se utilizó cuando se creó la relación por primera vez. Al propietario de una relación se le conceden *todos* los privilegios sobre esa relación. En SQL2, el DBA puede asignar un propietario a todo el esquema en su totalidad creando el esquema y asociando a ese esquema el identificador de autorización adecuado mediante el comando CREATE SCHEMA (consulte la Sección 8.1.1). El poseedor de la cuenta de propietario puede traspasar los privilegios sobre cualquiera de las relaciones que posee a otros usuarios mediante la **concesión** de privilegios a sus cuentas. En SQL se pueden conceder los siguientes tipos de privilegios a cada relación individual R :

- **Privilegio de selección (recuperación o lectura) sobre R .** Concede a la cuenta el privilegio para obtener información. En SQL, esto concede a la cuenta el privilegio de utilizar la instrucción SELECT para obtener tuplas de R .
- **Privilegio de modificación sobre R .** Concede a la cuenta la capacidad de modificar tuplas de R . En SQL, este privilegio se subdivide en privilegios sobre UPDATE, DELETE e INSERT para utilizar el correspondiente comando SQL sobre R . Adicionalmente, tanto el privilegio sobre INSERT como sobre UPDATE pueden especificar que sólo ciertos atributos de R pueden ser actualizados por la cuenta.
- **Privilegio de referencia de R .** Concede a la cuenta la posibilidad de referenciar la relación R al especificar restricciones de integridad. También se puede restringir este privilegio a determinados atributos de R .

Observe que para crear una vista, la cuenta debe tener privilegio de SELECT sobre *todas las relaciones* implicadas en la definición de la vista.

23.2.2 Especificación de privilegios mediante vistas

El mecanismo de **vistas** es un importante mecanismo de autorización discrecional por sí mismo. Por ejemplo, si el propietario A de una relación R quiere que otra cuenta B pueda obtener sólo algunos campos de R , enton-

ces A podría crear una vista V de R que incluyese sólo esos atributos y después autorizar SELECT sobre V a B . Lo mismo se aplica a la hora de limitar a B para que pueda obtener sólo determinadas tuplas de R ; se puede crear una vista V definiendo la vista por medio de una consulta que seleccione sólo las tuplas de R sobre las que A desee permitir el acceso a B . Ilustraremos esto mediante el ejemplo que aparece en la Sección 23.2.5.

23.2.3 Revocación de privilegios

En algunos casos, es deseable conceder un privilegio a un usuario de manera temporal. Por ejemplo, el propietario de una relación podría querer conceder privilegio a un usuario sobre SELECT para realizar una tarea determinada y, después, revocar ese privilegio una vez que la tarea haya finalizado. Según lo anterior, se necesita un mecanismo para **revocar** privilegios. SQL incluye un comando REVOKE con el propósito de retirar privilegios; en la Sección 23.2.5 veremos cómo se utiliza.

23.2.4 Propagación de los privilegios mediante GRANT OPTION

Siempre que el propietario A de una relación R concede un privilegio a otra cuenta B , dicho privilegio se puede conceder a B con o sin la opción **GRANT OPTION**. Si aparece esta opción, significa que B también puede conceder ese privilegio sobre R a otras cuentas. Supongamos que A concede la opción GRANT OPTION a B y que B entonces concede privilegio sobre R a una tercera cuenta C , también con la opción GRANT OPTION. De este modo, los privilegios sobre R se pueden **propagar** a otras cuentas sin el conocimiento del propietario de R . Si la cuenta propietaria A revoca a continuación el privilegio concedido a B , todos los privilegios que B propagó basados en ese privilegio deberían ser revocados de forma automática por el sistema.

Es posible que un usuario reciba determinados privilegios de dos o más fuentes. Por ejemplo, A_4 podría recibir un determinado privilegio UPDATE R de *ambos*, A_2 y A_3 . En este caso, si A_2 revoca este privilegio a A_4 , A_4 seguirá disfrutando del privilegio al haber sido concedido también por A_3 . Si A_3 revoca posteriormente el privilegio a A_4 , A_4 lo perderá totalmente. Por tanto, un DBMS que permita la propagación de privilegios debe seguir el rastro del modo en que fueron concedidos los privilegios para que la revocación de privilegios se pueda realizar correcta y convenientemente.

23.2.5 Un ejemplo

Supongamos que el DBA crease cuatro cuentas (A_1 , A_2 , A_3 y A_4) y quiere que sólo A_1 pueda crear relaciones base; en este caso, el DBA debería ejecutar el siguiente comando GRANT en SQL:

```
GRANT CREATETAB TO A1;
```

El privilegio CREATETAB (crear tabla) concede a la cuenta A_1 la posibilidad de crear nuevas tablas en la base de datos (relaciones base) y es, por tanto, un *privilegio de cuenta*. Este privilegio formaba parte de las primeras versiones de SQL, pero ahora se deja su definición a criterio de cada implementación del sistema.

En SQL2 se puede obtener el mismo resultado haciendo que el DBA ejecute el comando CREATE SCHEMA según se muestra a continuación:

```
CREATE SCHEMA EJEMPLO AUTHORIZATION A1;
```

Tras esto, la cuenta de usuario A_1 puede crear tablas bajo el esquema de nombre EJEMPLO. Para seguir con nuestro ejemplo, supongamos que A_1 crease las dos relaciones base EMPLEADO y DEPARTAMENTO que aparecen en la Figura 23.1; A_1 sería entonces el propietario de estas dos relaciones y, por tanto, tendría *todos los privilegios de relación* sobre cada una de ellas.

A continuación, supongamos que la cuenta A_1 desea conceder a la cuenta A_2 el privilegio para insertar y borrar tuplas en ambas relaciones. Sin embargo, A_1 no quiere que A_2 pueda propagar estos privilegios a otras cuentas. A_1 podría ejecutar el siguiente comando:

Figura 23.1. Esquemas para las dos relaciones EMPLEADO y DEPARTAMENTO.**EMPLEADO**

Nombre	<u>Dni</u>	FechaNac	Dirección	Sexo	Sueldo	Dno
--------	------------	----------	-----------	------	--------	-----

DEPARTAMENTO

<u>NúmeroDpto</u>	NombreDpto	DnDirector
-------------------	------------	------------

GRANT INSERT, DELETE ON EMPLEADO, DEPARTAMENTO TO A2;

Observe que la cuenta A1 propietaria de una relación posee de forma automática la opción GRANT OPTION, permitiéndole conceder privilegios sobre la relación a otras cuentas. Sin embargo, la cuenta A2 no puede conceder los privilegios INSERT y DELETE sobre las tablas EMPLEADO y DEPARTAMENTO, ya que a A2 no se le concedió la opción GRANT OPTION en el comando anterior.

A continuación, supongamos que A1 desea permitir a la cuenta A3 obtener información de cualquiera de las dos tablas y también poder propagar el privilegio SELECT a otras cuentas. A1 podría ejecutar el siguiente comando:

GRANT SELECT ON EMPLEADO, DEPARTAMENTO TO A3 WITH GRANT OPTION;

La cláusula WITH GRANT OPTION significa que A3 puede ahora propagar el privilegio a otras cuentas utilizando GRANT. Por ejemplo, A3 puede conceder el privilegio SELECT sobre la relación EMPLEADO a A4 ejecutando el comando siguiente:

GRANT SELECT ON EMPLEADO TO A4;

Observe que A4 no puede propagar el privilegio SELECT a otras cuentas ya que no se le concedió la opción GRANT OPTION.

Supongamos ahora que A1 decide revocar a A3 el privilegio SELECT sobre la relación EMPLEADO. A1 podría ejecutar este comando:

REVOKE SELECT ON EMPLEADO FROM A3;

El DBMS deberá entonces revocar también de forma automática el privilegio SELECT sobre EMPLEADO a A4, ya que A3 concedió ese privilegio a A4 y A3 ya no dispone de ese privilegio.

A continuación, supongamos que A1 desea devolver a A3 unos permisos limitados de SELECT sobre la relación EMPLEADO y desea permitir a A3 que pueda propagar el privilegio. La limitación consiste en poder obtener sólo los atributos Nombre, FechaNac y Dirección y sólo para las tuplas con el atributo Dno igual a 5. A1 podría entonces crear la siguiente vista:

```
CREATE VIEW A3EMPLEADO AS
SELECT Nombre, FechaNac, Dirección
FROM EMPLEADO
WHERE Dno = 5;
```

Una vez creada la vista, A1 podrá conceder a A3 el permiso para ejecutar SELECT sobre la vista A3EMPLEADO según se muestra a continuación:

GRANT SELECT ON A3EMPLEADO TO A3 WITH GRANT OPTION;

Por último, supongamos que A1 desea permitir a A4 que actualice sólo el atributo Sueldo de EMPLEADO. A1 podrá entonces ejecutar el siguiente comando:

GRANT UPDATE ON EMPLEADO (Sueldo) TO A4;

Los privilegios UPDATE o INSERT pueden especificar atributos en particular que puedan ser actualizados o insertados en una relación. Otros tipos de privilegios (SELECT, DELETE) no se aplican a atributos en concreto ya que esta funcionalidad se puede controlar fácilmente mediante la creación de las vistas adecuadas que incluyan sólo los atributos deseados y la concesión de los privilegios correspondientes sobre las vistas. Sin embargo, debido a que la actualización de las vistas no es siempre posible (consulte el Capítulo 8), a los privilegios UPDATE e INSERT se les da la opción de indicar qué atributos en particular de una relación base pueden ser actualizados.

23.2.6 Especificación de los límites a la propagación de privilegios

Se han desarrollado técnicas para limitar la propagación de privilegios, aunque aún no han sido implementadas en la mayoría de los DBMSs y tampoco forman parte de SQL. Limitar la **propagación horizontal** a un número entero i significa que una cuenta B a la que se ha dado la opción GRANT OPTION puede conceder el privilegio a , como mucho, otras i cuentas. La **propagación vertical** es más complicada; limita la profundidad de la concesión de los privilegios. Conceder un privilegio con una propagación vertical igual a cero es equivalente a conceder el privilegio sin ninguna opción GRANT OPTION. Si la cuenta A concede un privilegio a la cuenta B con una propagación vertical igual a un número entero $j > 0$ significa que la cuenta B tiene la opción GRANT OPTION sobre ese privilegio, pero B sólo puede conceder el privilegio a otras cuentas con una propagación vertical *menor que* j . De hecho, la propagación vertical limita la secuencia de GRANT OPTIONS que pueden ser concedidas de una cuenta a la siguiente en base a la única concesión de privilegios original.

Pondremos un ejemplo (que no está disponible actualmente en SQL u otros sistemas relacionales) para ilustrar los límites de propagación horizontal y vertical. Supongamos que A_1 concede SELECT a A_2 sobre la relación EMPLEADO con propagación horizontal igual a 1 y propagación vertical igual a 2. A_2 podría entonces conceder SELECT a, como mucho, una cuenta porque la limitación de propagación horizontal tiene un valor de 1. Además, A_2 no puede conceder el privilegio a otra cuenta excepto las que tienen propagación vertical igual a 0 (sin opción GRANT OPTION) o igual a 1, debido a que A_2 debe reducir la propagación vertical en, al menos, 1 al pasar el privilegio a otras cuentas. Según se muestra en este ejemplo, las técnicas de propagación horizontal y vertical están diseñadas para limitar el nivel de profundidad de la propagación de privilegios.

23.3 Control de acceso obligatorio y control de acceso basado en roles para la seguridad multinivel²

La técnica de control de acceso discrecional consistente en conceder y revocar privilegios sobre las relaciones ha sido tradicionalmente el principal mecanismo de seguridad en los sistemas de bases de datos relacionales. Se trata de un método de tipo “todo o nada”. Un usuario, o tiene o no tiene un determinado privilegio. En muchas aplicaciones se necesita una *política de seguridad adicional* que clasifique los datos y los usuarios basándose en clases de seguridad. Este modelo, conocido como **control de acceso obligatorio**, se *combina* habitualmente con los mecanismos de control de acceso discrecional descritos en la sección 23.2. Es importante observar que la mayoría de los DBMSs comerciales sólo proporcionan en la actualidad mecanismos para el control de acceso discrecional. Sin embargo, existe una necesidad de seguridad a varios niveles en las aplicaciones gubernamentales, militares y en las de los servicios de inteligencia, así como en muchas aplicaciones industriales y corporativas.

² Se agradece la contribución de Fariborz Farahmand en esta sección y en las siguientes.

Las **clases de seguridad** típicas son alto secreto (TS), secreto (S), confidencial (C) y no clasificado (U), siendo TS el nivel más alto y U el más bajo. Existen otros esquemas de clasificación de seguridad más complejos, en los cuales las clases de seguridad se organizan en forma de rejilla. Por simplicidad utilizaremos el sistema de cuatro niveles de clasificación de seguridad, en el que $TS \geq S \geq C \geq U$, para ilustrar nuestro debate. El modelo utilizado habitualmente para la seguridad a varios niveles, conocido como modelo Bell-LaPadula, clasifica cada **sujeto** (usuario, cuenta, programa) y **objeto** (relación, tupla, columna, vista, operación) en una de las clasificaciones de seguridad TS, S, C, o U. Llamaremos **clase(S)** al **nivel de autorización** (clasificación) de un sujeto S y **clase(O)** a la clasificación de un objeto O . Basándonos en las clasificaciones de sujeto/objeto podemos definir estas dos reglas:

1. A un sujeto S no se le permite el acceso de lectura a un objeto O a menos que la $\text{clase}(S) \geq \text{clase}(O)$. Esto se conoce como **propiedad de seguridad simple**.
2. A un sujeto S no se le permite escribir un objeto O a menos que la $\text{clase}(S) \leq \text{clase}(O)$. Esto se conoce como **propiedad estrella** (o propiedad-*).

La primera restricción es intuitiva y pone en evidencia la regla obvia que dice que ningún sujeto puede leer un objeto cuya clasificación de seguridad sea más alta que el nivel de autorización del sujeto. La segunda restricción es menos intuitiva. Impide que un sujeto escriba un objeto que se encuentre en una clasificación de seguridad menor que el nivel de autorización del sujeto. La violación de esta regla permitiría que la información fluyese desde clasificaciones más altas hacia las más bajas, lo cual viola uno de los principios básicos de la seguridad multinivel. Por ejemplo, un usuario (sujeto) con nivel de autorización TS puede hacer una copia de un objeto con clasificación TS y después volverlo a escribir como un nuevo objeto con clasificación U, haciendo que de este modo pase a ser visible en todo el sistema.

Para añadir conceptos de seguridad multinivel al modelo de base de datos relacional, es habitual considerar a los valores de atributos y a las tuplas como objetos de datos. De este modo, a cada atributo A se le asocia un **atributo de clasificación** C en el esquema, y a cada valor de atributo en una tupla se le asocia su correspondiente clasificación de seguridad. Además, en algunos modelos se añade un atributo de **clasificación de tupla** TC a los atributos de relación para proporcionar una clasificación a cada tupla en su totalidad. Según esto, un esquema de **relación multinivel** R con n atributos se representaría como

$$R(A_1, C_1, A_2, C_2, \dots, A_n, C_n, TC)$$

donde cada C_i representa el *atributo de clasificación* asociado a cada atributo A_i .

El valor de cada atributo TC en cada tupla t (que es *el más alto* de todos los valores de clasificación de atributos dentro de t) proporciona una clasificación general para la tupla, mientras que cada C_i proporciona una clasificación de seguridad más en detalle dentro de la tupla. La **clave aparente** de una relación multinivel es el conjunto de atributos que habrían formado parte de la clave primaria en una relación normal (a un nivel). Una relación multinivel contendrá aparentemente datos diferentes para los sujetos (usuarios) con distintos niveles de seguridad. En algunos casos, es posible almacenar una única tupla de la relación en un nivel de clasificación más alto y generar las tuplas correspondientes en un nivel de clasificación inferior mediante un proceso llamado **filtrado**. En otros casos, es necesario almacenar dos o más tuplas de niveles de clasificación diferentes con el mismo valor para la *clave aparente*. Esto nos lleva al concepto de **poliinstanciación**,³ según el cual varias tuplas pueden tener el mismo valor de clave aparente pero con distintos valores de atributo para los usuarios con diferentes niveles de clasificación.

Ilustraremos estos conceptos mediante el ejemplo sencillo de relación multinivel que se muestra en la Figura 23.2(a), en el cual aparecen los valores de atributos de clasificación junto a cada valor de atributo. Supongamos que el atributo Nombre es la clave aparente, y tomemos la consulta **SELECT * FROM EMPLEADO**. Un usuario con nivel de seguridad S vería la misma relación que se muestra en la Figura 23.2(a), ya que todas las clasificaciones de tupla son menores que o iguales a S. Sin embargo, un usuario con nivel de

³ Esto es similar a la idea de tener varias versiones en la base de datos para representar el mismo objeto del mundo real.

Figura 23.2. Una relación multinivel para ilustrar la seguridad multinivel. (a) Las tuplas originales de EMPLEADO. (b) Apariencia de EMPLEADO tras filtrar para los usuarios de clasificación C. (c) Apariencia de EMPLEADO tras filtrar para los usuarios de clasificación U. (d) Poliinstanciación de la tupla Gómez.

(a) **EMPLEADO**

Nombre	Sueldo	RendimientoTrabajo	TC
Gómez U	40000 C	Normal S	S
Campos C	80000 S	Bueno C	S

(b) **EMPLEADO**

Nombre	Sueldo	RendimientoTrabajo	TC
Gómez U	40000 C	NULL C	C
Campos C	NULL C	Bueno C	C

(c) **EMPLEADO**

Nombre	Sueldo	RendimientoTrabajo	TC
Gómez U	NULL U	NULL U	U

(d) **EMPLEADO**

Nombre	Sueldo	RendimientoTrabajo	TC
Gómez U	40000 C	Normal S	S
Gómez U	40000 C	Excelente C	C
Campos C	80000 S	Bueno C	S

seguridad C no estaría autorizado a ver los valores de salario de ‘Campos’ y la evaluación de rendimiento en el trabajo de ‘Gómez’, ya que tienen una clasificación más alta. Las tuplas serían filtradas para que aparecieran como se muestra en la Figura 23.2(b), donde Sueldo y RendimientoTrabajo *aparecerían* como valor nulo. Para un usuario con nivel de seguridad U, el filtro permite que sólo aparezca el atributo Nombre de ‘Gómez’, y los otros atributos como valores nulos (véase la Figura 23.2[c]). De este modo, el filtrado introduce valores nulos para los valores de atributo cuya clasificación de seguridad es más alta que el nivel de seguridad del usuario.

En general, la regla de **integridad de entidad** para las relaciones multinivel obliga a que todos los atributos que sean miembros de la clave aparente no puedan ser valores nulos y a que tengan la *misma* clasificación de seguridad en cada tupla individual. Además, el resto de valores de atributo de la tupla deben tener una clasificación de seguridad mayor que o igual a la de la clave aparente. Esta restricción garantiza que un usuario pueda ver la clave si al usuario se le permite ver cualquier parte de la tupla. Otras reglas de integridad, llamadas **integridad nula** e **integridad interinstancia** aseguran, informalmente, que si un valor de tupla en un determinado nivel de seguridad se puede filtrar (derivar) desde una tupla que está a un nivel de seguridad más alto, entonces es suficiente con almacenar en la relación multinivel la tupla que está a mayor nivel.

Para ilustrar la poliinstanciación con más detalle, supongamos que un usuario con nivel de seguridad C intenta actualizar el valor de RendimientoTrabajo de ‘Gómez’ de la Figura 23.2 a ‘Excelente’; esto se corresponde con la ejecución de la siguiente sentencia SQL de actualización:

```
UPDATE EMPLEADO
SET RendimientoTrabajo = 'Excelente'
WHERE Nombre = 'Gómez';
```


Ya que la vista que se muestra a los usuarios con nivel de seguridad C (véase la Figura 23.2[b]) permite una actualización de este tipo, el sistema no debería rechazarla; en caso contrario, el usuario podría deducir que existe algún valor no nulo para el atributo `RendimientoTrabajo` de 'Gómez' en lugar del valor nulo que aparece. Esto es un ejemplo de la inferencia de información a través de lo que se conoce como **canal oculto**, que no debería estar permitido en sistemas de alta seguridad (consulte la Sección 23.5.1). Sin embargo, al usuario no se le debería permitir sobrescribir el valor existente de `RendimientoTrabajo` en el nivel de clasificación más alto. La solución es crear una **poliinstanciación** de la tupla 'Gómez' en el nivel de clasificación más bajo C, según se muestra en la Figura 23.2(d). Esto es necesario ya que la nueva tupla no puede ser filtrada a partir de la tupla existente en el nivel de clasificación S.

Las operaciones básicas de actualización del modelo relacional (INSERT, DELETE, UPDATE) deben ser modificadas para tratar con situaciones parecidas a ésta, pero este aspecto del problema está fuera del ámbito de este texto. Los lectores interesados en el tema pueden dirigirse a la Bibliografía Seleccionada que se encuentra al final de este capítulo si desean más detalles.

23.3.1 Comparación del control de acceso discrecional y el control de acceso obligatorio

Las políticas de Control de acceso discrecional (DAC) se caracterizan por su alto grado de flexibilidad, lo cual las hace muy indicadas para una gran variedad de dominios de aplicación. El principal inconveniente de los modelos DAC es su vulnerabilidad a ataques maliciosos, como los caballos de Troya incluidos en programas de aplicación. La razón es que los modelos de autorización discrecional no imponen ningún control sobre cómo se propaga y se usa la información una vez que ha sido accedida por los usuarios autorizados a hacerlo. Por el contrario, las políticas obligatorias aseguran un alto grado de protección (dicho de otro modo, previenen cualquier flujo ilegal de información). Por tanto, son las indicadas para las aplicaciones de tipo militar, que requieren un alto grado de protección. Sin embargo, las políticas obligatorias tienen el inconveniente de ser demasiado rígidas, ya que requieren una clasificación estricta de los sujetos y de los objetos en niveles de seguridad y, por tanto, son aplicables a muy pocos entornos. En la práctica, en la mayoría de las situaciones se prefieren las políticas discrecionales, pues ofrecen una mejor relación entre seguridad y aplicabilidad.

23.3.2 Control de accesos basado en roles

El Control de acceso basado en roles (RBAC) surgió en muy poco tiempo en los años 90 como tecnología probada para la gestión y el reforzamiento de la seguridad en sistemas empresariales a gran escala. La idea básica es que los permisos están asociados a roles y a los usuarios se les asignan los roles apropiados. Los roles se pueden crear mediante los comandos `CREATE ROLE` y `DESTROY ROLE`. Los comandos `GRANT` y `REVOKE` que se vieron en el control de acceso discrecional (DAC) pueden ser usados para conceder y revocar privilegios a los roles.

El control de acceso basado en roles parece ser una alternativa viable a los controles de acceso discrecionales y obligatorios tradicionales; garantiza que sólo los usuarios autorizados tienen acceso a determinados datos o recursos. Los usuarios crean sesiones durante las cuales pueden activar un subconjunto de roles a los que pertenecen. Cada sesión se puede asignar a muchos roles, pero se corresponde con un único usuario o sujeto. Muchos DBMSs permiten el concepto de roles que consiste en asignar privilegios a los mismos.

La jerarquía de roles en RBAC es el modo natural de organizar los roles para que reflejen la jerarquía de autoridad y de responsabilidades en la organización. Por convención, los roles de menor responsabilidad de la parte inferior se conectan con roles de responsabilidad cada vez mayor a medida que se sube por la jerarquía. Los diagramas de jerarquía son órdenes parciales, por tanto son reflexivos, transitivos y antisimétricos.

Otra idea importante en los sistemas RBAC es la posibilidad de que existan restricciones temporales en los roles, como el tiempo y la duración de las activaciones de los roles, y el disparo temporizado de un rol median-

te la activación de otro rol. La utilización de un modelo RBAC es una meta a alcanzar para conseguir algo altamente deseable que es el tratamiento de los requisitos clave de seguridad en las aplicaciones web. Se pueden asignar roles a las tareas del flujo de trabajo para que se pueda autorizar a un usuario con uno de los roles relacionados con una tarea la ejecución de ésta y la actuación con un rol durante un determinado periodo de tiempo.

Los modelos RBAC tienen varias características interesantes, como la flexibilidad, la neutralidad en las políticas, un mejor soporte para la gestión de la seguridad y de la administración, y otros aspectos que los hacen candidatos adecuados para el desarrollo de aplicaciones web seguras. Por el contrario, a los modelos DAC y otros de control de acceso obligatorio (MAC) les faltan las capacidades necesarias para proporcionar los requisitos de seguridad de compañías emergentes y de aplicaciones web. Además, los modelos RBAC pueden representar políticas DAC y MAC tradicionales, así como políticas definidas por el usuario o específicas de una organización. De este modo, RBAC se convierte en un gran modelo que puede, a su vez, imitar el comportamiento de los sistemas DAC y MAC. Además, un modelo RBAC proporciona un mecanismo natural para el tratamiento de los temas de seguridad relacionados con la ejecución de tareas y flujos de trabajo. Otra razón para el éxito de los modelos RBAC ha sido la facilidad para el desarrollo en Internet.

23.3.3 Control de acceso XML

Debido a la extensión del uso de XML en las aplicaciones científicas y comerciales, se están realizando esfuerzos orientados al desarrollo de estándares de seguridad. Entre estos esfuerzos se encuentran las firmas digitales y los estándares de criptografía para XML. La especificación de sintaxis y procesamiento de firmas en XML describe una sintaxis XML para la representación de las asociaciones entre firmas criptográficas y documentos XML u otros recursos electrónicos. La especificación también incluye procedimientos para el cálculo y la verificación de firmas XML. Una firma digital XML se diferencia de otros protocolos de firma de mensajes como PGP (*Pretty Good Privacy*, un servicio de autenticación y confidencialidad que puede ser utilizado en el correo electrónico y en las aplicaciones de almacenamiento de archivos), en que proporciona la posibilidad de firmar sólo partes determinadas del árbol XML (consulte el Capítulo 26) en lugar del documento completo. Además, la especificación de firma XML define mecanismos de contrafirma y transformaciones (llamadas canonizaciones para asegurar que las dos instancias del mismo texto producen el mismo valor resumen para la firma, incluso si sus representaciones difieren ligeramente). La especificación de sintaxis y procesamiento de cifrado en XML define el vocabulario XML y las reglas de procesamiento para proteger la confidencialidad de los documentos XML en su totalidad o en parte y también la de los datos no XML. El contenido cifrado y la información de procesamiento adicional para el receptor se representan en XML “bien formado” de forma que el resultado pueda ser procesado posteriormente utilizando herramientas XML. A diferencia de otras tecnologías utilizadas habitualmente para la confidencialidad como SSL (*Secure Socket Layer*, un protocolo importante en la seguridad en Internet) y las redes privadas virtuales, el cifrado XML también se aplica a partes de los documentos que se encuentran en almacenamiento persistente.

23.3.4 Políticas de control de acceso para el comercio electrónico y la Web

Los entornos de comercio electrónico (E-commerce) se caracterizan por transacciones que se realizan de forma electrónica. Requieren políticas de control de acceso detalladas que van más allá de las de los DBMSs tradicionales. En los entornos de bases de datos convencionales, el control de acceso se realiza normalmente mediante un conjunto de autorizaciones definidas por los encargados de la seguridad o por los usuarios de acuerdo a determinadas políticas de seguridad. Este paradigma no se ajusta demasiado bien a un entorno dinámico como el comercio electrónico. Además, en un entorno de comercio electrónico los recursos a proteger no son sólo datos de tipo tradicional, sino que incluyen también el conocimiento y la experiencia. Estas peculiaridades requieren más flexibilidad al especificar políticas de control de acceso. El mecanismo de control de

acceso debe ser lo suficientemente flexible para proporcionar un amplio espectro de objetos heterogéneos a proteger.

Un segundo requisito relacionado con esto es la posibilidad de controlar el acceso basado en contenidos. El control de acceso basado en contenidos permite expresar políticas de control de acceso que tomen en cuenta el contenido del objeto a proteger. Para proporcionar el control de acceso basado en contenidos, las políticas de control de acceso deben permitir la inclusión de condiciones basadas en el contenido del objeto.

Un tercer requisito está relacionado con la heterogeneidad de los sujetos, lo cual necesita de políticas de control basadas en las características del usuario y sus cualificaciones, en lugar de basarse en características específicas e individuales (por ejemplo, los identificadores de usuario). Una posible solución para tener en cuenta los perfiles de usuario en la formulación de políticas de control de acceso es la existencia de la noción de credenciales. Una **credencial** es un conjunto de propiedades en relación con un usuario que son relevantes para los propósitos de seguridad (por ejemplo, la edad o el puesto dentro de la organización). Por ejemplo, al usar credenciales podemos simplemente formular políticas del tipo *Sólo los empleados fijos con cinco o más años de antigüedad pueden acceder a documentos relacionados con la organización interna del sistema*.

Se piensa que el lenguaje XML puede jugar un papel clave en el control de acceso en las aplicaciones de comercio electrónico⁴ ya que XML se está convirtiendo en el lenguaje de representación habitual en el intercambio de documentos en la Web y también en el lenguaje para el comercio electrónico. Según esto, por una parte existe la necesidad de hacer que las representaciones XML sean seguras proporcionando mecanismos de control de acceso diseñados específicamente para la protección de los documentos XML. Por otra parte, la información de control de acceso (es decir, las políticas de control de acceso y las credenciales de usuario) se puede expresar utilizando XML. El Lenguaje de marcado de servicios de directorio (DSML) proporciona una base para lo siguiente: un estándar para la comunicación con los servicios de directorio que serán responsables de proporcionar y autenticar las credenciales de usuario. La presentación uniforme de los objetos a proteger y de las políticas de control de acceso se puede aplicar a las propias políticas y credenciales. Por ejemplo, algunas propiedades de las credenciales (como el nombre de usuario) pueden ser accesibles a todo el mundo, mientras que otras propiedades pueden ser visibles sólo a ciertas clases restringidas de usuarios. Además, el uso de un lenguaje basado en XML para especificar credenciales y políticas de control de acceso facilita el envío seguro de las credenciales y la exportación de las políticas de control de acceso.

23.4 Introducción a la seguridad en bases de datos estadísticas

Las bases de datos estadísticas se utilizan principalmente para obtener estadísticas sobre poblaciones diversas. La base de datos puede contener información confidencial sobre individuos que debería ser protegida de su acceso por parte de los usuarios. Sin embargo, a los usuarios se les permite obtener información estadística sobre la población, como medias de valores, totales, contadores, máximos, mínimos y desviaciones estándar. Las técnicas que se han desarrollado para proteger la privacidad de la información de las personas se encuentran fuera del ámbito de este libro. Ilustraremos el problema mediante un ejemplo muy sencillo, basado en la relación que se muestra en la Figura 23.3. Se trata de una relación PERSONA con los atributos Nombre, Dni, Ingresos, Direcc, Ciudad, Estado, CP, Sexo y Estudios.

Figura 23.3. El esquema de la relación PERSONA para ilustrar la seguridad en bases de datos estadísticas.

PERSONA

Nombre	Dni	Ingresos	Direcc	Ciudad	Provincia	CP	Sexo	Estudios
--------	-----	----------	--------	--------	-----------	----	------	----------

⁴ Véase Thuraisingham y otros (2001).

Una **población** es un conjunto de tuplas de una relación (tabla) que satisfacen una condición de selección. Según esto, cada condición de selección en la relación PERSONA especificará una población en particular de tuplas de PERSONA. Por ejemplo, la condición Sexo = 'H' especifica la población de varones; la condición ((Sexo = 'F') AND (Estudios = 'Licenciado' OR Estudios = 'Doctor')) especifica la población femenina que tiene una titulación de Licenciado o Doctor; y la condición Ciudad = 'Madrid' especifica la población que vive en Madrid.

Las consultas estadísticas implican aplicar funciones estadísticas a una población de tuplas. Por ejemplo, podríamos querer obtener el número de individuos de una población o los ingresos medios de la población. Sin embargo, a los usuarios estadísticos no se les permite obtener datos individuales, como los ingresos de una persona determinada. Las técnicas de **seguridad en bases de datos estadísticas** deben prohibir la obtención de datos individuales. Esto se puede conseguir mediante la prohibición de consultas que obtengan valores de atributos y permitiendo sólo las consultas que impliquen funciones estadísticas de agregación como COUNT, SUM, MIN, MAX, AVERAGE y STANDARD DEVIATION. A las consultas de este tipo se les llama a veces **consultas estadísticas**.

Es responsabilidad de un sistema de gestión de bases de datos asegurar la confidencialidad de la información relacionada con los individuos y, a la vez, proporcionar a los usuarios resúmenes estadísticos de los datos acerca de esos individuos. Garantizar la **protección de la privacidad** de los usuarios en una base de datos estadística es lo más importante; su incumplimiento aparece documentado en el siguiente ejemplo.

En algunos casos es posible **deducir** los valores de tuplas individuales a partir de una secuencia de consultas estadísticas. Esto es particularmente cierto cuando las condiciones dan como resultado una población formada por un número pequeño de tuplas. Como muestra, supongamos las siguientes consultas estadísticas:

**C1: SELECT COUNT (*) FROM PERSONA
WHERE <condición>;**

**C2: SELECT AVG (Ingresos) FROM PERSONA
WHERE <condición>;**

Supongamos ahora que estamos interesados en encontrar el Sueldo de Ana Gómez y que sabemos que tiene el título de Doctora y que vive en la ciudad de Getafe, Madrid. Ejecutaríamos la consulta estadística C1 con la siguiente condición

(Estudios='Doctor' AND Sexo='F' AND Ciudad='Getafe' AND Provincia='Madrid')

Si obtenemos un resultado igual a 1 en esta consulta, podremos ejecutar la consulta C2 con la misma condición y descubrir el sueldo de Ana Gómez. Incluso si el resultado de C1 con la condición anterior no es 1 pero es un valor pequeño (por ejemplo, 2 ó 3), podremos ejecutar consultas estadísticas usando las funciones MAX, MIN y AVERAGE para identificar el posible rango de valores para el Sueldo de Ana Gómez.

La posibilidad de deducir información individual a partir de consultas estadísticas se reduce si no se permiten las consultas estadísticas siempre que el número de tuplas de la población especificada por la condición de selección esté por debajo de determinado nivel. Otra técnica para prohibir la obtención de información individual es imposibilitar las secuencias de consultas que hagan referencia repetidamente a la misma población de tuplas. También es posible introducir deliberadamente ligeras imperfecciones o *ruido* en los resultados de las consultas estadísticas para hacer que sea difícil deducir información individual a partir de los resultados. Otra técnica consiste en particionar la base de datos. El particionamiento implica que los registros se almacenan en grupos de un tamaño mínimo; las consultas pueden hacer referencia a cualquier grupo o conjunto de grupos completos, pero nunca a subconjuntos de registros dentro de un grupo. Los lectores interesados en el tema pueden dirigirse a la Bibliografía Seleccionada que se encuentra al final de este capítulo si desean una revisión de estas técnicas.

23.5 Introducción al control de flujo

El control de flujo regula la distribución o flujo de información entre objetos accesibles. Un flujo entre el objeto X y el objeto Y sucede cuando un programa lee valores de X y escribe valores en Y . Los **controles de flujo** comprueban que la información contenida en algunos objetos no fluye explícita o implícitamente hacia objetos menos protegidos. De este modo, un usuario no puede poner de forma indirecta en Y lo que no pueda obtener de forma directa en X . El control de flujo activo surgió a principios de los años 70. La mayoría de los flujos de control emplean algún concepto de clase de seguridad; la transferencia de información desde un emisor hasta un receptor sólo se permite si la clase de seguridad del receptor tiene al menos tantos privilegios como la del emisor. Entre los ejemplos de control de flujo podemos incluir el evitar que un programa de servicio guarde datos confidenciales de un cliente y el bloqueo de la transmisión de datos secretos militares a un usuario desconocido.

Una **política de flujo** especifica los canales a través de los cuales se permite que se mueva la información. La política de flujo más sencilla especifica sólo dos clases de información: confidencial (C) y no confidencial (N), y permite todos los flujos excepto aquellos que se dirigen desde la clase C hasta la clase N . Esta política puede resolver el problema de aislamiento que surge cuando un programa trata datos como información sobre clientes, algunos de los cuales pueden ser confidenciales. Por ejemplo, un programa de servicio para cálculo de impuestos podría estar autorizado a guardar la dirección del cliente y el importe del servicio prestado, pero no los ingresos de ese cliente o sus deducciones de impuestos.

Los mecanismos de control de acceso son los encargados de comprobar las autorizaciones de los usuarios para acceder a los recursos: sólo se ejecutan las operaciones autorizadas. Se pueden reforzar los controles de flujo mediante un mecanismo ampliado de control de acceso, lo cual implica la asignación de una clase de seguridad (llamada normalmente *autorización*) a cada programa en ejecución. Al programa se le permite leer un segmento de memoria en particular sólo si su clase de seguridad es tan alta como la del segmento. Se le permite escribir en un segmento sólo si su clase es de tan bajo nivel como la del segmento. Esto nos asegura, de forma automática, que la información transmitida por el usuario se puede mover desde una clase más alta hacia una clase más baja. Por ejemplo, un programa militar con una autorización de tipo secreto podría leer sólo objetos de tipo no clasificado o confidencial y escribir sólo en objetos de tipo secreto o alto secreto.

Se pueden distinguir dos tipos de flujo: *flujos explícitos*, que tienen lugar como consecuencia de instrucciones de asignación como $Y := f(X_1, X_n)$; y *flujos implícitos* generados por instrucciones condicionales, como $\text{if } f(X_{m+1}, \dots, X_n) \text{ then } y := f(X_1, X_m)$.

Los mecanismos de control de flujo deben comprobar que sólo se ejecutan los flujos autorizados, sean de tipo explícito o implícito. Para garantizar la seguridad en los flujos de información se deben cumplir un conjunto de reglas. Las reglas pueden ser expresadas mediante relaciones de flujo entre clases y pueden ser asignadas a la información para declarar los flujos autorizados dentro de un sistema. (Un flujo de información de A a B se produce cuando la información asociada a A afecta al valor de la información asociada a B . El flujo aparece como resultado de las operaciones que hacen que la información pase de un objeto al otro). Este tipo de relaciones puede definir, para el caso de una clase, el conjunto de clases en donde puede fluir la información (clasificada en esa clase), o puede especificar las relaciones específicas entre dos clases que tienen que ser comprobadas entre dos clases para permitir que la información fluya de la una a la otra. En general, los mecanismos de control de flujo implementan los controles asignando una etiqueta a cada objeto y especificando la clase de seguridad del objeto. Las etiquetas serán utilizadas posteriormente para verificar las relaciones de flujo definidas en el modelo.

23.5.1 Canales ocultos

Un canal oculto permite las transferencias de información que violan la seguridad o la política de seguridad. En concreto, un **canal oculto** permite que la información pase desde un nivel de clasificación más alto hacia

un nivel de clasificación más bajo a través de algún medio que no es el normal. Los canales ocultos se pueden clasificar en dos grandes grupos: canales de temporización y de almacenamiento. Lo que distingue a ambos es que en un **canal de temporización** la información se transmite tras una temporización de eventos o procesos mientras que los **canales de almacenamiento** no necesitan una sincronización temporal, ya que la información se transmite accediendo a información del sistema o a información que sería inaccesible al usuario de otro modo.

Como ejemplo sencillo de canal oculto, tomemos un sistema de base de datos distribuida en la cual dos nodos tienen niveles de seguridad de usuario igual a secreto (S) y no clasificado (U). Para que se ejecute una transacción, ambos nodos tienen que ponerse de acuerdo. Ambos en conjunto sólo pueden realizar operaciones que sean consistentes con la propiedad-*, que indica que en cualquier transacción, el elemento S no puede escribir o pasar información al elemento U. Sin embargo, si ambos sitios se ponen de acuerdo para establecer un canal oculto entre ellos, una transacción que contenga datos secretos podrá ser ejecutada incondicionalmente por el sitio U, pero el sitio S sólo podrá hacer lo mismo de algún modo predefinido y convenido para que determinada información pueda ser transferida desde el elemento S al elemento U, violando la propiedad-*. Esto se puede conseguir en los casos en los que la transacción se ejecute repetidamente y las acciones llevadas a cabo por el elemento S comuniquen información implícitamente al elemento U. Ciertas medidas, como el bloqueo, que vimos en los Capítulos 17 y 18, impiden la escritura concurrente de la información por parte de usuarios con distintos niveles de seguridad en los mismos objetos, impidiendo los canales ocultos de tipo almacenamiento. Los sistemas operativos y las bases de datos distribuidas proporcionan control sobre la multiprogramación de operaciones que permiten compartir recursos sin la posibilidad de que un proceso o programa se apropie de la memoria de otro o de los recursos del sistema, previniendo de este modo los canales ocultos de tipo temporización. En general, los canales ocultos no son un gran problema cuando las implementaciones de las bases de datos están bien construidas y son robustas. Sin embargo, ciertos esquemas pueden ser engañados por usuarios inteligentes que realicen transferencias de información de manera implícita.

Algunos expertos en seguridad creen que una forma de evitar los canales ocultos es impedir a los programadores obtener acceso a datos confidenciales que serán procesados por un programa una vez se encuentre en ejecución. Por ejemplo, un programador que trabaje para un banco no tiene necesidad de acceder a los nombres o al saldo de la cuenta de los clientes. Los programadores que trabajen para compañías de bolsa no necesitan saber qué órdenes de compra o de venta van a realizar los clientes. Durante las pruebas del programa podría estar justificado el acceso a partes de los datos reales o a datos de ejemplo, pero no una vez que se ha aceptado el programa para su uso en el entorno real.

23.6 Cifrado e infraestructuras de clave pública

Los anteriores métodos de acceso y de control de flujo, a pesar de ser medidas de control bastante fuertes, podrían no ser capaces de proteger a las bases de datos contra algunas amenazas. Supongamos que vamos a transmitir datos, pero estos caen en las manos de un usuario no autorizado. En esa situación, utilizando el cifrado podemos disfrazar el mensaje para que incluso en el caso de que la transmisión sea interceptada, el contenido del mensaje no sea descubierto. El cifrado es un medio de mantener la seguridad de los datos en un entorno no seguro. El cifrado consiste en aplicar un algoritmo de cifrado a los datos utilizando una clave de cifrado predefinida. Los datos resultantes tienen que ser descifrados mediante una clave de descifrado para recuperar los datos originales.

23.6.1 Los datos y estándares avanzados de cifrado

El estándar de cifrado de datos (DES) es un sistema desarrollado por el gobierno de Estados Unidos para su uso por el público en general. Es ampliamente aceptado como estándar de cifrado dentro y fuera de EE.UU. DES proporciona cifrado extremo a extremo en el canal existente entre el emisor A y el receptor B. El algo-

ritmo DES es una cuidadosa y compleja combinación de dos de los bloques fundamentales usados en la construcción del cifrado: la sustitución y la permutación (transposición). La fuerza del algoritmo deriva de la aplicación repetida de estas dos técnicas durante un total de 16 ciclos. El texto plano (la forma original del mensaje) se cifra en bloques de 64 bits. Aunque la clave tiene una longitud de 64 bits, en realidad puede ser cualquier número de 56 bits. Tras cuestionar la validez de DES, el Instituto Nacional de Estándares (NIST, *National Institute of Standards*) introdujo el algoritmo AES (*Advanced Encryption Standards*). Este algoritmo tiene un tamaño de bloque de 128 bits, a diferencia de los 56 bits del bloque de DES, y puede utilizar claves de 128, 192 o 256 bits, a diferencia de los 56 bits de la clave de DES. AES introduce más claves posibles, a diferencia de DES y, por tanto, el tiempo necesario para romperlo es mayor.

23.6.2 Cifrado con clave pública

En 1976, Diffie y Hellman propusieron un nuevo tipo de sistema de cifrado, al cual llamaron **cifrado con clave pública**. Los algoritmos de clave pública se basan en funciones matemáticas en lugar de patrones de bits. También requieren el uso de dos claves independientes, a diferencia del cifrado convencional que utiliza una única clave. El uso de dos claves puede tener grandes consecuencias en el ámbito de la confidencialidad, la distribución de claves y la autenticación. A las dos claves que se usan para el cifrado con clave pública se les llama **clave pública** y **clave privada**. Invariablemente, la clave privada se mantiene en secreto, pero se le denomina *clave privada* en lugar de *clave secreta* (la clave que se usa en el cifrado convencional) para evitar confusiones con el cifrado convencional.

Un esquema, o infraestructura, de cifrado con clave pública tiene seis componentes:

1. **Texto plano o sin formato.** Son los datos o el mensaje legible que se toma como entrada al algoritmo.
2. **Algoritmo de cifrado.** Este algoritmo realiza diversas transformaciones al texto plano.
3. y 4. **Claves pública y privada.** Son un par de claves que han sido elegidas de forma que si una de ellas se utiliza para cifrar, la otra se podrá usar para descifrar. Las transformaciones realizadas por el algoritmo de cifrado dependen de la clave pública o privada que se suministre como entrada.
5. **Texto cifrado.** El mensaje ininteligible generado como resultado. Depende del texto plano y de la clave. Para un mensaje en concreto, dos claves diferentes producirán dos textos cifrados diferentes.
6. **Algoritmo de descifrado.** Este algoritmo toma como entrada el texto cifrado y la clave correspondiente y genera el texto plano original.

Como indica su nombre, la clave pública del par se hace pública a los demás para que la utilicen, mientras que la clave privada sólo la conoce su propietario. Un algoritmo de cifrado con clave pública de propósito general se basa en una clave para el cifrado y otra diferente, pero relacionada con la anterior, para descifrar. Los pasos más importantes se muestran a continuación:

1. Cada usuario genera un par de claves a utilizar en el cifrado y descifrado de mensajes.
2. Cada usuario coloca una de las dos claves en un registro público u otro archivo accesible. Ésta es la clave pública. La otra clave se mantendrá en privado.
3. Si un emisor desea enviar un mensaje privado a un receptor, el emisor cifrará el mensaje utilizando la clave pública del receptor.
4. Cuando el receptor reciba el mensaje, lo descifrá utilizando la clave privada del receptor. Ningún otro destinatario puede descifrar el mensaje, ya que sólo el receptor conoce su clave privada.

Algoritmo RSA de cifrado con clave pública. Uno de los primeros esquemas de clave pública fue introducido en 1978 por Ron Rivest, Adi Shamir y Len Adleman del MIT y se llama esquema RSA siguiendo sus iniciales. El esquema RSA ha sido desde entonces uno de los más extendidos y uno de los modelos de cifrado con clave pública más implementados. El algoritmo de cifrado RSA incorpora nociones de la teoría numé-

rica combinadas con la dificultad de obtención de los factores primos de un elemento. El algoritmo RSA también trabaja con la aritmética modular ($\text{mod } n$).

Se utilizan dos claves, d y e para el cifrado y descifrado. Una propiedad importante es que son intercambia-bles entre sí. Se elige n como el entero más grande que sea el producto de dos números primos grandes y distintos, a y b . La clave de cifrado e es un número elegido al azar entre 1 y n que sea primo relativo a $(a - 1) \times (b - 1)$. El bloque de texto plano P se cifra como $P^e \text{ mod } n$. Ya que la exponenciación se realiza en módulo n , resulta difícil factorizar P^e para descubrir el texto plano que ha sido cifrado. Sin embargo, la clave de cifrado d se elige cuidadosamente para que $(P^e)^d \text{ mod } n = P$. La clave de cifrado d se puede calcular a partir de la condición $d \times e = 1 \text{ mod } ((a - 1) \times (b - 1))$. Por tanto, al receptor autorizado que conozca d le basta con calcular $(P^e)^d \text{ mod } n = P$ y recuperará P sin tener que factorizar P^e .

23.6.3 Firmas digitales

Una firma digital es un ejemplo del uso de las técnicas de cifrado para proporcionar servicios de autenticación en las aplicaciones de comercio electrónico. Al igual que en el caso de la firma escrita a mano, una **firma digital** es un mecanismo para asociar una identificación única para un individuo a un cuerpo de texto. La marca debe ser perdurable, en el sentido de que otros deberían ser capaces de comprobar que la firma procede de su emisor.

Una firma digital está formada por una secuencia de símbolos. Si la firma digital de una persona fuese siempre la misma para todos los mensajes, alguien podría falsificarla fácilmente con sólo copiar la secuencia de símbolos. Por tanto, las firmas deben ser diferentes en cada uso. Esto se puede conseguir haciendo que cada firma digital se genere en función del mensaje que se está firmando, junto con una marca de tiempo. Para que sea única para cada usuario y a prueba de falsificaciones, cada firma digital debe depender también de algún número secreto que sea único para cada firmante. Según esto, en general, una firma digital a prueba de falsificaciones debe depender del mensaje y de un número secreto único para el firmante. Sin embargo, a la hora de comprobar la firma, no debe ser necesario conocer ningún número secreto. Las técnicas de clave pública son el mejor medio de creación de firmas digitales que cumplan estas propiedades.

23.7 Mantenimiento de la privacidad

Mantener la privacidad es un desafío cada vez mayor para la seguridad en las bases de datos y para los expertos en privacidad. En cierto modo, para mantener la privacidad de los datos deberíamos incluso limitar la ejecución de los análisis y obtención de datos a gran escala. Las técnicas utilizadas más habitualmente para tratar estos temas son evitar la construcción de grandes almacenes de datos centralizados en un único repositorio con toda la información importante. Otra posible medida es la modificación o la distorsión intencionada de los datos.

Si todos los datos se encuentran disponibles en un único almacén, la violación de la seguridad en sólo un único repositorio podría dejar accesibles todos los datos. Si se evitan los almacenes de datos centralizados y se utilizan algoritmos distribuidos de obtención de datos se minimiza el intercambio de datos necesarios para el desarrollo de modelos válidos a nivel global. Si se modifican, distorsionan o se hacen anónimos los datos, podemos también mitigar los riesgos en la privacidad asociados a la minería de datos. Esto se puede conseguir eliminando la información de identidad en los datos obtenidos e inyectando ruido en ellos. Sin embargo, al utilizar estas técnicas deberíamos prestar atención a la calidad de los datos resultantes en la base de datos, ya que podrían quedar sometidos a demasiadas modificaciones. Debemos ser capaces de calcular los errores que puedan ser introducidos por estas modificaciones.

La privacidad es un área importante en el desarrollo de las investigaciones en gestión de bases de datos. Resulta complicado debido a su naturaleza multidisciplinar y a los temas relacionados con la subjetividad en la interpretación de la privacidad, fiabilidad, etc. Como ejemplo, tomemos los registros y las transacciones

médicas y de tipo legal, que deben mantener ciertos requisitos de privacidad cuando son definidos y reforzados. Actualmente, también se están dedicando esfuerzos a proporcionar control de acceso y privacidad en los dispositivos móviles. Los DBMS tienen la necesidad de técnicas sólidas para el almacenamiento eficaz de información relevante en cuanto a seguridad sobre dispositivos de pequeño tamaño, y también de técnicas de negociación de relaciones de confianza. Aún resulta un problema importante saber dónde almacenar información relacionada con identidades de usuario, perfiles, credenciales y permisos, y cómo utilizarla para la identificación fiable de los usuarios. Ya que en esos entornos se genera un gran flujo de datos, es necesario desarrollar técnicas eficientes para el control de accesos que se encuentren integradas con las técnicas de procesamiento continuo de consultas. Por último, se debe asegurar la privacidad de datos relacionados con la localización física de usuarios que hayan sido adquiridos a través de detectores y redes de comunicaciones.

23.8 Retos en la seguridad en las bases de datos

Pensemos en el gran crecimiento en volumen y en la rapidez de aparición de las amenazas a las bases de datos y a la información de valor; los esfuerzos en investigación se deben orientar a los siguientes temas: calidad de los datos, derechos de propiedad intelectual y supervivencia en bases de datos.

23.8.1 Calidad de los datos

La comunidad de bases de datos necesita técnicas y soluciones organizativas para evaluar y demostrar la calidad de los datos. Estas técnicas pueden incluir mecanismos sencillos como los sellos de calidad que se insertan en los sitios web. Necesitamos también técnicas que proporcionen reglas semánticas de integridad más efectivas y herramientas para la evaluación de la calidad de los datos basadas en técnicas como el enlace de registros. Se necesitan también técnicas de recuperación a nivel de aplicación para la reparación automática de datos incorrectos. Actualmente, para estos temas se está intentando el uso de las herramientas ETL (extraer, transformar, cargar) usadas ampliamente en la carga de datos en los almacenes de datos (consulte la Sección 29.4).

23.8.2 Derechos de propiedad intelectual

Con la extensión del uso de Internet y de las tecnologías Intranet, los aspectos legales e informativos de los datos se están convirtiendo en una de las principales preocupaciones de las organizaciones. Para tratar estos temas, se han propuesto recientemente unas técnicas de marca de agua digital para bases de datos. El propósito principal de las marcas de agua digitales es proteger el contenido frente a la duplicación y distribución no autorizadas mediante la creación de una prueba sobre la propiedad del contenido. Tradicionalmente, esto se ha conseguido basándose en la disponibilidad de un amplio dominio de ruido dentro del cual se puede alterar el objeto manteniendo, a su vez, sus propiedades esenciales. Sin embargo, es necesario seguir trabajando para evaluar la robustez de estas técnicas y para descubrir diferentes modelos orientados a prevenir la violación de los derechos de propiedad intelectual.

23.8.3 Supervivencia en las bases de datos

Los sistemas de bases de datos necesitan mantenerse operativos y seguir desarrollando sus funciones a pesar de que se produzcan sucesos que les afecten negativamente, como los ataques contra la información que contienen. Un DBMS, además de estar preparado contra un ataque y de poder detectarlo en caso de que suceda, debería ser capaz de realizar lo siguiente:

- **Aislamiento.** Llevar a cabo acciones inmediatas para eliminar el acceso del atacante al sistema y aislar o contener el problema para evitar que se extienda.
- **Evaluación de los daños.** Determinar el alcance del problema, incluyendo las funciones afectadas y los datos corrompidos.
- **Reconfiguración.** Reconfigurar para permitir que continúe en funcionamiento aún estando dañado mientras se lleva a cabo la recuperación.
- **Reparación.** Reparación de los datos perdidos o corrompidos y reparación o reinstalación de las funciones dañadas del sistema para establecer un nivel normal de funcionamiento.
- **Tratamiento de los fallos.** Hasta el punto que sea posible, identificar las debilidades que han sido aprovechadas por el ataque y tomar medidas para prevenir que vuelva a ocurrir.

El objetivo de un atacante de un sistema es dañar el funcionamiento de la organización y cumplir su misión mediante la interrupción de sus sistemas de información. El objetivo específico de un ataque puede ser bien el sistema o bien sus datos. Aunque los ataques que dejan el sistema fuera de servicio son muy graves, deben ser planificados para poder conseguir el objetivo del atacante, ya que esos ataques recibirán una atención inmediata y enérgica para volver a poner el sistema en estado operativo, para diagnosticar cómo se produjo el ataque y para instalar medidas preventivas.

A fecha de hoy, los temas relacionados con la supervivencia de las bases de datos aún no han sido investigados suficientemente. Se necesita mucha más investigación dedicada a las técnicas y metodologías que aseguren la supervivencia de las bases de datos.

23.9 Resumen

En este capítulo hemos visto varias técnicas para reforzar la seguridad en las bases de datos. Hemos presentado diferentes amenazas a las bases de datos en términos de pérdida de integridad, disponibilidad y confidencialidad. Revisamos los tipos de medidas de control que tratan con estos problemas: control de accesos, control de inferencias, control de flujo y cifrado.

El reforzamiento de la seguridad tiene relación con el control de los accesos al sistema de base de datos en su totalidad y con el control de la autorización para acceder a determinadas partes de una base de datos. Lo primero se lleva a cabo habitualmente mediante la asignación de cuentas con contraseñas a los usuarios. Lo último se puede realizar utilizando un sistema de concesión y revocación de privilegios a cuentas individuales para acceder a partes concretas de la base de datos. A este modelo se le denomina, por lo general, control de acceso discrecional. Mostramos algunos comandos SQL para conceder y revocar privilegios, e ilustramos su uso mediante ejemplos. Posteriormente, hicimos un repaso a los mecanismos de control de acceso obligatorio que refuerzan la seguridad multinivel. Éstos requieren la clasificación de los usuarios y de los valores de los datos en clases de seguridad y refuerzan las reglas que prohíben el flujo de información desde niveles de seguridad altos hacia los más bajos. Se presentaron algunos de los conceptos clave que subyacen al modelo relacional multinivel, incluidos el filtrado y la poliinstanciación. Se mostró el control de acceso basado en roles, en el cual se asignan privilegios basados en roles desempeñados por los usuarios. Repasamos brevemente el problema del control de acceso a las bases de datos estadísticas para proteger la privacidad de la información de los individuos a la vez que se proporciona acceso estadístico a las poblaciones de registros. Los temas relacionados con el control de flujo y los problemas asociados a los canales ocultos se revisaron a continuación. También vimos el cifrado de datos, incluyendo la infraestructura de clave pública y las firmas digitales. Resaltamos la importancia de los asuntos relacionados con la privacidad y dimos unas ideas sobre técnicas de salvaguardia de la privacidad. Revisamos varios retos a la seguridad entre los que incluimos la calidad de los datos, los derechos de propiedad intelectual y la supervivencia de los datos.

Preguntas de repaso

- 23.1. Comente qué se entiende por cada uno de los siguientes términos: autorización en bases de datos, control de acceso, cifrado de datos, cuentas privilegiadas (de sistema), auditorías de bases de datos y registro de auditoría.
Comente los tipos de privilegios a nivel de cuenta y a nivel de relación.
- 23.2. ¿Qué cuenta es la que se designa como propietaria de la relación? ¿Qué privilegios tiene el propietario de una relación?
- 23.3. ¿Cómo se usa el mecanismo de vista como mecanismo de autorización?
- 23.4. ¿Qué significa conceder un privilegio?
- 23.5. ¿Qué significa revocar un privilegio?
- 23.6. Comente el sistema de propagación de privilegios y las restricciones impuestas por los límites de propagación horizontal y vertical.
- 23.7. Enumere los tipos de privilegios disponibles en SQL.
- 23.8. ¿Cuál es la diferencia entre control de acceso *obligatorio* y *discrecional*?
- 23.9. ¿Cuáles son las clasificaciones habituales de seguridad? Comente la propiedad de seguridad simple y la propiedad-*, y explique qué justificación hay tras estas reglas para reforzar la seguridad multinivel.
- 23.10. Describa el modelo de datos relacional multinivel. Defina los siguientes términos: *clave aparente*, *poliinstantiación* y *filtrado*.
- 23.11. ¿Cuáles son los beneficios de utilizar DAC o MAC?
- 23.12. ¿Qué es el control de acceso basado en roles? ¿De qué modo es superior a DAC y MAC?
- 23.13. ¿Qué es una base de datos estadística? Comente los problemas de seguridad en las bases de datos estadísticas.
- 23.14. ¿Qué relación tiene la privacidad con la seguridad en bases de datos estadísticas? ¿Qué medidas se pueden tomar para asegurar cierto grado de privacidad en las bases de datos estadísticas?
- 23.15. ¿Qué es el control de flujo como medida de seguridad? ¿Qué tipos de control de flujo existen?
- 23.16. ¿Qué son los canales ocultos? Ponga un ejemplo de canal oculto.
- 23.17. ¿Cuál es el objetivo del cifrado? ¿Qué proceso está involucrado en el cifrado de los datos y su posterior recuperación en el otro extremo?
- 23.18. Proporcione un ejemplo de un algoritmo de cifrado y explique cómo funciona.
- 23.19. Repita la pregunta anterior para el conocido algoritmo RSA.
- 23.20. ¿Qué es el esquema de infraestructura de clave pública? ¿Cómo proporciona seguridad?
- 23.21. ¿Qué son las firmas digitales? ¿Cómo funcionan?

Ejercicios

- 23.22. ¿Cómo se puede mantener la privacidad de los datos en una base de datos?
- 23.23. ¿Cuáles son los mayores retos actuales para la seguridad en las bases de datos?
- 23.24. Observe el esquema de base de datos relacional de la Figura 5.5. Suponga que todas las relaciones fueron creadas por (y, por tanto, propiedad de) el usuario *X*, que desea conceder los siguientes privilegios a las cuentas de usuario *A*, *B*, *C*, *D* y *E*:
 - a. La cuenta *A* puede modificar o acceder a cualquier relación excepto SUBORDINADO y puede otorgar cualquiera de estos privilegios a otros usuarios.

- b. La cuenta *B* puede acceder a todos los atributos de EMPLEADO y DEPARTAMENTO excepto a Sueldo, DniDirector, y FechaIngresoDirector.
 - c. La cuenta *C* puede modificar o acceder a TRABAJA_EN, pero sólo puede acceder a los atributos Nombre, Apellido1, Apellido2 y Dni de EMPLEADO y a los atributos NombreProyecto y NumProyecto de PROYECTO.
 - d. La cuenta *D* puede acceder a cualquier atributo de EMPLEADO o SUBORDINADO y puede modificar SUBORDINADO.
 - e. La cuenta *E* puede acceder a cualquier atributo de EMPLEADO, pero sólo a las tuplas de EMPLEADO que tengan Dno = 3.
 - f. Escriba sentencias SQL para conceder estos privilegios. Utilice vistas en donde sea adecuado.
- 23.25.** Supongamos que se va a dar el privilegio (a) del Ejercicio 23.1 con la opción GRANT OPTION pero sólo para que la cuenta *A* pueda concederlo a, como mucho, cinco cuentas, y cada una de estas cuentas podrá propagar el privilegio a otras cuentas pero *sin* el privilegio GRANT OPTION. ¿Cuáles serían los límites de propagación horizontal y vertical en este caso?
- 23.26.** Observe la relación que aparece en la Figura 23.2(d). ¿Cómo se mostraría a un usuario con clasificación U? Supongamos que un usuario de clasificación U intenta actualizar el salario de 'Gómez' a 50.000; ¿cuál sería el resultado de esta acción?

Bibliografía seleccionada

La autorización basada en la concesión y revocación de privilegios se propuso en el DBMS experimental SYSTEM R y se muestra en Griffiths y Wade (1976). Varios libros revisan la seguridad en bases de datos y en sistemas de computadores en general, entre los cuales se encuentran los libros de Leiss (1982a) y Fernández y otros (1981).

En muchos documentos se revisan distintas técnicas para el diseño y la protección de las bases de datos estadísticas. Entre ellos podemos citar McLeish (1989), Chin y Ozsoyoglu (1981), Leiss (1982), Wong (1984), y Denning (1980). Ghosh (1984) revisa el uso de bases de datos estadísticas para el control de la calidad. También hay muchos documentos en los que se discute la criptografía y el cifrado de datos, entre los que están Diffie y Hellman (1979), Rivest y otros (1978), Akl (1983), Pfleeger (1997), Omura y otros (1990), Stallings (2000), y Iyer y otros (2004).

La seguridad multinivel se repasa en Jajodia y Sandhu (1991), Denning y otros (1987), Smith y Winslett (1992), Stachour y Thuraisingham (1990), Lunt y otros (1990), y Bertino y otros (2001). Una visión general de temas de investigación en seguridad en bases de datos se muestra en Lunt y Fernández (1990), Jajodia y Sandhu (1991), Bertino y otros (1998), Castano y otros (1995), y Thuraisingham y otros (2001). Los efectos de la seguridad multinivel en el control de la concurrencia se muestran en Atluri y otros (1997). La seguridad en las bases de datos de la siguiente generación, la semántica y las bases de datos orientadas a objetos aparece en Rabbiti y otros (1991), Jajodia y Kogan (1990), y Smith (1990). Oh (1999) presenta un modelo para la seguridad discrecional y obligatoria. Los modelos de seguridad para las aplicaciones web y el control de acceso basado en roles se muestra en Joshi y otros (2001). Temas de seguridad para gestores en el contexto de las aplicaciones de comercio electrónico y la necesidad de modelos de evaluación de riesgos para la selección de medidas de control de seguridad adecuadas aparecen en Farahmand y otros (2005).

Los avances más recientes, así como los desafíos futuros en seguridad y privacidad de bases de datos, se discuten en Bertino y Sandhu (2005). El lenguaje XML y el control de accesos se revisan en Naedele (2003). Se puede encontrar más detalles sobre técnicas de mantenimiento de la privacidad en Vaidya y Clifton (2005), derechos de propiedad intelectual en Sion y otros (2004), y supervivencia de bases de datos en Jajodia y otros (1999).

